C++在线课程巅峰 二十年磨一剑

新高度 | 新方法 | 新经典

C++新经典

对象模型

王健伟◎编著
Wang Jianwei

C++
PROGRAMMING
LANGUAGE

OBJECT MODEL

清華大學出版社

C++
PROGRAMMING
LANGUAGE

OBJECT MODEL

C++新经典

对象模型

王健伟◎编著
Wang Jianwei

清華大學出版社
北京

内 容 简 介

本书是一部介绍C++对象模型知识的书，意在探究C++幕后的秘密，讲述C++编译器的故事。内容涉及很多不被常人所知的C++对象内部工作原理以及底层的一些具体实现机制方面的知识。

本书共分为7章：第1章介绍书中范例所涉及的演示环境搭建、如何创建基本的C++程序，以及对程序进行跟踪和断点调试的方法。第2章的主题是对象，包括以对象所占内存大小为讲解切入点详述对象结构的发展和演化；构造、拷贝构造、移动构造函数语义；深浅拷贝、成员初始化列表等内容。第3章的主题是虚函数，包括虚函数表指针、虚函数表的概念和细节；更进一步涉及多重继承的虚函数表分析和各种辅助工具的讲解。第4章的主题是数据语义学，包括类中的数据成员布局、数据成员存取，涉及单一继承、单类单继承、多重继承等情况下的数据成员布局；二层、三层结构时虚基类表内容、与this指针调整等问题。第5章的主题是函数语义学，包括普通成员函数、虚函数、静态成员函数调用方式；静态、动态绑定以及多态性的详细体现分析和析构函数必须是虚函数原因的着重解释；多重继承第二虚基类详谈、RTTI实现原理、inline函数细说等内容。第6章的主题是对象构造语义学，包括对象构造顺序，对象复制、析构函数语义；局部、全局对象的构造和析构，临时对象详细探讨等内容。第7章的主题是模板实例化语义学，重点对模板的实例化进行了详细的分析。

这是一本C++进阶类图书，要求学习者具备一定的C++开发基础，适合作为C++编程人员的参考读物。

本书封面贴有清华大学出版社防伪标签，无标签者不得销售。
版权所有，侵权必究。侵权举报电话：010-62782989　13701121933

图书在版编目(CIP)数据

C++新经典：对象模型/王健伟编著. —北京：清华大学出版社，2020.4
ISBN 978-7-302-55126-3

Ⅰ. ①C…　Ⅱ. ①王…　Ⅲ. ①C++语言—程序设计　Ⅳ. ①TP312.8

中国版本图书馆CIP数据核字(2020)第049481号

责任编辑：盛东亮　钟志芳
封面设计：李召霞
责任校对：梁　毅
责任印制：宋　林

出版发行：清华大学出版社
网　　址：http://www.tup.com.cn，http://www.wqbook.com
地　　址：北京清华大学学研大厦A座　　**邮　　编**：100084
社 总 机：010-62770175　　**邮　　购**：010-62786544
投稿与读者服务：010-62776969，c-service@tup.tsinghua.edu.cn
质量反馈：010-62772015，zhiliang@tup.tsinghua.edu.cn
课件下载：http://www.tup.com.cn，010-83470236
印 装 者：北京鑫海金澳胶印有限公司
经　　销：全国新华书店
开　　本：185mm×260mm　　**印　　张**：16.5　　**字　　数**：404千字
版　　次：2020年6月第1版　　**印　　次**：2020年6月第1次印刷
印　　数：1～2000
定　　价：79.00元

产品编号：086124-01

前言

PREFACE

本书是在读完了笔者的《C++新经典》一书后，建议阅读的第二本有关C++的书，是一本足以颠覆你以往许许多多认知的书，一本你绝不应该错过的书。

章节主题概要

C++对象模型知识久负盛名，在C++界具有很高的美誉度，这方面知识的学习更是被诸多颇具开发实力的行业前辈倾力推荐！

这本书的内容涉及很多不被常人所知的C++对象内部工作原理、底层的一些具体实现机制方面的知识，探究C++幕后的秘密，讲述C++编译器的故事。这是一本“提高个人C++开发修养，练习C++编程内功”的图书，定位为进阶类C++图书，这意味着本书的学习难度很高，需要学习者有中高级C++开发基础，也意味着书中的知识是想成为C++高级开发者甚至大师级开发者必须掌握的。

如果你想成为一名既会开车又会修车的老司机，如果你在C++面试中不想丢掉进入世界级大公司的机会，如果你想挑战一些有难度的技术，或者希望晋级、成长为C++高手甚至大师，C++对象模型方面的知识是必学的。

在读者对C++对象模型有了一个笼统和宽泛的概念后，就可以进一步了解C++对象模型研究的是哪方面知识了。

读者都知道，C++是一门支持面向对象程序设计的语言，而面向对象程序设计的三大特性是封装性、继承性、多态性。这些特性都需要系统内部(底层)的支持。C++编译器在程序员的背后默默地做了很多事情，例如，生成类对象时调用类的构造函数、释放类对象时调用类的析构函数；当系统认定需要某些成员函数(如拷贝构造函数)而程序员没有定义该成员函数时，编译器会自动合成出该成员函数等。

本书研究的就是系统内部都做了哪些事情，面向对象三大特性中的继承性和多态性的具体实现细节是怎么样的，实现起来开销如何等。既然是针对对象模型方面知识的研究探索，那么基本都是围绕类、对象来讨论问题。

如果按照正常的学习C++语言开发的学习旅程来说，可以不学这些知识，或者说也学不到这些知识，所以，笔者通过写这本书，专门讲解这方面的知识。

C++对象模型方面知识的掌握对于无形中提高程序员自身整体素质会起到一种非常关键的作用。本书定位为一本C++进阶图书，针对C++初学者和仅希望用C++应付普通开发的人，并不合适。但是，如果读者想成为大家公认的C++开发高手甚至C++专家或者大师级的人，C++对象模型知识是必需的。或者换句话说，这本书主要针对C++编程经验比较丰富并希望进一步提高自己的认知，希望进一步了解系统底层细节的中高级C++程序员。

阅读本书的基础要求

阅读本书要求熟练掌握 C++98(传统 C++)标准，如对类、对象、继承、虚函数、虚基类等概念都要有比较清晰的了解和认识，希望读者至少用过 1～2 年 C++，当对自身的能力进行比较客观的评估时，认为自己能够达到中级应用水平之后，再开始看这本书。如果读者认为自己当前的知识积累还不太够，那么建议先把基础的 C++语言知识学好，然后再阅读本书才更加合适。

在此，强烈推荐笔者所著的《C++新经典》一书，这是一本不可多得的打 C++语言编程基础的书。

本书面向的读者

这本经典、有趣并能带给读者意想不到精彩知识的书，适合有以下几种意愿的读者：

(1) 希望提升自身 C++开发内功。

本书能够在很大程度上拓展读者的眼界和思路，让读者了解在 C++语言之下，C++编译器究竟做了什么。通过认真研读本书，读者的认知层次、能力、对自己 C++编程的自信心都会提升一大截，请读者认真体会这种感受。

(2) 希望在面试中获得良好表现。

很多大公司 C++开发岗位的面试会问及对象模型方面的知识(读者不妨看一看以往中国三个著名互联网公司(BAT)的 C++面试题)，例如，“一个不包含任何成员的空类，sizeof 值是多少?”“C++如何通过虚函数表实现多态?”“一个对象访问普通成员函数和虚函数哪个更快?”“在什么情况下，析构函数需要是虚函数?”等。如果面试者此前从未接触过这方面的知识，很可能无法给出正确答案，从而直接导致丢掉这份工作机会。

(3) 希望获得成就感。

当听到别人夸赞自己或者自身表现更加突出的时候总会产生成就感，这种成就感也是让自己不断前进的最佳动力。对象模型方面的知识就是那种容易让人产生这种感觉的知识。掌握对象模型知识的人，也往往更容易成为他人眼中高大上的牛人。

本书特色

“C++对象模型”这个概念，并不是所有人都非常熟悉。但笔者相信，熟悉的人一定都读过《深度探索 C++对象模型》，这是一本在 C++界颇受赞誉的书，也正是借着这本书，“C++对象模型”这个概念被广大读者慢慢熟悉。

C++对象模型方面的知识比较分散，也比较抽象，理解起来有一定的难度。《深度探索 C++对象模型》是一本让人敬畏的书，出版至今已有 20 年，读过的人对它有着很高的评价。虽然瑕不掩瑜，但这本书的缺点也很突出：①语言晦涩，难以理解；②偏重理论，缺乏实践验证。这也让很多学习者望而却步或读完后一知半解，并未有特别大的收获，成为了很多学习者心中的遗憾。

很久以前，笔者就想写一本专门讲解 C++对象模型知识的书，但因各种原因未能成行，终于在不久前狠下决心，放下手头一切工作，集中精力做这件事。在本书中，使用大量的图解、范例代码进行演示，参考大量资料并将笔者多年一线开发实战经验的积累和体会融入书中，生动和全方位地阐述 C++对象模型的魅力所在，极大地降低学习者的理解和掌握门槛，相信凡是参与本书学习的 C++从业者，都会有一种由表及里整体实力、信心的全方位提升的感受，而作者本人，在写这本书的过程中，也有一种自身往更高层次迈进的喜悦之感！

翻看目录，本书从一个类对象所占用的内存空间大小为突破口讲起，涉及的主题包括对象、虚函数、数据语义学、函数语义学、对象构造语义学、模板实例化语义学等，每一章的内容都非常精彩，既长知识，又开眼界。目录的组织结构笔者参考了《深度探索 C++对象模型》一书，因为觉得以这种目录组织结构讲述问题的难度比较适中，学习顺序也比较合理。

大量学习过“设计模式”这个话题内容的读者，总是感觉学完“设计模式”后不会运用，说到根上，设计模式本身考验的是抽象思维能力，找出设计中的易变部分用虚函数去实现，就是如此简单。而在本书中，对虚函数的工作原理和过程有着非常深入的探讨。所以，通过本书的阅读，对于读者学习向来比较晦涩的设计模式类知识，也将会有着妙不可言的好处！

在成书的过程中，笔者尽最大努力通过网络寻找一切对本书有用的资料，尽量不遗漏任何重要的知识，笔者的心愿是通过自己的表达把这些知识更通俗地传达给读者，可能让读者只花费以往 1/5 甚至 1/10 的力气就能获得同等甚至更高质量的知识。书中没有哪个内容是笔者自己独创的，都是参考了大量资料，经过实践和整合，付出了数月的时间和极大的心血而写成的。在这里，笔者发自内心地感谢诸多 C/C++前辈在这个行业中多年的耕耘及对知识的积累和传播，感谢你们对他人的奉献。

本书阅读进度

不排除有基础和学习速度特别快的读者可以很快把本书的内容阅读完毕并掌握得很好，但通常来讲，笔者建议本书的阅读速度如下：

整本书一共 48 节(如 1.1 这样的标题算一节)，笔者建议大概每周平均学习两节。对于上班的读者，可以尽量将学习时间安排在晚上和周末，对于学习速度较快的读者，可以每周学习更多节数。整本书的学习难度比较平均。笔者认为每周学习两节还是能够做到的，这意味着整本书大概需要 160 天的阅读时间，如果加快学习进度，每周能学习三节，读者可以立争在 100 天(3 个月多一点)左右时间内将本书读完。

在阅读过程中，极其强烈地推荐读者跟着笔者一起实践书中的每个范例，验证范例的正确性。实践是加深知识理解和掌握的一个极其重要、不可省略的过程，这样的学习效果将比只用耳朵听和用眼睛看要强 3～5 倍。千万不要偷懒，既然决定走程序开发这条路，现在的勤奋就是为了让自己以后少一些被动和困境。

本书内容全是重点，没有非重点，建议读者将整本书全部仔细地读完，一节都不要错过。通过这样仔细地阅读，本书包含哪些内容，在读者心中就会有一个比较深刻的印象，此时，这本书就可以当作读者的贴身伙伴和工具书了——忘记了哪个知识点，随时翻开查阅。

对于知识点千万不要死记硬背，随着读者学习知识的增多，会面临着学过的知识很快忘记的情形。这很正常，所以在大多数情况下，我们的头脑里往往只记录一个知识点的大概思路和解决方法的索引(位置)，需要解决类似问题的时候能找到这个位置，通过查阅资料来解决问题就非常好。

运行环境

本书的范例全部在 Windows 操作系统下的 Visual Studio 2019 集成开发环境下调试通过。笔者之所以选择这个开发环境，是因为它对开发者相当友好，开发和调试程序特别方便，极大地简化了学习难度。

书中详细阐述了在 Windows 操作系统下安装 Visual Studio 2019 的过程与进行程序调试的方法。

根据讲解需要，笔者也会在 Linux 操作系统下进行一定的程序演示操作(使用 g++编译程序)。基本上，书中所有范例都可以跨操作系统平台运行(既可以在 Windows 操作系统下运行，又可以在 Linux 操作系统下运行)。

资料获取

本书有完整的配套学习资料(源码)，读者可以免费获取。获取方法如下：

(1) 查找并关注“程序员速成”微信公众号。

(2) 在微信公众号中，输入“对象模型”四个字，就可以获取配套学习资料下载链接。

读者评论

笔者在出版本书之前，有数月的时间，通过网络推出本书所讲述的内容，收到了大量学习者的反馈和极高的评价，99%的评论都是五星好评。这里摘录一部分评论留言：

- 王老师牛。
- 看过的最好的系列教程，没有之一。
- 很好，感觉自己的 C++能力上升了一个层次。
- 课程很好的，就算以前学过的人也值得买来复习一遍，很棒的课程，期待实战课。
- 只有学了才知道好的课程。
- 这课程让我长了不少见识。
- 跟着老师学了很多课了，一直跟下来，感觉收获特别多。
- 老师的课程讲得非常好！让我学到了以前从来没学到的东西，例如，this 指针调整偏移、虚函数表指针等！让我更深地认识到 C++在背后做了哪些事情！
- 课讲得很有激情，也很细，学到了以前很多不了解的知识！
- 功力深厚的老师，无话可说，大写的厉害，说的非常详细，理解得很深入，期待老师其他的课程。

致谢

20 几年前，笔者写过一本《Crystal Reports 水晶报表设计与开发实务》，这本书的写作实在是太辛苦，通宵达旦，熬了有 3 个多月，每天大概要写到后半夜 3 点才能休息。书稿完成之后，整个人瘦了好几圈。

虽然该书有一定的销量，但是跟付出感觉不成比例。所以，从那时候开始，笔者就暗下决心，从此以后再也不写书了。

人算不如天算，2018 年下半年，笔者通过网络，逐步发布一些 C/C++类网络课程，让人意外的是，这些课程获得了学员极好的口碑，隐隐有超越市面绝大多数 C/C++类课程的趋势。在这个时候，清华大学出版社电子信息教材事业部主任、首席策划盛东亮先生找到笔者，希望笔者能够写一套优秀的 C/C++类图书。

因为多年前出书的辛劳产生的心理阴影，笔者当时本能地拒绝了写书这件事，但在笔者的心里却从此埋下了一颗写书的种子。

几个月之后，笔者终于想通了写书这件事，有两点好处：

(1) 帮助更多的读者掌握更多的 C++开发知识，这是为笔者自己增加福报，也是为社会尽一点自己的微薄之力，做一个对社会有用的人。

(2) 扩大自己的知名度，增加收入。毕竟，笔者也需要养家，通过自己的劳动赚取收入，这是光荣的。

当笔者把出书的想法说给盛东亮先生听的时候，他非常高兴并与笔者做了深入的沟通，我们二人年纪相仿，聊得很投机，在写书这件事情上观点也非常一致——希望这是一本高质量的、经典的C++图书，是一本可以摆在书架上10年甚至20年的书籍。我相信，读者从书名中也能感受到笔者对这本书的期盼。

在出书的交流过程中，盛东亮先生给予了笔者相当多的支持与鼓励，甚至可以说，如果不是盛东亮先生当初的接洽，如果不是后续顺畅愉快的交流，如果不是许多实际的工作都能在最短的时间敲定并逐一落实，这套书也许根本不会出现在读者面前。所以，这份感谢，笔者要送给你——盛东亮先生。

C++知识体系庞杂，而C++对象模型方面的知识又具有一定的深度和难度，限于水平，虽然笔者非常尽心尽力，但书中错误在所难免，恳请各位读者不吝指教。

编著者

2019年8月17日

目 录

CONTENTS

第 1 章　演示环境介绍和准备 …… 1

1.1　演示环境搭建说明 …… 1
1.2　创建最基本的能运行的 C++程序 …… 3
1.3　Visual Studio 中程序的调试方法 …… 6

第 2 章　对象 …… 9

2.1　类对象所占用的空间 …… 9
2.2　对象结构的发展和演化 …… 12
2.3　this 指针调整 …… 15
2.4　分析 obj(目标文件)与构造函数语义 …… 18
2.5　拷贝构造函数语义与移动构造函数语义 …… 30
2.5.1　拷贝构造函数语义 …… 30
2.5.2　移动构造函数语义 …… 36
2.6　程序转换语义 …… 37
2.6.1　定义时初始化对象 …… 38
2.6.2　参数的初始化 …… 39
2.6.3　返回值初始化 …… 40
2.7　程序的优化 …… 42
2.7.1　从开发者层面的优化 …… 42
2.7.2　从编译器层面的优化 …… 44
2.8　程序优化续、拷贝构造函数续与深浅拷贝 …… 47
2.8.1　程序优化续 …… 47
2.8.2　拷贝构造续 …… 49
2.8.3　深浅拷贝 …… 50
2.9　成员初始化列表 …… 52
2.9.1　何时必须用成员初始化列表 …… 52
2.9.2　使用初始化列表的优势 …… 54
2.9.3　初始化列表的细节探究 …… 57

第 3 章　虚函数 …… 59

3.1　虚函数表指针位置分析 …… 59

3.2 继承关系作用下虚函数的手工调用 …… 60
3.3 虚函数表分析 …… 63
3.4 多重继承虚函数表分析 …… 66
3.5 辅助工具与 vptr、vtbl 创建时机 …… 71
3.5.1 使用辅助工具查看虚函数表 …… 71
3.5.2 虚函数表的创建时机 …… 73
3.6 单纯的类不纯时引发的虚函数调用问题 …… 74

第 4 章 数据语义学 …… 82
4.1 数据成员绑定时机 …… 82
4.1.1 成员函数函数体的解析时机(整个类定义完毕后) …… 82
4.1.2 成员函数参数类型的确定时机(最近碰到原则) …… 83
4.2 进程内存空间布局 …… 85
4.3 数据成员布局 …… 88
4.3.1 观察成员变量地址规律 …… 88
4.3.2 边界调整与字节对齐 …… 89
4.3.3 成员变量偏移值的打印 …… 91
4.4 数据成员的存取 …… 93
4.4.1 静态成员变量的存取 …… 93
4.4.2 非静态成员变量的存取 …… 95
4.5 单一继承下的数据成员布局 …… 98
4.6 单类单继承虚函数下的数据成员布局 …… 102
4.6.1 单个类带虚函数的数据成员布局 …… 103
4.6.2 单一继承父类带虚函数的数据成员布局 …… 105
4.6.3 单一继承父类不带虚函数的数据成员布局 …… 106
4.7 多重继承数据布局与 this 调整深谈 …… 107
4.7.1 单一继承数据成员布局 this 指针偏移知识补充 …… 107
4.7.2 多重继承且父类都带虚函数的数据成员布局 …… 108
4.8 虚基类问题的提出和初探 …… 112
4.8.1 虚基类(虚继承/虚派生)问题的提出 …… 112
4.8.2 虚基类初探 …… 116
4.9 两层结构时虚基类表内容分析 …… 121
4.9.1 虚基类表内容之 5～8 字节内容分析 …… 122
4.9.2 各种形式的继承 …… 124
4.9.3 虚基类表内容之 1～4 字节内容分析 …… 128
4.10 三层结构时虚基类表内容分析与虚基类设计原由 …… 128
4.10.1 三层结构时虚基类表内容分析 …… 128
4.10.2 虚基类为什么这样设计 …… 132
4.11 成员变量地址、偏移与指针等重申 …… 134

4.11.1 对象成员变量内存地址及其指针 …… 134
4.11.2 成员变量的偏移值及其指针 …… 135
4.11.3 成员变量指针和不指向任何成员变量的成员变量指针 …… 135

第 5 章 函数语义学 …… 138

5.1 普通成员函数调用方式 …… 138
5.2 虚成员函数与静态成员函数调用方式 …… 141
5.2.1 虚成员函数调用方式 …… 141
5.2.2 静态成员函数调用方式 …… 143
5.3 虚函数地址问题的 vcall 引入 …… 146
5.4 静动态类型、绑定、坑点与多态体现深谈 …… 149
5.4.1 静态类型和动态类型 …… 149
5.4.2 静态绑定和动态绑定 …… 150
5.4.3 继承的非虚函数坑 …… 151
5.4.4 虚函数的动态绑定 …… 151
5.4.5 重新定义虚函数的缺省参数坑 …… 152
5.4.6 C++中的多态性 …… 153
5.5 单继承虚函数趣味性测试和回顾 …… 155
5.5.1 单继承下的虚函数 …… 155
5.5.2 回顾和一些小试验 …… 157
5.6 多继承虚函数深释、第二基类与虚析构必加 …… 158
5.6.1 多继承下的虚函数 …… 158
5.6.2 如何成功删除用第二基类指针 new 出来的子类对象 …… 159
5.6.3 父类非虚析构函数时导致的内存泄露演示 …… 167
5.7 多继承第二基类虚函数支持与虚继承带虚函数 …… 170
5.7.1 多重继承第二基类对虚函数支持的影响(this 指针调整作用) …… 170
5.7.2 虚继承下的虚函数 …… 173
5.8 RTTI(运行时类型识别)回顾与存储位置简介 …… 177
5.8.1 RTTI(运行时类型识别)简单回顾 …… 177
5.8.2 RTTI 实现原理 …… 178
5.8.3 vptr、vtbl 与 RTTI 的 type_info 信息创建时机 …… 181
5.9 函数调用与继承关系性能 …… 182
5.9.1 函数调用中编译器的循环代码优化 …… 182
5.9.2 继承关系深度增加,开销也增加 …… 185
5.9.3 继承关系深度增加,虚函数导致的开销增加 …… 187
5.10 指向成员函数的指针和 vcall …… 189
5.10.1 不用类对象能否调用类的虚函数和普通成员函数 …… 189
5.10.2 指向成员函数的指针 …… 190
5.10.3 指向虚成员函数的指针和 vcall …… 191

5.10.4 vcall 在继承关系中的体现 …… 194
5.11 inline 函数回顾和扩展 …… 196
5.11.1 inline 函数回顾 …… 196
5.11.2 inline 扩展 …… 198
第 6 章 对象构造语义学 …… 201
6.1 继承体系下的对象构造 …… 201
6.1.1 对象的构造顺序 …… 201
6.1.2 虚函数 …… 204
6.1.3 构造函数中对虚函数的调用 …… 206
6.2 对象复制语义学与析构函数语义学 …… 209
6.2.1 对象的默认复制行为 …… 209
6.2.2 拷贝赋值运算符与拷贝构造函数 …… 209
6.2.3 如何禁止对象的拷贝构造和赋值 …… 211
6.2.4 析构函数语义 …… 211
6.3 局部对象、全局对象的构造和析构 …… 216
6.3.1 局部对象的构造和析构 …… 216
6.3.2 全局对象的构造和析构 …… 218
6.4 局部静态对象、对象数组构造析构和内存分配 …… 220
6.4.1 局部静态对象的构造和析构 …… 220
6.4.2 局部静态对象数组的内存分配 …… 225
6.5 new、delete 运算符与内存高级话题 …… 229
6.6 临时性对象的详细探讨 …… 231
6.6.1 拷贝构造函数相关的临时性对象 …… 231
6.6.2 拷贝赋值运算符相关的临时性对象 …… 233
6.6.3 直接运算产生的临时性对象 …… 236
第 7 章 模板实例化语义学 …… 240
7.1 模板及其实例化详细分析 …… 240
7.1.1 函数模板 …… 240
7.1.2 类模板的实例化分析 …… 241
7.1.3 多个源文件中使用类模板 …… 245
7.2 炫技写法 …… 248
7.2.1 不能被继承的类 …… 249
7.2.2 类外调用私有虚成员函数 …… 250
后记 …… 252

第1章

演示环境介绍和准备

本章会搭建范例演示环境并创建一个最基本的能运行的C++程序，同时，还会详细解释在Visual Studio中程序的调试方法，为后面内容的讲解做好充分的准备。

（扫码获取资料）

1.1 演示环境搭建说明

本书中所展示的各种范例会以Windows操作系统平台为主，但考虑到知识的复杂性，当Windows和Linux这两大主要的操作系统平台差异比较明显的时候，也会在Linux操作系统平台上做适当的演示。

对于Windows操作系统平台上的演示环境要求不高，一般的开发环境就可以，所以，Visual Studio 2005、2008、2010、2012任一版本，甚至更老的Visual C++ 2、4、6版本等，只要能用起来，都不必再安装其他的C++开发环境（当然，不排除在太老的开发环境下会出现某些范例无法成功演示的窘境，所以建议还是使用Visual Studio 2005之后的版本）。笔者采用的演示环境是Visual Studio 2019集成开发环境，该环境使用简单，调试方便，这意味着讲解和演示会变得特别方便，同时也能大大降低读者对一些复杂知识的理解难度。为了让读者顺利地开始学习，笔者在这里将详细地阐述Visual Studio 2019的安装过程（在线安装，需要网络支持）。

Visual Studio 2019是微软公司目前推出的最新版本集成开发环境，可以直接访问网站https://visualstudio.microsoft.com/zh-hans/。这个页面就有Visual Studio 2019的下载地址，分为三个版本：社区版（Community）、专业版（Professional）和企业版（Enterprise），如图1.1所示。其中社区版是可以免费使用的，下载社区版即可。

图1.1 Visual Studio 2019可供下载的三个版本

在图1.1中单击Community 2019，此时会将一个大概不到2MB的可执行文件下载到自己的计算机上。这个可执行文件实际是一个下载器，运行该下载器，它会按提示下载和安装Visual Studio 2019。因这种安装方式属于在线下载和安装，整个过程可能会持续半小时到数小时之间，安装时长主要取决于网速和计算机的

速度。

安装过程中会出现复选框，让用户选择安装哪些组件，如图 1.2 所示。

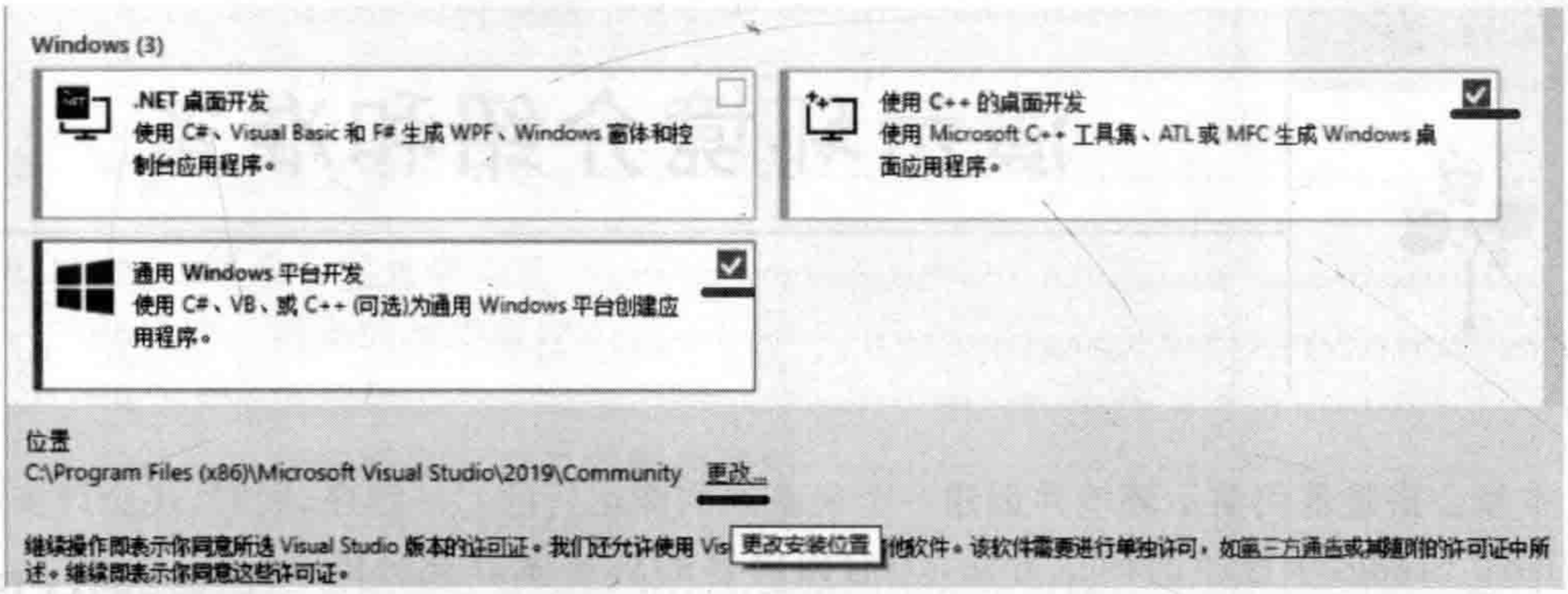

图 1.2　Visual Studio 2019 安装选项

这里选择“使用 C++的桌面开发”和“通用 Windows 平台开发”。在选择这两个选项时，安装界面右侧会出现详细的安装信息，其中有很多复选框，保持默认设置即可。请记住一个原则：只安装看上去和 C++开发有关的选项，即便错过了一些选项，以后可以重复这个步骤补充安装，但切不可图省事而完全安装，因为那可能会耗费数十甚至上百 GB 的磁盘空间，完全没有必要。

在图 1.2 左侧靠下的“位置”处，可以单击“更改”，尽量把安装位置设置到非 C 盘(非系统盘)的位置以尽量减少对系统盘空间的耗费。系统盘空间非常宝贵，一旦空间耗尽可能会导致计算机运行变慢甚至崩溃等各种问题，这一点也请切记！

安装完成后，很可能在计算机的桌面上看不到 Visual Studio 2019 程序图标，此时必须到操作系统左下角，单击“开始”按钮，然后往下翻，一直找到 Visual Studio 2019 图标，如图 1.3 所示。单击并按住图形部分拖动到桌面上，创建桌面快捷方式，下次双击桌面上的该图标即可运行 Visual Studio 2019。

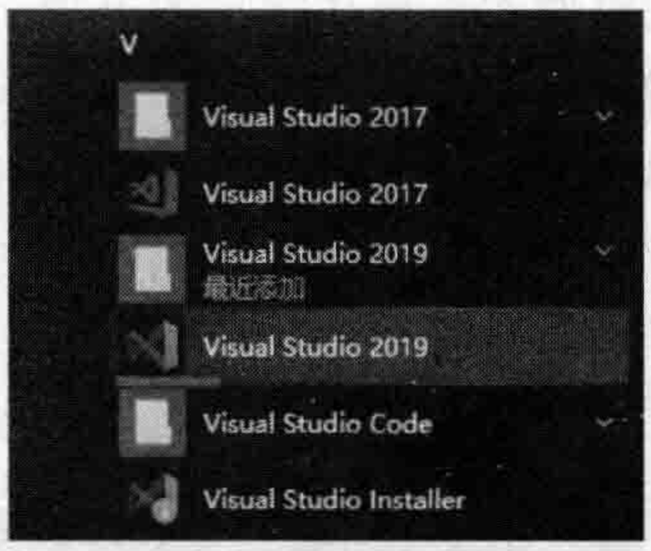

图 1.3　“开始”菜单中的 Visual Studio 2019 运行图标

双击 Visual Studio 2019 图标即可运行 Visual Studio 2019，启动界面如图 1.4 所示。

单击图 1.4 右下角的“继续但无须代码”超链接直接进入开发环境中。因为这是一个集成开发环境，可以开发很多种计算机编程语言所编写的代码，所以第一步先设置开发环境为 C++语言。在开发环境中，进行如下操作：

(1) 选择“工具”→“导入和导出设置”命令。

(2) 在弹出的对话框中，选择最下面的“重置所有设置”选项并单击“下一步”按钮。

(3) 选择下面的“否，仅重置设置，从而覆盖当前设置”项并单击“下一步”按钮。

(4) 选择“Visual C++”选项并单击“完成”按钮。

等待数秒时间，设置完成后单击“关闭”按钮并退出整个 Visual Studio 2019，就完成了

图 1.4　Visual Studio 2019 的启动界面

将开发环境设置为 C++语言的操作步骤。

Visual Studio 2019 会不定时更新，当需要更新时，在 Visual Studio 2019 界面上会有提示，单击提示会出现一些操作步骤，按照操作步骤进行操作就可以在线更新。值得一提的是，可能需要注册一个账号才能进行正常的在线更新，根据系统提示进行注册即可。

对于 Linux 操作系统平台上的演示环境，笔者安装了一个虚拟机（VMware Workstation Pro），在虚拟机上安装了乌班图 Linux 操作系统，在这个操作系统里安装了 gcc 和 g++编译器。不熟悉 Linux 操作系统的读者，笔者不建议去搭建 Linux 开发环境，因为有一定的复杂性，可能会花费很多时间，看笔者在书中的演示和描述也能达到很好的学习效果。当然，自己能搭建起来 Linux 开发环境更好。在这里笔者并不准备带着读者一起搭建 Linux 下的 C++开发环境，因为本书的重点并不是 Linux 操作系统上的 C++开发。

1.2　创建最基本的能运行的 C++程序

为了后续做范例演示的方便，要做的第一件事就是创建一个最基本的能运行的 C++程序。在 Visual Studio 2019 中，只需要选择几下菜单，就能创建出一个最基本的能运行的 C++程序，其他的 Visual Studio 版本操作也类似。下面的步骤供参考：

（1）启动 Visual Studio 2019。

（2）在启动界面，单击右下角“创建新项目”选项，会弹出如图 1.5 所示的对话框，选择“控制台应用”选项，并单击“下一步”按钮。

（3）让系统新创建一个项目。请记住，Visual Studio 2019 中，任何一个可执行程序都是通过新建一个项目的手段得来，所以新建一个项目是必须的。在图 1.6 中填写一些项目的配置信息：

- 项目名称：为创建的项目起的名字，例如输入 MyProject。

图 1.5　Visual Studio 2019 创建新项目

- 位置：保存此项目的位置，可以直接输入一个目录路径或单击后面的“...”按钮选择一个已存在的目录名，这里导航到事先创建好的路径 C:\Users\KuangXiang\Desktop\C++(读者可根据需要自由选择路径)。
- 解决方案名称：一个解决方案里可以包含多个项目，Visual Studio 开发环境硬性要求一个项目必须被包含在一个解决方案里，同时，一个项目最终可以生成一个可执行程序，所以创建 MyProject 项目时，Visual Studio 2019 会连带创建一个解决方案并让 MyProject 项目包含在该解决方案里，解决方案名称这里输入 MySolution。

(4) 单击图 1.6 右下角的“创建”按钮，系统开始创建项目，几秒钟后，系统创建好了一个叫 MyProject 的项目，位于 MySolution 解决方案之下，如图 1.7 所示。因版本不断升级变化，读者的界面内容可能会略有差异，这不要紧，不要随意改动内容以免出错。

如果读者使用其他 Visual Studio 版本，创建项目的步骤大同小异，只要能创建一个基于控制台的 C++程序项目供后续学习使用即可。如果读者对自己使用的 Visual Studio 版本不确定如何创建项目，通过搜索引擎搜索诸如“Visual Studio 2019 创建新 C++项目”这样的关键词组合就能找到详细答案。

展开图 1.7 左侧的“源文件”文件夹的树状分支，其中包含一个 MyProject.cpp 文件，这是系统依据图 1.6 所起的项目名称生成的一个源码文件，里面已经包含一些 C++源码，其实目前系统生成的该项目已经能够编译并运行了。

项目要先编译、链接、生成可执行程序，然后才能运行，这一整套动作用快捷键 Ctrl+F5 就可以完成，按住 Ctrl 键，再按 F5 键即可，该快捷键在很多 Visual Studio 版本中通用，

图 1.6　新项目的一些配置信息

图 1.7　成功创建了一个新项目

记住它。如果出现一个提示窗口，可以单击提示窗口中的 Yes 按钮，也可以直接按 Enter 键进行确认。

如果按 Ctrl＋F5 键之后 Visual Studio 2019 没任何反应，可能是这个快捷键被其他软件所占用，此时可以用 Visual Studio 2019 中的菜单命令代替，依次单击如图 1.8 所示的“调试”→“开始执行(不调试)”命令也能达到编译、链接、生成可执行程序并开始执行的效果。

图 1.8　编译生成可执行程序并执行

可执行程序运行起来后，出现一个背景为黑色的窗口，其中显示的 Hello World! 字符串如图 1.9 所示。因为刚才创建项目时选择的是“控制台应用”，这种“控制台应用”项目运行后显示的正是一个黑色窗口，该窗口中会显示程序执行的结果，通过该窗口显示运行结果完全能够满足本书的学习要求。

图 1.9　可执行程序的执行结果

此时按任意键，这个黑窗口关闭，预示着该可执行程序执行结束。可执行程序执行结果中之所以会显示 Hello World!，是因为在 MyProject.cpp 源码文件中有如图 1.10 所示的代码行，代码的含义是属于 C++的基础开发知识，相信读者会非常熟悉，在这里就不多说了。

图 1.10　输出语句 std::cout 向屏幕输出字符串 Hello World!

1.3　Visual Studio 中程序的调试方法

1. 普通的断点调试(跟踪调试)

首先介绍一下如何在 Visual Studio 2019(其他 Visual Studio 版本也类似)中进行程序调试，程序调试对于日后顺利进行范例演示和讲解将起到极其重要的作用，同时也对读者理解所讲解的知识起到极其重要的作用，所以一定要掌握好程序调试的方法。

(1) 快捷键 F9(对应“调试”→“切换断点”命令)，用于给光标所在的行增加断点(设置断点)或取消该行已有的断点，断点行最前面会有一个红色的小圆球表示该行有一个断点，如图 1.11 所示，可以通过将光标定位到多个行并每次都按 F9 键来为多个行增加断点。

(2) 快捷键 F5(对应“调试”→“开始调试”命令)，用于开始执行程序，遇到第一个断点行就停下来，如图 1.12 所示，程序停到了第 8 行，这个红色圆球中间多了一个向右指向的黄色小箭头，表示程序执行流程停止到了这一行(虽然停止到了这一行，但是此刻这一行还没有被执行，表示即将要执行)。

图 1.11　给某行增加断点后该行前面出现红色小圆球

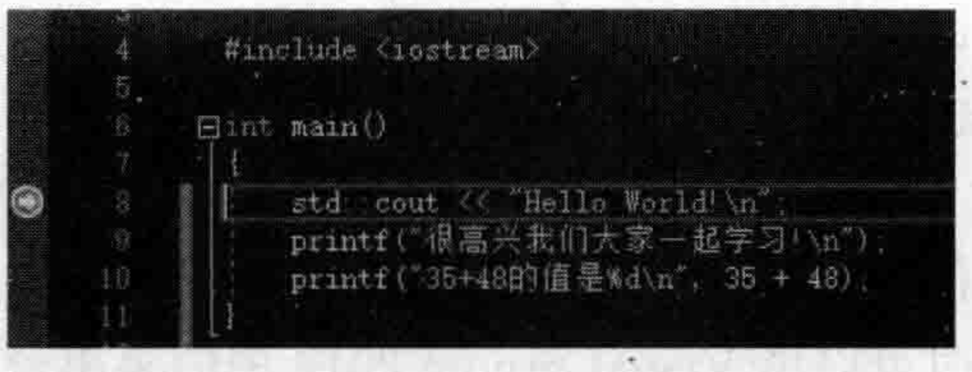

图 1.12　断点停止到了第 8 行

(3) 此时，因为程序**执行流程已经停了下来**，处于程序员(我们)的控制之中，所以就在此刻，可以多次使用快捷键 F10(对应“调试”→“逐过程”命令)，从当前停下来的这行开始一行一行地执行下去，一边逐行执行，一边可以观察程序的执行走向(例如，如果是条件语句就会因为不同的条件执行不同的分支)以及各种变量的值，从而达到调试的目的。

(4) 如果断点停在了一个自定义函数调用行，并且希望跟踪到这个函数里面的语句行中去，使用快捷键 F11(对应“调试”→“逐语句”命令)就可以跳到函数中继续跟踪调试。

如果想从当前所在的函数跳出去，使用快捷键 Shift＋F11（对应“调试”→“跳出”命令），就能够跳回到该函数的调用处并继续往下跟踪调试。

2. 学会调试时查看内存中的内容

在调试程序时学会查看内存中的内容对于深入掌握 C/C++语言编程好处巨大。下面是两行演示代码，按 F9 键把断点设置到如下的 printf 行：

```
char aaa[1000] = "safasdfa\0def";
printf(aaa);
```

按 F5 键执行整个程序，使**断点停在** printf 行上，则此时此刻就处于了调试程序中，如图 1.13 所示。

图 1.13　正处于调试程序状态中

在此种状态下，按下快捷键 Alt＋6 或者选择“调试”→“窗口”→“内存”→“内存 1”命令，则在整个 Visual Studio 2019 的下方，就打开了内存查看窗口，如图 1.14 所示。

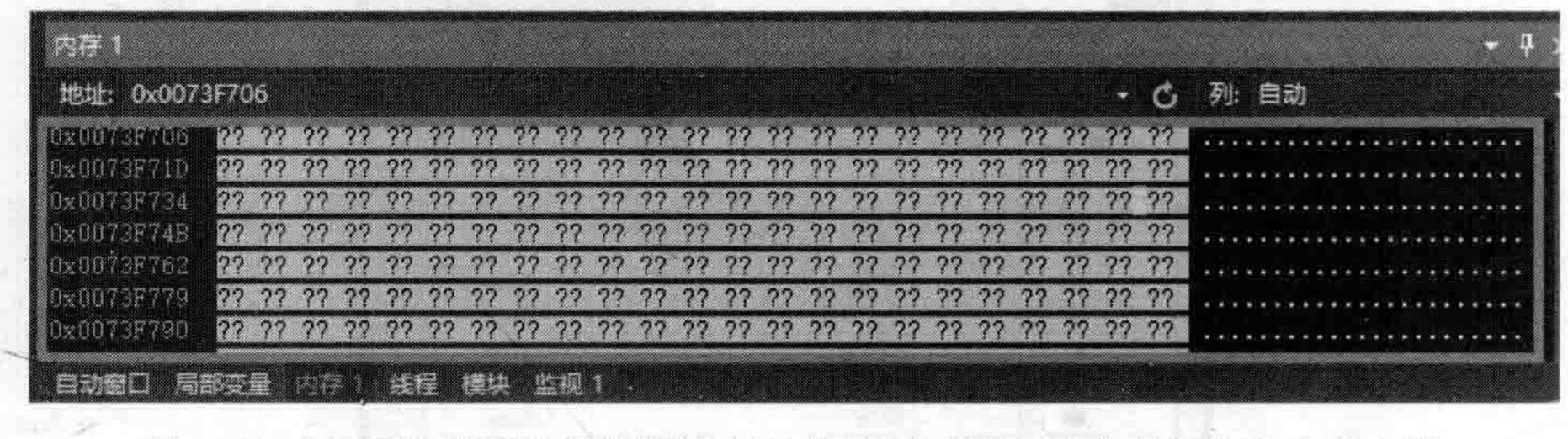

图 1.14　在调试时可以打开调试窗口中的“内存”窗口来查看内存中的内容

只需要在图 1.14 左上角位置“地址”右侧的编辑框中输入地址符“&”，后面跟要查看的变量名，然后按一下 Enter 键（如这里输入 &aaa 并按 Enter 键），就可以看该变量的地址所代表的内存中的内容。当然，在图 1.13 所示的第 78 行，选中 aaa 这个变量名，直接按住鼠标左键往图 1.14 中左上角位置“地址”右侧的编辑框中拖动并按 Enter 键，也能达到查看 aaa 变量所代表的内存中内容的目的，如图 1.15 所示。

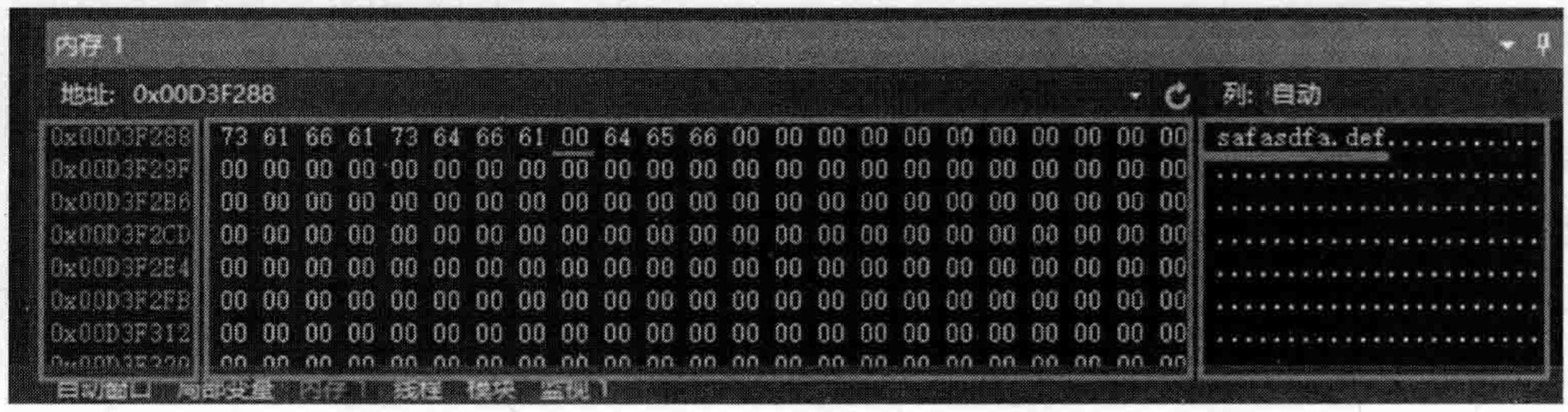

图 1.15　变量 aaa 在内存中的内容

图 1.15 所示的“内存”窗口中，左上角的“地址”部分显示的 0x00D3F288 是变量 aaa 所代表的内存地址。往下看，分成三部分，左侧部分显示的内存地址，是变量 aaa 的地址和附近的内存地址；中间部分显示的是内存地址中保存的十六进制数字内容（是的，内存中保存的数据都是二进制数据，为了方便观察，Visual Studio 2019 中把这些二进制数据以十六进

制形式显示出来，四位二进制数字显示为一位十六进制数字）；右侧部分显示的是内存中的十六进制数字所代表的一些字符，有些可显示字符会显示出来，对于不可显示的字符就用.来代替。

程序代码中的“safasdfa\0def”字样，与在内存中的内容进行比较，可以看到，“\0”这个转义字符在内存中显示的十六进制数字为 00，其他的字符比如“a”这个字符在内存中显示的十六进制数字为 61，十六进制的 61 正好是十进制的 97，而十进制的 97 正好就是字符“a”的 ASCII 码，所以在内存中存放一个字符时，存放的其实就是该字符的 ASCII 码。

3. 快速监视窗口

当运行着的程序**停到断点处**时，也可以按 Shift＋F9 键（对应“调试”→“快速监视”命令），并在其中输入要监视的变量内容，这样也是可以看到变量中所保存的数据的。例如，输入 &aaa 并按 Enter 键，可以看到 aaa 的地址，地址中的内容也是清晰可见，如图 1.16 所示。

图 1.16 “快速监视”窗口用于快速查看变量的值、变量的地址等

掌握了 Visual Studio 2019 中程序的调试方法，就可以在后面的学习中利用这些手段帮助自己快速掌握新知识。

从下一章开始，就要真正进入到 C++对象模型知识的学习和探索中去。在此之前，强烈建议读者把 Visual Studio 开发环境搭建好，因为本书所讲解的内容，如果读者能够亲自动手进行实践，那么学习效果可能要好上许多许多！

第 2 章

对　　象

本章的主题定位在“对象”。首先会介绍类对象所占用的空间问题，由这个基本问题引出对象结构的发展和演化。接着，介绍多重继承下的 this 指针调整，详细分析构造函数、拷贝构造函数、移动构造函数的语义。然后，探讨程序转化语义、程序的优化、深浅拷贝等话题。最后，详细讲解构造函数成员初始化列表的作用以及存在的意义。

2.1　类对象所占用的空间

看看如下范例。在 MyProject.cpp 的上面，增加如下类 A 的定义代码：

```
class A
{
public:
};
```

在 main 主函数中，增加如下代码：

```
A a;
//sizeof(): 返回一个对象所占的字节数字
int ilen = sizeof(a);                    //等价于 sizeof(A),结果为 1
printf("ilen =  %d\n", ilen);            //ilen = 1
```

执行起来，看一看结果：

```
ilen = 1
```

为什么 sizeof(a)的结果是 1？如果按照直觉来讲，类 A 是一个空类，里面没有任何成员变量和成员函数，所以 sizeof(a)的结果应该是 0 才对，但实际情况是 sizeof(a)的结果为 1，为什么？

因为都知道，对象是有地址的。例如，A a;代码行定义了类 A 的对象 a，那么，&a 代表对象 a 的首地址，也就是该对象所对应内存的起始地址。设置断点行并跟踪调试，看一下这个对象 a 的首地址，如图 2.1 所示。

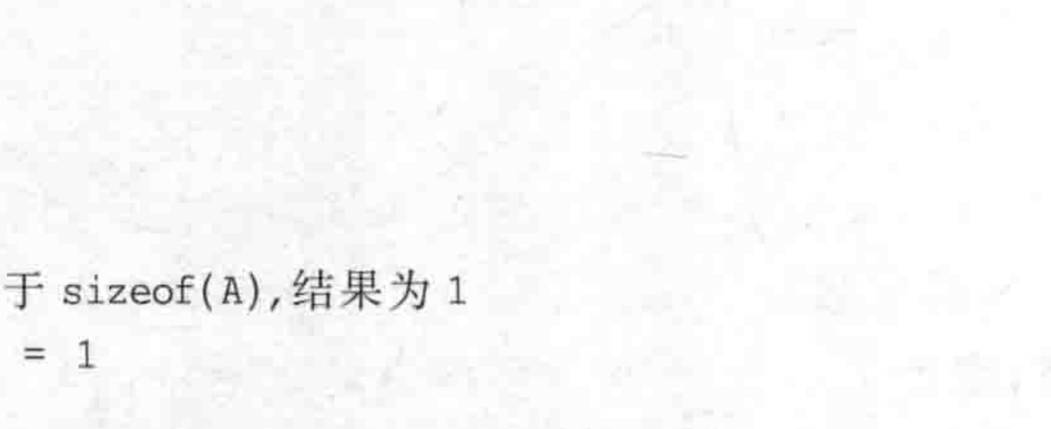

图 2.1　在快速监视窗口中查看一个对象的地址

可以看到，对象 a 是有地址的。内存中

的一个地址单元**里面保存的是1个字节的内容**。所以说，即便是一个空类的对象，但是因为在内存中是有起始地址的，既然这个地址属于该对象，该对象必然最少能存得下一个字节，所以，即便 sizeof(空类)或者 sizeof(空类对象)，得到的结果也不可能是0，而是1。这就好像买房子一样，别人问你买的房子有多大面积，你回答说是0平方米，那可能吗？只要是一个真实存在的房子，它肯定占空间，面积一定是大于0平方米。房子的面积和计算机的内存一个道理，只要有内存地址，那就表示至少能存下一个字节。综合而言，sizeof(a)值为1，这个1的确切含义是：产生一个类A的对象时，这个对象所占用的内存大小是1个字节。

现在，向类A中增加3个成员函数。增加后的类A代码如下：

```
class A
{
public:
    void func() {};
    void func1() {};
    void func2() {};
};
```

再次计算一下 sizeof(a)，发现结果依旧是1字节。由此可以得到一个结论：**类中的成员函数是不占用类对象内存空间的**。注意这里笔者重点强调不占用的是“类对象的内存空间”。

现在，继续往类A中增加一个成员变量 ab：

```
public:
    char ab;
```

请想一想，这回 sizeof(a)是多少个字节？

现在已知两点：

(1) 成员函数不占类对象的内存空间。

(2) 一个类对象至少占1字节内存空间。

再次计算 sizeof(a)，发现结果依旧是1字节。可能读者会觉得奇怪，空类的时候是1字节，增加了一个 char 类型成员变量后依旧是1字节，那么这次这1字节空间是谁的？当然是成员变量 ab 的，因为 ab 是 char 类型，char 类型是占1个字节的。

现在，在 main 主函数中增加一行给对象 a 的成员变量 ab 赋值的语句。完整的 main 主函数中的代码如下：

```
A a;
//sizeof(): 返回一个对象所占的字节数
int ilen = sizeof(a);                    //等价于 sizeof(A),结果为 1
a.ab = 'c';
printf("ilen = %d\n", ilen);             //ilen = 1
```

在 printf 行设置断点并进行调试，如图 2.2 所示。

如果类A中的成员变量 ab 不是 char 类型而是 int 类型呢？把类A中的成员变量 ab 修改为 int 类型：

```
public:
    int ab;
```

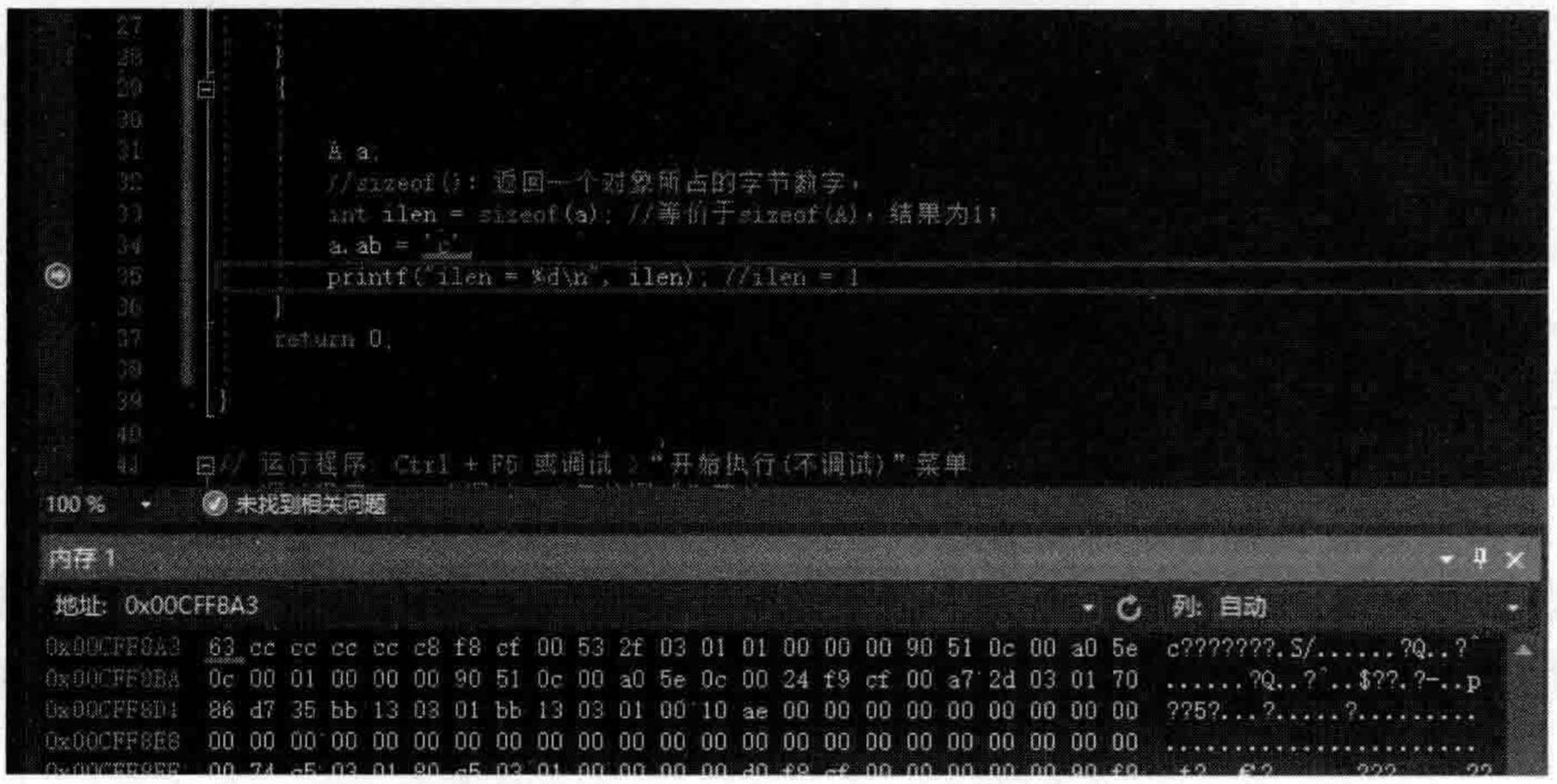

图 2.2　只占 1 个字节内存的对象 a 在内存中的内容是 63('c')

再次修改 main 主函数中的给 ab 成员变量赋值的代码行。修改如下：

```
a.ab = 12;
```

执行起来，看一看结果：

```
ilen = 4
```

这次 sizeof(a)等于 4，那么这 4 字节的空间是谁的呢？当然是成员变量 ab 的，因为 ab 是 int 类型，int 类型是占 4 个字节的，如图 2.3 所示。

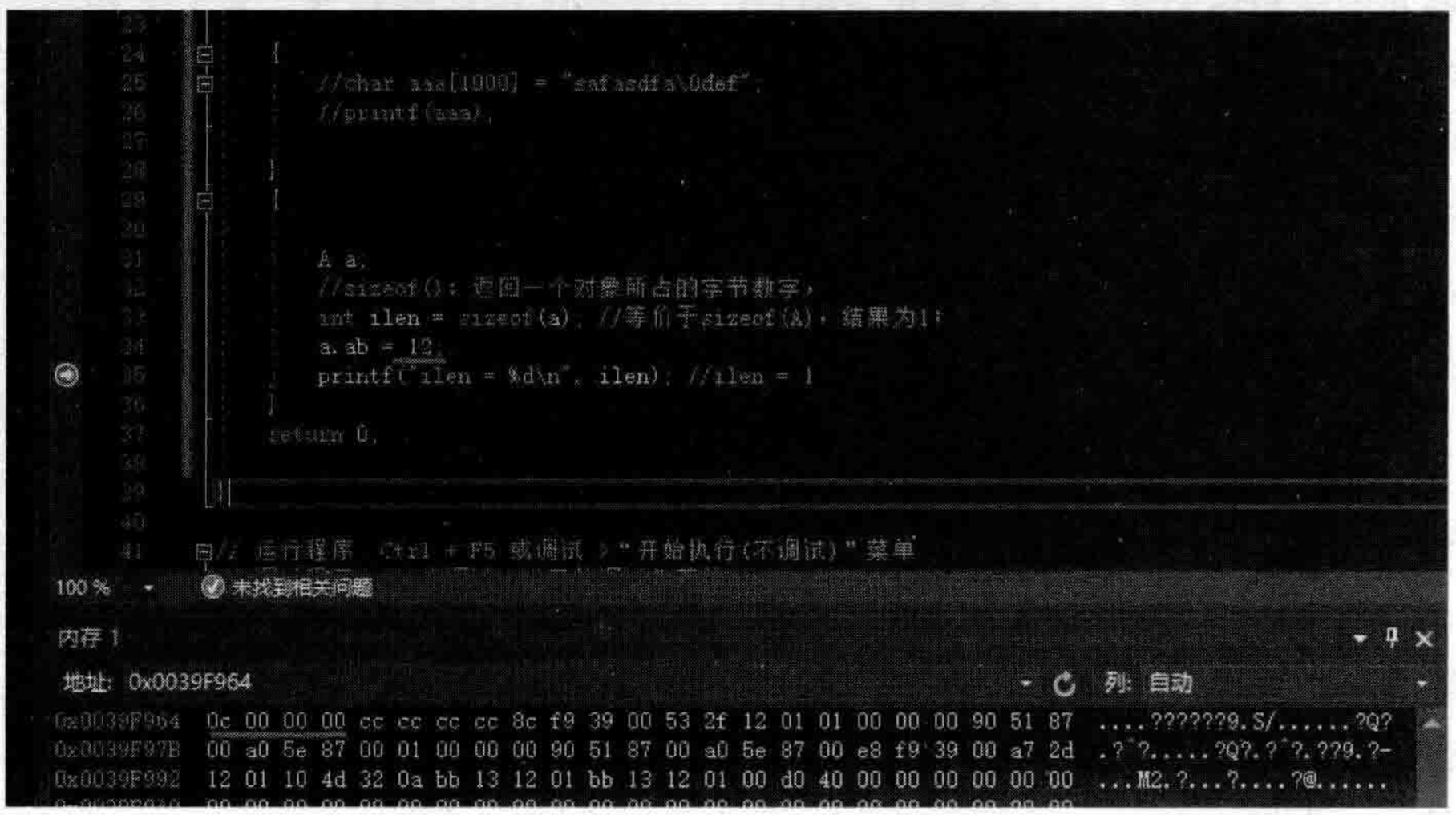

图 2.3　占 4 个字节内存的对象 a 在内存中的内容是 0c 00 00 00(12)

通过这些实践，能够得出一些结论：

(1) 成员变量是包含在每个对象中的，占字节的。

(2) 成员函数虽然也写在类定义中，但成员函数不占类对象的字节空间。也就是说，成员函数是跟着类走的，跟类对象没有关系，不管用这个类产生了多少个该类的对象。

2.2 对象结构的发展和演化

C++对象的结构经过了多年的发展，诸多设计者提出了各种不同的见解和想法，取其精华，去其糟粕，C++对象模型也就逐渐地建立起来了。

以往在发展过程中的C++对象模型演化步骤在这里就不多谈了。下面主要谈的是当前正在采用的C++对象模型是一个什么样子。

1. 非静态的成员变量跟着对象走

所谓非静态的成员变量跟着对象走，指的是非静态的成员变量保存在对象内部。也就是说，每个类对象都有自己的成员变量，什么叫都有自己的成员变量？也就是说，如果在类中定义了这个成员变量，那么sizeof这个类对象时，这个类对象占的内存就会因为这个成员变量的存在而增长。

看看如下范例：

```
class A
{
    int a = 100;
};
A aobj;                                    //sizeof(aobj) = 4
```

设置断点并进行跟踪调试，看看aobj所占用的4个字节内存，如图2.4所示。

图2.4 对象aobj所占用的4个字节内存

2. 静态成员变量跟对象没有关系

静态成员变量跟对象没什么关系，所以不会保存在对象内部，而是保存在对象外面。这里笔者使用类对象外面这种用词，表达所占用的内存空间和类对象无关的意思。

看看如下范例：

```
class A
{
    static int a;
    static int b;
};
A aobj;     //sizeof(aobj) = 1，不管几个静态成员变量，sizeof的大小不增加
```

3. 成员函数

不管静态的还是非静态的，全部都保存在类对象之外。所以不管几个成员函数，不管是否是静态成员函数，对象的sizeof值的大小都不增加。

看看如下范例：

```
class A
{
    static void sfunc() {};
    void myfunc() {};
};
A aobj;                                    //sizeof(aobj) = 1
```

4. 虚函数

看看如下范例：

```
class A
{
    virtual void myfunc3() {};
    virtual void myfunc4() {};
};
A aobj;                                    //sizeof(aobj) = 4
```

类中因为有了一个或者多个虚函数的存在，对象的 sizeof 会增加 4 字节，也就是该类对象所占的内存会多出 4 个字节。那为什么会多出 4 个字节？这 4 个字节是用来做什么的？看一看这个虚函数。

首先，说类本身。一个类中只要有一个虚函数（或者说至少有一个虚函数），这个类就会产生一个指向虚函数的指针，有两个虚函数，这个类就会产生两个指向虚函数的指针。当然是每个虚函数一个指针，两个虚函数不就两个指针嘛！那如果这个类有一堆虚函数，那就有一堆指向虚函数的指针。一个指针也好，一堆指针也好，这些指针总得有个地方放。放哪里呢？放在一个表里，这个表称为**虚函数表**（virtual table，简称 vtbl），这个虚函数表一般保存在最后生成的可执行文件中，在程序执行的时候载入到内存中。

再说一说类对象。刚才已经看到，无论往类中增加几个虚函数，类对象的 sizeof 只会增加 4 个字节。这 4 个字节的增加，其实是因为虚函数的存在，导致系统往类对象中**添加了一个指针**，这个指针正好就**指向这个虚函数表**，这个指针的名字叫作虚函数表指针（virtual table pointer，简称为 vptr）。什么时候这个 vptr 会有值以及什么时候这个值会重置等等，系统内部会做这些事。在哪些函数中做呢？一般就是在构造函数、析构函数、拷贝赋值运算符里等。

总结一下：

（1）静态成员变量不计算在对象的 sizeof 内。

（2）普通成员函数和静态成员函数不计算在 sizeof 内。

（3）虚函数不计算在对象的 sizeof 内，但虚函数会让对象的 sizeof 增加 4 字节以容纳指向虚函数表的指针。

（4）虚函数表是基于类的（跟着类走，不是基于对象的）。

（5）如果类中有多个数据成员（成员变量），那么为了访问速度和性能考虑，某些编译器可能会将数据成员之间的内存占用比例进行调整。例如类中有两个成员变量：

```
char a;                                    //1 字节
int b;                                     //4 字节
```

两者本来共占 5 字节，但因为挨在一起，可能系统就会给 char 分配成 4 字节（32 位），这两个成员共占 8 字节（也叫内存字节对齐：都往 4 字节对齐），以后读者见到这种情况不用感到奇怪。

（6）不管什么类型指针 char * p、int * q 等，指针占用的内存大小是固定的。一般要么是 4 个字节，要么是 8 个字节，具体取决于是 x86 平台还是 x64 平台（32 位操作系统还是 64 位操作系统）。看看如下范例：

```
int ilen2 = sizeof(char*);          //4,如果在 Linux 操作系统下可能会是 8 字节
int ilen3 = sizeof(int*);           //4
```

下面通过一个完整的代码整理一下讲过的知识。在 MyProject.cpp 的上面，定义一个名字叫作 myobject 的类。

```
class myobject
{
public:
    myobject() {};                          //构造函数
    virtual ~myobject() {};                 //析构函数
    float getvalue() const                  //普通成员函数
    {
        return m_value;
    }
    static int s_getcount()                 //静态成员函数
    {
        return ms_scount;
    }
    virtual void vfrandfunc() {};           //虚函数
protected:
    float m_value;                          //普通成员变量
    static int ms_scount;                   //静态成员变量
};
```

在 main 主函数中，加入如下代码：

```
myobject obj;
int ilen = sizeof(obj);
cout << ilen << endl;                       //8
```

从结果可以看到，ilen 的值为 8。因为(以下笔者的描述均是在 x86 平台下)：

(1) float m_value;占 4 字节。

(2) 因为有虚函数在(本类有两个虚函数)，系统会往类对象中添加一个指向虚函数表的指针，这个指针占 4 个字节。

然后，看一看对象模型到底长什么样，如图 2.5 所示。

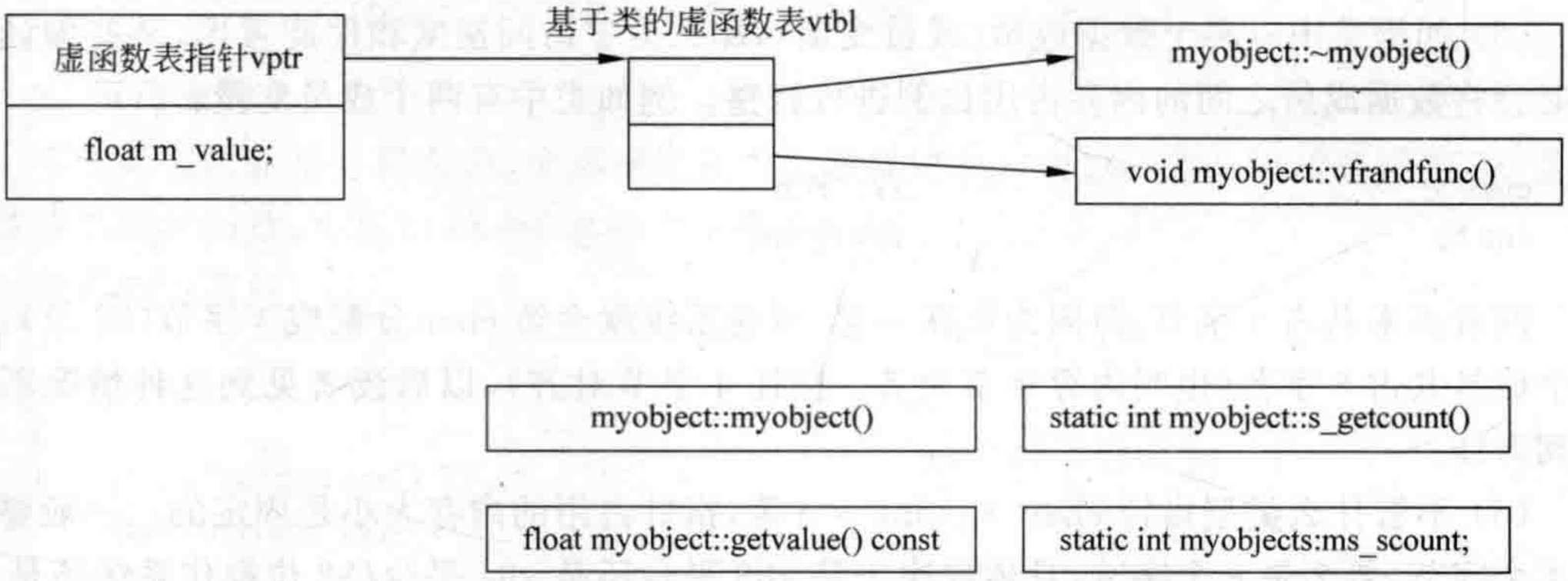

图 2.5　对象 obj 的对象模型

如果读者看不懂图 2.5,不要紧,随着后面的学习,慢慢地就可以看懂该图。

在图 2.5 中,左上角是对象 obj,占 8 个字节,包含一个虚函数表指针 vptr 和一个成员变量 m_value。虚函数表指针指向一个虚函数表 vtbl,vtbl 中有两项(两个指针)分别指向类 myobject 的两个虚函数的首地址。类 myobject 中其他的成员函数和成员变量绘制在图 2.5 的下面偏右的位置。当类的对象要调用虚函数的时候,实际上是通过虚函数表指针找到类的虚函数表,通过类的虚函数表就能够调用到类的虚函数。虚函数表的这种设计是因为要支持面向对象程序设计的三大特性之一“多态性”,后面会慢慢讲解。

属于对象 obj 的只有成员 m_value(占 4 字节)和一个虚函数表指针 vptr(占 4 字节),其他的全部都属于类 myobject。

这种布局(模型)的优点是它的空间占用适当,数据存取效率比较高。成员函数有独立的内存地址,与类对象无关,可以直接调用(成员函数的调用还是需要类对象参与的)。成员变量数据就保存在对象自己的内存中,所以存取效率是比较高的。

总结一下一个类对象大小的组成(一个类对象的 sizeof 值):

(1) 非静态成员变量所占的内存总量和这些成员变量之间内存字节对齐所额外占用的内存。

(2) 若有虚函数,则会产生虚函数表指针 vptr。

当然,如果是类之间的多重继承关系在,并且每个父类都有虚函数,那么子类的情况还有不同,这个后面章节会继续讲解。

2.3 this 指针调整

this 指针调整这件事,一般存在于**多重继承**的情况下。

这里笔者直接进行代码演示。在 MyProject.cpp 前面,增加两个基类和一个派生类:

```
class A
{
public:
    int a;
    A()
    {
        printf("A::A() 的 this 指针是: %p!\n", this);
    }
    void funcA() { printf("A::funcA: this =  %p\n", this); }
};

class B
{
public:
    int b;
    B()
    {
        printf("B::B() 的 this 指针是: %p!\n", this);
    }
```

```
    void funcB() { printf("B::funcB: this =  %p\n", this); }

};

class C : public A, public B
{
public:
    int c;
    C()
    {
        printf("C::C() 的 this 指针是: %p!\n", this);
    }
    void funcC() { printf("C::funcC: this =  %p\n", this); }
};
```

在 main 主函数中，加入如下代码：

```
cout << sizeof(A) << endl;
cout << sizeof(B) << endl;
cout << sizeof(C) << endl;
C myc;
myc.funcA();
myc.funcB();
myc.funcC();
```

执行起来，看一看结果：

```
4
4
12
A::A() 的 this 指针是: 009FFD68!
B::B() 的 this 指针是: 009FFD6C!
C::C() 的 this 指针是: 009FFD68!
A::funcA: this = 009FFD68
B::funcB: this = 009FFD6C
C::funcC: this = 009FFD68
```

分析一下上面的结果：

(1) 前 3 行结果是 4、4、12。这是因为，A 类对象(这里也可以说成是 A 类)占 4 字节(int a;成员)，B 类对象占 4 个字节(int b;成员)，而 C 类，因为继承自 A 类和 B 类，并且 C 类还有一个自己的成员变量 c，因此 C 类对象占 4(A 类)＋4(B 类)＋4(成员变量 c)＝12 字节。

(2) 接着 3 行的结果是执行代码行 C myc;所致。这里要注意，执行的 3 个类的构造函数，父类(基类)A 类构造函数的 this 指针与父类 B 类构造函数的 this 指针不同，差了 4 个字节，而 C 类构造函数的 this 指针与 A 类构造函数的 this 指针相同。

(3) 然后调用 funcA、funcB、funcC 三个成员函数，在其中打印出来的 this 指针也不同，执行 funcA 的时候得到的结果是 A::funcA: this = 009FFD68(A 类子对象空间)。而执行 funcB 的时候得到的结果是 B::funcB: this = 009FFD6C(B 类子对象空间)。读者不难

发现，在执行 funcB 的时候，**this 指针从原来的 009FFD68 调整成了 009FFD6C**，这属于编译器内部自动调整这个 this 指针。而在执行 funcC 的时候，通过得到的结果 C::funcC: this = 009FFD68（C 类对象空间，也是 A 类对象空间）可以看到，this 指针又调整回 009FFD68 了。

读者注意，子类 C 同时继承了父类 A 和父类 B，注意继承顺序。看代码行 class C : public A, public B 可以知道，C 类是**先继承 A 类，再继承 B 类**，这个继承顺序很重要。所以子类 C 的 this 指针才跟父类 A 一样（因为先继承父类 A）。绘制一张图看一看这种多重继承关系的感受，如图 2.6 所示。

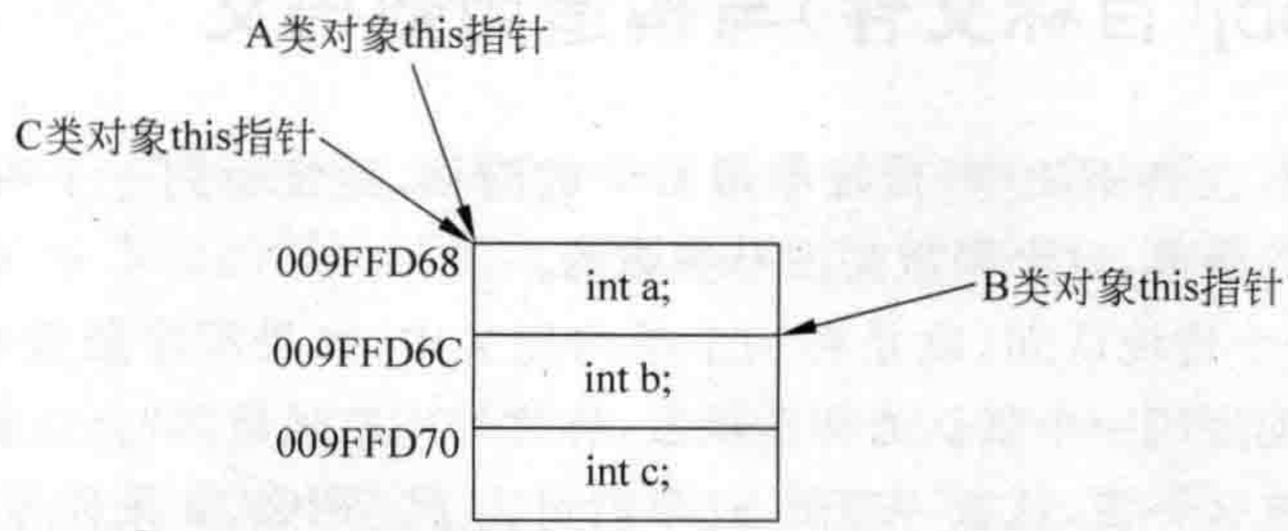

图 2.6　多重继承时对象布局中 this 指针的调整

读者都知道，派生类（子类）对象是包含着基类子对象的。如果这个派生类只从一个基类继承，那么，这派生类对象的地址和基类子对象地址相同。但如果派生类对象同时继承多个基类，那么就要注意一下，第一个基类子对象的起始地址和派生类对象起始地址相同，后续的这些基类子对象的起始地址和派生类对象的起始地址相差多少呢？相差前面那些基类子对象所占用的内存空间（单位：字节）。所以在图 2.6 中，可以注意到，B 类子对象的起始地址是加了 4 个字节的，因为 A 类子对象是占 4 个字节的。

（4）从图 2.6 能观察到 this 指针的调整问题。当执行 myc.funcA();和 myc.funcC();代码行时，this 指针是 009FFD68，这两行的 this 值相同，这是因为基类 A 子对象和子类 C 对象的起始地址相同，也就是 this 指针相同。当执行 myc.funcB();代码行时，因为 funcB 是属于基类 B 的成员函数，那么基类 B 子对象的 this 指针就被编译器自动调整成了 009FFD6C，与子类 C 对象的 this 指针相差 4 字节，这 4 字节正好是基类 A 子对象的大小。

那么，同理，如果在 C 类中覆盖 B 类的同名函数 funcB：

```
public:
    void funcB() { printf("C::funcB: this =  %p\n", this); } //定义在 C 类中
```

在 main 主函数中，在 myc.funcB();代码行之后，增加如下一行代码：

```
myc.B::funcB();
```

执行起来，看一看结果：

```
4
4
12
A::A() 的 this 指针是: 009FFD68!
B::B() 的 this 指针是: 009FFD6C!
```

```
C::C() 的 this 指针是: 009FFD68!
A::funcA: this = 009FFD68
C::funcB: this = 009FFD68
B::funcB: this = 009FFD6C
C::funcC: this = 009FFD68
```

总结一下：调用哪个子类的成员函数，这个 this 指针就会被编译器自动调整到对象内存布局中对应该类子对象的起始地址那去。

2.4 分析 obj(目标文件)与构造函数语义

提起构造函数，读者不陌生，传统学习 C++的时候，经常听到一个叫作“默认构造函数(缺省构造函数)”的函数，这个函数是没有参数的。

以往一直有一个传统认知，就是在一个类的定义中，如果程序员没有定义任何构造函数，编译器会隐式地生成一个默认的构造函数，称这种构造函数为“合成默认构造函数”。

一般来讲，听这么一说，这么一理解，这事就过去了。那么，如果程序员没有定义任何构造函数，编译器到底是否会生成一个“合成默认构造函数”呢？本节就探究一下这件事。

首先，笔者先给出结论：这种“合成默认构造函数”，只有在必要的时候，编译器才会为程序员合成出来，而不是必然或者必须合成出来。所以，问题的焦点在于“必要”这个词，这就是本节研究的重点：必要的时候是什么时候。

先讲解一点前提知识：

在 C++中，要生成一个可执行文件，需要经过编译、链接的步骤。一个.cpp **源**码文件(也可能是其他扩展名的源码文件)，经过编译，就生成了一个.obj(或者.o)文件，这个文件也叫作**目标**文件，也就是说，.obj 文件是编译的结果文件，而最终生成的可执行文件是把这些.obj 文件连接起来生成的。

所以，一般来说，一个.cpp 文件，编译一下，就生成了一个.obj 文件。

.cpp 文件是 C++源码文件，这个读者应该都了解，那么.obj 文件呢?.obj 文件既然是编译器编译.cpp 文件生成的，那有的时候不免也希望看一看编译器生成的.obj 文件长什么样子。.obj 文件属于二进制文件，不能直接看，但可以借助一些工具来看。

这里笔者以 Windows 平台工具为例，介绍一下.obj 文件如何查看。Linux 等平台也能找到相关的工具查看，请读者注意寻找。

看看如下范例。在 MyProject.cpp 的上面定义两个类，代码如下：

```
class MATX
{
public:
    MATX() //默认构造函数
    {
        cout << "goodHAHAHA" << endl;
    }
};
class MBTX
{
```

```
public:
    int m_i;
    int m_j;

    void funct()
    {
        cout << "Iamverygood" << endl;
    }
};
```

在 main 主函数中，加入如下代码行：

```
MBTX myb;
```

然后，通过“生成”→“生成解决方案”命令来编译一下当前的项目。等待几秒到几十秒，当 Visual Studio 2019 下面的“输出”窗口提示“成功 1 个”字样的时候，就代表一个.obj 文件(MyProject.obj)生成了。

下面就是找到这个.obj 文件的位置。在 Visual Studio 2019 中右击 MyProject.cpp 文件名标签并在弹出的快捷菜单中选择“打开所在的文件夹”命令，如图 2.7 所示。

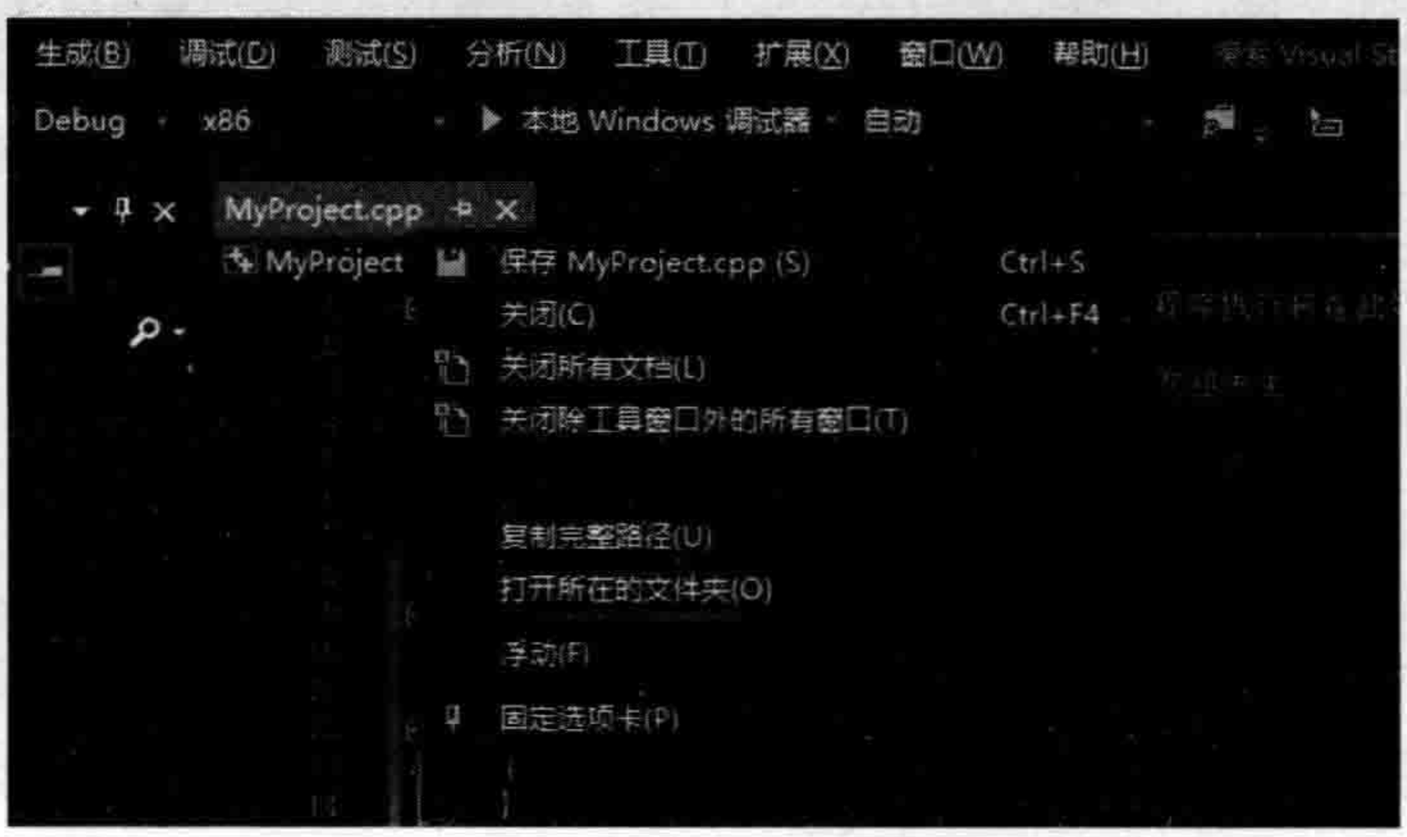

图 2.7　寻找 MyProject.cpp 文件所在的文件夹

此时，操作系统会弹出“文件资源管理器”窗口，并定位到 MyProject.cpp 所在的文件夹，进入其中的 Debug 子文件夹，就可以找到 MyProject.obj 文件，如图 2.8 所示。

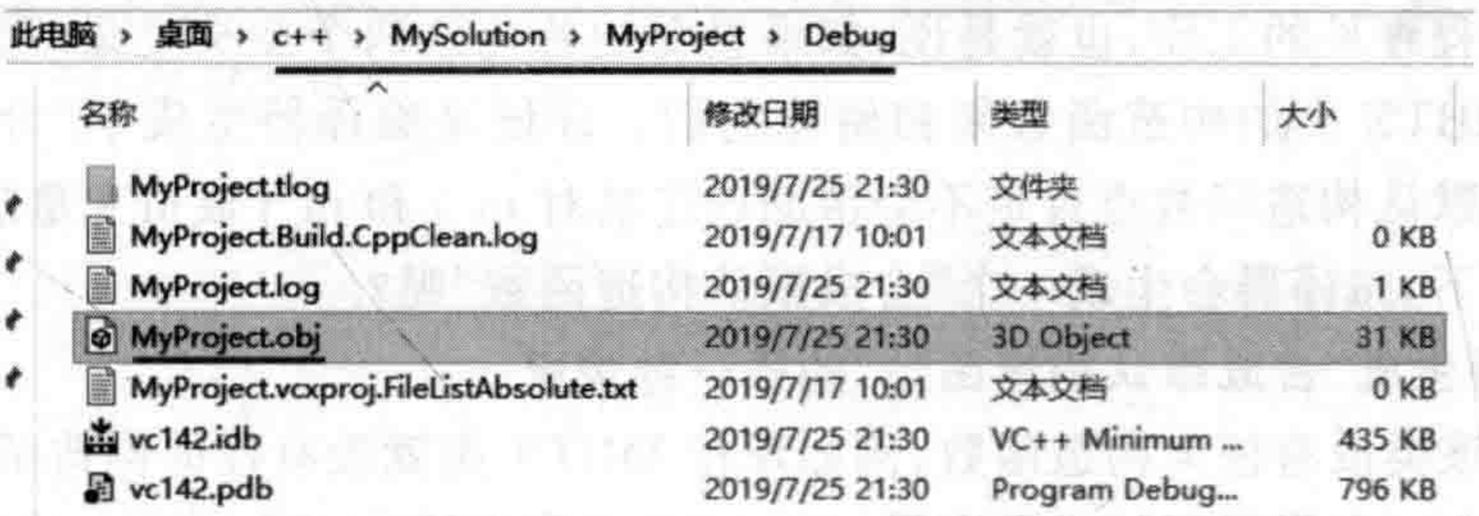

图 2.8　MyProject.obj 文件所在的文件夹

MyProject.obj 文件就是目标文件(在 Linux 平台下扩展名也许是.o)。在 Linux 下使用 g++编译器的读者可以使用-c 参数来生成.o 文件。

然后需要打开**开发人员命令行工具**(Developer Command Prompt…),安装了 Visual Studio 2019(或者其他版本的 Visual Studio)一般都会带有该命令行工具,可能需要在 Windows 操作系统左下角的“开始”菜单中进行寻找,如图 2.9 所示。

图 2.9 Visual Studio 2019 命令行工具

在这个命令行提示符下可以直接使用 dumpbin 命令行工具,不然还要自己去查找 dumpbin 这个工具的位置。

在命令行提示符下进入到 MyProject.obj 文件所在的路径,并输入 dumpbin /all MyProject.obj > MyProject.txt,如图 2.10 所示。

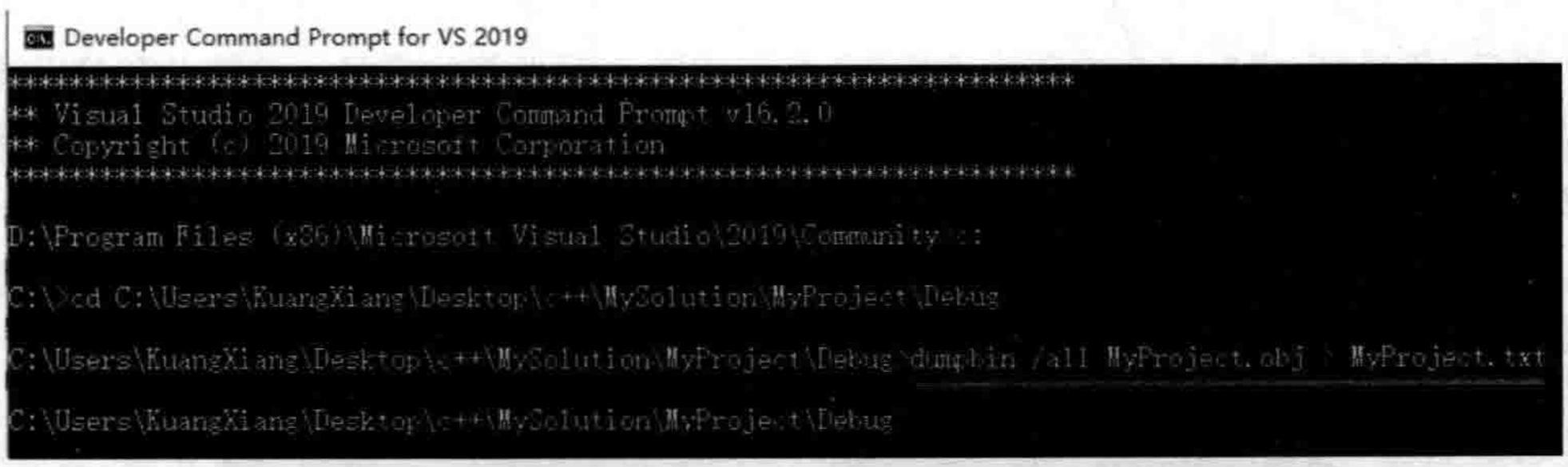

图 2.10 在 Visual Studio 2019 命令行工具中用 dumpbin 将.obj 文件的可视化信息导出到.txt 文本文件中

此时,在与 MyProject.obj 相同的文件夹下,就会产生一个 MyProject.txt 文件。这样,就把 MyProject.obj 二进制文件中的可视化信息导入到了 MyProject.txt 文本文件中。这个文本文件可以直接使用 Visual Studio 打开查看。

.obj 文件的格式一般会被认为是一种 COFF——通用对象文件格式(Common Object File Format)。这里不详细研究.obj 文件的格式。

通过查看 MyProject.txt 文件中的内容,能看到.obj 文件中保存的诸多信息。

在 MyProject.txt 文件中,按 Ctrl+F 快捷键查找 MBTX::MBTX 字样看是否有 MBTX 的构造函数,此时并没有发现,也就是说,编译器根本就没有合成出该函数来。

那么现在就要回归话题:编译器在哪些必要的时候会帮助程序员生成“合成默认构造函数”?

这里笔者要提醒读者注意类 MBTX 中的 m_i 和 m_j 这两个成员变量的初始化,这个初始化工作是程序员的工作,也就是说,如果想要初始化这两个成员变量,那么程序员就要自己手工写 MBTX 类的构造函数来初始化它们。即便是编译器生成了“合成默认构造函数”,这个合成默认构造函数也肯定不会帮助程序员对 m_i 和 m_j 成员变量进行初始化。

什么情况下,编译器会生成一个“合成默认构造函数”呢?

1. 编译器生成“合成默认构造函数”的第一种情况

(1) 如果该类没有任何构造函数,例如现在 MBTX 类就没有任何构造函数。

(2) **但包含一个类类型的成员变量**。在 MBTX 类中增加一个 public 修饰的 MATX 类型的成员变量:

```
public:
    MATX ma;                                    //类类型成员变量
```

(3) 而 ma 成员变量所属的 MATX 类有一个**默认构造函数**。笔者已经看到，当前在 MATX 类中确实已经有了一个默认构造函数。

编译链接，执行程序，看一看结果：

```
goodHAHAHA
```

这里可以注意到，MATX 的构造函数被执行(被调用)了。

根据上面的结果可以断定，编译器为类 MBTX 生成了一个“合成默认构造函数”，生成这个“合成默认构造函数”的目的是在这个“合成默认构造函数”中安插(增加)代码(编译器自动安插的代码，不需要程序员干预)来调用 MATX 的默认构造函数。

通过“生成”→“生成解决方案”命令来编译一下当前的项目，重复上述的步骤，使用 dumpbin 命令将 MyProject. obj 的可视化信息导出到 MyProject. txt 中，再次查找 MBTX:: MBTX 字样，这回可以找到了，如图 2.11 所示。

```
2364       86D8 file pointer to raw data (000086D8 to 0000871C)
2365       871D file pointer to relocation table
2366          0 file pointer to line numbers
2367          2 number of relocations
2368          0 number of line numbers
2369   60501020 flags
2370            Code
2371            COMDAT; sym= "public: __thiscall MBTX::MBTX(void)" (??0MBTX@@QAE@XZ)
2372            16 byte align
2373            Execute Read
2374
2375   RAW DATA #B
2376     00000000: 55 8B EC 81 EC CC 00 00 00 53 56 57 51 8D BD 34  U.ì.ìÌ...SVWQ.?4
2377     00000010: FF FF FF B9 33 00 00 00 B8 CC CC CC CC F3 AB 59  ???¹3...¸ÌÌÌÌó«Y
2378     00000020: 89 4D F8 8B 4D F8 83 C1 08 E8 00 00 00 00 8B 45  .M?.M?.Á.è.....E
2379     00000030: F8 5F 5E 5B 81 C4 CC 00 00 00 3B EC E8 00 00 00  ?_^[.?Ì...;ìè...
2380     00000040: 00 8B E5 5D C3                                   .å]Ã
2381
2382   RELOCATIONS #B
2383                                                  Symbol    Symbol
2384    Offset    Type              Applied To       Index     Name
2385    --------  ----------------  ---------------  --------  ------
2386    0000002A  REL32                 00000000         C3  ??0MATX@@QAE@XZ (public: __thiscall MATX::MATX(void))
2387    0000003D  REL32                 00000000         D1  __RTC_CheckEsp
```

图 2.11　编译器为 MBTX 生成了“合成默认构造函数”并调用了其他构造函数

在图 2.11 中，因为对这里面的格式不太了解，所以可以边看边猜，MBTX::MBTX(编译器生成的合成默认构造函数)里面，调用了 MATX::MATX 构造函数(看图中 RELOCATIONS 中的 MATX::MATX 字样，表示的就是调用关系)。

现在再丰富一下整个范例。在 MyProject. cpp 的前面，类 MBTX 上面，增加一个新类 M0TX 的定义，该类也有默认构造函数：

```
class M0TX
{
public:
    M0TX()                              //默认构造函数
    {
        cout << "0good!" << endl;
    }
};
```

接着，在 MBTX 类中，再增加一个 public 修饰的 M0TX 类型的成员变量。现在，在

MBTX 类中，就有了两个类类型的成员变量：

```
public:
    MATX ma;                          //类类型成员变量
    MOTX m0;                          //注意定义顺序，先定义的 ma，再定义的 m0
```

执行起来，看一看结果：

```
goodHAHAHA
Ogood!
```

再次重复上面的 dumpbin 步骤，查看此时生成的 MyProject.txt 中的内容，并再次查找 MBTX::MBTX 字样，当然还是可以找到，如图 2.12 所示。

```
    895D file pointer to raw data (0000895D to 000089AC)
    89AD file pointer to relocation table
       0 file pointer to line numbers
       3 number of relocations
       0 number of line numbers
60501020 flags
         Code
         COMDAT; sym= "public: __thiscall MBTX::MBTX(void)" (??0MBTX@@QAE@XZ)
         16 byte align
         Execute Read

RAW DATA #D
  00000000: 55 8B EC 81 EC CC 00 00 00 53 56 57 51 8D BD 34  U.ì.ì....SVWQ.?4
  00000010: FF FF FF B9 33 00 00 00 B8 CC CC CC CC F3 AB 59  ???3...?ÌÌÌÌó?Y
  00000020: 89 4D F8 8B 4D F8 83 C1 08 E8 00 00 00 00 8B 4D  .M?.M?.Á.è.....M
  00000030: F8 83 C1 09 E8 00 00 00 00 8B 45 F8 5F 5E 5B 81  ?.Á.è.....E?_^[.
  00000040: C4 CC 00 00 00 3B EC E8 00 00 00 00 8B E5 5D C3  ?Ì...;ìè.....?]?

RELOCATIONS #D
                                                Symbol    Symbol
 Offset    Type              Applied To         Index     Name
 --------  ----------------  -----------------  --------  ------
 0000002A  REL32                     00000000        CC  ??0MATX@@QAE@XZ (public: __thiscall MATX::MATX(void))
 00000035  REL32                     00000000        C9  ??0MOTX@@QAE@XZ (public: __thiscall MOTX::MOTX(void))
 00000048  REL32                     00000000        D8  __RTC_CheckEsp
```

图 2.12　编译器为 MBTX 生成了“合成默认构造函数”并调用了其他构造函数

在图 2.12 中，不但看到编译器生成了“合成默认构造函数”(MBTX::MBTX)，在 RELOCATIONS 里还看到了 MBTX::MBTX 调用了 MATX::MATX 和 MOTX::MOTX。

注意调用顺序。这里 MBTX::MBTX 先调用的是 MATX::MATX，然后才调用 MOTX::MOTX，之所以是这样的调用顺序，是因为在 MBTX 类中，成员变量 ma 的定义在 m0 之前。如果调整 MBTX 类中这两个成员变量的定义顺序如下：

```
public:
    MOTX m0;                          //调换成员变量定义顺序
    MATX ma;                          //类类型成员变量
```

再次编译项目，并用 dumpbin 将 MyProject.obj 的信息导出到 MyProject.txt 中并在该.txt 中查找 MBTX::MBTX，如图 2.13 所示。

在图 2.13 中可以注意到，这回 MBTX::MBTX 先调用 MOTX::MOTX，后调用 MATX::MATX。

执行起来，看一看结果：

```
Ogood!
goodHAHAHA
```

```
      895D file pointer to raw data (0000895D to 000089AC)
      89AD file pointer to relocation table
         0 file pointer to line numbers
         3 number of relocations
         0 number of line numbers
  60501020 flags
           Code
           COMDAT; sym= "public: __thiscall MBTX::MBTX(void)" (??0MBTX@@QAE@XZ)
           16 byte align
           Execute Read

RAW DATA #D
  00000000: 55 8B EC 81 EC CC 00 00 00 53 56 57 51 8D BD 34  U.ì.Ìì...SVWQ.?4
  00000010: FF FF FF B9 33 00 00 00 B8 CC CC CC CC F3 AB 59  ???¹3...?ÌÌÌÌó?Y
  00000020: 89 4D F8 8B 4D F8 83 C1 08 E8 00 00 00 00 8B 4D  .M?.M?.Á.è.....M
  00000030: F8 83 C1 09 E8 00 00 00 00 8B 45 F8 5F 5E 5B 81  ?.Á.è.....E?_^[.
  00000040: C4 CC 00 00 00 3B EC E8 00 00 00 00 8B E5 5D C3  ?Ì...;ìè.....?]?

RELOCATIONS #D

                                                Symbol    Symbol
Offset    Type              Applied To          Index     Name
--------  ----------------  ------------------  --------  ------
0000002A  REL32                     00000000         C9  ??0MQTX@@QAE@XZ (public: __thiscall MQTX::MQTX(void))
00000035  REL32                     00000000         CC  ??0MATX@@QAE@XZ (public: __thiscall MATX::MATX(void))
00000048  REL32                     00000000         D8  __RTC_CheckEsp
```

图 2.13 编译器为 MBTX 生成了“合成默认构造函数”并调用了其他构造函数

2. 编译器生成“合成默认构造函数”的第二种情况

(1) 父类带有默认构造函数。

(2) 子类没有任何构造函数。

当创建一个子类对象时,父类这个默认的构造函数是要被调用的,所以,编译器会为该子类生成一个“合成默认构造函数”,用以向其中安插代码来调用父类的默认构造函数。

看看如下范例。这里就拿 MBTX 作为子类,然后引入 MBTXPARENT 作为父类,在 MyProject.cpp 的前面,增加 MBTXPARENT 类的定义:

```
class MBTXPARENT
{
public:
    MBTXPARENT()                          //默认构造函数
    {
        cout << "MBTXPARENT()构造函数执行了" << endl;
    }
};
```

改造 MBTX 类,注释掉其中的 m0 和 ma 成员变量并继承自 MBTXPARENT 类:

```
class MBTX:public MBTXPARENT
{
public:
    int m_i;
    int m_j;

    void funct()
    {
        cout << "Iamverygood" << endl;
    }
};
```

main 主函数中,保持下面这行代码不变:

```
MBTX myb;
```

执行起来，看一看结果：

```
MBTXPARENT()构造函数执行了
```

从结果可以看到，程序是调用了父类的构造函数的。看一看调用关系，编译，用dumpbin导出 MyProject.obj 对应的 MyProject.txt 文件并观察，如图 2.14 所示。

```
2280       84E3 file pointer to raw data (000084E3 to 00008524)
2281       8525 file pointer to relocation table
2282          0 file pointer to line numbers
2283          2 number of relocations
2284          0 number of line numbers
2285   60501020 flags
2286            Code
2287            COMDAT; sym= "public: __thiscall MBTX::MBTX(void)" (??0MBTX@@QAE@XZ)
2288            16 byte align
2289            Execute Read
2290
2291   RAW DATA #9
2292     00000000: 55 8B EC 81 EC CC 00 00 00 53 56 57 51 8D BD 34  U.ì.ì....SVWQ.?4
2293     00000010: FF FF FF B9 33 00 00 00 B8 CC CC CC CC F3 AB 59  ???13...?ÌÌÌÌó«Y
2294     00000020: 89 4D F8 8B 4D F8 E8 00 00 00 00 8B 45 F8 5F 5E  .M?.M?è.....E?_^
2295     00000030: 5B 81 C4 CC 00 00 00 3B EC E8 00 00 00 00 8B E5  [.?Ì...;ìè.....?
2296     00000040: 5D C3                                            ]?
2297
2298   RELOCATIONS #9
2299                                                    Symbol    Symbol
2300    Offset    Type              Applied To          Index     Name
2301    --------  ----------------  ------------------  --------  ------
2302    00000027  REL32                     00000000          C5  ??0MBTXPARENT@@QAE@XZ (public: __thiscall MBTXPARENT::MBTXPARENT(void))
2303    0000003A  REL32                     00000000          D1  __RTC_CheckEsp
```

图 2.14　编译器为 MBTX 生成了“合成默认构造函数”并调用了父类 MBTXPARENT 的构造函数

在图 2.14 中，MBTX::MBTX 调用了父类的构造函数 MBTXPARENT::MBTXPARENT。读者可以动手实践一下：如果 MBTX 类不继承自 MBTXPARENT，编译器是否还会为 MBTX 类生成“合成默认构造函数”？

3. 编译器生成“合成默认构造函数”的第三种情况

(1) 一个类含有虚函数。

(2) 该类没有任何构造函数。

改造 MBTX 类，改造后的 MBTX 类代码如下：

```
class MBTX
{
public:
    int m_i;
    int m_j;

    void funct()
    {
        cout << "Iamverygood" << endl;
    }
    virtual void mvirfunc()          //虚函数
    {
        cout << "mvirfunc" << endl;
    }
};
```

编译，用 dumpbin 导出 MyProject.obj 对应的 MyProject.txt 文件观察，如图 2.15 所示。

```
40301040 flags
         Initialized Data
         COMDAT; sym= "const MBTX::`vftable'" (??_7MBTX@@6B@)
         4 byte align
         Read Only

RAW DATA #33
  00000000: 00 00 00 00 00 00 00 00                        ........

RELOCATIONS #33

                                                 Symbol    Symbol
 Offset    Type              Applied To          Index     Name
 --------  ----------------  ----------------    --------  ------
 00000000  DIR32                     00000000       116  ??_R4MBTX@@6B@ (const MBTX::`RTTI Complete Object Locator')
 00000004  DIR32                     00000000        C5  ?mvirfunc@MBTX@@UAEXXZ (public: virtual void __thiscall MBTX::mvirfunc(void))
```

图 2.15 编译器为 MBTX 类生成了虚函数表

请注意，因为类 MBTX 中虚函数的存在，编译器会**为该类生成一个虚函数表**(vftable/vtbl)，里面记录该类中各个虚函数的首地址。虚函数相关的概念会在后面章中详细介绍，所以本节内容如果读者看不太懂可以做个标记，等把虚函数这一章学完，回头再看本节内容，可能就一目了然了。

同时，也是因为类 MBTX 中虚函数的存在，编译器为 MBTX 类生成了“合成默认构造函数”，如图 2.16 所示。

```
      8627 file pointer to raw data (00008627 to 0000865C)
      865D file pointer to relocation table
         0 file pointer to line numbers
         1 number of relocations
         0 number of line numbers
60501020 flags
         Code
         COMDAT; sym= "public: __thiscall MBTX::MBTX(void)" (??0MBTX@@QAE@XZ)
         16 byte align
         Execute Read

RAW DATA #9
  00000000: 55 8B EC 81 EC CC 00 00 00 53 56 57 51 8D BD 34  U.ì.ÌÌ...SVWQ.?4
  00000010: FF FF FF B9 33 00 00 00 B8 CC CC CC CC F3 AB 59  ???13...?ÌÌÌÌó«Y
  00000020: 89 4D F8 8B 45 F8 C7 00 00 00 00 00 8B 45 F8 5F  .M?.E??......E?_
  00000030: 5E 5B 8B E5 5D C3                                ^[.?]?

RELOCATIONS #9

                                                 Symbol    Symbol
 Offset    Type              Applied To          Index     Name
 --------  ----------------  ----------------    --------  ------
 00000028  DIR32                     00000000       10E  ??_7MBTX@@6B@ (const MBTX::`vftable')
```

图 2.16 编译器为有虚函数的 MBTX 类生成了“合成默认构造函数”

编译器同时为这个合成默认构造函数安插了代码，代码其实是一个赋值语句，用于**把类 MBTX 的虚函数表地址赋给类对象的虚函数表指针**(读者可以把虚函数表指针看成是一个隐藏的成员变量，隶属于类对象)。因为虚函数的调用存在“多态”的问题，所以需要用到虚函数表指针，后面章节会详细介绍虚函数表和虚函数表指针，这里先有个大概印象就行。

现在，再次改造 MBTX 类，让 MBTX 类依旧继承自 MBTXPARENT，然后给 MBTX 类增加一个默认构造函数。这里注意观察，看一看编译器往程序员自己写的这个默认构造函数中安插了什么代码，用来做什么事情。改造后的 MBTX 类代码如下：

```
class MBTX:public MBTXPARENT
{
public:
    int m_i;
    int m_j;
```

```
MBTX()                                  //默认构造函数
{
    m_i = 15;
}

void funct()
{
    cout << "Iamverygood" << endl;
}
virtual void mvirfunc()                 //虚函数
{
    cout << "mvirfunc" << endl;
}
};
```

编译，用 dumpbin 导出 MyProject.obj 对应的 MyProject.txt 文件并观察，如图 2.17 所示。

```
      8777 file pointer to raw data (00008777 to 000087D8)
      87D9 file pointer to relocation table
         0 file pointer to line numbers
         5 number of relocations
         0 number of line numbers
60501020 flags
         Code
         COMDAT; sym= "public: __thiscall MBTX::MBTX(void)" (??0MBTX@@QAE@XZ)
         16 byte align
         Execute Read

RAW DATA #9
  00000000: 55 8B EC 81 EC CC 00 00 00 53 56 57 51 8D BD 34  U.ì.Ì...SVWQ.?4
  00000010: FF FF FF B9 33 00 00 00 B8 CC CC CC CC F3 AB 59  ???3...?ÌÌÌÌó?Y
  00000020: 89 4D F8 B9 00 00 00 00 E8 00 00 00 00 8B 4D F8  .M?1....è.....M?
  00000030: 83 C1 04 E8 00 00 00 00 8B 45 F8 C7 00 00 00 00  .Á.è.....E??....
  00000040: 00 8B 45 F8 C7 40 04 0F 00 00 00 8B 45 F8 5F 5E  ..E??@......E?_
  00000050: 5B 81 C4 CC 00 00 00 3B EC E8 00 00 00 00 8B E5  [.?Ì...;ìè.....?
  00000060: 5D C3                                            ]?

RELOCATIONS #9

                                                Symbol    Symbol
 Offset    Type              Applied To         Index     Name
 --------  ----------------  -----------------  --------  ------
 00000024  DIR32                     00000000        30  __F0B5FBBB_MyProject@cpp
 00000029  REL32                     00000000        D5  @__CheckForDebuggerJustMyCode@4
 00000034  REL32                     00000000        CB  ??0MBTXPARENT@@QAE@XZ (public: __thiscall MBTXPARENT::MBTXPARENT(void))
 0000003D  DIR32                     00000000       119  ??_7MBTX@@6B@ (const MBTX::`vftable')
 0000005A  REL32                     00000000        D8  __RTC_CheckEsp
```

图 2.17　编译器向程序员自己写的默认构造函数中安插代码调用父类构造函数和给虚函数表指针赋值

前面说过，因为类 MBTX 中虚函数的存在，所以编译器为 MBTX 类生成了一个虚函数表，然后，编译器往 MBTX 的默认构造函数中安插了代码。这些代码看起来做了下面的事情：

(1) 调用了父类构造函数 MBTXPARENT::MBTXPARENT。

(2) 因为 MBTX 类中有虚函数存在，把 MBTX 类的虚函数表地址赋值给该类对象的虚函数表指针。

不过在图 2.17 中体现不到(看不到)代码行 m_i = 15;的执行，可以考虑换一种方式观看该代码行的执行情况。

在 MBTX 类的构造函数中，在代码行 m_i = 15;设置一个断点，按 F5 键执行程序，当断点停留在 m_i = 15;代码行时，切换到反汇编窗口(菜单命令："调试"→"窗口"→"反汇编")，不懂汇编语言也没关系，猜一猜这些汇编语言代码也同样可以达到学习效果，如

图 2.18 所示。

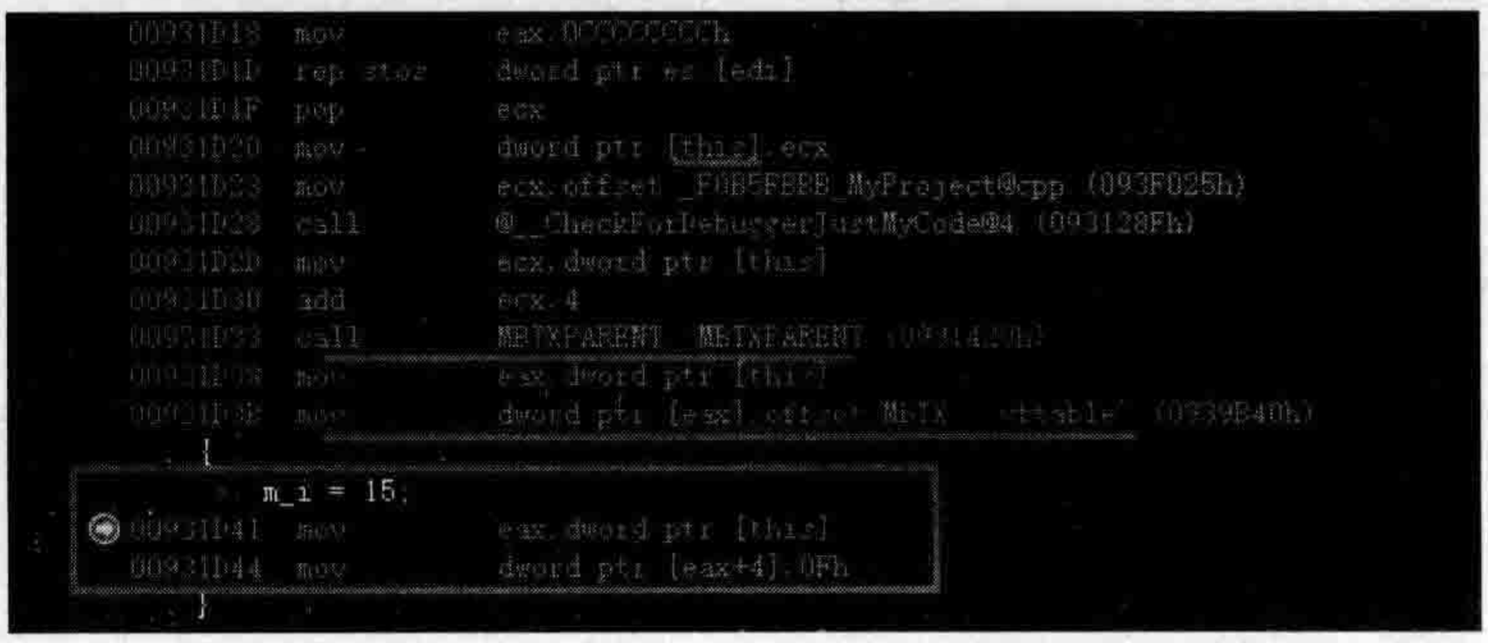

图 2.18 观察反汇编窗口中 MBTX 类构造函数内所执行的代码行

从图 2.18 可以看到，当程序员书写了自己的默认构造函数时，编译器会根据需要扩充该构造函数，向其中安插必要的代码，例如加入调用父类 MBTXPARENT 构造函数的代码，加入给虚函数表指针赋值的代码。

所以，编译器是默默地做了很多事情的：在程序员没有书写默认构造函数时会根据需要决定是否帮助程序员生成“合成默认构造函数”。当程序员书写了自己的默认构造函数时会根据需要安插必要的代码来完成一些特定的事情。

4. 编译器生成“合成默认构造函数”的第四种情况

如果一个类带有虚基类，编译器也会为它生成一个“合成默认构造函数”。

可以把以往的代码注释掉，专门写一段虚基类相关的代码。

读者都知道，虚基类实际是通过两个直接基类继承同一个间接基类。所以一般是图 2.19 的样子，是一个三层结构，有爷爷类 Grand，有两个父亲类 A 和 A2，有孙子类 C，这样才能构成虚基类（虚继承）。

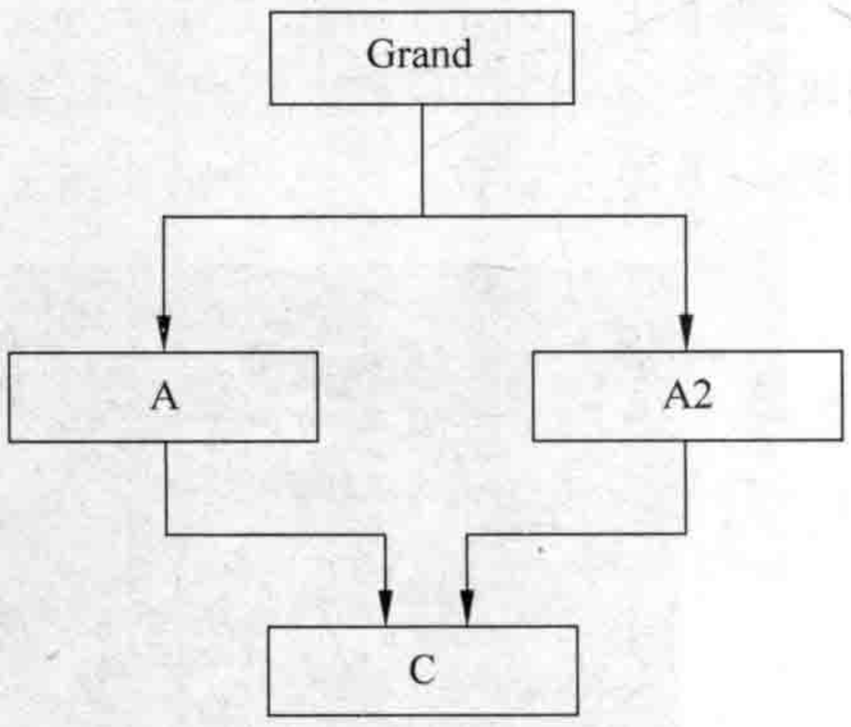

图 2.19 虚继承的继承层次关系图

想一想，在默认（常规）情况下，派生类对象中含有继承链上的每个类的子对象。那么，如果某个类（Grand）在派生过程中多次出现，那么，派生类（C）对象就会包含该类子对象多次，这样，不但多余，占空间，还可能产生名字冲突等问题。

例如，如果定义了一个 C 类对象，想访问 Grand 类的成员 m_grand，访问这个成员可以走路线 C→A→Grand::m_grand，也可以走路线 C→A2→Grand::m_grand，这种访问编译器也会直接报错，访问不明确（也就是名字冲突），**因为 Grand 类被继承了两次**，因此，在 C 类对象中**有两个 Grand 类子对象**（一次是从 A 继承来的，一次是从 A2 继承来的）。

而且，必须要明确的是，因为有了这个 C 类，并且生成了 C 类的对象，才导致 Grand 类被继承两次。

所以引入了虚基类的概念，这个概念的特点就是无论 Grand 类在继承体系中出现多少次，在派生类 C 对象中都只会包含唯一一个该虚基类（Grand）的子对象内容。所以，如果 Grand 类有这个特点，那么 Grand 就是一个虚基类。派生类 C 对象中只包含一个 Grand 子对象。

这样，如果想访问 Grand 类的成员 m_grand，不管走路线 C→A→Grand::m_grand，还是走路线 C→A2→Grand::m_grand，都是没问题的，反正 Grand 子对象只有一个。

那么虚基类(虚继承)的代码怎样写呢？

在 MyProject.cpp 的上面，增加如下代码，一共定义 4 个类：

```
class Grand //爷爷类
{
public:
};
class A : virtual public Grand          //注意 virtual
{
public:
};
class A2 : virtual public Grand         //注意 virtual
{
public:
};
class C :public A, public A2            //这里不需要 virtual
{
public:
};
```

在 main 主函数中，增加如下代码：

```
C cc;                                   //生成 C 类对象
```

编译，用 dumpbin 导出 MyProject.obj 对应的 MyProject.txt 文件并观察，利用查找功能进行查找，既可以找到 A 类构造函数 A::A，也可以找到 A2 类构造函数 A2::A2，还可以找到 C 类构造函数 C::C，分别如图 2.20～图 2.22 所示。

```
      8AAD file pointer to raw data (00008AAD to 00008AEA)
      8AEB file pointer to relocation table
         0 file pointer to line numbers
         1 number of relocations
         0 number of line numbers
 60501020 flags
            Code
            COMDAT; sym= "public: __thiscall A::A(void)" (??0A@@QAE@XZ)
            16 byte align
            Execute Read

RAW DATA #B
  00000000: 55 8B EC 81 EC CC 00 00 00 53 56 57 51 8D BD 34  U.ì.ìÌ...SVWQ.?4
  00000010: FF FF FF B9 33 00 00 00 B8 CC CC CC CC F3 AB 59  ???¹3...¸ÌÌÌÌó«Y
  00000020: 89 4D F8 83 7D 08 00 74 09 8B 45 F8 C7 00 00 00  .M?.}..t..E?Ç...
  00000030: 00 00 8B 45 F8 5F 5E 5B 8B E5 5D C2 04 00        ..‹E?_^[.?]Â..

RELOCATIONS #B

                                                Symbol    Symbol
 Offset    Type              Applied To          Index     Name
 --------  ----------------  ----------------  --------  ------
 0000002E  DIR32                     00000000       137  ??_8A@@7B@ (const A::`vbtable')
```

图 2.20　编译器为类 A 生成了“合成默认构造函数”

前面涉及一个 vftable(vtbl/虚函数表)，这里又涉及一个 vbtable(虚基类表)，vbtable 是跟虚基类有关的表，后面章节讲到虚基类的时候再详细阐述，这里先不做过多说明。

但是请注意，这种虚基类结构，编译器会为子类和父类都生成“合成默认构造函数”。

```
      89AB file pointer to raw data (000089AB to 000089E8)
      89E9 file pointer to relocation table
         0 file pointer to line numbers
         1 number of relocations
         0 number of line numbers
60501020 flags
         Code
         COMDAT; sym= "public: __thiscall A2::A2(void)" (??0A2@@QAE@XZ)
         16 byte align
         Execute Read

RAW DATA #9
  00000000: 55 8B EC 81 EC CC 00 00 00 53 56 57 51 8D BD 34  U.ì.ÌÌ...SVWQ.?4
  00000010: FF FF FF B9 33 00 00 00 B8 CC CC CC CC F3 AB 59  ???13...?ÌÌÌÌó?Y
  00000020: 89 4D F8 83 7D 08 00 74 09 8B 45 F8 C7 00 00 00  .M?.}..t..E??...
  00000030: 00 00 8B 45 F8 5F 5E 5B 8B E5 5D C2 04 00        ...E?_^[.?]?..

RELOCATIONS #9
                                                Symbol    Symbol
 Offset    Type              Applied To         Index     Name
 --------  ----------------  -----------------  --------  ------
 0000002E  DIR32                     00000000       13B  ??_8A2@@7B@ (const A2::`vbtable')
```

图 2.21 编译器为类 A2 生成了“合成默认构造函数”

```
      8BAB file pointer to raw data (00008BAB to 00008C16)
      8C17 file pointer to relocation table
         0 file pointer to line numbers
         5 number of relocations
         0 number of line numbers
60501020 flags
         Code
         COMDAT; sym= "public: __thiscall C::C(void)" (??0C@@QAE@XZ)
         16 byte align
         Execute Read

RAW DATA #D
  00000000: 55 8B EC 81 EC CC 00 00 00 53 56 57 51 8D BD 34  U.ì.ÌÌ...SVWQ.?4
  00000010: FF FF FF B9 33 00 00 00 B8 CC CC CC CC F3 AB 59  ???13...?ÌÌÌÌó?Y
  00000020: 89 4D F8 83 7D 08 00 74 13 8B 45 F8 C7 00 00 00  .M?.}..t..E??...
  00000030: 00 00 8B 45 F8 C7 40 04 00 00 00 00 6A 00 8B 4D  ...E??@.....j..M
  00000040: F8 E8 00 00 00 00 6A 00 8B 4D F8 83 C1 04 E8 00  ?è....j..M?.Á.è.
  00000050: 00 00 00 8B 45 F8 5F 5E 5B 81 C4 CC 00 00 00 3B  ....E?_^[.?Ä...;
  00000060: EC E8 00 00 00 00 8B E5 5D C2 04 00              ìè.....?]?..

RELOCATIONS #D
                                                Symbol    Symbol
 Offset    Type              Applied To         Index     Name
 --------  ----------------  -----------------  --------  ------
 0000002E  DIR32                     00000000       13F  ??_8C@@7BA@@@ (const C::`vbtable'{for `A'})
 00000038  DIR32                     00000000       143  ??_8C@@7BA2@@@ (const C::`vbtable'{for `A2'})
 00000042  REL32                     00000000        E0  ??0A@@QAE@XZ (public: __thiscall A::A(void))
 0000004F  REL32                     00000000        E1  ??0A2@@QAE@XZ (public: __thiscall A2::A2(void))
 00000062  REL32                     00000000        ED  __RTC_CheckEsp
```

图 2.22 编译器为类 C 生成了“合成默认构造函数”

熟悉汇编语言的读者，想研究一下编译器到底在调用类 C 构造函数之前做了什么事，当然是可以的。这里可以给类 C 手工增加一个默认构造函数。完整的类 C 定义修改成下面的样子：

```
class C :public A, public A2          //这里不需要 virtual
{
public:
    C()                               //默认构造函数
    {
        cout << "C::C()默认构造函数执行了" << endl;
    }
};
```

按 F9 键将断点设置在类 C 默认构造函数中 cout 语句行，按 F5 键运行程序，当断点停到类 C 默认构造函数中的 cout 语句行时，切换到反汇编窗口(菜单命令："调试"→"窗口"→"反汇编")，看一看编译器向构造函数中额外增加了什么代码，如图 2.23 所示。

图 2.23　编译器向程序员自己写的类 C 默认构造函数中安插代码

看起来这些额外增加的代码就是给虚基类表指针赋值以及调用父类的构造函数。其他的就不多说。

5. 编译器生成"合成默认构造函数"的第五种情况

如果在定义成员变量的时候赋初值，C++ 11 支持这种新语法。

在 MyProject.cpp 前面，增加一个类 Time 的定义：

```
class Time
{
public:
    int Second{ 0 };                    //定义并初始化成员变量
};
```

在 main 主函数中，加入如下代码：

```
Time mytime;
```

编译，用 dumpbin 导出 MyProject.obj 对应的 MyProject.txt 文件并观察，利用查找功能进行查找，可以找到 Time::Time 构造函数，这也是编译器为 Time 类生成的"合成默认构造函数"，其目的，并不难想象：在其中安插代码，初始化 Second 成员变量的值为 0(因为代码中{}内给的初值是 0)。

还有哪些情况编译器会在程序员没有写自己的构造函数时为程序员生成"合成默认构造函数呢"？这个问题就留给读者进一步探索和发现了。

2.5　拷贝构造函数语义与移动构造函数语义

2.5.1　拷贝构造函数语义

拷贝构造函数都不陌生，尤其是当用一个类对象初始化另外一个类对象时，需要调用的是类的拷贝构造函数。

传统上，读者可能有这样一个概念：如果程序员没有定义自己的拷贝构造函数，编译器会帮助程序员合成一个拷贝构造函数。

实际情况也同默认构造函数一样，如果程序员没有定义自己的拷贝构造函数，编译器会在**必要**的时候合成出一个拷贝构造函数。

看看如下范例。在 MyProject.cpp 前面，增加一个类 A 的定义：

```
class A
{
public:
    int m_test;
};
```

在 main 主函数中，加入如下代码：

```
A mya1;
mya1.m_test = 15;
A mya2 = mya1;     //这属于拷贝构造动作,但编译器没有合成拷贝构造函数
```

设置断点并运行起来观察，对象 mya2 的 m_test 成员变量确实被赋予了 15，但使用 dumpbin 导出 MyProject.obj 对应的 MyProject.txt 文件并观察，并没有看到类 A 的拷贝构造函数产生。

程序员没有书写自己的拷贝构造函数，编译器也没有产生拷贝构造函数，但是，mya2.m_test 却有了 15 这个值。为什么呢？这其实就是**编译器内部**的一个手法：成员变量初始化手法，如 int 这种简单类型，直接就**按值拷贝**过来了，编译器在**不需要合成拷贝构造函数**的情况下就能够帮助程序员做到这一点。

在 MyProject.cpp 前面，增加一个 ASon 新类的定义，位置放在 A 类的上面：

```
class ASon
{
public:
    int m_testson;
};
```

类 A 增加一个成员变量，修改后的类 A 代码如下：

```
class A
{
public:
    int m_test;
    ASon asubobj;
};
```

在 main 主函数中，注释掉以往代码，加入全新的如下代码：

```
A mya1;
mya1.m_test = 15;
mya1.asubobj.m_testson = 120;
A mya2 = mya1;
```

设置断点并跟踪调试，可以发现，mya2.asubobj.m_testson 的值也会被设置为 120。

上面这段代码，总结一下：

(1) 代码行 A mya2 = mya1；用于拷贝构造一个对象。

(2) 程序员并没有书写类 A 的拷贝构造函数，编译器也没有帮助程序员生成类 A 的拷贝构造函数。

(3) 通过设置断点并调试运行，观察发现 mya1 对象的成员变量值的确被复制到了 mya2 中，这是编译器内部的一些**直接复制数据的实现手法**，如果类 A 中有类类型(ASon)成员变量(asubobj)，也会**递归式**地去复制这个类 ASon 的每个成员变量。

但是，在某些情况下，如果程序员不书写自己的拷贝构造函数，编译器会帮助程序员生成出一个拷贝构造函数来。那么，在什么情况下编译器会帮助程序员生成拷贝构造函数，而这个生成的拷贝构造函数要做什么事呢？

1. 编译器生成“拷贝构造函数”的第一种情况

(1) 如果一个类 A 没有拷贝构造函数。

(2) 但该类含有一个类类型(CTB 类，见下)的成员变量，类 CTB 含有拷贝构造函数。

那么当代码中有涉及类 A 对象的复制构造时，编译器会为类 A 生成一个拷贝构造函数。

在 MyProject.cpp 前面，增加一个 CTB 新类的定义，该类中含有一个拷贝构造函数：

```
class CTB
{
public:
    CTB(const CTB&)                    //拷贝构造函数
    {
        cout << "CTB 类的拷贝构造函数执行了" << endl;
    }
    CTB()                              //默认构造函数
    {
    }
};
```

在类 A 中，增加一个 public 修饰的类类型成员变量如下：

```
public:
    CTB m_ctb;
```

执行起来，看一看结果：

```
CTB 类的拷贝构造函数执行了
```

那么，CTB 的拷贝构造函数是谁调用的呢？是当运行到 A mya2 = mya1；代码行时，编译器因为 CTB 类拷贝构造函数的存在而为类 A 生成了一个拷贝构造函数。为什么编译器要生成这个拷贝构造函数呢？其实就是为了向其中嵌入代码调用类 CTB 的拷贝构造函数。

编译，用 dumpbin 导出 MyProject.obj 对应的 MyProject.txt 文件并观察，如图 2.24 所示。

然后再设置断点进行一下跟踪调试。可以看到，就算是编译器为类 A **生成了拷贝构造**

```
    8623 file pointer to raw data (00008623 to 00008686)
    8687 file pointer to relocation table
       0 file pointer to line numbers
       2 number of relocations
       0 number of line numbers
60501020 flags
         Code
         COMDAT; sym= "public: __thiscall A::A(class A const &)" (??0A@@QAE@ABV0@@Z)
         16 byte align
         Execute Read

RAW DATA #9
  00000000  55 8B EC 81 EC CC 00 00 00 53 56 57 51 8D BD 34  U.ì.ìÌ...SVWQ.?4
  00000010  FF FF FF B9 33 00 00 00 B8 CC CC CC CC F3 AB 59  ???13...?ÌÌÌÌó?Y
  00000020  89 4D F8 8B 45 F8 8B 4D 08 8B 11 89 10 8B 45 08  .M?.E?.M......E.
  00000030  8B 48 04 8B 55 F8 89 4A 04 8B 45 08 83 C0 08 50  .H..U?.J..E..À.P
  00000040  8B 4D F8 83 C1 08 E8 00 00 00 00 8B 45 F8 5F 5E  .M?.Á.è.....E?_
  00000050  5B 81 C4 CC 00 00 00 3B EC E8 00 00 00 00 8B E5  [.Ä)...;ìè.....?
  00000060  5D C2 04 00                                      ]?..

RELOCATIONS #9

                                                 Symbol    Symbol
 Offset    Type              Applied To          Index     Name
 --------  ----------------  ------------------  --------  ------
 00000047  REL32                     00000000        CF  ??0CTB@@QAE@ABV0@@Z (public: __thiscall CTB::CTB(class CTB const &))
 0000005A  REL32                     00000000        DF  __RTC_CheckEsp
```

图 2.24 编译器为类 A 生成拷贝构造函数并安插代码调用类 CTB 的拷贝构造函数

函数，原来的一些成员变量值的复制都不受影响。例如，在执行了代码行 A mya2 = mya1；后，mya2 的各个成员变量的值依然和 mya1 中相同。

所以，可以得到一个结论：编译器生成拷贝构造函数往往都是有一些特殊事情要做。如果**只是一些类成员变量的值复制这些简单的事情，编译器不用专门生成拷贝构造函数来做，编译器内部就可以完成**。

2. 编译器生成“拷贝构造函数”的第二种情况

(1) 如果一个类 CTBSon 没有拷贝构造函数，但是它有一个父类 CTB。

(2) 父类 CTB 有拷贝构造函数。

当代码中有涉及类 CTBSon 对象的复制构造时，编译器会为类 CTBSon 生成一个拷贝构造函数。

在 MyProject.cpp 前面，增加一个 CTBSon 新类的定义：

```
class CTBSon :public CTB
{
public:
};
```

在 main 主函数中，注释掉以往代码，加入全新的如下代码：

```
CTBSon myctbson1;
CTBSon myctbson2 = myctbson1;
```

执行起来，看一看结果：

```
CTB 类的拷贝构造函数执行了
```

可以注意到，父类 CTB 的拷贝构造函数被调用。是谁调用的呢？是编译器生成了 CTBSon 类的拷贝构造函数，目的是通过向该函数中嵌入代码调用其父类 CTB 的拷贝构造函数。

编译，用 dumpbin 导出 MyProject.obj 对应的 MyProject.txt 文件并观察，如图 2.25 所示。

```
      896A file pointer to raw data (0000896A to 000089B1)
      89B2 file pointer to relocation table
         0 file pointer to line numbers
         2 number of relocations
         0 number of line numbers
60501020 flags
         Code
         COMDAT; sym= "public: __thiscall CTBSon::CTBSon(class CTBSon const &)" (??0CTBSon@@QAE@ABV0@@Z)
         16 byte align
         Execute Read

RAW DATA #D
  00000000: 55 8B EC 81 EC CC 00 00 00 53 56 57 51 8D BD 34  U.ì.ÌÌ...SVWQ.?4
  00000010: FF FF FF B9 33 00 00 00 B8 CC CC CC CC F3 AB 59  ???13...?ÌÌÌÌó«Y
  00000020: 89 4D F8 8B 45 08 50 8B 4D F8 E8 00 00 00 00 8B  .M?.E.P.M?è.....
  00000030: 45 F8 5F 5E 5B 81 C4 CC 00 00 00 3B EC E8 00 00  E?_^[.?Ä...;ìè..
  00000040: 00 00 8B E5 5D C2 04 00                          ..?å]?..

RELOCATIONS #D
                                                 Symbol    Symbol
 Offset    Type              Applied To          Index     Name
 --------  ----------------  ------------------  --------  ------
 0000002B  REL32                     00000000          CF  ??0CTB@@QAE@ABV0@@Z (public: __thiscall CTB::CTB(class CTB const &))
 0000003E  REL32                     00000000          DF  __RTC_CheckEsp
```

图 2.25　编译器为类 CTBSon 生成拷贝构造函数并安插代码调用类 CTB 的拷贝构造函数

3. 编译器生成“拷贝构造函数”的第三种情况

(1) 如果一个类 CTBSon 没有拷贝构造函数。

(2) 但是该类定义了虚函数或者该类的父类定义了虚函数。

当代码中涉及类 CTBSon 对象的拷贝构造时，编译器会为类 CTBSon 生成一个拷贝构造函数。

修改 CTBSon 类，不再继承自 CTB 类并增加一个虚函数定义：

```
class CTBSon
{
public:
    virtual void mvirfunc() {};      //定义一个虚函数
};
```

main 主函数中代码保持不变。

编译，用 dumpbin 导出 MyProject.obj 对应的 MyProject.txt 文件并观察，如图 2.26 所示。

```
      6D54 file pointer to raw data (00006D54 to 00006D89)
      6D8A file pointer to relocation table
         0 file pointer to line numbers
         1 number of relocations
         0 number of line numbers
60501020 flags
         Code
         COMDAT; sym= "public: __thiscall CTBSon::CTBSon(void)" (??0CTBSon@@QAE@XZ)
         16 byte align
         Execute Read

RAW DATA #6
  00000000: 55 8B EC 81 EC CC 00 00 00 53 56 57 51 8D BD 34  U.ì.ÌÌ...SVWQ.?4
  00000010: FF FF FF B9 33 00 00 00 B8 CC CC CC CC F3 AB 59  ???13...?ÌÌÌÌó«Y
  00000020: 89 4D F8 8B 45 F8 C7 00 00 00 00 00 8B 45 F8 5F  .M?.E??......E?_
  00000030: 5E 5B 8B E5 5D C3                                ^[.?]?

RELOCATIONS #6
                                                 Symbol    Symbol
 Offset    Type              Applied To          Index     Name
 --------  ----------------  ------------------  --------  ------
 00000028  DIR32                     00000000          77  ??_7CTBSon@@6B@ (const CTBSon::`vftable')
```

图 2.26　编译器为类 CTBSon 生成拷贝构造函数并安插代码给虚函数表指针赋值

可以看到，编译生成了类 CTBSon 的拷贝构造函数，并向这个拷贝构造函数中安插代码来给被复制的该类对象的虚函数表指针赋值。对于虚函数表指针的概念，在后面章节会详细讲述。

现在，看一看父类有虚函数的情形。修改 CTB 类，修改后的代码如下：

```
class CTB
{
public:
    virtual void mvirfunc() {}
};
```

修改子类 CTBSon，继承自父类 CTB，并去掉虚函数：

```
class CTBSon :public CTB
{
public:
};
```

main 主函数中代码保持不变。

编译，用 dumpbin 导出 MyProject.obj 对应的 MyProject.txt 文件并观察。不难发现编译器不但生成了 CTBSon 类的拷贝构造函数，也生成了 CTB 类的拷贝构造函数。

CTBSon 类的拷贝构造函数被安插代码调用父类 CTB 的构造函数，同时用来给被复制的该类对象的虚函数表指针赋值。而 CTB 类的拷贝构造函数被安插代码用来给被复制的该类对象的虚函数表指针赋值(做了两次赋值似乎不应该，后面章节会详细分析这个问题)。

4. 编译器生成“拷贝构造函数”的第四种情况

(1) 如果一个类没有拷贝构造函数。

(2) 但是该类含有虚基类。

当代码中有涉及类对象的复制构造时，编译器会为该类合成一个拷贝构造函数。

把以往的类代码注释掉，在 MyProject.cpp 的前面，重新写几个类：

```
class Grand                              //爷爷类
{
public:
};
class A : virtual public Grand
{
public:
};
class A2 : virtual public Grand
{
public:
};
class C :public A, public A2
{
public:
};
```

在 main 主函数中，增加如下代码：

```
C cc;
C cc2 = cc;
```

编译，用 dumpbin 导出 MyProject.obj 对应的 MyProject.txt 文件并观察。实际上，A、A2、C 类，编译器都生成了拷贝构造函数。这些拷贝构造函数里面的内容和上一节讲解的合成默认构造函数内容也都很类似，也是与 vbtable(虚基类表)有关。虚基类的内容，后面章节再讲解，这里也是不做过多说明。

还有哪些情况编译器会在程序员没有写自己的拷贝构造函数时 为程序员生成拷贝构造函数呢？这个问题就留给读者探索。

另外，如类中的**赋值运算符**，如果程序员没有重载，那么在某些情况下(如该类中有个其他类类型的对象，而这个其他类类型中却重载了一个赋值运算符)，编译器就会为该类重载赋值运算符。道理都是相通的，读者有兴趣可以自己尝试。

2.5.2 移动构造函数语义

在 C++11 新标准中，引入了“移动构造函数”和“移动赋值运算符”的概念。如果读者不太熟悉，可以通过搜索引擎搜索，也可以阅读笔者所著的《C++新经典》，该书中有详细介绍。

其实，在某些情况下，如果程序员没有书写自己的“移动构造函数”和“移动赋值运算符”，编译器也会生成(合成)出“移动构造函数”和“移动赋值运算符”。

(1) 如果一个类定义了自己的拷贝构造函数、拷贝赋值运算符或者析构函数(这三者之一，表示程序员要自己处理对象的复制或者释放问题)，编译器就不会为该类生成出移动构造函数和移动赋值运算符。这说明，只要程序员有自己复制对象和释放对象的倾向，编译器就不会帮助程序员生成移动动作的相关函数(所以有一些类是没有移动构造函数和移动赋值运算符的)，这样就可以防止编译器合成出一个完全不是程序员自己想要的移动构造函数或者移动赋值运算符。

(2) 只有一个类没定义任何自己版本的拷贝构造函数、拷贝赋值运算符、析构函数，且类的每个非静态成员都可以移动时，编译器才会为该类合成移动构造函数或者移动赋值运算符。那什么叫成员可以移动呢？

① 内置类型(如整型、实型等)的成员变量可以移动。

② 如果成员变量是一个类类型，如果这个类有对应的移动操作相关的函数，则该成员变量可以移动。

此时编译器就会依据具体的代码来智能地决定是否生成移动构造函数和移动赋值运算符。看看如下范例。

在 MyProject.cpp 前面，增加一个 TC 新类的定义：

```
struct TC
{
    int i;                          //内置类型可以移动
    std::string s;                  //string 类型定义了自己的移动操作
};
```

在 main 主函数中，加入如下代码：

```
TC a;
a.i = 100;
a.s = "I Love China!";
const char * p = a.s.c_str();
TC b = std::move(a);  //导致结构/类 TC 移动构造函数的执行,数据移动不是 std::move 所为,
                      //而是 string 的移动构造函数所为
const char * q = b.s.c_str();
```

增加断点,调试观察,可以看到,执行完上面的代码后,a.s 已经为空(""),这是执行了 string 类移动构造函数的结果。同时,注意到 b.s 结果为"I Love China!"(如果此时读者向 TC 结构中增加一个析构函数,就会发现 a.s 不再为空,因为系统不会为 TC 生成移动构造函数)。

另外,p 和 q 指向的内存位置也是不同的,这应该是 string 类的特性所导致。所以,这里虽然执行了 string 的移动构造函数,但是似乎也没节省什么性能(没体现出移动构造函数节省性能的特性)。

其实,从上面的代码看,编译器之所以会为 TC 合成移动构造函数,完全是因为代码行 TC b = std::move(a);的存在,编译器为 TC 合成移动构造函数的目的是往该函数中安插代码以调用 string 类的移动构造函数。

编译,用 dumpbin 导出 MyProject.obj 对应的 MyProject.txt 文件并观察,如图 2.27 所示。

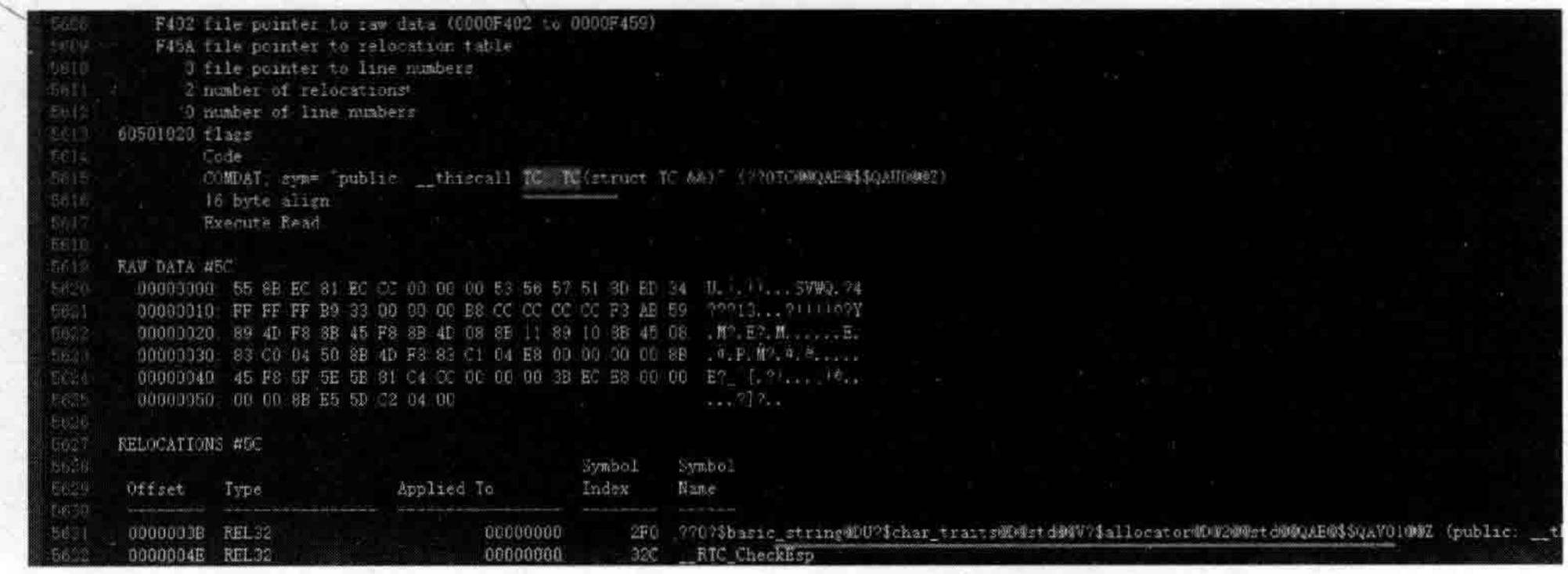

图 2.27 编译器为类 TC 生成移动构造函数并安插代码调用 string 类的移动构造函数

编译器合成移动赋值运算符的情况与合成移动构造函数非常类似,笔者在这里同样不多说,读者可以根据上面所学的内容,自行尝试。

经过多年的发展,编译器的功能已经非常强大,而且也非常智能,为了特定的需要,编译器会为程序员生成(合成)很多类中的成员函数并向其中安插代码,并不仅仅限于上面讲述的这些成员函数,请读者在学习的时候举一反三,灵活地去理解。

2.6 程序转换语义

程序转换语义,就是程序员书写的一些代码,编译器在处理的时候,会进一步把这些代码拆分,拆分成更容易被理解和被实现的代码。

换句话说，现在要研究的是程序员所写的代码，编译器会如何去解析的问题。所以本节内容不但会站在程序员的角度，也会站在编译器的角度，研究一下编译器的做法，通过这样的分析，对编译器的工作步骤有更进一步的了解。

本节需要在“程序员看代码”的视角和“编译器看代码的视角”之间不断切换。

2.6.1 定义时初始化对象

看看如下范例。在 MyProject.cpp 前面，增加一个 X 类的定义：

```
class X
{
public:
    int m_i;
    X(const X& tmpx)
    {
        m_i = tmpx.m_i;
        cout << "X 类的拷贝构造函数被调用" << endl;
    }
    X()
    {
        m_i = 0;
        cout << "X 类的构造函数被调用" << endl;
    }
};
```

在 main 主函数中，加入如下代码：

```
X x0;
x0.m_i = 15;
X x1 = x0;
X x2(x0);
X x3 = x0;
```

执行起来，看一看结果：

```
X 类的构造函数被调用
X 类的拷贝构造函数被调用
X 类的拷贝构造函数被调用
X 类的拷贝构造函数被调用
```

因为读者对 C++都很熟悉，所以上面的结果不难理解。这里就以 X x3 = x0;代码行为例，谈一下这种定义时初始化的操作（下面把 x3 换名为 x3_2 以方便代码演示）。

编译器遇到这种代码时，它会拆分成两个步骤。切换成编译器的视角看一看。

（1）定义一个对象，为对象分配内存，但这里编译器内部并没有调用构造函数。如果以编译器的眼光看 X x3_2，则不会调用构造函数，除非编译器主动增加 x3_2.X::X();这种代码行到现有的代码中：

```
X x3_2;
```

(2) 直接调用对象的拷贝构造函数：

```
x3_2.X::X(x0);
```

读者可以把上面这两行代码输入到程序中，运行起来注意观察结果。可以看到，在原有结果的基础之上，多了两行结果：

```
X类的构造函数被调用
X类的拷贝构造函数被调用
```

这两行结果中的第一行多调用了一个构造函数，因为这是程序员自己写的代码。实际编译器在处理的时候并不会调用X类的构造函数，所以实际上是看不到第一行结果的。最终能看到的是第二行结果“X类的拷贝构造函数被调用”。

2.6.2　参数的初始化

继续给X类增加一个public修饰的析构函数：

```
public:
    ~X()
    {
        cout << "X类的析构函数被调用" << endl;
    }
```

再增加一个普通函数：

```
void func(X tmpx)
{
    return;
}
```

在main主函数中，注释掉以往代码，增加如下代码：

```
X x0;
func(x0);                                    //程序员视角(现代编译器视角)
```

执行起来，看一看结果：

```
X类的构造函数被调用
X类的拷贝构造函数被调用
X类的析构函数被调用
X类的析构函数被调用
```

从上面这个结果不难理解，当把实参传递给形参时，调用了X类的拷贝构造函数。

上面这两行代码的执行结果与下面这两行代码一样(都是调用了一次构造函数和一次拷贝构造函数)：

```
X x0;
X tmpx = x0;
```

对于代码行func(x0);，**编译器**是在func这个函数的**空间内**构造出来了tmpx对象，然

后在 func 函数返回之前，再把 tmpx 对象析构掉。

对于比较老(早期)的编译器，上面对 func 函数的调用可能就更加烦琐，**站在老编译器的视角**看一看(再次提醒：站在编译器视角来说，类的构造、复制构造、析构等函数都不会自动去调用)：

```
//以下这些可以认为是伪代码,并不代表在当代编译器上能够正确运行,只是借此了解编译器内部具体都做了什么
X tmpobj;                                    //编译器产生出来一个临时对象
tmpobj.X::X(x0);                             //调用拷贝构造函数
func(tmpobj);                                //用临时对象调用 func
tmpobj.X::~X();                              //func()被调用完成后,析构函数被调用
```

func 函数的参数站在老编译器的角度也会做出调整，变成了引用：

```
void func(X &tmpx)
{
    return;
}
```

上面这种老编译器视角相当于把 func 函数的参数率先在调用 func 函数之前就构造出来，然后再传递到 func 函数中。

所以，读者可以看到，程序员写的**一行**代码，编译器往往会**转换成多行**代码实现相应的功能。同时也感觉到了老编译器实现一些代码的笨拙和编译器的不断进化。

2.6.3 返回值初始化

注释掉以往的代码，在 MyProject.cpp 前面，增加如下代码：

```
X func()
{
    X x0;
    //.....
    return x0;
}
```

在 main 主函数中，增加如下代码：

```
X my = func();
```

执行起来，看一看结果：

```
X类的构造函数被调用
X类的拷贝构造函数被调用
X类的析构函数被调用
X类的析构函数被调用
```

读者可以设置断点跟踪调试并观察结果。笔者假设读者对 C++的开发知识掌握的已经比较好。所以，对上面的结果也会很明了。

在上面代码的 func 函数中，因为 return x0;语句行的出现，产生了临时对象，这个临时对象就直接构造到了 my 中，也就是说可以把 my 看成是这个临时对象。

站在程序员的角度，上面的解释和理解是对的。那么，看一看编译器是怎样理解和处理这些代码的。

切换成编译器视角，如下是一种编译器的理解方法：

```
void func(X& extra) //1: 安插一个引用参数
{
    X x0;     //这个之所以能调用构造函数，是因为编译器内部有诸如 x0.X::X()这种代码
    //.....
    extra.X::X(x0);                    //2: return 之前安插一个对拷贝构造函数的调用
    //return x0;
    return;
}
```

main 主函数中代码从编译器的视角看应该是这样的：

```
X my;  //从编译器的视角看，这里是不调用构造函数的，除非编译器主动增加代码 my.X::X();
func(my);
```

观看从编译器的视角生成的这些代码，感觉更容易理解。

现在，再给 X 类增加一个 public 修饰的成员函数：

```
public:
    void functest()
    {
        cout << "functest()成员函数被调用" << endl;
    }
```

现在，切换回程序员视角。在 main 主函数中，加入下面代码行：

```
func().functest();
```

func 函数的程序员视角内容保持不变(前面的 X func(){…})。

此时，看一看编译器视角下的 main 主函数中的代码变成什么样子：

```
X my;
(func(my), my).functest();  //逗号表达式，先求解表达式 1，再求解表达式 2，整个表达式的值
                            //是表达式 2 的值
```

func 函数的编译器视角内容保持不变(前面的 void func(X& extra){…})。

继续，切换回程序员视角。在 main 主函数中，加入下面代码行：

```
X( * pf)();                         //定义函数指针
pf = func;
pf().functest();
```

此时，看一看编译器视角下的 main 主函数中的代码变成什么样子：

```
X my;
void ( * pf)(X&);
pf = func;
pf(my);
my.functest();
```

2.7 程序的优化

本节谈一谈程序的优化，笔者准备从两个方面谈：①从开发者层面；②从编译器层面。

2.7.1 从开发者层面的优化

看看如下范例。在 MyProject.cpp 前面，增加如下类 CTempValue 的定义代码：

```
class CTempValue
{
public:
    int val1;
    int val2;
public:
    CTempValue(int v1 = 0, int v2 = 0) : val1(v1), val2(v2) //构造函数
    {
        cout << "调用了 CTempValue 类的构造函数!" << endl;
        cout << "val1 = " << val1 << endl;
        cout << "val2 = " << val2 << endl;
    }
    CTempValue(const CTempValue& t) :val1(t.val1),val2(t.val2)//拷贝构造函数
    {
        cout << "调用了 CTempValue 类的拷贝构造函数!" << endl;
    };
    virtual ~CTempValue()              //析构函数
    {
        cout << "调用了 CTempValue 类的析构函数!" << endl;
    };
};
```

再增加一个函数定义(**这是 Double 函数的第一个版本**)：

```
CTempValue Double(CTempValue& ts)     //第一个版本
{
    CTempValue tmpm;                  //这里会消耗一个构造和一个析构函数的调用
    tmpm.val1 = ts.val1 * 2;
    tmpm.val2 = ts.val2 * 2;
    return tmpm;  //断点设置到这里,发现调用了拷贝构造函数和析构函数,这表示生成了临时对象
}
```

在 main 主函数中，增加如下代码：

```
CTempValue ts1(10, 20);
Double(ts1);
```

执行起来，看一看结果：

```
调用了 CTempValue 类的构造函数!
val1 = 10
val2 = 20
```

```
调用了 CTempValue 类的构造函数!
val1 = 0
val2 = 0
调用了 CTempValue 类的拷贝构造函数!
调用了 CTempValue 类的析构函数!
调用了 CTempValue 类的析构函数!
调用了 CTempValue 类的析构函数!
```

这里读者可以设置断点并跟踪调试，详细查看哪些语句行导致构造函数、拷贝构造函数和析构函数的执行。

上面的代码中，因为 Double 函数返回的临时对象(return tmpm;)没有接收者，所以这个临时对象会在 Double 函数调用后被立即释放(调用一次析构函数)，如果 Double 返回的临时对象有接收者，如代码行修改为这样：

```
CTempValue ts2 = Double(ts1);
```

那么，Double 函数返回的临时对象就不会被立即析构，而是会直接构造到 ts2 中，并且一直到 ts2 作用域结束后才会析构该临时对象。

分析一下程序，不难想象，在 Double 函数里，CTempValue tmpm;代码行会消耗一个构造和一个析构函数的调用。而 return tmpm;代码行生成了临时对象，所以最终它又占用了一个拷贝构造函数和一个析构函数的调用。

那么针对上面这段 Double 函数中的代码，是否有优化空间呢？当然是有的，笔者把这两行调用构造函数的代码想办法合成一行，改造一下 Double 函数(**这是 Double 函数的第二个版本**)。新的 Double 函数代码如下：

```
CTempValue Double(CTempValue& ts)                    //第二个版本
{
    return CTempValue(ts.val1 * 2, ts.val2 * 2);     //临时对象
}
```

执行起来，看一看结果：

```
调用了 CTempValue 类的构造函数!
val1 = 10
val2 = 20
调用了 CTempValue 类的构造函数!
val1 = 20
val2 = 40
调用了 CTempValue 类的析构函数!
调用了 CTempValue 类的析构函数!
```

通过本次与上次执行结果的对比可以看到，修改(优化)后的代码少调用了一次拷贝构造函数和一次析构函数。

上面优化后的代码是处于程序员的视角，那么，在编译器的视角中，优化后的 Double 函数代码应该是什么样子呢(注意代码中的注释)？

```
void Double(CTempValue &tmpobj, CTempValue &ts1) //编译器会插入一个 tmpobj 参数
```

```
{
    tmpobj.CTempValue::CTempValue(ts1.val1 * 2, ts1.val2 * 2);  //编译器手工调用构造函数
    return;
}
```

在编译器的视角中，main 主函数中的代码应该又是什么样子呢？

```
CTempValue ts1;
ts1.CTempValue::CTempValue(10, 20);
CTempValue tmpobj;
Double(tmpobj, ts1);
```

2.7.2 从编译器层面的优化

为了研究从编译器层面的优化问题，笔者切换到 Linux 操作系统，因为在 Linux 操作系统中比较容易观察到编译器做了哪些优化。在 Linux 操作系统下，输入刚才没有优化过的代码(参考本节前面 Double 函数的**第一个**版本，这里不再重复这些代码)。

用 g++编译并运行起来，可以发现，得到的结果与在 Visual Studio 2019 中得到的结果并不一样，但却与在 Visual Studio 2019 中优化了 Double 函数后(参考本节前面 Double 函数的第二个版本)得到的结果相同。

这种情况可以认为是 g++编译器的优化，属于编译器层面帮助程序员进行了优化。这里，g++编译器直接把 Double 函数的第一个版本的代码优化成了 Double 函数的第二个版本的代码。也就是说，编译器把 Double 函数第一个版本中的 tmpm 对象优化没了，被 g++编译器用 Double 函数第二个版本中的 tmpobj 取代了。

因为编译器的这种优化是针对返回临时对象的情况所进行的优化，所以有些资料上也称之为 NRV 优化(Named Return Value 优化)或者也有叫 RVO(Return Value Optimization)，这些叫法，读者大概了解，并不需要太详细研究。

g++编译器的这种优化也是可以**关闭**的，使用的是-fno-elide-constructors 选项。例如：

```
g++ -fno-elide-constructors MyProject.cpp -o MyProject
```

关闭优化选项后，再次执行程序，得到的结果就与在 Visual Studio 2019 中第一次运行得到的结果相同了(未优化前得到的结果)。

Visual Studio 2019(以及其他的 Visual Studio 版本)可以做这种优化吗？可以尝试着找一找优化选项。

在优化之前，先**将代码恢复**到本节刚开始展示的优化之前的 Double 函数的版本(Double 函数的第一个版本)以验证优化选项的设置效果，然后开始进行优化选项的设置：

(1) 在 Visual Studio 2019 中右击“解决方案资源管理器”中项目名 MyProject 并在弹出的快捷菜单中选择“属性”命令，这将打开“MyProject 属性页”对话框，在对话框左侧选择“配置属性”→C/C++→“优化”，对话框右侧的“优化”当前的选项是“已禁用(/Od)”，这里选择“**最大优化(优选速度)(/O2)**”选项，如图 2.28 所示(其他优化选项读者可以根据需要自行研究)。

(2) 依旧是在“MyProject 属性页”对话框，在对话框左侧选择“配置属性”→C/C++→“代码生成”，对话框右侧的“基本运行时检查”当前的选项是“两者(/RTC1，等同于

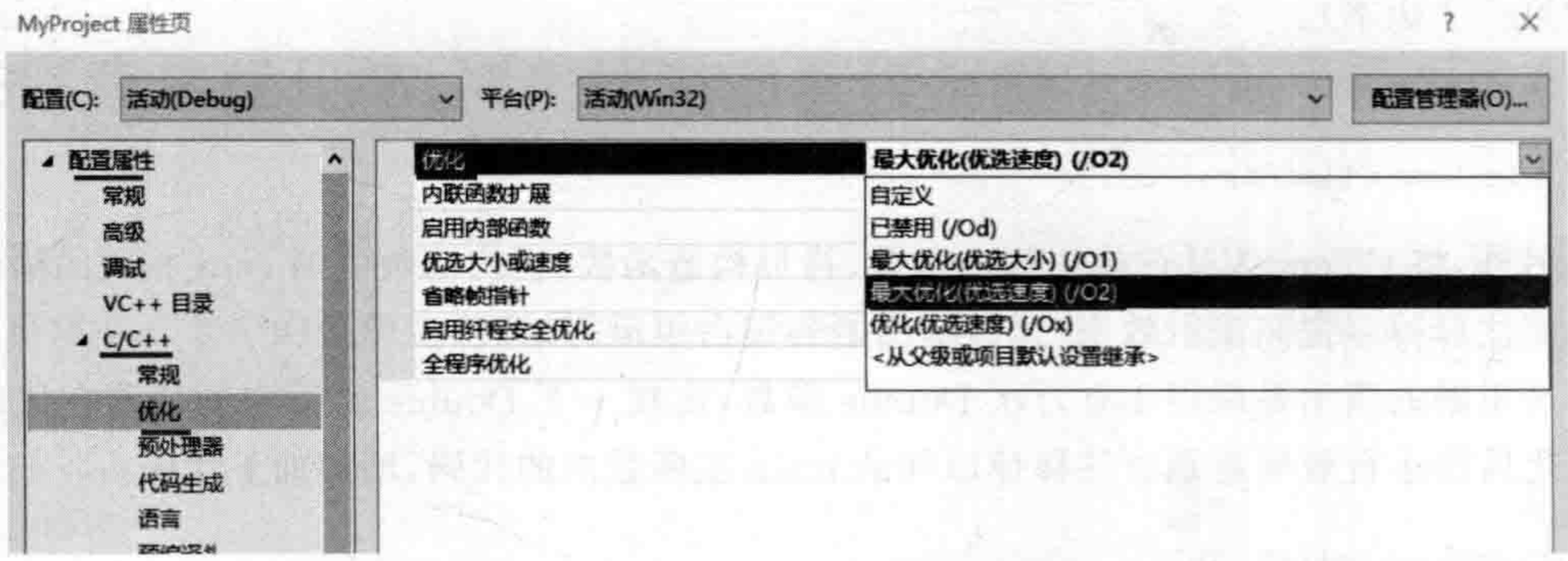

图 2.28 开启 Visual Studio 2019 中编译器的优化选项

/RTCsu)（/RTC1)”，这个选项是一种用于发现运行时代码问题的手段，如有的时候程序有栈溢出、指针问题等，编译器都能报异常，但是，如果使用了优化选项编译程序，就不能开启这个 RTC1，这里选择“**默认值**”选项，如图 2.29 所示。

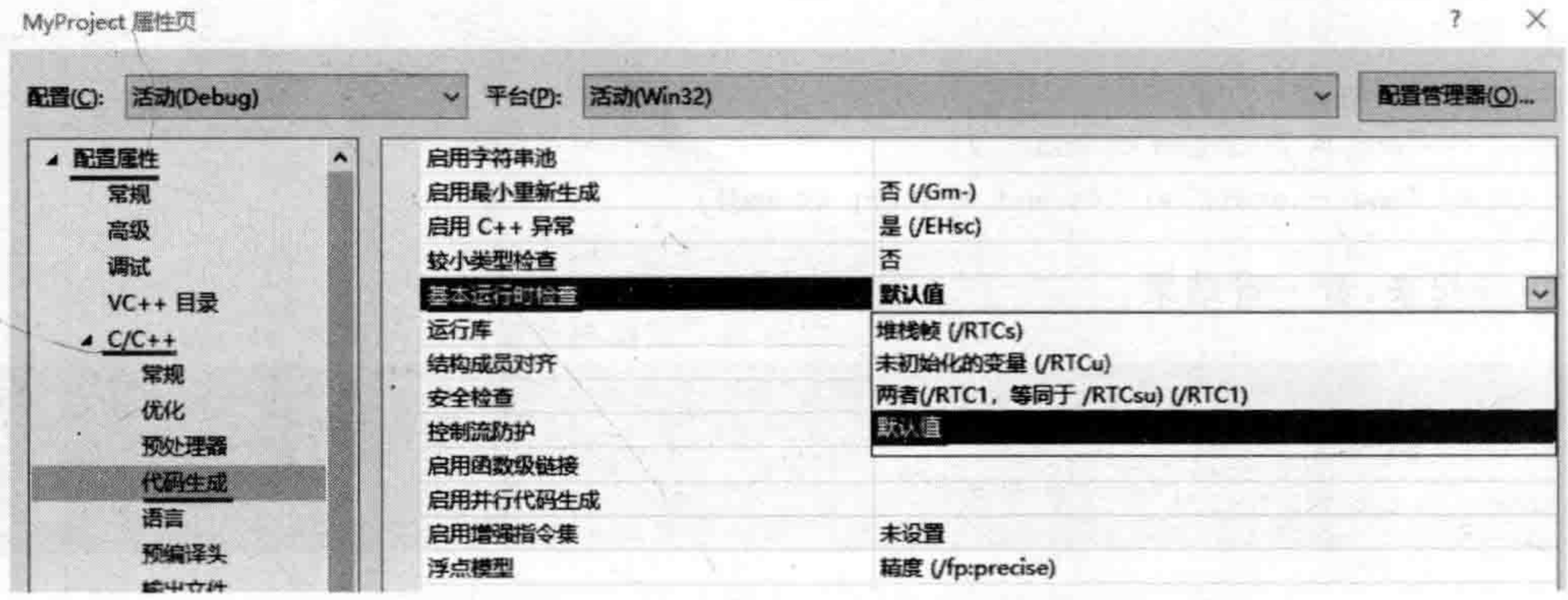

图 2.29 关闭 Visual Studio 2019 中基本运行时检查选项

执行起来，看一看结果：

```
调用了 CTempValue 类的构造函数!
val1 = 10
val2 = 20
调用了 CTempValue 类的构造函数!
val1 = 0
val2 = 0
调用了 CTempValue 类的析构函数!
调用了 CTempValue 类的析构函数!
```

可以看到，优化选项发挥了作用，实际结果少调用了一次拷贝构造函数和一次析构函数。现在，**把优化选项还原回去**以免影响后续的一些测试工作。

针对 Double 函数代码在改造（优化）代码前后的执行效率对比，如果调用 Double 函数的次数少，可能感觉不到效率的提升，但是如果频繁地调用 Double 函数（如调用几百上千万次），那么效率的提升就会比较明显。

为了做适当的演示，先将 Double 函数的代码保持在最初的未优化时的版本（Double 函

数的第一个版本）。

在 MyProject.cpp 开头位置增加对 time.h 头文件的包含：

```
#include <time.h>
```

另外，将 CTempValue 类中构造函数、拷贝构造函数、析构函数中的 cout 相关的输出语句全部注释掉以提高演示效率（这些输出语句很占用运行时间）。

这里做的演示是调用 100 万次 Double 函数，比较一下 Double 函数中的代码在优化前和优化后的运行效率差别。注释掉以往的 main 主函数中的代码，增加如下代码：

```
CTempValue ts1(10, 20);
clock_t start, end;
start = clock();                          //程序开始到本行执行时所用的毫秒数
cout << "start = " << start << endl;
for (int i = 0; i < 1000000; i++)
{
    Double(ts1);
}
end = clock();
cout << "end = " << end << endl;
cout << "end - start = " << end - start << endl;
```

执行起来，看一看结果：

```
start = 0
end = 163
end - start = 163
```

多次执行，end 与 start 的时间差在 140～210。

继续，将 Double 函数的版本调整为优化之后的版本（Double 函数的第二个版本）。

再次执行起来，看一看结果：

多次执行，end 与 start 的时间差在 80～110。可以看到，修改后的 Double 函数，会使执行效率有一定的提升。

优化这种事不太好把握，针对**编译器的优化**（非程序员改动代码进行优化），要说明几点：

（1）编译器是否真优化了。如果不做各种测试，可能也不知道编译器是否真做了某些优化，而且往往编译器优化的能力更强悍（比程序员改动代码的优化效果更明显）。

（2）如果程序员写的函数内容比较复杂，可能编译器会选择索性不优化了。

（3）不要过度优化。有的时候用特别高优化度的优化选项也许会影响程序的功能甚至使程序运行出现错误，在应用优化选项前要搞清楚优化选项可能带来的弊端。

（4）另外，优化可能使程序员犯错误。例如刚才在 Linux 中演示时，如果程序员不使用-fno-elide-constructors 选项，那么程序执行时就不调用拷贝构造函数，如果程序员的本意是期望调用拷贝构造函数做一点什么事情，那此时程序员期望的事情就没有发生（期望落空）。

2.8 程序优化续、拷贝构造函数续与深浅拷贝

2.8.1 程序优化续

借用 2.6 节中的范例进行演示，并对代码进行了一些扩充。完整的代码如下：

```
class X
{
public:
    int m_i;
    X(const X& tmpx)
    {
        m_i = tmpx.m_i;
        cout << "X 类的拷贝构造函数被调用" << endl;
    }
    X()
    {
        m_i = 0;
        cout << "X 类的构造函数被调用" << endl;
    }
    ~X()
    {
        cout << "X 类的析构函数被调用" << endl;
    }
    X(int value) :m_i(value)          //类型转换构造函数(带有一个形参的拷贝构造函数)
    {
        cout << "X 类的 X(int)构造函数被调用" << endl;
    }
};
```

在 main 主函数中加入如下代码(注意每行语句的写法)：

```
cout << " ---- begin ---- " << endl;
X x10(1000);
cout << " -------- " << endl;
X x11 = 1000;
cout << " -------- " << endl;
X x12 = X(1000);
cout << " -------- " << endl;
X x13 = (X)1000;
cout << " ---- end ---- " << endl;
```

执行起来，看一看结果：

```
---- begin ----
X 类的 X(int)构造函数被调用
--------
X 类的 X(int)构造函数被调用
--------
```

```
X类的 X(int)构造函数被调用
--------
X类的 X(int)构造函数被调用
---- end ----
X类的析构函数被调用
X类的析构函数被调用
X类的析构函数被调用
X类的析构函数被调用
```

在笔者 Visual Studio 2019 的开发环境中，看起来 main 主函数中每行创建对象的代码调用的都是构造函数。但是，凭借编程经验，感觉这 4 行创建对象的代码执行结果应该不同（如 X x12 = X(1000)；代码行看起来更应该调用拷贝构造函数）。是否是 Visual Studio 2019 编译器内部还做了优化？笔者找了一下优化选项，似乎都关闭了，没有开优化选项，是没找到还是压根没有？读者也可以尝试着探索一下。

带着这个怀疑，笔者来到 Linux 操作系统下进行测试，希望发现一些新情况。为了防止 Linux 下的 g++编译器对程序进行优化，特加入了-fno-elide-constructors 选项，示意编译器不做任何优化：

```
g++ -fno-elide-constructors MyProject.cpp -o MyProject
```

程序代码在 Linux 上完全不变，看一看在 Linux 操作系统上经过 gcc 编译器编译连接后的执行结果：

```
---- begin ----
X类的 X(int)构造函数被调用
--------
X类的 X(int)构造函数被调用
X类的拷贝构造函数被调用
X类的析构函数被调用
--------
X类的 X(int)构造函数被调用
X类的拷贝构造函数被调用
X类的析构函数被调用
--------
X类的 X(int)构造函数被调用
X类的拷贝构造函数被调用
X类的析构函数被调用
---- end ----
X类的析构函数被调用
X类的析构函数被调用
X类的析构函数被调用
X类的析构函数被调用
```

可以注意到，与在 Visual Studio 2019 中执行的结果相比，出现了差异和不同。后三个创建对象的代码行都是先构造了一个临时对象，然后把临时对象复制给了等号左侧所定义的对象（所以调用了拷贝构造函数），然后又把临时对象给析构掉了。

在编译器不参与优化的情况下，只有第一行创建对象的代码似乎是最有效率的。因为

在编译器的视角看来，第一行相当于：

```
X x10;                                    //编译器视角是不调用构造函数的
x10.X::X(1000);
```

后面几行创建对象的代码，感觉就不太好了，在编译器的视角看来可能大概就是这样了：

```
X _tmp0;                                  //编译器生成的临时对象
_tmp0.X::X(1000);                         //带一个参数的构造函数被调用
X x12;
x12.X::X(_tmp0);                          //拷贝构造函数被调用
_tmp0.X::~X();                            //调用析构
```

而且 X x11 = 1000;代码行的写法和 X x12 = X(1000);或者 X x13 = (X)1000;代码行的写法又不一样。因为 X x11 = 1000;代码行涉及隐式类型转换(涉及先把 1000 这个数字转换成一个临时对象)，这种转换成功靠的是 X(int value) :m_i(value){…}这种类型转换构造函数。

要验证上面隐式类型转换这个说法也很容易，如果在类型转换构造函数前面增加 explicit，如下：

```
explicit X(int value) :m_i(value){…}
```

那么代码行 X x11 = 1000;就会报错，无法编译通过，这表示 1000 这个数字无法转换成 X 类型对象。

总结：当编译器面临用一个类对象(哪怕是临时对象)作为另一个类对象初值的情形时，很多编译器的表现各不相同，但各个厂商的编译器都是为更高的执行效率进行努力的。所以，上面的代码写法，程序员没有办法确定所使用的编译器一定会调用拷贝构造函数。

2.8.2 拷贝构造续

那么，拷贝构造函数必须有吗？结论是不一定，视情况而定。如果只是一些简单数据类型的成员变量，如 int、double 类型，你会发现根本不需要什么拷贝构造函数，编译器内部本身就支持这种成员变量的所谓 bitwise(按位)复制。

改造一下代码，注释掉 X 类中的拷贝构造函数。在 main 主函数中，注释掉原有代码，增加如下代码：

```
X x0;
x0.m_i = 150;
X x1(x0);
cout << x1.m_i << endl;
```

可以发现，即使类 X 没有拷贝构造函数，x1.m_i 的值仍旧变成了 150，这就是编译器本身支持的针对成员变量的 bitwise 复制。读者也可以在 g++编译器上尝试，效果应该是一样的。

那么，什么时候程序员需要书写自己的拷贝构造函数呢？就是当处理一些很复杂的成员变量类型的时候，最简单的例子就是**指针变量作为类的成员变量**。例如，上面的类 X 有

一个指针类型的成员变量 int * p_m 指向一块内存，那么，用 x0 对象生成 x1 对象的时候，就会导致 x1 里的 p_m 也指向 x0 的 p_m 所指向的内存。那么试想，如果 x0 对象被析构的时候，x0 有责任把自己的 p_m 成员所指向的内存给释放掉，那就会导致 x1 的 p_m 指向的内存非法。对于这种情况，程序员就有必要书写自己的拷贝构造函数，然后在其中为 p_m 开辟内存空间，把另外一个对象 p_m 所指向的内存中的内容复制过来等诸如此类操作。这就是读者常常听说的“深拷贝，浅拷贝”的话题。

现在举一个实际例子，将类 X 拷贝构造函数的注释取消，并且，类 X 的拷贝构造函数中只保持有如下一行代码：

```
X(const X& tmpx)
{
    cout << "X 类的拷贝构造函数被调用" << endl;
}
```

main 主函数中代码不变，将程序执行起来，看一看结果：

```
X 类的构造函数被调用
X 类的拷贝构造函数被调用
-858993460
X 类的析构函数被调用
X 类的析构函数被调用
```

从结果中可以看到，打印出来的 x1.m_i 值已经不对了。

本来，不加拷贝构造函数时 x1.m_i 值是对的，但是加了之后，该值反而不对了。看起来就是因为程序员**加了拷贝构造函数导致编译器原本内置的 bitwise（按位）复制能力失效**。所以结论就是：**程序员往类中增加了拷贝构造函数后，就要对类中各个成员变量的初始化负责了**。

2.8.3 深浅拷贝

笔者在这里谈一下深浅拷贝问题，因为笔者不敢确定所有读者都听过这个话题，一旦没有听过，以后遇到类似问题掉到坑里，那么笔者也会非常自责。

再次把类 X 的拷贝构造函数注释掉，并且向类 X 中增加若干代码。请读者留意，完整的 X 类当前如下：

```
class X
{
public:
    int m_i;
    int * p_mi;                          //整型指针，其他类型指针都可以，道理相同
    X()
    {
        m_i = 0;
        p_mi = new int(100);
        cout << "X 类的构造函数被调用" << endl;
    }
    ~X()
```

```
    {
        delete p_mi;
        cout << "X 类的析构函数被调用" << endl;
    }
    X(int value) :m_i(value)          //类型转换构造函数(带有一个形参的拷贝构造函数)
    {
        p_mi = new int(100);
        cout << "X 类的 X(int)构造函数被调用" << endl;
    }
};
```

main 主函数中，代码不变。

将程序执行起来，发现执行出现异常。为什么？要分析一下。将断点设置到 main 主函数中的 cout << x1. m_i << endl;代码行，按快捷键 F5 运行程序。因为类 X 没有拷贝构造函数，编译器执行了所谓的 bitwise(按位)复制。所以，执行完代码行 X x1(x0);，就导致 x0 和 x1 两个对象的成员变量值都一样了。这里重点观察 p_mi 这个指针类型成员变量的值，如图 2.30 所示。

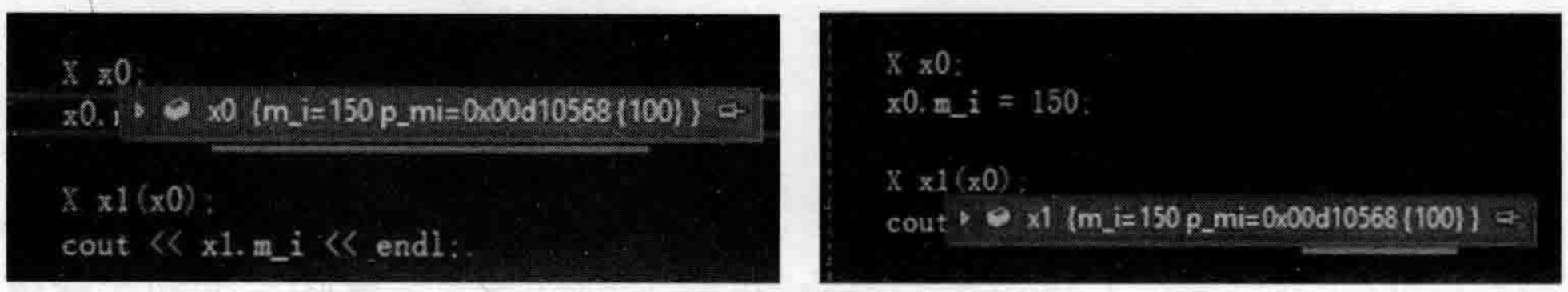

图 2.30　两个对象的指针类型成员变量居然指向同一块内存 0x00d10568

不难发现，x0 和 x1 对象的成员变量 p_mi 指向的是同一块内存。这肯定是不行的，这块内存是对象 x0 构造时开辟的，如果 x0 对象被释放了，那么 x0 对象的 p_mi 所指向的内存会在类 X 的析构函数中执行 delete p_mi;代码行被释放，这就等价于 x1 的 p_mi 指针失效了。当释放 x1 对象的时候，类 X 的析构函数再次执行 delete p_mi;代码行把已经被释放的内存再次释放，编译器就会报异常，所以，x1 对象的 p_mi 也**必须指向 x1 对象自己分配的内存**。因为 x1 对象是通过类 X 的拷贝构造函数并借助对象 x0 复制构造出来的，所以这时程序员必须要书写自己的拷贝构造函数，来额外处理 p_mi 的内存分配问题。

对于类定义中有指针成员变量的情况，只复制指针值(上面的代码就是这种情形)，指针所指向的地址编译器并没有额外做处理，这就叫**浅拷贝**，是一种很坑人的复制。

取消类 X 中拷贝构造函数的注释并详细修改其中的代码，请读者注意观察：

```
X(const X& tmpx)
{
    p_mi = new int(100);                    //为指针类型成员变量单独分配内存
    memcpy(p_mi, tmpx.p_mi, sizeof(int));
    m_i = tmpx.m_i;
    cout << "X 类的拷贝构造函数被调用" << endl;
}
```

上面这段代码，在拷贝构造函数中，为指针类型成员变量 p_mi 创建自己的内存，并复制形参 tmpx 中的成员变量 p_mi 的值到该内存中，这种行为就叫**深拷贝**。当释放这个被复

制对象(x1)的时候，X 类的析构函数中执行的代码行 delete p_mi；释放的就是在 X 类拷贝构造函数中分配的内存。

这种深拷贝实现了每个不同的 X 类对象的指针类型成员变量 p_mi 都指向一块完全不同的内存，释放 X 类对象回收 p_mi 所指内存时这些对象之间完全没有瓜葛，不会彼此影响。

2.9 成员初始化列表

2.9.1 何时必须用成员初始化列表

看看如下范例。在 MyProject.cpp 的上面，增加如下类 A 的定义代码：

```
class A
{
public:
    int m_x;
    int m_y;
    A() :m_x(0), m_y(0)
    {
    }
};
```

上面代码中，m_x(0)、m_y(0)就是初始化列表。当然，在这种情形下，可以不使用初始化列表，像下面这样书写类 A 的构造函数也可以：

```
A()
{
    m_x = 0;
    m_y = 0;
}
```

上面这种写法，抛开效率等说法，单纯从要实现的功能上来讲，和使用初始化列表是一样的。

这里要思考的第一个问题是，在什么场合下必须要用初始化列表来初始化成员？

(1) 如果成员变量是引用类型

在类 A 中增加一个引用类型的成员变量：

```
public:
    int &m_yy;
```

那么，在构造类 A 对象时，这个引用就要绑定到一个 int 类型变量上去。这种情况下，一般来说类 A 的构造函数都会带有一个形参。修改一下类 A 的构造函数，修改后的代码如下：

```
A(int &tmpvalue):m_yy(tmpvalue)
{
    //m_yy = tmpvalue;                    //错误,不可以,必须在初始化列表中进行绑定
    m_x = 0;
```

```
    m_y = 0;
    m_yy = 160;                          //就等于修改外界的 abc 变量值
}
```

在 main 主函数中，加入如下代码：

```
int abc = 1;
A a(abc);
```

执行起来，结果正常。显然上面的代码虽然语法正确，但比较多余，因为引入 m_yy 成员变量看起来并没有什么实际意义。笔者引入该成员变量的主要目的只是举例说明，如果成员变量是一个引用类型，则必须在初始化列表中进行初始化。

至于引用类型成员变量到底有什么实际用途，读者可以在今后的学习和实践中多阅读别人的代码，进一步体会，这里笔者就不多说。

(2) 如果成员变量是 const 类型

在类 A 中增加一个 const 类型的成员变量：

```
public:
    const int m_myc;
```

这种情况同样要在初始化列表中初始化 m_myc 成员变量：

```
A(int &tmpvalue):m_yy(tmpvalue), m_myc(tmpvalue){…}
```

(3) 如果该类继承自一个基类，并且基类中有构造函数，这个构造函数里还有参数

在 MyProject.cpp 的上面，增加如下类 Base 的定义代码：

```
class Base
{
public:
    int ba;
    int bb;
    Base(int tmpa, int tmpb)             //构造函数
    {
    }
};
```

修改类 A，让其继承自 Base 类，注意类 A 构造函数中初始化列表的改动。完整的类 A 代码如下：

```
class A :public Base
{
public:
    int m_x;
    int m_y;
    int &m_yy;
    const int m_myc;
    A(int& tmpvalue) :m_yy(tmpvalue), m_myc(tmpvalue), Base(tmpvalue, tmpvalue)
    {
        m_x = 0;
        m_y = 0;
```

```
            m_yy = 160;
        }
};
```

(4) 如果类中成员变量的类型是某个类类型，而这个类类型所指类的构造函数带有参数

在 MyProject.cpp 的上面，增加如下类 CSub 的定义代码：

```
class CSub
{
public:
    CSub(int tmpvalue)
    {

    }
};
```

修改类 A，增加一个 CSub 类型的成员变量，并在初始化列表中对其进行初始化。完整的类 A 代码如下：

```
class A :public Base
{
public:
    int m_x;
    int m_y;
    int &m_yy;
    const int m_myc;
    CSub cmysub;

    A(int& tmpvalue) :m_yy(tmpvalue), m_myc(tmpvalue), Base(tmpvalue, tmpvalue), cmysub
(tmpvalue)
    {
        m_x = 0;
        m_y = 0;
        m_yy = 160;
    }
};
```

2.9.2 使用初始化列表的优势

除了必须使用初始化列表的场合，程序员使用初始化列表还有什么目的？其实就是一句话——**提高程序运行效率**。

这里使用两个类进行演示。在 MyProject.cpp 的上面，增加如下类 X 的定义代码：

```
class X
{
public:
    int m_i;
    X(int value = 0) :m_i(value)      //类型转换构造函数
    {
```

```
        printf("this = %p", this);
        cout << "X类的X(int)构造函数被调用" << endl;
    }
    X(const X& tmpv)
    {
        printf("this = %p", this);
        cout << "X类的拷贝构造函数被调用" << endl;
    }
    X& operator = (const X& tmpTime)
    {
        printf("this = %p", this);
        cout << "X类的拷贝赋值运算符被调用" << endl;
        return *this;
    }
    ~X()
    {
        printf("this = %p", this);
        cout << "X类的析构函数被调用" << endl;
    }
};
```

继续增加如下类 XA 的定义代码：

```
class XA
{
public:
    X xobj;
    int m_test;
    XA(int tmpvalue)
    {
        xobj = 1000;
        m_test = 500;
    }
};
```

在 main 主函数中，注释掉以往的代码，增加如下代码：

```
XA myaobj(1000);
```

从上面的代码可以注意到，类 XA 中成员变量 xobj 的初始化放在了类 XA 的构造函数里。

执行起来，看一看结果：

```
this = 00B3F9B4X类的X(int)构造函数被调用
this = 00B3F7F8X类的X(int)构造函数被调用
this = 00B3F9B4X类的拷贝赋值运算符被调用
this = 00B3F7F8X类的析构函数被调用
this = 00B3F9B4X类的析构函数被调用
```

可以通过设置断点并跟踪调试观察。结果中最上面一行“X(int)构造函数被调用”是执行代码行 XA(int tmpvalue)时产生的。**说明在执行类 XA 构造函数的函数体之前就构造了**

xobj 这个对象，所以这次构造占用了一次调用类 X 构造函数的机会。

站在编译器的角度相当于执行了下面两行代码：

```
X xobj;
xobj.X::X();
```

继续跟踪调试，进入到 XA 构造函数的函数体。不难发现，仅仅执行函数体中 xobj = 1000；这行代码就会产生 3 条输出结果(下面 3 条结果取自上面 5 条结果中的第 2～4 条)：

```
this = 00B3F7F8X 类的 X(int)构造函数被调用
this = 00B3F9B4X 类的拷贝赋值运算符被调用
this = 00B3F7F8X 类的析构函数被调用
```

根据这三条输出结果，分析分析。站在编译器的视角来看，针对 xobj = 1000；代码行，很可能生成出下面这种代码：

```
X tmpobj;                                    //生成一个临时对象
tmpobj.X::X(1000);                           //临时对象调用构造函数
xobj.X::operator = (tempobj);                //调用 xobj 的拷贝赋值运算符
tempobj.X::～X();                            //调用析构函数摧毁临时对象
```

但是，如果把 xobj 对象的初始化放在 XA 构造函数的初始化列表中呢？

```
XA(int tmpvalue) : xobj(1000)
{
    m_test = 500;
}
```

执行起来，看一看结果：

```
this = 007EF8C8X 类的 X(int)构造函数被调用
this = 007EF8C8X 类的析构函数被调用
```

原来输出的结果是 5 行，现在只剩 2 行了。省掉了一次构造函数、一次拷贝赋值运算符、一次析构函数。

此时，站在编译器的视角，看看编译器是如何初始化 xobj 的：

```
X xobj;
xobj.X::X(1000);                             //直接用 1000 调用 X 类的构造函数
```

其实，初始化列表中的代码可以理解成(相当于)往构造函数中插入代码，也就是说，可以把这些初始化列表中的代码看成是构造函数的函数体中代码的一部分。

通过上面的讲解可以看到，通常，诸如 **xobj 这种类类型成员变量的初始化，放在构造函数的初始化列表中进行初始化比放在构造函数函数体中初始化效率更高**。但是，如果只是一个简单类型的成员变量(如一个 int 类型的成员变量)，其实放在初始化列表里初始化与放在构造函数函数体里初始化没什么差别。

将来读者面试的时候，可能有些考官很在意初始化成员变量时将它们放在构造函数初始化列表里还是放在构造函数函数体里，请**尽量往初始化列表里放**，这样可能会更受到考官

的认可，就算是简单类型的成员变量放在初始化列表中并不提高程序执行性能，也显得更专业。

2.9.3 初始化列表的细节探究

其实初始化列表的语法看起来还是挺奇怪的，如：xobj(1000)，m_test(500)，有冒号，也有逗号，看起来像函数调用一样。

有几点说明：

(1) 初始化列表中的代码可以看作被编译器安插在构造函数中的。

(2) 初始化列表中的代码是在构造函数的函数体代码执行之前被执行。

(3) 初始化列表中成员变量的初始化顺序看的是这些成员变量在类中定义的顺序，而不是看在初始化列表中出现的顺序：

假如在类 XA 中，定义了三个成员变量：

```
public:
  X xobj;
  int m_test;
  int m_test2;                          //定义在后面,则后得到值
```

而 XA 类的构造函数中的初始化列表如果像下面这样写：

```
XA(int tmpvalue):xobj(1000),m_test2(500),m_test(m_test2){…}
```

则是错误的，因为 m_test 在类 XA 中定义在前面，会先得到值，根据构造函数初始化列表，m_test 的值取决于 m_test2，但 m_test2 此时没值，所以 m_test 值变得不可预料(不确定)了。读者可以设置断点并跟踪调试来查看，可以查看反汇编代码，如图 2.31 所示。

```
class XA
{
public:
    X  xobj;
    int m_test;
    int m_test2;  //定义在后面，则后得到值
    //XA(int tmpvalue)
    //XA(int tmpvalue) : xobj(1000)
    XA(int tmpvalue) :xobj(1000), m_test2(500), m_test(m_test2)
00D91A0D  push        3E8h
00D91A12  mov         ecx,dword ptr [this]
00D91A15  call        X::X (0D91519h)
00D91A1A  mov         eax,dword ptr [this]
00D91A1D  mov         ecx,dword ptr [this]
00D91A20  mov         edx,dword ptr [ecx+8]
00D91A23  mov         dword ptr [eax+4],edx
00D91A26  mov         eax,dword ptr [this]
00D91A29  mov         dword ptr [eax+8],1F4h
    {
    //xobj = 1000;
    m_test = 500;
00D91A30  mov         eax,dword ptr [this]
00D91A33  mov         dword ptr [eax+4],1F4h
    }
```

图 2.31 成员变量得到值的顺序是根据它们在类中定义的顺序来的

在图 2.31 中，给成员变量值是按照它们在类中定义的顺序来的，先给 eax+4(m_test)值，再给 eax+8(m_test2)值。

读者千万注意上面这种问题，不要犯这种错误，因为这种错误不好检查，不容易发现。编译器本身也不会针对这类错误出现警告之类的提示(笔者测试 Visual Studio 2019 和

g++都不提示这类问题）。

因此，笔者不建议在初始化列表中进行两个成员变量之间的初始化。例如，下面这种用成员变量 m_test2 来初始化成员变量 m_test。

```
A(int tmpvalue):xobj(1000),m_test2(500),m_test(m_test2){…}
```

哪怕是一个成员变量放在初始化列表中初始化，另一个成员变量放在构造函数的函数体内初始化，这样的执行顺序看起来也很清晰（初始化列表中的代码先执行）：

```
A(int tmpvalue):xobj(1000),m_test2(500)
{
    m_test = m_test2;
}
```

第3章

虚 函 数

本章会涉及一个最重要的话题：虚函数。这里将详细阐述虚函数表指针、虚函数表等概念与虚函数的工作原理。在开始本章的学习之前，读者应该确保对虚函数的概念和用法有比较好的掌握。

掌握虚函数表指针、虚函数表两大话题，是程序员自身开发素养和开发能力的体现，也是C++面试中最常考的两个话题，读者不可不会，否则非常容易丢掉工作机会，尤其是一些高级开发职位的工作机会。

3.1 虚函数表指针位置分析

读者已经知道，一个类中如果有虚函数，则针对这个类会产生一个虚函数表。生成这个类对象的时候，该对象里就有一个指针(虚函数表指针)用来指向这个虚函数表的起始地址。这个指针可以看成类中的一个成员变量，也占一定的字节数(一般是4个或者8个字节)。既然是成员变量，那么它在对象内存中就会有一个位置，它有可能位于对象内存的开头，有可能位于对象内存的末尾，具体在什么位置取决于编译器。现在写一个范例，范例的目的是让读者感知一下虚函数表和虚函数表指针的存在。

看看如下范例。在MyProject.cpp的上面，增加如下类A的定义代码：

```
class A
{
public:
    int i;                          //一个成员变量,4字节
    virtual void testfunc() {}      //虚函数
};
```

在main主函数中，加入如下代码：

```
A a;
int ilen = sizeof(a);  //8: i占4字节,虚函数表指针占4字节(皆在x86平台下)
```

现在可以确定一件事情，a对象里有一个虚函数表指针(vptr)，还有一个成员变量i，成员变量是跟着对象走的，也就是说是属于该对象的。可以以图2.5做参照。

在main主函数中继续增加如下的代码，用以感知一下虚函数表指针的存在。注意观察代码中的注释：

```
char * p1 = reinterpret_cast<char *>(&a); //0x006ff730 类型转换,这属于硬转,a 是对象
                                          //首地址
char * p2 = reinterpret_cast<char *>(&(a.i)); //0x006ff734
if (p1 == p2) //说明 a.i 和 a 位置相同,则成员变量 i 在 a 对象内存的上面位置,那么虚函数表
              //指针在下面位置
{
    cout << "虚函数表指针位于对象内存的末尾" << endl;
}
else
{
    cout << "虚函数表指针位于对象内存的开头" << endl; //本条件会成立
}
```

执行起来,看一看结果:

```
虚函数表指针位于对象内存的开头
```

上面的代码比较简单,不难看到,经过测试,**虚函数表指针位于对象内存的起始(上面开头)位置**。在 Linux 操作系统下编译并运行程序,得到的结果也相同(虚函数表指针也位于对象内存的起始位置)。对象 a 的内存结构示意图如图 3.1 所示。

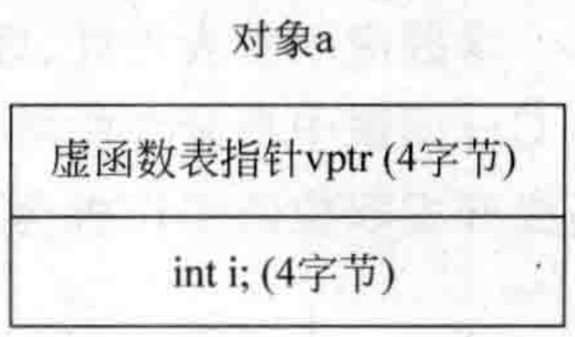

图 3.1 对象 a 的内存结构示意图

通过上面这个范例,感知到了虚函数表指针的存在,后续就要通过这个虚函数表指针来尝试调用一下虚函数。

3.2 继承关系作用下虚函数的手工调用

通过上一节感知了一下虚函数表指针的存在,本节将把问题深入一点,创建一个父类,创建一个子类,然后,尝试着通过虚函数表指针来调用一下虚函数。

看看如下范例,注意程序的写法,因为研究 C++对象模型知识的时候程序的写法往往都是比较独特和奇怪的。在 MyProject.cpp 的上面,增加如下类 Base 和类 Derive 的定义代码:

```
class Base
{
public:
    virtual void f() { cout << "Base::f()" << endl; }
    virtual void g() { cout << "Base::g()" << endl; }
    virtual void h() { cout << "Base::h()" << endl; }
};
class Derive : public Base {
public:
    void g() { cout << "Derive::g()" << endl; }
};
```

在 main 主函数中,加入如下代码(请读者仔细分析这些代码并密切注意其中的注释):

```
cout << sizeof(Base) << endl;      //4 字节,说明虚函数表的指针在这里的 x86 平台是 4 字节的
(vptr),g++编译器是 8 字节
cout << sizeof(Derive) << endl; //4 字节,和 Base 类一个道理
```

```
Derive * d = new Derive();//派生类指针,其实用基类指针指向派生类也一样 Base * d = new Derive();
long * pvptr = (long * )d; //指向对象 d 的指针转成 long * 型。大家注意,目前 d 对象里只有虚函数表指针
long * vptr = (long * )( * pvptr); //( * pvptr)表示 pvptr 指向的对象,也就是 Derive 对象本身。这个对象 4 字节,这个 4 字节是虚函数表地址。例如, * pvptr 可能等于 15440724(0x00eb9b54),这个其实就是虚函数表地址。把一个虚函数表地址用(long * )一转,则本行的意思就是让 vptr 代表 Derive 对象虚函数表指针,用于指向类 Derive 的虚函数表
```

上面几行代码请读者好好理解,在这里,笔者绘制图 3.2,帮助理解上面的代码。

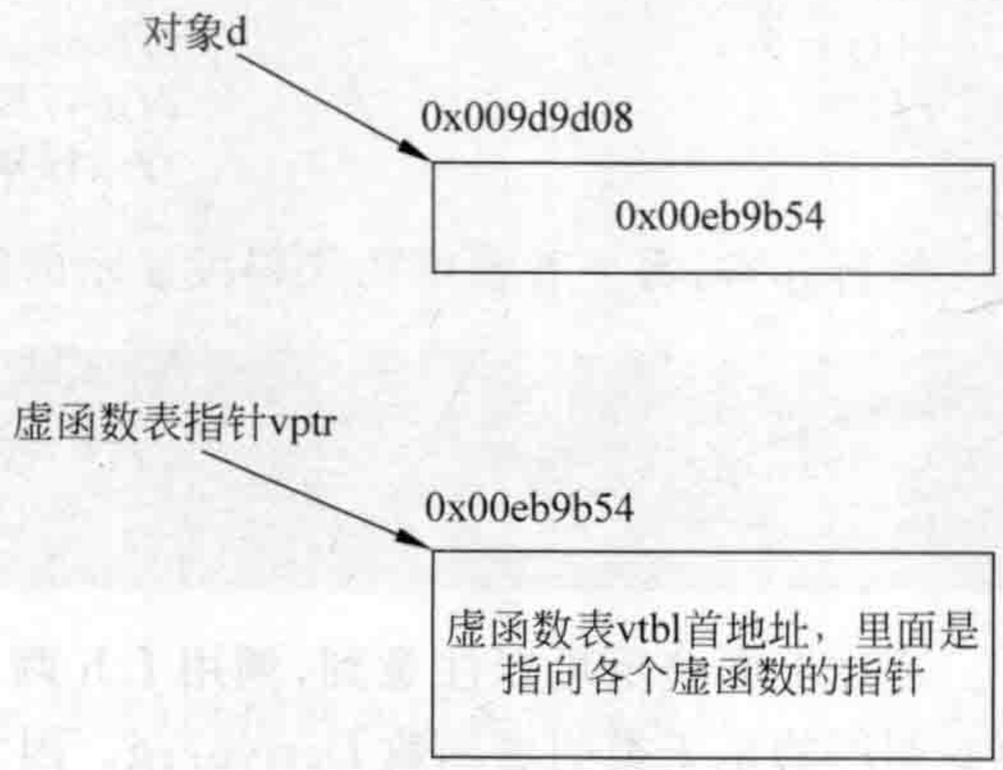

图 3.2 通过代码获取类 Derive 虚函数表首地址,并让 vptr 指针变量指向该地址

读者可以参考图 2.5,因为这个图比较直观。现在可以估计一下,类 Derive 有三个虚函数(两个继承自父类,分别是 f 和 g,另一个覆盖了父类的同名函数 h),所以,预计虚函数表里会有三项,这里输出一下该虚函数表里指向各个虚函数的指针的值。这里为了观察到更多信息,打印出五项来。在 main 主函数中,继续增加如下代码:

```
for (int i = 0; i <= 4; i++) //打印五项出来
{
    printf("vptr[ %d] = 0x: %p\n", i, vptr[i]); //这样打印可能会打印到非法内存,但好在程序不崩溃,使我们能看到结果
}
```

执行起来,看一看结果:

```
4
4
vptr[0] = 0x:00811569
vptr[1] = 0x:0081157D
vptr[2] = 0x:00811573
vptr[3] = 0x:0081A7D4
vptr[4] = 0x:00811230
```

可以注意到,vptr[0]、vptr [1]、vptr [2]这前三项指向的应该是三个虚函数首地址。而后面的 vptr[3]和 vptr[4]应该就不是有效内容了(而且内存地址看起来与前三项离得也比较远),这里可以不理会这两项。有必要提醒一下,不同的编译器 vptr[3]的结果可能不同,如在 Visual Studio 2017 中,vptr[3]的内容可能是 0x000000000。

既然拿到了 vptr[0]、vptr [1]、vptr [2]这三个虚函数的首地址,那么就尝试调用一下这三个虚函数。在本范例中这些虚函数的调用格式都比较统一,因为它们没有形参也没有返回值,这就给写测试代码带来了便利(不用传递多余的东西进去就能达到调用的效果)。现在就开始写代码来调用这三个虚函数看看效果。在 main 主函数中,继续增加如下代码:

```
typedef void( * Func)(void); //typedef 后面是定义一个函数指针,加上 typedef 就代表 Func 是一个函数指针类型,也就是它能当类型用
//Func 当类型用后就可以像下面这样
```

```
Func f = (Func)vptr[0]; //f、g、h就是函数指针变量,vptr[0]指向第一个虚函数{project4.exe!
Base::f(void)}
Func g = (Func)vptr[1];          //vptr[1]指向第二个虚函数{project4.exe!Derive::g(void)}
Func h = (Func)vptr[2];          //vptr[2]指向第三个虚函数{project4.exe!Base::h(void)}
Func i = (Func)vptr[3];          //vptr[3]
Func j = (Func)vptr[4];          //vptr[4]
f();                             //Base::f
g();                             //Derive::g  子类覆盖父类的虚函数
h();                             //Base::h
//i();                           //运行异常
//j();                           //运行异常
```

执行起来,看一看新增的代码段显示的结果:

```
Base::f()
Derive::g()
Base::h()
```

通过上面演示可以注意到,调用 f、h 调用的是基类的虚函数 Base::f 和 Base::h,而调用 g 调用的是子类的虚函数 Derive::g。因为子类能够覆盖父类的虚函数,所以上述结果是正确的。

上面演示了生成一个子类对象,并通过该对象寻找到子类 Derive 的虚函数表,根据这个虚函数表调用对应的虚函数。

下面继续在 main 主函数中增加代码,创建一个父类(Base)对象,通过这个父类对象的虚函数表指针找到父类的虚函数表,并调用该虚函数表里面的虚函数:

```
Base * dpar = new Base();
long * pvptrpar = (long * )dpar;
long * vptrpar = (long * )( * pvptrpar);
for (int i = 0; i <= 4; i++)
{
    printf("vptr_Base [ %d] = 0x: %p\n", i, vptrpar[i]); //这样打印可能会打印到非法内存,
但好在程序不崩溃
}
Func fpar = (Func)vptrpar[0];    //project4.exe!Base::f(void)}
Func gpar = (Func)vptrpar[1];    //project4.exe!Base::g(void)}
Func hpar = (Func)vptrpar[2];    //project4.exe!Base::h(void)}
Func ipar = (Func)vptrpar[3];
Func jpar = (Func)vptrpar[4];
fpar();                          //Base::f
gpar();                          //Base::g
hpar();                          //Base::h
//ipar();                        //运行异常
//jpar();                        //运行异常
```

执行起来,看一看整个程序的运行结果:

```
4
4
vptr[0] = 0x:00241569
```

```
vptr[1] = 0x:0024159B
vptr[2] = 0x:00241573
vptr[3] = 0x:0024A7D4
vptr[4] = 0x:00241230
Base::f()
Derive::g()
Base::h()
vptr_Base[0] = 0x:00241569
vptr_Base[1] = 0x:00241596
vptr_Base[2] = 0x:00241573
vptr_Base[3] = 0x:72747076
vptr_Base[4] = 0x:5D64255B
Base::f()
Base::g()
Base::h()
```

观察上面的结果，可以注意到几点，总结一下：

(1) 父类对象调用的虚函数都是父类的。

(2) 注意两个虚函数表(Base 类的和 Derive 类的)所指向的虚函数首地址：下标[0]和下标[2]的相同，都代表 Base::f()和 Base::h()的首地址。而下标[1]不同，子类对象的下标[1]代表的是 Derive::g()的首地址，父类对象的下标[1]代表的是 Base::g()的首地址。

3.3 虚函数表分析

结合上一节的代码，绘制一下父类对象和子类对象内存布局示意图，如图 3.3 所示。

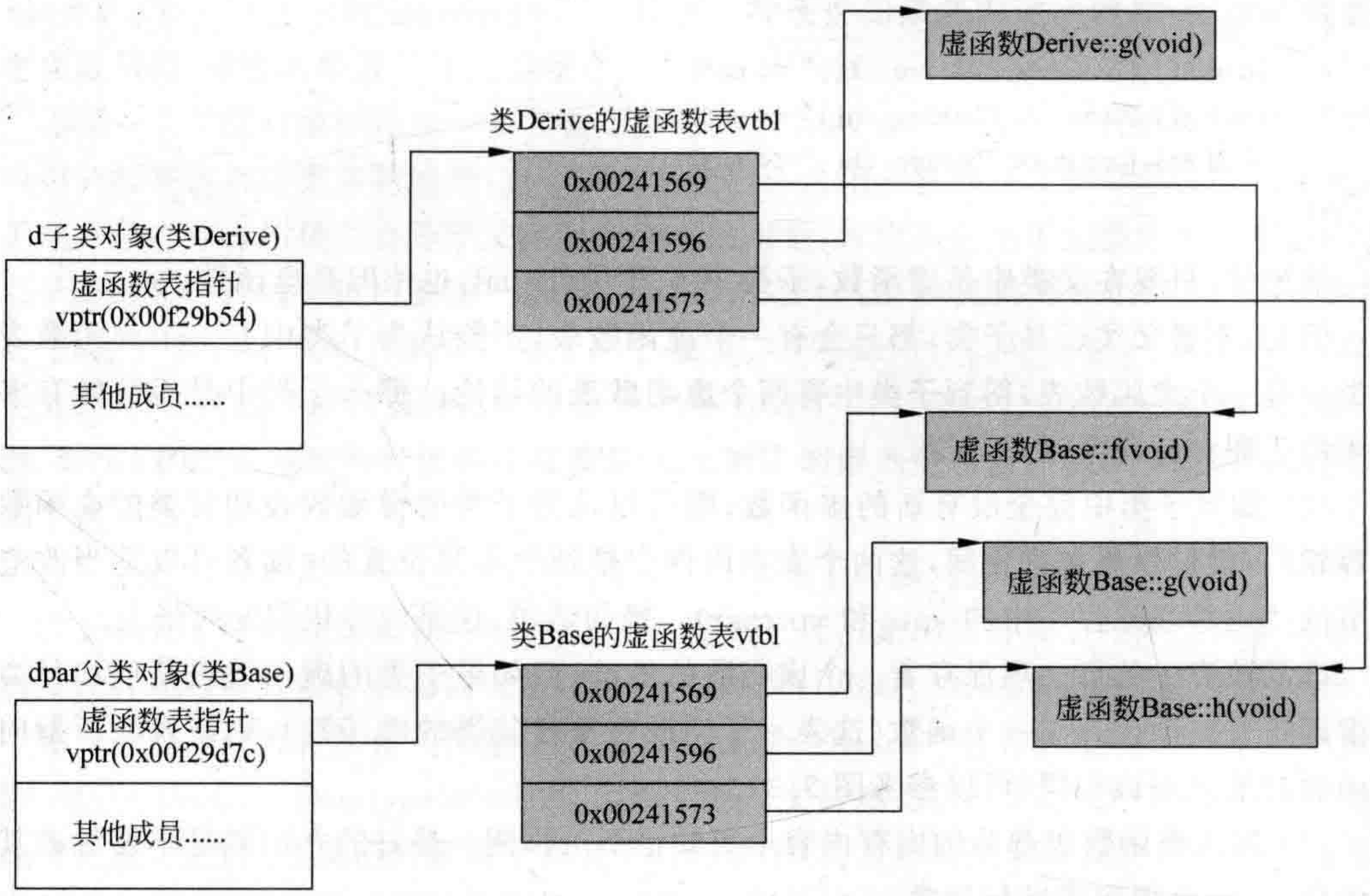

图 3.3 父类对象和子类对象内存布局示意图(虚函数)

结合上一节的具体代码，上面这个图不难理解。这里总结一下：

(1) 包含虚函数的类才会有虚函数表，同属于一个类的对象共享这个虚函数表，但是每个对象都有各自的 vptr(虚函数表指针)，当然该指针所指向的地址(虚函数表首地址)相同。

例如，增加如下这行代码：

```
Derive * d2 = new Derive(); //d2 和 d 虽然都指向不同的地址(代表不同的对象),但是这两个地址中的四个字节的内容相同,因为这四个字节代表的是 Derive 类虚函数表的首地址
```

(2) 父类中有虚函数就等于子类中有虚函数。换句话说，父类中有虚函数表，则子类中肯定也会有虚函数表，即便不覆盖父类的任何虚函数。

读者可以尝试一下，即便注释掉 Derive 类中的虚函数 g，使该类变成一个空类：

```
class Derive : public Base {
public:
    /* void g()
    {
        cout << "Derive::g()" << endl;
    } */
};
```

sizeof(Derive)的值依旧是 4。这个 4 就代表虚函数表指针所占用的内存空间。

也可能有人认为，如果在子类中把所覆盖的父类中的虚函数的 virtual 修饰符去掉，这些函数就不再是虚函数了。例如如下：

```
class Derive :public Base
{
    void f() { cout << "Derive::f()" << endl; }
    void g() { cout << "Derive::g()" << endl; }
    void h() { cout << "Derive::h()" << endl; }
};
```

请记住，只要在父类中是虚函数，子类中不用写 virtual，也依旧是虚函数。

但是，不管父类还是子类，都只会有一个虚函数表，不能认为子类中有一个虚函数表＋父类中有一个虚函数表，**得到子类中有两个虚函数表**的结论。那么子类中是否可能有多个虚函数表呢？后面会逐步介绍。

(3) 如果子类中完全没有新的虚函数，则可以认为子类的虚函数表和父类的虚函数表内容相同，但**仅仅是内容相同**，这两个表在内存中是处于不同位置的(读者可以适当改造上一节的代码并观察程序中的 vptr 和 vptrpar)。换句话说，这是内容相同的两张表。

虚函数表中的每一项保存着一个虚函数的地址，但如果子类的虚函数表某项和父类中的虚函数表某项代表同一个函数(这表示子类没有覆盖父类的虚函数)，则该表项所指向的该函数的地址应该相同(可以参考图 3.3)。

(4) 超出虚函数表部分的内存内容不可知也不可预测。最好的办法就是不要触碰其中的内容，以免出现程序运行异常。

本节的范例接着上一节，但在继续讲解新知识之前，请将 main 主函数中原有的代码全

部注释掉，加入如下新代码：

```
typedef void( * Func)(void);
Derive derive;
long * pvptrderive = (long * )(&derive);
long * vptrderive = (long * )( * pvptrderive);
Func f1 = (Func)vptrderive[0];   //project4.exe!Base::f(void)
Func f2 = (Func)vptrderive[1];   //project4.exe!Derive::g(void)
Func f3 = (Func)vptrderive[2];   //project4.exe!Base::h(void)
```

现在，观察一下 vptrderive 的值，并记录下来，该值就是类 Derive 的虚函数表的地址。然后，再生成一个子类对象，继续在 main 主函数中加入如下代码(注意代码中的注释)：

```
Derive derive2 = derive;          //复制构造
long * pvptrderive2 = (long * )(&derive2);
long * vptrderive2 = (long * )( * pvptrderive2); //所指向的地址和 vptrderive 一样，说明两个对象指向的是同一个虚函数表(子类 Derive 的虚函数表)
```

继续在 main 主函数中加入如下代码：

```
Base base = derive;               //直接用子类对象给父类对象值
long * pvptrbase = (long * )(&base);
long * vptrbase = (long * )( * pvptrbase);
Func fb1 = (Func)vptrbase[0];     //project4.exe!Base::f(void)
Func fb2 = (Func)vptrbase[1];     //project4.exe!Base::g(void)
Func fb3 = (Func)vptrbase[2];     //project4.exe!Base::h(void)
```

观察上面的代码可以看到，base 对象的数据是从 derive 复制过来的。那为什么 base 中的 vptrbase 和 derive 中的 vptrderive 不一样呢？**按理说如果复制过来数据，则虚函数表指针也应该相同**，系统内部做了什么处理呢？

当用一个子类对象初始化一个父类对象时，如 Base base = derive;代码行，因为子类对象内存一般来讲都有更多的成员，所以子类对象内存占用一般都比父类对象内存占用大，最差子类对象内存占用量也会等于父类对象内存占用量，肯定不会小于父类对象内存占用量。所以子类对象(内存)会被切割出一部分(父类子部分)复制到父类对象(内存)中去，所以，Base base = derive;代码行其实是做了两个事情：①生成一个 base 对象；②用 derive 来初始化 base 对象的值。所以编译器在这里为程序员做了一个选择——**derive 初始化 base 对象时，derive 的虚函数表指针值并没有覆盖 base 对象的虚函数表指针值**，编译器帮助程序员做到了这一点。

这里笔者额外要谈一下 OO(面向对象)和 OB(基于对象)的概念。

读者都有 C++编程基础，所以都知道 C++是通过类对象的指针和引用来支持多态的，这种程序设计风格就是常说的面向对象模型——OOM(Object-Oriented Model)。

还有一种程序设计风格叫作基于对象 OB(Object-Based)，这种风格以前叫作抽象数据模型 ADT(Abstract Datatype Model)，这种模型不支持多态，所以在这种模型下的对象运行速度更快，因为函数调用的解析不需要运行时决定(没有多态或者说没有虚函数)，而是编译期间就解析完成，内存空间紧凑程度上更紧凑，因为没有虚函数指针和虚函数表的概念。但是显然 OB 的这种设计灵活性上与 OO 比就要差一些。

C++既支持面向对象程序设计，也支持基于对象程序设计，继承机制和多态性机制就摆在那里。不用，就是基于对象的程序设计；用了，就是面向对象的程序设计。弹性和效率方面如何选择，取决于程序员。

3.4 多重继承虚函数表分析

通过前面的学习已经知道，虚函数表指针跟着对象走，一个虚函数表指针在当前 Visual Studio 2019 演示中占 4 个字节。

本节将展示一下在多重继承时，虚函数表的样子。

看看如下范例。在 MyProject.cpp 的上面，增加如下三个类(两个父类一个子类)的定义代码：

```
//父类 1
class Base1
{
public:
    virtual void f() {
        cout << "Base1::f()" << endl;
    }

    virtual void g() {
        cout << "Base1::g()" << endl;
    }
};

//父类 2
class Base2
{
public:
    virtual void h() {
        cout << "Base2::h()" << endl;
    }
    virtual void i() {
        cout << "Base2::i()" << endl;
    }
};

//子类
class Derived : public Base1, public Base2
{
public:
    virtual void f() {
        cout << "Derived::f()" << endl; //只覆盖 Base1 的 f
    }

    virtual void i() {
```

```
    cout << "Derived::i()" << endl; //只覆盖 Base2 的 i
}

//如下三个是本类自己的虚函数
virtual void mh() {
    cout << "Derived::mh()" << endl;
}
virtual void mi() {
    cout << "Derived::mi()" << endl;
}
    virtual void mj() {
        cout << "Derived::mj()" << endl;
    }
};
```

在 main 主函数中，加入如下代码(请详细阅读代码行中的注释)：

```
cout << sizeof(Base1) << endl;      //4
cout << sizeof(Base2) << endl;      //4
cout << sizeof(Derived) << endl;    //8 字节,这说明 Derived 里有两个虚函数表指针(2 个 vptr)

Derived ins;                        //定义一个子类对象
Base1& b1 = ins;
Base2& b2 = ins;
Derived& d = ins;

typedef void( * Func)(void);

long * pderived1 = (long * )(&ins);
long * vptr1 = (long * )( * pderived1); //第一个虚函数表指针,根据 public 中的继承顺序,应该
是和 Base1 对应

long * pderived2 = pderived1 + 1;        //往后走 4 个字节
long * vptr2 = (long * )( * pderived2);  //第二个虚函数表指针

//这些条目有规律,先从第一个继承的父类里找函数,如果自己覆盖的,就弄成自己的; 如果自己没
覆盖的,就弄成父亲的
//接着是自己的虚函数首地址
Func f1 = (Func)vptr1[0];        //0x00f31550 {project100.exe!Derived::f(void)}
Func f2 = (Func)vptr1[1];        //0x00f31596 {project100.exe!Base1::g(void)}
Func f3 = (Func)vptr1[2];        //0x00f31578 {project100.exe!Derived::mh(void)}
Func f4 = (Func)vptr1[3];        //0x00f31582 {project100.exe!Derived::mi(void)}
Func f5 = (Func)vptr1[4];        //0x00f3157d {project100.exe!Derived::mj(void)}
Func f6 = (Func)vptr1[5]; //0x0029aa64 {project100.exe!const Derived::`RTTI Complete Object
Locator'{for `Base2'}}
Func f7 = (Func)vptr1[6];        //后面先不管
Func f8 = (Func)vptr1[7];        //后面先不管

//这里找继承的另外一个父类,如果自己覆盖的,就弄成自己的; 如果自己没覆盖的,就弄成父亲的
```

```
Func f11 = (Func)vptr2[0];      //0x00291587 {project100.exe!Base2::h(void)}
Func f22 = (Func)vptr2[1];      //0x00291591 {project100.exe!Derived::i(void)}
Func f33 = (Func)vptr2[2];      //非法
Func f44 = (Func)vptr2[3];      //非法

b1.f(); //Derived::f(),父类引用,但引用的是子类,所以这里执行子类所覆盖的父类的虚函数
b2.i(); //Derived::i(),父类引用,但引用的是子类,所以这里执行子类所覆盖的父类的虚函数
d.f(); //Derived::f()
d.i(); //Derived::i()
d.mh(); //Derived::mh()
```

执行起来,看一看结果:

```
4
4
8
Derived::f()
Derived::i()
Derived::f()
Derived::i()
Derived::mh()
```

继续在 main 主函数中增加虚函数的一些直接调用代码如下:

```
cout << "--------------------" << endl;
f1();                           //Derived::f()
f2();                           //Base1::g()
f3();                           //Derived::mh()
f4();                           //Derived::mi()
f5();                           //Derived::mj()
//f6();                         //异常,报错
cout << "--------------------" << endl;
f11();                          //Base2::h()
f22();                          //Derived::i()
cout << "--------------------" << endl;
```

执行起来,看一看新增的代码段显示的结果:

```
--------------------
Derived::f() ------------------------ : 执行 f1();
Base1::g() -------------------------- : 执行 f2();
Derived::mh() ----------------------- : 执行 f3();
Derived::mi() ----------------------- : 执行 f4();
Derived::mj() ----------------------- : 执行 f5();
--------------------
Base2::h()
Derived::i()
--------------------
```

注意到上述结果,执行代码行 f6();会导致执行出现异常。

如果将上述代码放到 Linux 操作系统去执行，则 f6();代码行不会报异常，可以正常执行。所以，在 Linux 操作系统上，**取消对 f6();代码行的注释，让其正常执行**。在 Linux 操作系统下完整的执行结果如下：

```
8
8
16
Derived::f()
Derived::i()
Derived::f()
Derived::i()
Derived::mh()
--------------------
Derived::f() ------------------------ : 执行 f1();
Base1::g() -------------------------- : 执行 f2();
Derived::i() ------------------------ : 执行 f3();
Derived::mh() ----------------------- : 执行 f4();
Derived::mi() ----------------------- : 执行 f5();
Derived::mj() ----------------------- : 执行 f6();
--------------------
Base2::h()
Derived::i()
--------------------
```

可以看到，在 Linux 操作系统下，除上面三个数字显示的分别是 8、8、16(因为是 x64 平台)，与 Windows 操作系统下的 Visual Studio 2019 显示的结果不一样外，一些表项看起来也不太一样：Linux 操作系统下的第一个虚函数表项**多出一个** Derived::i()来。

这表示 Visual Studio 2019 编译器在虚函数表方面的设计和实现与 g++编译器略有不同。

有几点说明：

(1) 一个对象，如果它所属的类有多个基类，则有多个虚函数表指针(注意是多个虚函数表指针，而不是多个虚函数表，不要搞混)。

代码行 cout << sizeof(Derived) << endl;输出的结果为 8(Visual Studio 2019 的 x86 平台下)，这说明 Derived 里有两个虚函数表指针(2 个 vptr)。

在多重继承中，对应各个基类的 vptr 按继承顺序(class Derived : public **Base1**, public **Base2**;)依次放置在类内存空间中，且子类与第一个基类共用同一个 vptr(第二个基类有自己的 vptr)。本例中类 Derived 先继承了 Base1，后继承了 Base2。

(2) 补充，图 3.4 适合 Visual Studio 2019 编译器，不代表也适合其他的 Visual Studio 版本，也不代表适合 Linux 下的 g++编译器(如刚才看到了 g++编译器的表现就不太一样)。但这没有任何关系，读者只要知道原理，知道笔者写的这些代码的目的，那么在任何编译器上，只要输入这些代码并进行适当的分析，都能很清楚地了解针对这个编译器的多重继承虚函数表特性，请读者举一反三。通过观察上面程序的输出结果，绘制出了多重继承下的子类对象内存布局示意图，如图 3.4 所示。

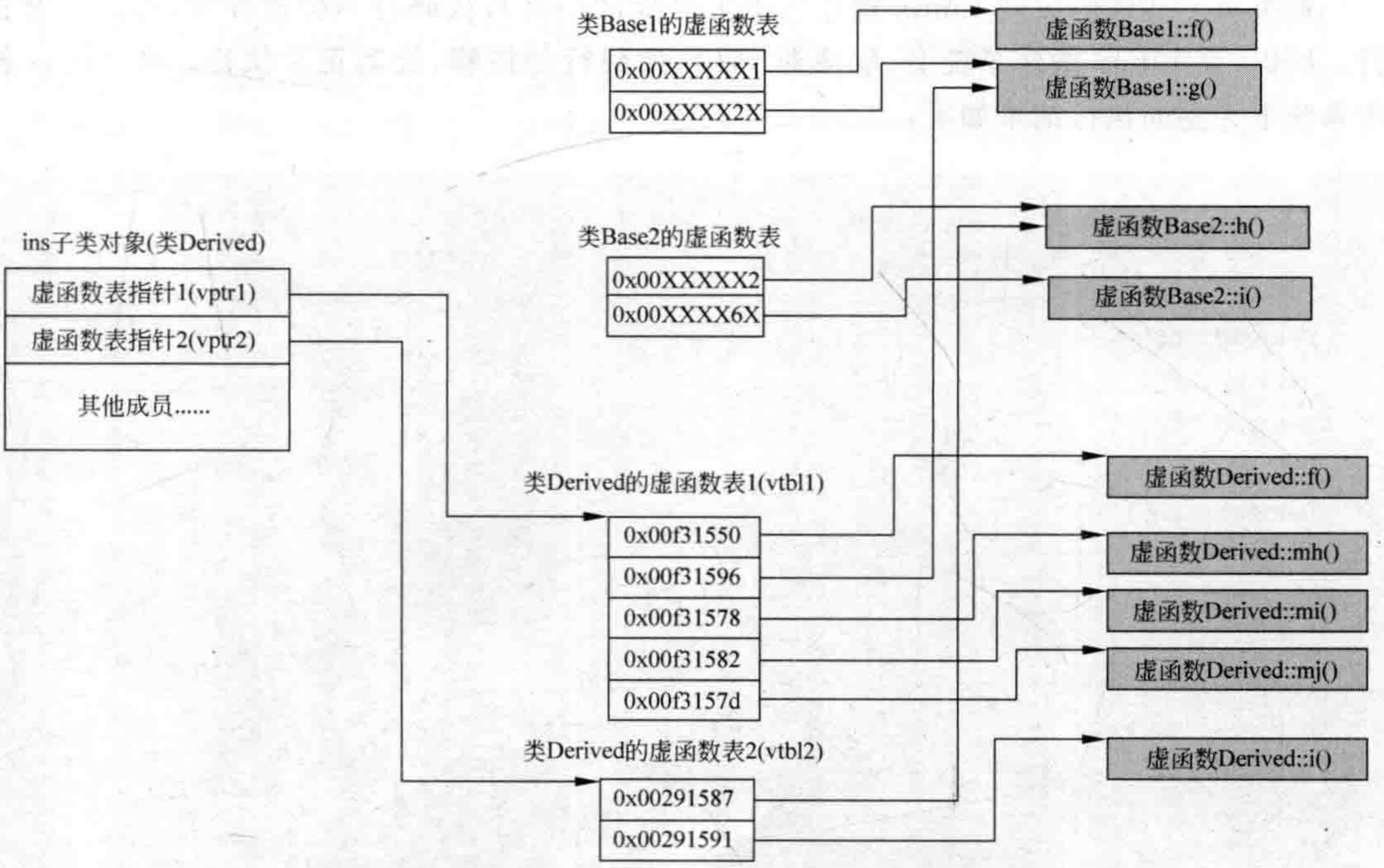

图 3.4　多重继承下的子类对象内存布局示意图(虚函数)

从图 3.4 可以看到：

(1) 子类对象 ins 有两个虚函数表指针——vptr1 和 vptr2。

(2) 类 Derived 有两个虚函数表，因为它继承自两个基类。

(3) 子类和第一个基类共用一个 vptr(所以也可以说子类和第一个基类共用一个虚函数表 vtbl1)，因为可以注意到类 Derived 的虚函数表 1(vtbl1)里面的 5 个函数中，g 正好是类 Base1 里的虚函数。

(4) 子类中的虚函数覆盖了父类中的同名虚函数，如 Derived::f 和 Derived::i。

另外，Visual Studio 编译器也提供了基本的查看虚函数表的能力，但笔者感觉这些信息不全，只能查看到一部分，供读者参考。

设置断点并进行调试，当程序执行流程停到断点行时，该断点行前面定义过的各种类对象都可以用鼠标放上去，展开进行观察，如图 3.5 所示。

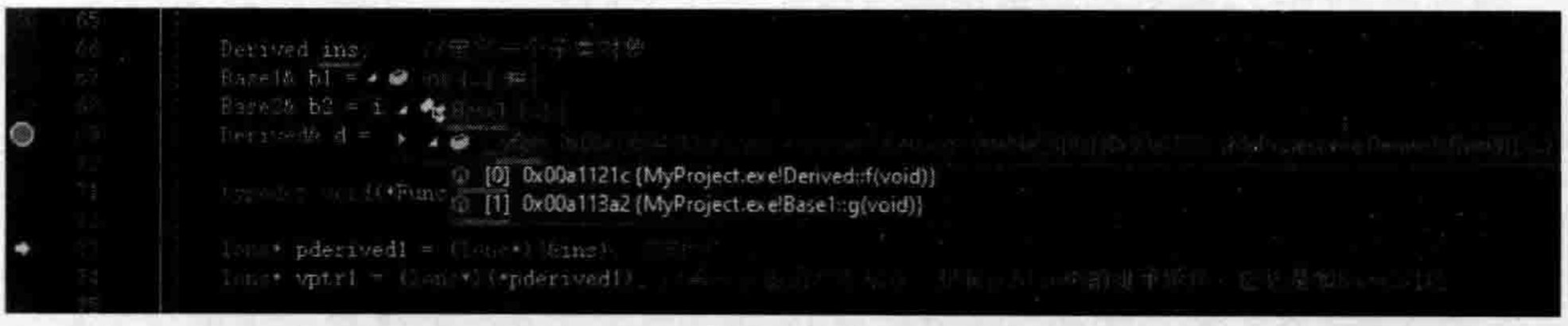

图 3.5　通过 Visual Studio 编译器观察某个类对象的虚函数表信息

或者选中某个类对象，然后按下 Shift＋F9 键，也可以查看。利用这种手段辅助分析，如图 3.6 所示。

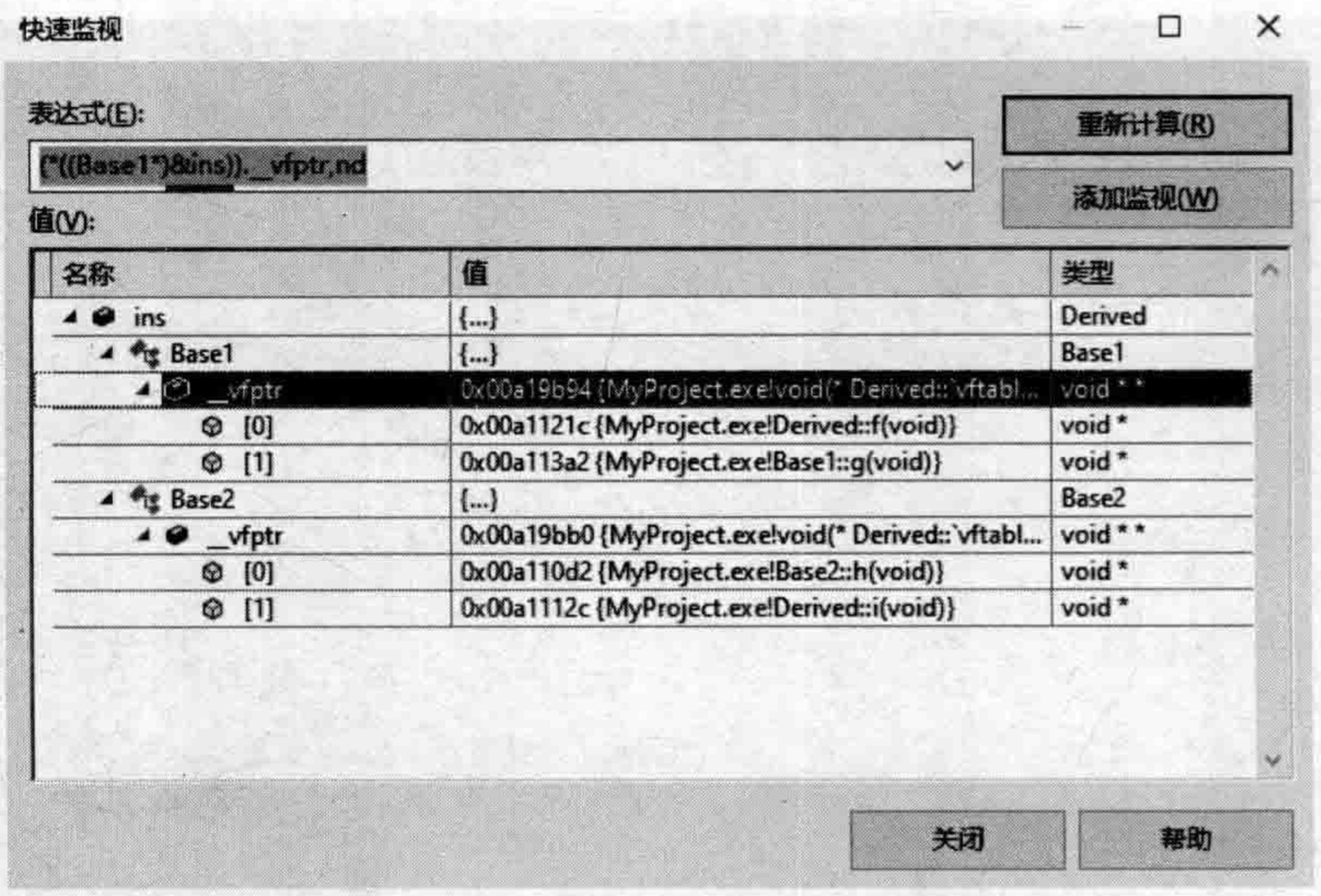

图 3.6 通过快速监视窗口观察某个类对象的虚函数表信息

3.5 辅助工具与 vptr、vtbl 创建时机

3.5.1 使用辅助工具查看虚函数表

上一节讲解的范例代码,在本节保持不变。

前面对于虚函数表,已经通过写代码的方式了解了不少虚函数表的组织结构。那么,有没有什么能借助的工具来辅助分析一下虚函数表的结构呢?还是有的。

在 Visual Studio 2019(或其他 Visual Studio 版本)中打开**开发人员命令行工具**,忘记如何打开的读者可以参考图 2.9。在这个命令行提示符下,可以直接使用 cl 命令行工具(cl 是编译链接工具)。

在命令行提示符下,进入到 MyProject.cpp 文件所在的路径,并输入 cl /d1 reportSingleClassLayout**Derived** MyProject.cpp,如图 3.7 所示。

注意上面的这个命令:

(1) 区分 cl 和 d1,一个是小写字母 l,一个是数字 1,两者看上去比较像。

(2) reportSingleClassLayout 后面跟的是要查看的类名。

(3) 最后面跟的是要查看的类所在的.cpp 源文件名。

注意比较图 3.7 和图 3.4。图 3.7 也明显指出了类 Derived 对象中包含两个虚函数表指针(vfptr/vptr),并分别列出了所指向的两个虚函数表和其中的表项。

在 Linux 操作系统下,使用 g++编译器,可以利用-fdump-class-hierarchy 选项来导出类的层次结构。命令行如下:

```
g++ -fdump-class-hierarchy -fsyntax-only MyProject.cpp
```

此时,系统会生成一个扩展名为.class 的文件,该文件里就有类的布局信息。可以利用

```
D:\Program Files (x86)\Microsoft Visual Studio\2019\Community>c:

C:\>cd C:\Users\KuangXiang\Desktop\c++\MySolution\MyProject

C:\Users\KuangXiang\Desktop\c++\MySolution\MyProject>cl /d1 reportSingleClassLayoutDerived MyProject.cpp
用于 x86 的 Microsoft (R) C/C++ 优化编译器 19.22.27905 版
版权所有(C) Microsoft Corporation。保留所有权利。

MyProject.cpp

class Derived   size(8):
        +---
 0      | +--- (base class Base1)
 0      | | {vfptr}
        | +---
 4      | +--- (base class Base2)
 4      | | {vfptr}
        | +---
        +---

Derived::$vftable@Base1@:
        | &Derived_meta
        |  0
 0      | &Derived::f
 1      | &Base1::g
 2      | &Derived::mh
 3      | &Derived::mi
 4      | &Derived::mj

Derived::$vftable@Base2@:
        | -4
 0      | &Base2::h
 1      | &Derived::i

Derived::f this adjustor: 0
Derived::i this adjustor: 4
Derived::mh this adjustor: 0
Derived::mi this adjustor: 0
Derived::mj this adjustor: 0
```

图 3.7　Windows 操作系统下使用 cl 工具查看类的虚函数表组织结构图

vim 来查看这个.class 文件的内容。这个.class 文件很长，大概有接近 2000 行内容，就看最后的位置就行，因为最后位置显示的就是 Derived 类的虚函数指针和指向的虚函数表信息，如图 3.8 所示。

```
Vtable for Derived
Derived::_ZTV7Derived: 12u entries
0     (int (*)(...))0
8     (int (*)(...))(& _ZTI7Derived)
16    (int (*)(...))Derived::f
24    (int (*)(...))Base1::g
32    (int (*)(...))Derived::i
40    (int (*)(...))Derived::mh
48    (int (*)(...))Derived::mi
56    (int (*)(...))Derived::mj
64    (int (*)(...))-8
72    (int (*)(...))(& _ZTI7Derived)
80    (int (*)(...))Base2::h
88    (int (*)(...))Derived::_ZThn8_N7Derived1iEv

Class Derived
   size=16 align=8
   base size=16 base align=8
Derived (0x0x7fa0803b3a80) 0
    vptr=((& Derived::_ZTV7Derived) + 16u)
```

图 3.8　Linux 操作系统下查看类的虚函数表组织结构图

图 3.8 与图 3.7 所显示的内容虽然有些区别，但是大同小异，核心的内容相信读者都已经理解了。

3.5.2　虚函数表的创建时机

现在，请想一想，vptr(虚函数表指针)是什么时候创建的呢？

读者已经知道，vptr 是跟着对象走的，对象创建出来了，才存在这个 vptr。对象什么时候创建出来的呢？当然是程序运行的时候。

实际上，对于这种有虚函数的类，在编译时候，**编译器会往类的构造函数中安插为 vptr 赋值的语句**，这是在编译期间做的。这是编译器默默在背后为程序员做的事情，程序员并不知道。

当程序运行的时候，遇到创建该类对象的语句时，会执行该类的构造函数，因为构造函数中有给 vptr(要创建的对象所属的 vptr)赋值的语句，从而让这个对象的 vptr 变得有有效值。

谈完了 vptr 是什么时候创建的，现在谈一谈虚函数表 vtbl 是什么时候创建的。实际上，编译器在**编译期间**(不是运行期间)**就为每个类确定好了对应的虚函数表 vtbl 的内容**，然后也是在编译期间在相应的类构造函数中添加给 vptr 赋值的语句，这样程序运行的时候，当运行到生成类对象的代码时，会调用类的构造函数，执行到类的构造函数中的给 vptr 赋值的语句时，这个类对象的 vptr 就有值了。

下面再进一步展开一点讨论，让读者更清晰一些。

当书写一个 C++程序，经过编译、链接的步骤，最终会生成一个可执行程序(文件)。可执行程序遵循一定的文件格式，这样当执行这个可执行程序时，操作系统会把这个可执行程序根据约定的格式装载到内存中并开始执行。

一般来讲，一个可执行程序必然有“代码段”和“数据段”用于存储各种要执行的代码和各种必要的数据。

当这个可执行程序装载到内存后，它的代码段、数据段也一并被装载到了内存中，同时，操作系统还会根据一定的规则为这个可执行程序分配各种内存空间(如果读者真的对这方面知识特别有兴趣，笔者推荐《程序员的自我修养——链接、装载与库》一书)，如堆、栈之类的。所以，当一个可执行程序被装载到内存执行的时候，看起来大概如图 3.9 所示。

在图 3.9 中可以看到，可执行程序中的数据段、代码段被分别载入内存，同时还有栈区和堆区用于在程序执行的过程中分配内存。

如果执行下面这样一行代码，看一看流程是什么样，内存是什么样：

```
Derived * obj = new Derived();
```

上面这行代码(参考图 3.10 辅助理解)，定义了一个 obj 指针，这个指针当然是在栈上分配的 4 字节内存(这个 obj 本身是一个局部变量，是在栈上分配内存的)，当 new 一个类 Derived 对象时，obj 就会指向一个堆内存(堆区)，new 出来的 Derived 对象实例就保存在堆区中，该对象实例中的虚函数表指针 vptr 在执行了类 Derived 的构造函数代码后，就指向了存在于数据段中的类 Derived 的虚函数表首地址，而虚函数表中的每一项都指向代码段中某个虚函数的代码。

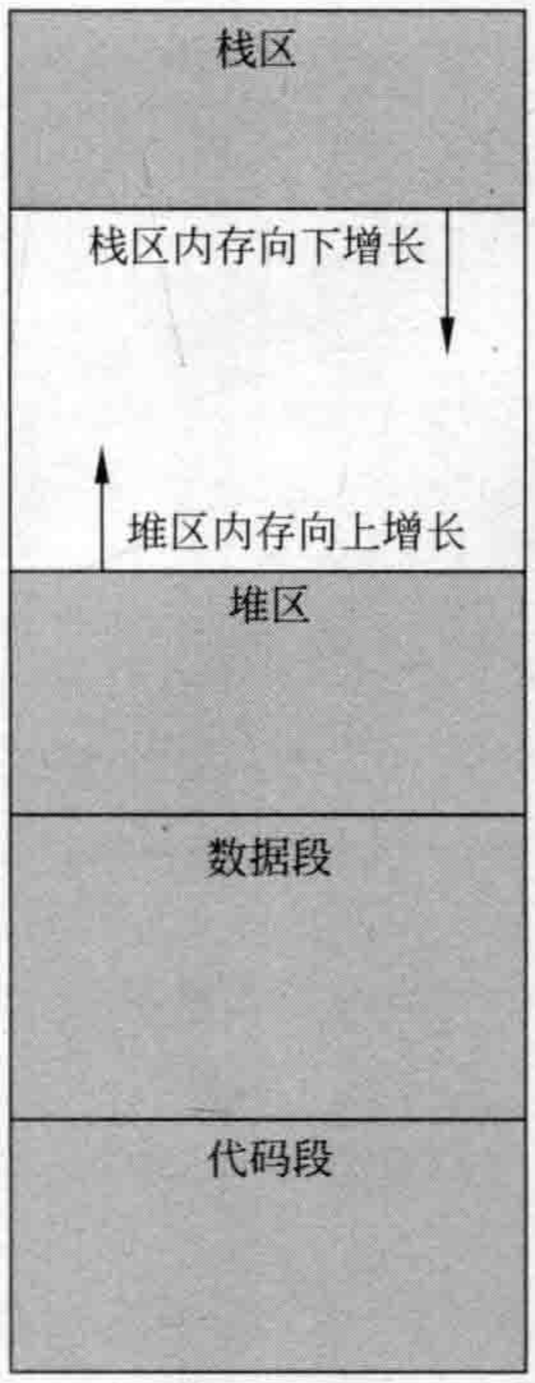

图 3.9 可执行程序被载入到内存时的内存结构图

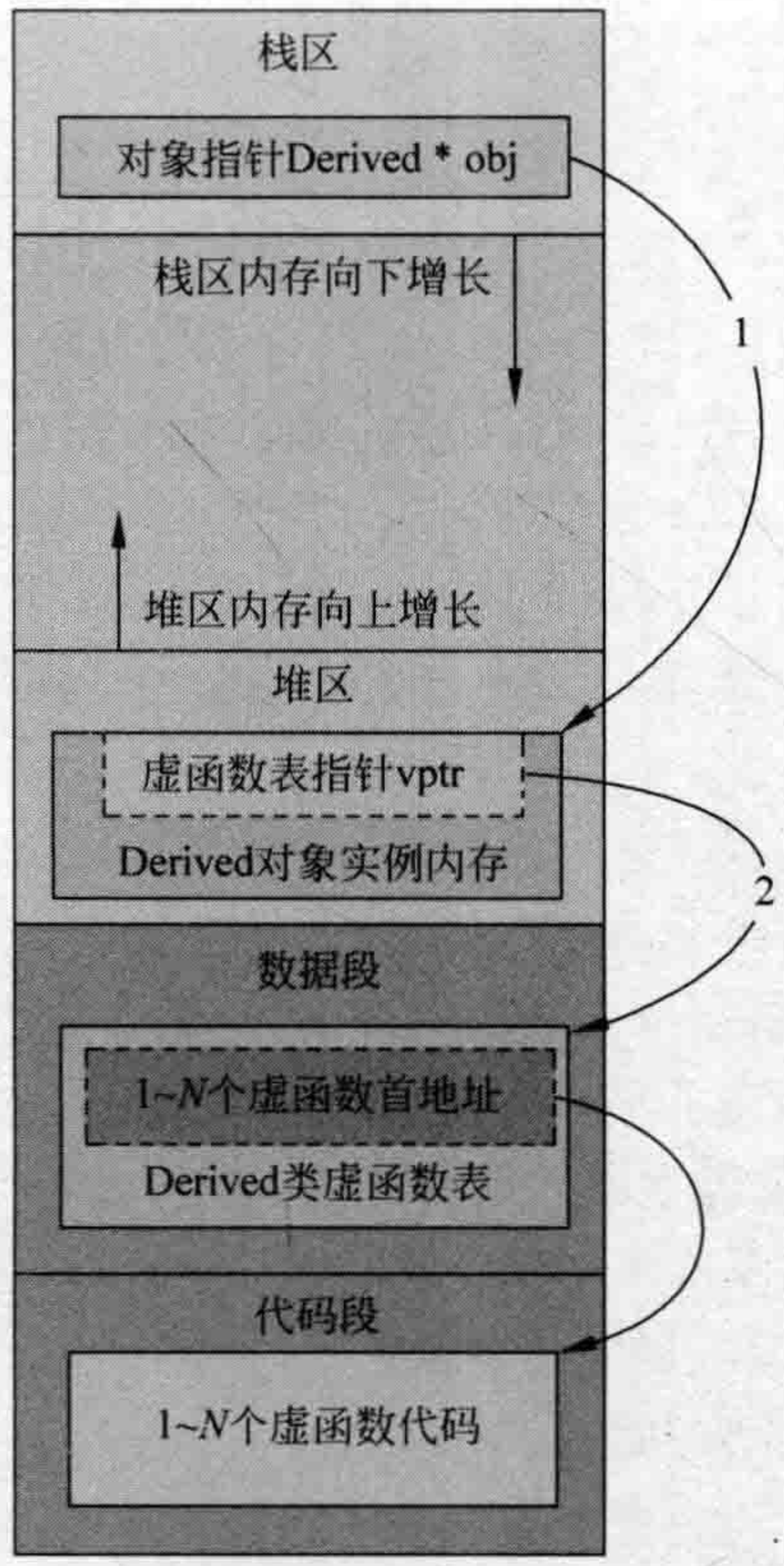

图 3.10 new 一个类对象时虚函数表指针、虚函数表、虚函数内存结构图

3.6 单纯的类不纯时引发的虚函数调用问题

看看如下范例。在 MyProject.cpp 的上面，增加类 X 的定义代码：

```
class X
{
public:
    int x;
    int y;
    int z;
    X() :x(0), y(0), z(0)
    {
        cout << "X 类的构造函数被执行" << endl;
    }
    X(const X& tm) :x(tm.x), y(tm.y), z(tm.z)
    {
        cout << "X 类的拷贝构造函数被执行" << endl;
    }
};
```

在 main 主函数中，加入如下代码：

```
X x0;                                    //调用构造函数
x0.x = 100;
x0.y = 200;
x0.z = 300;
X x1(x0);                                //调用拷贝构造函数
cout << "x1.x = " << x1.x << " x1.y = " << x1.y << " x1.z = " << x1.z << endl;
```

执行起来，看一看结果：

```
X 类的构造函数被执行
X 类的拷贝构造函数被执行
x1.x = 100 x1.y = 200 x1.z = 300
```

可以看到，X 这种类只有一些简单的成员变量，所以是很单纯的类。这种单纯的类，拷贝构造函数可以写的更有效率一点。修改类 X 的拷贝构造函数，修改后的代码如下：

```
X(const X& tm)
{
    memcpy(this, &tm, sizeof(X));
    cout << "X 类的拷贝构造函数被执行" << endl;
}
```

这种改造后的拷贝构造函数是通过调用 memcpy 直接进行内存复制，与原来的拷贝构造函数效果一样。

另外，在类 X 的构造函数中，如果只有一些数值类型的成员变量，并且它们的初始值都为 0，那么可以这样来修改类 X 的构造函数：

```
X()
{
    memset(this, 0, sizeof(X));
    cout << "X 类的构造函数被执行" << endl;
}
```

笔者相信，现实项目中，会有这样写程序的人。再次执行程序，结果没有发生变化。

如果这个类(类 X)像现在这样**单纯**，那么，上述这种使用 memcpy、memset 函数并将 this 作为第一个参数的写法都没有问题。但是，如果这个类并**不单纯**，那么上面这种用 memcpy、memset 函数的写法就是错误的，会导致程序崩溃。那么什么叫类并不单纯呢？

那就是，在某些情况下，编译器会往类内部增加一些程序员看不见但真实存在的成员变量(隐藏的成员变量)，有了这种变量的类，就不单纯了。

同时，这种隐藏的成员变量的赋值时机往往都是**在执行构造函数或者拷贝构造函数的函数体之前**进行的。那么，请想一想：

(1) 构造一个类对象时，编译器给一些隐藏的成员变量赋值，然后在构造函数的函数体中，一执行 memset，很可能把这些值覆盖(清零)了。

(2) 而复制构造一个对象时，往往对象与对象之间(复制的源对象与目标对象之间)，这些隐藏的成员变量的值可能并不相同，在执行拷贝构造函数时，一执行 memcpy，就会造成

源对象中隐藏的成员变量值覆盖了目标对象中隐藏的成员变量值。

例如，如果类中有一个虚函数，系统就会默认往类对象里增加一个虚函数表指针，这就是一个隐藏的成员变量。

请注意，现在笔者向类 X 中增加一个 public 修饰的**虚析构**函数、一个普通的**虚**成员函数和一个普通的成员函数：

```
public:
    virtual ~X()
    {
        cout << "X 类的析构函数被执行" << endl;
    }
    virtual void virfunc()
    {
        cout << "虚函数 virfunc()执行了" << endl;
    }
    void ptfunc()
    {
        cout << "普通函数 ptfunc()执行了" << endl;
    }
```

那么，现在类 X 中有了虚函数，编译的时候，编译器就会往类 X 的构造函数插入诸如下面的代码行：

```
vptr = vtbl; //把类 X 的虚函数表 vtbl 的首地址赋值给隶属于某个类的 vptr(虚函数表指针)
```

或者是：

```
this = vtbl; //this 代表对象的首地址，而虚函数表指针就位于对象的首地址开始的四个字节里，
所以这也相当于让对象的虚函数表指针 vptr 指向类 X 的虚函数表 vtbl
```

其实读者只要设置一个断点在类 X 的构造函数所在行，按 F5 键执行程序并开始调试，当程序执行停留在断点行时，切换到反汇编窗口，不难看到**编译器插入的用于给 vptr 赋值的语句**，如图 3.11 所示。

图 3.11　编译器插入的用于给虚函数表指针 vptr 赋值的语句

问题来了，如果紧接着，在类 X 的构造函数的函数体中，执行 memset(this，0，sizeof(X))；代码行，则会导致刚刚被赋值的 vptr 的值被清零(内存内容变成了 0)，可以设置断点调试一

下观察 vptr 被清零的情形(这次断点设置在 main 主函数中的 X x0;语句行的下一行),如图 3.12 所示。

图 3.12 对 memset 的误用导致类 X 对象的虚函数表指针 vptr 被清零

这个时候,按照理解,vptr 被无意中清零,程序执行起来应该就会有问题,尤其是对虚函数调用要通过虚函数表指针 vptr 来进行,现在 vptr 值被破坏,还能正常调用虚函数吗?

运行整个程序,根据执行结果,可以发现 x0 这个类对象的析构函数还是能正常执行。析构函数是虚函数,而 vptr 值被破坏,按道理来说调用虚函数会出现问题,但为什么 x0 类对象在程序终止时看起来却能够正常执行类 X 的析构函数(虚函数)呢?

读者可以再次运行程序,结果如下,看起来程序运行结果是正常的,没有受到 vptr 值被破坏的影响,析构函数能够正常执行:

```
类 X 的构造函数被执行
类 X 的拷贝构造函数被执行
x1.x = 100 x1.y = 200 x1.z = 300
类 X 的析构函数被执行
类 X 的析构函数被执行
```

如果在 main 主函数中增加代码行来调用一个虚函数,发现也能正常调用:

```
x0.virfunc();
```

```
虚函数 virfunc()执行了
```

既然程序代码已经把虚函数指针清零了,为什么还能成功调用虚函数?

于是,笔者修改一下写法。在 main 主函数中,注释掉原有代码,加入如下代码:

```
X * px0 = new X();
px0 -> ptfunc();                    //可以正常调用普通函数
px0 -> virfunc();                   //无法正常调用虚函数
delete px0;                         //无法正常调用析构函数
```

运行起来,会发现普通函数可以正常调用,而 virfunc 虚函数无法正常调用,析构函数也无法正常调用(因为析构函数也是一个虚函数),总之一句话:虚函数变得没法正常执行了。

现在看到的现象是:生成一个类 X 的局部对象(对象在栈中),不影响通过该对象调用虚函数以及不影响该对象的正常析构,但如果用 new 在堆中生成一个类 X 的对象实例并用对象指针指向该对象实例,再用这个对象指针调用虚函数,或者 delete 该指针,程序执行都

变得不再正常(虚函数无法被调用,指针指向的堆中的对象也无法调用虚析构函数)。这是为什么呢?

读者对多态、虚函数、父类、子类这些概念都很清楚,也应该知道,虚函数的应用场景主要是父类指针指向子类对象的情形(有父子关系的情况下,虚函数才能体现出其作用来)。而目前的范例程序代码,只有一个 X 类,并不存在父类子类这种说法,只不过是在类 X 中有虚函数而已。那么,只有虚函数,没有继承关系时,虚函数和普通函数从功能上来说有什么区别呢? 这种情况下虚函数和普通函数并没有什么区别。

现在,把以往 main 主函数中的代码注释掉,并增加如下新代码行,来演示一个新内容:

```
int i = 9;
printf("i 的地址 = %p\n", &i); //会发现每次 i 的地址输出出来都不一样
```

将程序多执行几次,发现**每次输出的 i 的地址都会改变**。

读者已经知道,函数是跟着类走的,虚函数也一样,也是跟着类走的。所以,继续向 main 主函数中增加代码,输出一下函数地址:

```
X x0;
printf("ptfunc 的地址 = %p\n", &X::ptfunc); //正常函数地址可以这样输出
```

虚函数的地址也可以输出,在 3.2 节演示过,这里再写一次,继续向 main 主函数中增加代码,输出一下虚函数地址:

```
long* pvptr = (long*)(&x0);
long* vptr = (long*)(*pvptr);
printf("vptr[1] = %p\n", vptr[1]);
```

上面这三行代码,因为在类 X 的构造函数中存在 memset 语句,把虚函数表指针清理成零导致程序没法正常运行,因此,现在把类 X 的构造函数中的 memset(this, 0, sizeof(X)); 语句行暂时注释掉。

执行起来,看一看结果:

```
i 的地址 = 008FF7A8
X 类的构造函数被执行
ptfunc 的地址 = 001614F1
vptr[1] = 001614E7
X 类的析构函数被执行
```

成功看到了 vptr[1]的地址后,取消 X 构造函数中 memset 语句行的注释,然后把上面三行打印虚函数地址的代码注释掉,并继续往 main 函数中增加如下两行代码:

```
x0.ptfunc();
x0.virfunc();                    //虚函数是可以正常调用的
```

执行起来,看一看结果:

```
i 的地址 = 004FFDB4
X 类的构造函数被执行
ptfunc 的地址 = 004614F1
```

```
普通函数 ptfunc()执行了
虚函数 virfunc()执行了
X 类的析构函数被执行
```

将程序多执行几次，发现**每次输出的 i 的地址都会改变**。但 **ptfunc 的地址都不发生变化（都是 004614F1）**。

如果把断点设置到 x0.virfunc();行的下一行，按 F5 键开始运行程序，当程序运行到断点行并停下来时，切换到反汇编窗口，如图 3.13 和图 3.14 所示。

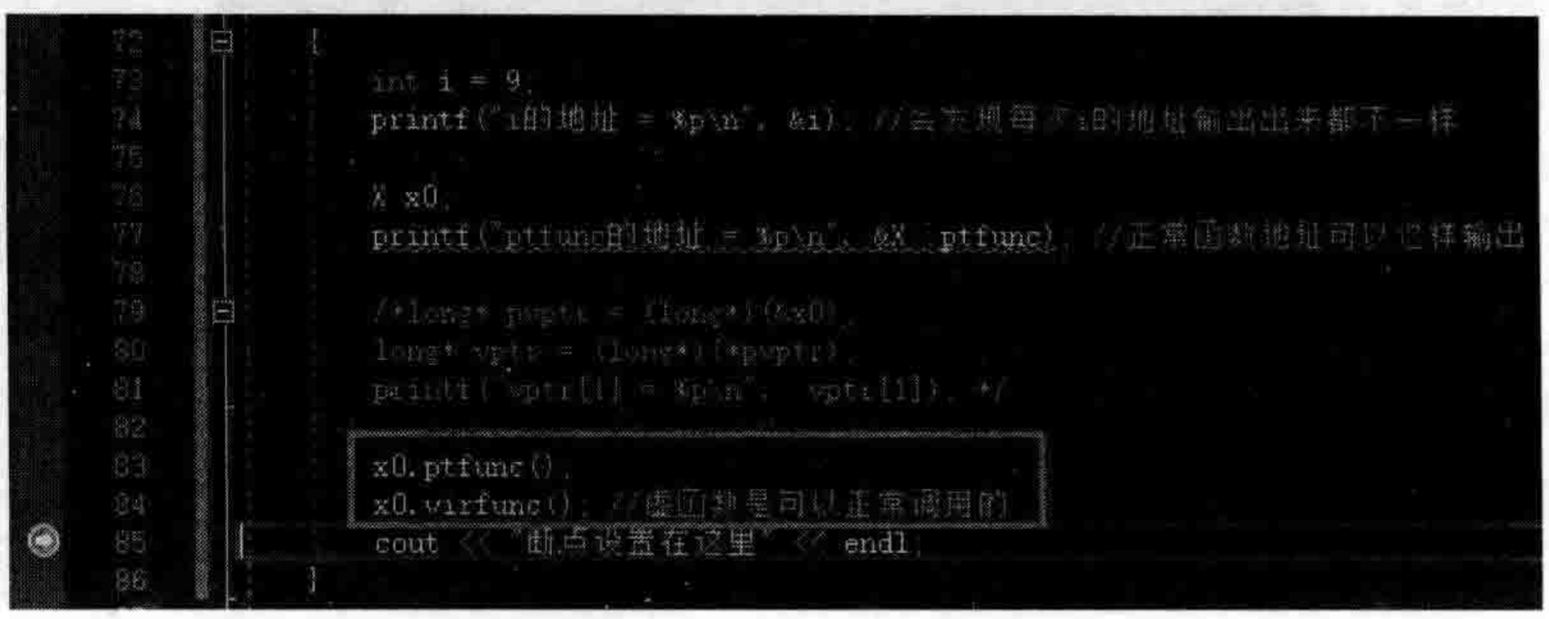

图 3.13　设置断点并运行程序后程序运行停止到断点行

图 3.14　程序运行停止到断点行后通过反汇编窗口观察函数的入口地址

图 3.14 中显示的 04614F1h 和 04614E7h 正是普通函数 ptfunc 和虚函数 virfunc 的地址。多次执行程序或者设置断点调试，每次运行这两个地址的值都不会改变。据此可以推断，该普通函数的地址和虚函数的地址是在编译的时候就确定了的。

现在引入“静态联编”和“动态联编”这两个概念：

（1）静态联编：编译的时候就能确定调用哪个函数，把调用语句和被调用函数绑定到一起。

（2）动态联编：在程序运行的时候，根据实际情况，动态地把调用语句和被调用函数绑定到一起，动态联编一般只有在多态和虚函数情况下才存在。

有了这种概念作为铺垫，就不难分析出来，其实：

```
X x0;
x0.ptfunc();
x0.virfunc();
```

这几行代码就属于静态联编，别看调用的是虚函数，因为无论普通函数还是虚函数，在编译的时候都是有固定地址的，所以，这种直接以定义对象的方式来生成一个对象并通过该对象调用虚函数时**根本用不到虚函数表**，所以，即使把对象中的虚函数表指针清零，也不影响对虚函数的调用，因为编译时对这些函数的调用就已经确定了，绑定了。

反观如下代码：

```
X * pX0 = new X();
pX0->ptfunc();
pX0->virfunc();
```

可以仿照上面这样设置断点位于虚函数调用这一行，按 F5 键执行程序，并在停止到断点行时查看反汇编代码，如图 3.15 所示(请读者注意比较图 3.14 和图 3.15 中的针对虚函数调用的汇编代码行的区别)。

图 3.15　程序运行停止到断点行后通过反汇编窗口观察 new 的对象进行虚函数调用时对应的汇编代码

观察图 3.15，不难发现，pX0 调用普通函数 ptfunc，它是直接调用(call)的，因为普通函数有固定的地址，是在编译的时候确定好的(静态联编)。

而 pX0 调用虚函数 virfunc，可不是直接调用，而是执行了一堆代码，这堆代码就是为了实现多态用的，做的事情是通过虚函数表指针去找虚函数表，然后从中找到虚函数 virfunc 的地址并进行调用。但可惜的是，虚函数表指针已经在类 X 的构造函数中通过 memset 被清零了，所以这段代码不可能找到虚函数表地址，更不可能找到虚函数 virfunc 的地址。所以执行到调用 virfunc 虚函数代码行时程序会报异常，如图 3.16 所示。

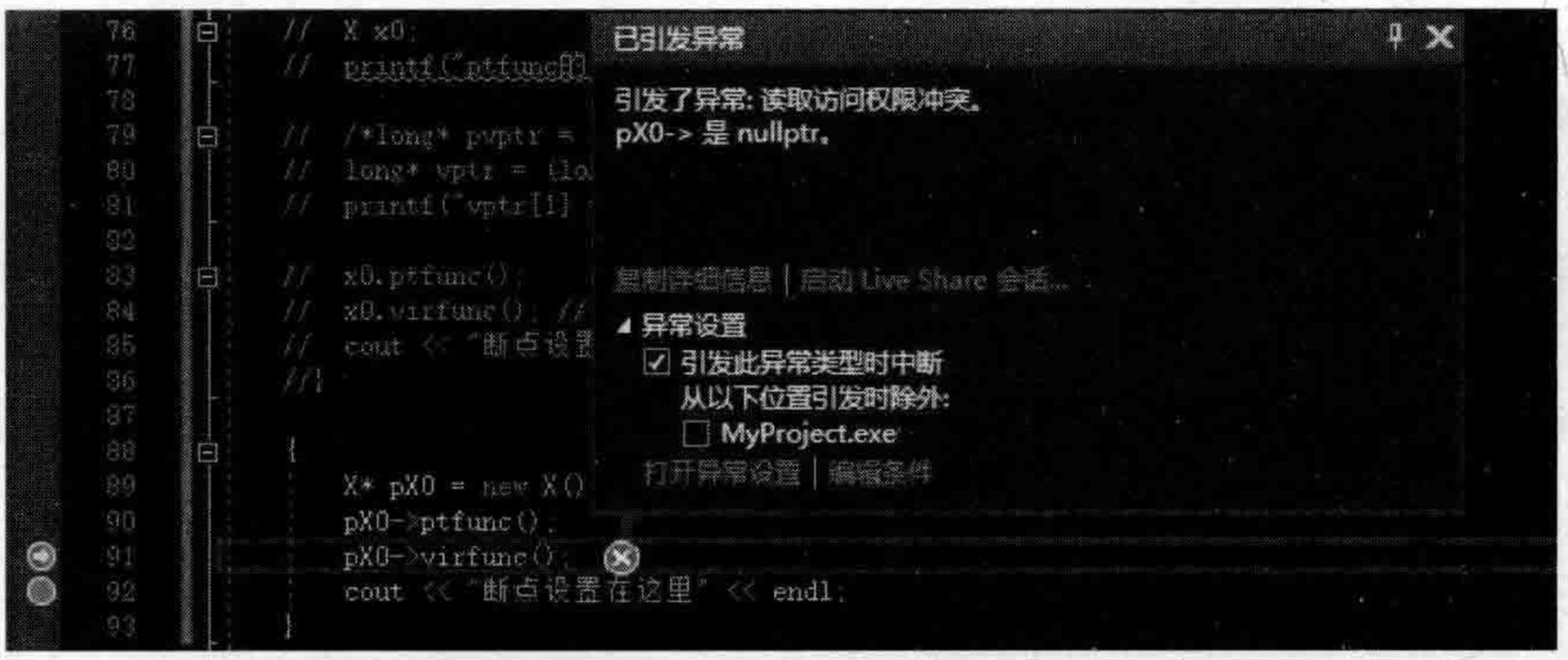

图 3.16　因为虚函数表指针被清零导致无法正确调用虚函数

这就是在栈上生成一个对象：

```
X x0;
```

和利用 new 在堆上 new 一个对象：

```
X * pX0 = new X();
```

它们调用虚函数时的不同：一个是直接调用虚函数，一个是通过虚函数表指针找到虚函数表，在虚函数表中查询虚函数地址再调用该虚函数。所以读者更明白了，其实虚函数、多态这种概念是专门给**指针**和**引用**来使用的。

上面的代码中，pX0 是一个指针，如果换一种写法（下面的 x1y 就是一个引用）：

```
X x1;
X& x1y = x1;                          //引用
x1y.virfunc();//也报异常，因为虚函数表指针被清零导致无法找到虚函数表，从而也无法执行虚函数
```

当然，这种多态要真正发挥作用，还需要类继承关系存在（也就是有父类有子类）的情况下才能体现出来：一般都是父类指针指向子类的对象，或者是父类引用引用了子类对象。

本例中没有子类，所以多态的效果体现不出来，但是通过上面的分析，无疑，它**实际上就是多态的实现手段**：用到了虚函数表指针和虚函数表来进行虚函数的调用。

所以，回归到本节开始，请切记：

一个类，一旦变得不再单纯，就千万不能在构造函数中使用诸如 memset 或在拷贝构造函数中使用 memcpy 等来覆盖类对象所在内存空间中的内容，否则，一旦用到该类对象指针或者引用去调用虚函数，那么一定会出问题甚至导致程序执行崩溃。

读者已经知道，一个类中，一旦增加了**虚函数**，就变得不再单纯，编译器要往类中插入代码。那么同理，如果一个类中有**虚基类**，这个类也将变得不再单纯，编译器也同样要往类中插入代码，所以，有虚基类的场合，也千万不要在构造函数中使用诸如 memset 或在拷贝构造函数中使用 memcpy 等来覆盖类对象所在内存空间中的内容，否则，一样会面临程序不知何时会执行崩溃的局面。

有关虚基类的问题，后面章节会详细阐述。

第4章

数据语义学

本章主要讲解类对象的数据布局，着重讲解虚函数存在的情况下类对象的数据布局。同时，本章也详细描述了虚基类表指针和虚基类表的概念与实现细节，并继续详述虚基类存在时类对象的数据布局。

通过本章的学习，读者可以详细地了解一个类对象在内存中如何存储，成员的存储顺序是怎样的，在单一继承与多重继承的情况下，this 指针如何调整，以及成员变量偏移值等概念。

4.1 数据成员绑定时机

4.1.1 成员函数函数体的解析时机(整个类定义完毕后)

看看如下范例。在 MyProject.cpp 的上面，增加如下代码行：

```
string myvar;                          //定义全局量,字符串型
//定义一个类
class A
{
public:
    int myfunc()
    {
        return myvar;
    }
private:
    int myvar;                         //与全局量同名的成员变量
};
```

编译一下，没有报错。看一下这个 myfunc 成员函数，代码中，它返回的是 myvar。现在的问题是，有一个全局量叫 myvar，而类 A 中也有一个成员变量叫 myvar，而这两者的类型并不相同。显然，在 myfunc 成员函数中返回的 myvar 编译器认为是类 A 的成员变量 myvar，因为 myfunc 返回类型是 int 类型，而只有类 A 的成员变量 myvar 是 int 类型。

编译器为什么不认为成员函数 myfunc 返回的 myvar 变量是全局的 myvar 变量呢?

这是因为，**编译器对 myfunc 的解析是整个类 A 定义完毕后才开始的**，所以，对这个 myvar 成员变量的解析和绑定操作，也是在这个类定义完成之后才发生。这个类定义完成了，编译器会优先在类中找 myvar。所以，这个 myvar 指的是类 A 中定义的 myvar 成员变量。

现在修改一下代码，将 myfunc 成员函数的函数体写在类定义的外面：

```
int A::myfunc()
{
    return myvar;
}
```

那这里的 myvar 是全局的 myvar 还是类中的 myvar 成员变量呢?

可能许多读者认为是全局的 myvar 变量,实际上还是类 A 中的成员变量 myvar,编译器很聪明,遇到成员函数体内的变量时,会先到类中去找,如果找不到,才会到全局范围去找。

那么,如果要在 A::myfunc 的函数体中访问全局的 myvar 变量该怎么做? 只需要在 myvar 前面增加::就可以了,这就表示全局的 myvar 变量。当然,全局的 myvar 变量是 string 类型,所以在 A::myfunc 的函数体中返回全局的 myvar,编译肯定不通过,会报错。

如果再写一个全局函数,也返回 myvar,看以下代码:

```
int myfunc()
{
    return myvar;
}
```

编译一下,发现会报错,因为在这个函数中找到的是全局的 myvar,把鼠标放到 myvar 上,看弹出的浮动提示,发现编译器找到的确实是 string myvar。但 myfunc 全局函数要求返回的是 int 类型,所以编译肯定报错。

编译器确实挺智能的,在成员函数中,它找的就是类定义里面的成员变量;在全局函数中,它找的就是全局范围的成员变量。

以上所讲,最核心的要意是一句话:

编译器**对成员函数 myfunc 的解析是整个 A 类定义完毕后才开始的**。因为只有整个类 A 定义完毕后,编译器才能看到类 A 中的 myvar,才能根据实际的需要把出现 myvar 的场合做上述的适当的解析(成员函数中解析成类中的 myvar,全局函数中解析成全局的 myvar)。

4.1.2　成员函数参数类型的确定时机(最近碰到原则)

上面看了一下成员函数的返回值的问题,现在看一下参数列表的问题。为了简单起见,把以往所有的演示代码全部注释掉,下面的演示重新写代码。

在 MyProject.cpp 的上面,增加如下代码行:

```
typedef string mytype;
class A
{
public:
    void myfunc(mytype tmpvalue)      //这个 mytype 是 string 类型
    {
        m_value = tmpvalue;
    }

private:
    typedef int mytype;
    mytype m_value;                   //这个 mytype 是 int 类型
};
```

编译一下,报错,分析一下会发现 myfunc 中的 tmpvalue 是 string 类型,而 m_value 是

int 类型，string 类型变量不能给 int 类型变量赋值。

现在修改一下代码，将 myfunc 成员函数的函数体写在类定义的外面：

```
void A::myfunc(mytype tmpvalue) //这个 mytype 又变成了 int 类型,但若在类中定义 myfunc,那么参数中出现的 mytype 是 string 类型
{
    m_value = tmpvalue;
}
```

编译一下，依然会报错，因为现在的情况是，在类 A 内部声明的 myfunc 的形参 tmpvalue 是 string 类型，而在类 A 外面实现的 myfunc 代码中的形参 tmpvalue 却是 int 类型，显然两者形参类型不一致。

但如果是一个普通的全局函数，编译就不会报错：

```
void myfunc(mytype tmpvalue)//这个 mytype 就是 string 类型
{
    string mvalue = tmpvalue;
}
```

所以为什么说编译器挺智能的，因为遇到类成员函数，它就把 mytype 解析成类内的 int 类型，否则它就解析成全局的 string 类型。

现在问题的焦点在于类内声明的这行代码：

```
void myfunc(mytype tmpvalue);
```

将光标放到 mytype 上，根据弹出的浮动提示，mytype 的类型被解析成了 string 类型，现在的编译错误是因为 mytype 被解析成了 string 类型引发的，那么，为什么 mytype 被解析成了 string 类型。

通过种种观察，可以得到一些结论：

成员函数的参数类型是在**编译器最近一次遇到 mytype 时决定的，从上往下看代码**，最近一次遇到 mytype 的时候，编译器只看到了：

```
typedef string mytype;
```

没有看到类中的：

```
typedef int mytype;
```

所以，为了在类中“**最近**”看到类型 mytype，类中的这种**类型定义语句**(typedef)**一定要挪到类内的开始位置来定义**，这样，后面的成员函数遇到这个类型时，它就本着**最近碰到的类型的原则**来应用最近碰到的类型。以后读者写程序的时候也要注意并遵守这个规则。

所以，根据最近碰到的类型的原则，只需要在类 A 中把 mytype 类型的定义提到类 A 的最前面，一切问题就迎刃而解：

```
class A
{
    typedef int mytype;
    ...
};
```

当然，对于全局的 myfunc 函数的形参类型，依旧会解释成 string 类型，因为是全局函数，不可能解释成类 A 内的 int 类型。

4.2　进程内存空间布局

因为本章讲的是数据语义学，这就意味着将更多地涉及数据。数据在内存中总要有保存位置，而且不同的数据有不同的保存时机和不同的保存位置，所以，有必要先学习一下进程的内存空间布局，看看内存是分几块，各种不同的数据可能会保存在这些内存空间中的什么位置。这就是本节的学习目的。

当把一个**可执行文件加载到内存**后，就变成了一个进程。进程的虚拟地址空间一般如图 4.1 所示。

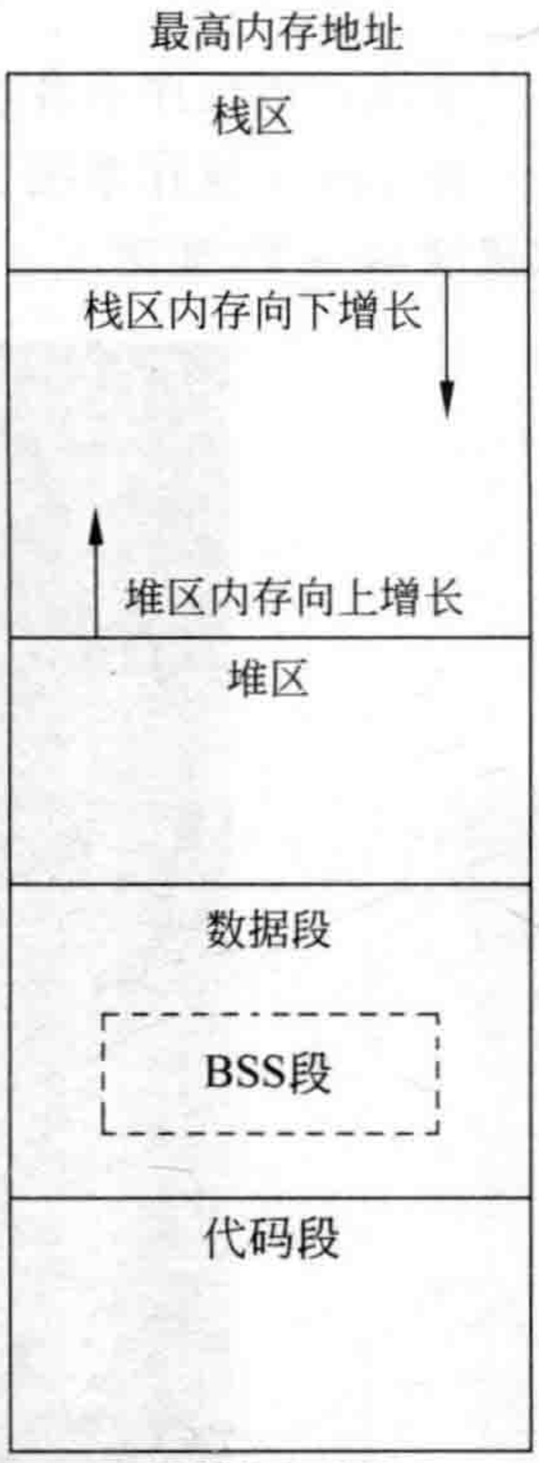

图 4.1　进程的虚拟地址空间

这个虚拟空间(内存)大概分成以下几部分：

(1) 栈(堆栈/栈区)：局部变量等放在这里。

(2) 堆(堆区)：new、malloc 等申请的内存空间从这里分配。

(3) BSS 段：未初始化的全局变量，初始化为 0 的全局变量等放在这里。

(4) 数据段：已初始化的全局变量。从图 4.1 可以看到，BSS 段其实是被包含到数据段里的。

(5) 代码段：存放程序执行代码的一块内存区域。

进程的虚拟空间大概就分这 5 个部分，其中所保存的内容许多资料上都有介绍，介绍的内容也都大同小异。

那么，一个可执行文件，**没有加载到内存的时候，其实也是分很多部分，如也有数据段、BSS 段等**。

可以写一段程序，验证一下。在 MyProject.cpp 的上面，增加如下代码行：

```
int * ptest = new int(120);
int g1;
int g2;

int g3 = 12;
int g4 = 32;
int g5;
int g6 = 0;
static int g7;
static int g8 = 0;
static int g9 = 10;
void mygfunc()
{
    return;
}
//定义一个类
class MYACLS
{
public:
```

```
    int m_i;
    static int m_si;                    //这是声明不是定义
    int m_j;
    static int m_sj;
    int m_k;
    static int m_sk;
};
int MYACLS::m_sj = 12;                  //这才是定义
```

在 main 主函数中，增加如下代码：

```
int e_mytest;                           //该变量在程序运行时临时放在栈中
e_mytest = 9;
```

上面这段程序笔者也在 Linux 操作系统上编译、连接并生成了可执行文件。

在 Linux 操作系统下有一个 nm 命令，能够**列出可执行文件中全局变量等存放的位置**。这里演示一下，如图 4.2 所示。

```
kuangxiang@bogon:/mnt/hgfs/linux$ g++ MyProject.cpp -o MyProject
kuangxiang@bogon:/mnt/hgfs/linux$ nm MyProject
0000000000601060 B __bss_start
0000000000601060 b completed.7594
                 U __cxa_atexit@@GLIBC_2.2.5
0000000000601040 D __data_start
0000000000601040 W data_start
0000000000400640 t deregister_tm_clones
00000000004006c0 t __do_global_dtors_aux
0000000000600e08 t __do_global_dtors_aux_fini_array_entry
0000000000601048 D __dso_handle
0000000000600e18 d _DYNAMIC
0000000000601060 D _edata
0000000000601090 B _end
0000000000400804 T _fini
00000000004006e0 t frame_dummy
0000000000600df8 t __frame_dummy_init_array_entry
00000000004009b0 r __FRAME_END__
0000000000601070 B g1
0000000000601074 B g2
0000000000601050 D g3
0000000000601054 D g4
0000000000601078 B g5
000000000060107c B g6
0000000000601000 d _GLOBAL_OFFSET_TABLE_
0000000000400774 t _GLOBAL__sub_I_ptest
                 w __gmon_start__
0000000000400814 r __GNU_EH_FRAME_HDR
0000000000400578 T _init
0000000000600e08 t __init_array_end
0000000000600df8 t __init_array_start
0000000000400810 R _IO_stdin_used
                 w _ITM_deregisterTMCloneTable
                 w _ITM_registerTMCloneTable
0000000000600e10 d __JCR_END__
0000000000600e10 d __JCR_LIST__
                 w _Jv_RegisterClasses
0000000000400800 T __libc_csu_fini
0000000000400790 T __libc_csu_init
                 U __libc_start_main@@GLIBC_2.2.5
000000000040070d T main
0000000000601068 B ptest
0000000000400680 t register_tm_clones
0000000000400610 T _start
0000000000601060 D __TMC_END__
000000000040071f t _Z41__static_initialization_and_destruction_0ii
0000000000400706 T _Z7mygfuncv
0000000000601084 b _ZL2g7
0000000000601088 b _ZL2g8
0000000000601058 d _ZL2g9
000000000060105c D _ZN6MYACLS4m_sjE
                 U _ZNSt8ios_base4InitC1Ev@@GLIBCXX_3.4
                 U _ZNSt8ios_base4InitD1Ev@@GLIBCXX_3.4
                 U _Znwm@@GLIBCXX_3.4
0000000000601080 b _ZStL8__ioinit
```

图 4.2　可执行文件中全局变量存放的地址信息

从图 4.2 中可以看到 bss 段、data(数据)段在可执行文件中的开始位置(地址)、一些全局变量的地址，甚至类的静态成员变量也在里面，还有全局函数的入口地址等。

在 Windows 操作系统上要查看一个可执行文件(.exe)的格式需要专门的工具，这里就不过多研究，虽然 Windows 操作系统和 Linux 操作系统上可执行文件格式有一定差异，但是大同小异，意思都差不多。

再回到 Windows 操作系统下的 Visual Studio 2019 中来，做一些测试。在 main 主函数中增加如下代码：

```
printf("ptest 地址 = %p\n", &ptest);
printf("g1 地址 = %p\n", &g1);
printf("g2 地址 = %p\n", &g2);
printf("g3 地址 = %p\n", &g3);
printf("g4 地址 = %p\n", &g4);
printf("g5 地址 = %p\n", &g5);
printf("g6 地址 = %p\n", &g6);
printf("g7 地址 = %p\n", &g7);
printf("g8 地址 = %p\n", &g8);
printf("g9 地址 = %p\n", &g9);
printf("MYACLS::m_sj 地址 = %p\n", &(MYACLS::m_sj));
printf("mygfunc()地址 = %p\n", mygfunc);
printf("main()地址 = %p\n", main);
```

执行起来，看一看结果：

```
ptest 地址 = 0092B2F8
g1 地址 = 0092B2E8
g2 地址 = 0092B2EC
g3 地址 = 0092B000
g4 地址 = 0092B004
g5 地址 = 0092B2F0
g6 地址 = 0092B2F4
g7 地址 = 0092B2FC
g8 地址 = 0092B300
g9 地址 = 0092B00C
MYACLS::m_sj 地址 = 0092B008
mygfunc()地址 = 00921410
main()地址 = 00921316
```

多次执行，发现每次输出的地址值都相同。同时，注意观察地址数字，其中，g1、g2、g5、g6、g7、g8 的地址值离的很近(看起来是：没给初值或者初值为 0 的全局量聚集在一起)，而 g3、g4、g9 的地址值离的很近(看起来是：给了初值但初值不为 0 的全局量聚集在一起)。

上述这些输出内容表明，一些全局量(全局变量、全局函数、类静态成员变量等)的地址值在生成可执行文件时就已经是确定好的，不会发生改变，它们的存放位置通常在 bss 段、数据段等，一旦加载(映射)到内存时这些地址值都不发生变化。

如果在 main 主函数中定义一个局部变量，例如：

```
int i = 7;
printf("i 地址 = %p\n", &i);
```

每次程序执行时，这种局部变量 i 的内存都会从栈中分配，而且，每次执行程序时，变量 i 的地址都会发生变化。

4.3 数据成员布局

4.3.1 观察成员变量地址规律

看看如下范例。在 MyProject.cpp 的上面，增加如下代码行：

```
class MYACLS
{
public:
    int m_i;
    static int m_si;                    //这是声明不是定义
    int m_j;
    static int m_sj;
    int m_k;
    static int m_sk;
};
int MYACLS::m_sj = 0;                   //这才是定义
```

在 main 主函数中，加入如下代码：

```
MYACLS myobj;
cout << sizeof(myobj) << endl;          //12
```

有几点说明：

(1) sizeof(myobj)在这里应该是 12，因为整型成员变量有 3 个，每个 4 字节，3 个一共 12 字节，而静态成员变量是跟着类走的，所以，不占类对象的内存空间(或者说和对象无关，不属于对象布局的一部分)。实际上这些静态成员变量是存储在整个程序的数据段中的，4.2 节已经详细讲解过，也看到了静态成员变量存放在其中。

(2) 在类对象的内存布局中，这些普通成员变量的存储顺序是按照在类中定义的顺序从上到下来的。在 main 主函数中增加如下代码，并查看 myobja 对象内存中数据排列如图 4.3 所示。

```
myobj.m_i = 2;
myobj.m_j = 5;
myobj.m_k = 8;
```

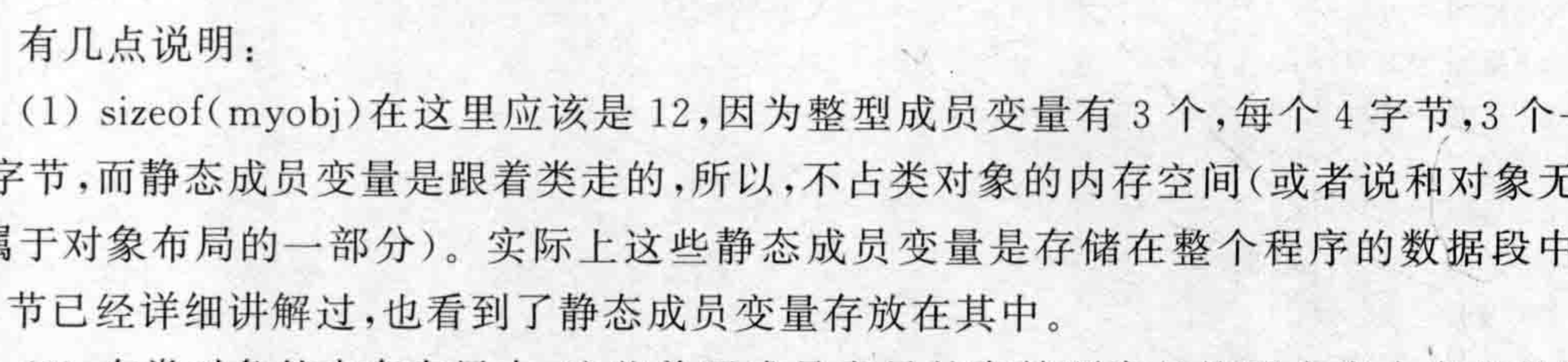

图 4.3 类中普通成员变量的存储顺序是按照定义顺序来的

在图 4.3 中，注意内存中存储的数据都是按照成员变量的定义顺序来存储的：02000000（代表 m_i）、05000000（代表 m_j）、08000000（代表 m_k）。

（3）输出一下变量的内存地址，观察一下这些地址的规律。注释掉 main 主函数中原有的代码，加入新代码用来分别在栈和堆上创建对象：

```
MYACLS myobj;
myobj.m_i = myobj.m_j = myobj.m_k = 0;
cout << sizeof(myobj) << endl;
printf("myobj.m_i =  %p\n", &myobj.m_i);
printf("myobj.m_j =  %p\n", &myobj.m_j);
printf("myobj.m_k =  %p\n", &myobj.m_k);

MYACLS * pmyobj = new MYACLS();
printf("pmyobj->m_i =  %p\n", &pmyobj->m_i);
printf("pmyobj->m_j =  %p\n", &pmyobj->m_j);
printf("pmyobj->m_k =  %p\n", &pmyobj->m_k);
```

执行起来，看一看结果：

```
12
myobj.m_i = 00EFF7F8
myobj.m_j = 00EFF7FC
myobj.m_k = 00EFF800
pmyobj->m_i = 00F9DFD8
pmyobj->m_j = 00F9DFDC
pmyobj->m_k = 00F9DFE0
```

不难看到，类对象所占的内存当然是一块连续的内存。上述代码中，myobj 代表的是在栈上分配的一块内存，而 pmyobj 所指向的是在堆中分配的一块内存。myobj 的三个成员变量的内存地址紧挨着，都间隔 4；pmyobj 所指向的对象的三个成员变量的内存地址也紧挨着，也都间隔 4。而且，这些地址也有规律：比较晚出现的成员变量在内存中有更大的地址值，所以，对象所属成员变量 m_i 的地址值肯定没有 m_k 的地址值大。

另外还要注意，一个类或者类对象的 sizeof 值与在类定义里面有几个 public、private、protected 没有什么关系。多个 public、private、protected 对程序性能也没什么额外的影响。

4.3.2　边界调整与字节对齐

在类 MYACLS 中，现在看到的情况是，每个成员变量正好都占 4 字节，并且是连续排列的。

那么，有些因素会导致成员之间排列不连续，也就是说，成员的边界会进行调整。边界调整就是往成员变量之间填补一些字节，使类对象的 sizeof 值凑成 4 或 8 的整数倍。边界调整的主要目的就是提高程序执行效率，而且编译器会自动去做，不需要程序员干预。

在类 MYACLS 中的最后面，继续增加两个成员变量：

```
public:
    char m_c;                    //1 字节
    int m_n;                     //4 字节
```

在 main 主函数中也额外增加代码，打印出这两个成员变量的地址：

```
printf("myobj.m_c =  %p\n", &myobj.m_c);
printf("myobj.m_n =  %p\n", &myobj.m_n);
...
printf("pmyobj->m_c =  %p\n", &pmyobj->m_c);
printf("pmyobj->m_n =  %p\n", &pmyobj->m_n);
```

执行起来，看一看结果：

```
20
myobj.m_i = 0073FD48
myobj.m_j = 0073FD4C
myobj.m_k = 0073FD50
myobj.m_c = 0073FD54
myobj.m_n = 0073FD58
pmyobj->m_i = 00865780
pmyobj->m_j = 00865784
pmyobj->m_k = 00865788
pmyobj->m_c = 0086578C
pmyobj->m_n = 00865790
```

观察一下结果，不难得到一些结论：类成员变量定义中间要是有这种 1 字节的成员，编译器可能就要往 4 字节凑，等等。不同的编译器，可能凑的字节数还不一样。

做网络编程的读者可能体会更深，编译器不统一是很麻烦的事，有可能某个类在这个编译器上的 sizeof 值是 20 字节，跑到另外一个编译器上 sizeof 值就是 24 字节。所以，必须要保证该类的 sizeof 值无论在哪个编译器上都必须一样，不然没有办法正常通信（传输同一个类对象在两个计算机上的长度却不同）。

所以就引入了 1 字节对齐（也叫“不对齐”）的概念。所谓 1 字节对齐，就是针对该类（结构），要求其各个成员之间紧密排列，不允许做边界调整，不允许在成员之间留任何空隙。

如果需要将某个类按 1 字节对齐，则在该类定义之前，使用如下代码行：

```
#pragma pack (1)                    //1 字节对齐方式
```

接着，进行类的定义。定义完需要 1 字节对齐的 1 个或者多个类之后，使用如下代码行来结束 1 字节对齐方式：

```
#pragma pack() //取消指定的 1 字节对齐，恢复默认对齐（原来是多少就恢复成多少）
```

现在，修改一下代码，用 #pragma pack 将类 MYACLS 的定义包起来。修改后的代码大概如下：

```
#pragma pack (1)
class MYACLS
{
    ...
};
#pragma pack()
```

执行起来，看一看结果：

```
17
myobj.m_i = 00B8F79C
myobj.m_j = 00B8F7A0
myobj.m_k = 00B8F7A4
myobj.m_c = 00B8F7A8
myobj.m_n = 00B8F7A9
pmyobj->m_i = 00E04EC8
pmyobj->m_j = 00E04ECC
pmyobj->m_k = 00E04ED0
pmyobj->m_c = 00E04ED4
pmyobj->m_n = 00E04ED5
```

从结果中不难看到，sizeof 值从 20 变成 17 了，而且原来结果中，m_c 与 m_n 成员变量的地址间隔是 4 字节，现在采用 1 字节对齐后，m_c 与 m_n 成员变量的地址间隔是 1 字节了（紧密挨在一起）。

4.3.3　成员变量偏移值的打印

成员变量偏移值，就是指这个成员变量的地址离对象首地址偏移了多少字节。

在类 MYACLS 的最后，增加两个 private 修饰的成员变量的定义：

```
private:
    int m_pria;
    int m_prib;
```

接着，再增加一个 public 修饰的成员函数：

```
public:
    void printMemPoint() //因为要打印私有的成员变量偏移值，只能用成员函数打印
    {
        cout << "打印成员变量偏移值 ----------------" << endl;
        printf("MYACLS::m_i =  %d\n", &MYACLS::m_i); //类名不要省略，必须要有，打印的才是偏移值，否则打印的就是对象的该成员变量的物理地址
        printf("MYACLS::m_j =  %d\n", &MYACLS::m_j);
        printf("MYACLS::m_k =  %d\n", &MYACLS::m_k);
        printf("MYACLS::m_c =  %d\n", &MYACLS::m_c);
        printf("MYACLS::m_n =  %d\n", &MYACLS::m_n);

        printf("MYACLS::m_pria =  %p\n", &MYACLS::m_pria);
        printf("MYACLS::m_prib =  %p\n", &MYACLS::m_prib);
        cout << "----------------------------" << endl;
    }
```

在 main 主函数中，增加如下代码行：

```
pmyobj->printMemPoint();
```

执行起来，看一看新增加的代码行输出的结果：

```
打印成员变量偏移值----------------
MYACLS::m_i = 0
MYACLS::m_j = 4
MYACLS::m_k = 8
MYACLS::m_c = 12
MYACLS::m_n = 13
MYACLS::m_pria = 17
MYACLS::m_prib = 21
------------------------
```

从结果可以看到，因为 m_i 在对象地址空间的最上面，所以该成员变量的地址离对象首地址偏移了 0 字节(对象的首地址就是 m_i 成员变量的地址)；向后移动 4 字节，就是 m_j 成员变量的首地址，以此类推。

打印这种偏移值还有一种比较奇异的写法，在 MyProject.cpp 的最上面增加一个叫作 GET 的宏定义：

```
#define GET(A,m) (int)(&((A*)0)->m)   //A 代表类名，m 代表成员名，这里用了一个 0，违反常规
                                      //的写法，但能输出一个类成员变量的偏移值
```

然后，如果想打印 m_prib 的偏移值，可以这样写，在 printMemPoint 成员函数的末尾，增加如下语句行：

```
cout << "MYACLS::m_prib = " << GET(MYACLS, m_prib) << endl; //利用 GET 宏定义来打印 m_prib 成
员变量的偏移值
```

执行起来，看一看新增加的代码行输出的结果：

```
MYACLS::m_prib = 21
```

现在，笔者往类 MYACLS 定义的末尾，增加一个 public 修饰的**虚函数**的定义：

```
public:
    virtual void mfv() {}
```

因为虚函数的存在，编译器会为 MYACLS 类增加虚函数表指针 vptr 这种隐藏的成员变量。这个成员变量当前占 4 字节，并且增加在类定义的开头位置。这样看来，上面的所有成员变量的偏移值都会往下移动 4 字节，而且，整个类的 sizeof 也会多出 4 字节。

执行起来，看一看结果：

```
29
myobj.m_i = 0098F8B4
myobj.m_j = 0098F8B8
myobj.m_k = 0098F8BC
myobj.m_c = 0098F8C0
myobj.m_n = 0098F8C1
pmyobj->m_i = 00BA515C
pmyobj->m_j = 00BA5160
```

```
pmyobj->m_k = 00BA5164
pmyobj->m_c = 00BA5168
pmyobj->m_n = 00BA5169
打印成员变量偏移值----------------
MYACLS::m_i = 4
MYACLS::m_j = 8
MYACLS::m_k = 12
MYACLS::m_c = 16
MYACLS::m_n = 17
MYACLS::m_pria = 21
MYACLS::m_prib = 25
-------------------------
MYACLS::m_prib = 25
```

根据上面的结果，可以看到，myobj 对象的 sizeof 值是 29(没增加虚函数之前是 25)字节，而且 MYACLS::m_i 的偏移值已经是 4 了。

实际上，用**成员变量指针**也能打印成员变量的偏移值。在 main 主函数中，增加如下代码：

```
int MYACLS::*mypoint = &MYACLS::m_n;
printf("pmyobj->m_n 偏移地址 = %d\n", mypoint);
```

执行起来，看一看新增加的代码行输出的结果：

```
pmyobj->m_n 偏移地址 = 17
```

4.4　数据成员的存取

4.4.1　静态成员变量的存取

看看如下范例。在 MyProject.cpp 的上面，增加如下代码行：

```
class MYACLS
{
public:
    int m_i;
    static int m_si;                    //这是声明不是定义
    int m_j;
};
int MYACLS::m_si = 0;                   //这才是定义
```

其实，静态成员变量可以当作一个全局变量。但是它只有在类的空间内可见，也就是说，引用它的时候，一般需要带上“类名::”。

另外，通过 4.2 节的学习，读者已经知道，静态成员变量是保存在可执行文件的数据段的。

静态成员变量的引用，可以用类名，也可以用对象名，还可以用对象指针，用类名则后面

要接“::”，用对象名后边要接“.”，用对象指针后边要接“->”。读者可以在 main 主函数中增加如下代码试一试：

```
MYACLS myobj;
MYACLS * pmyobj = new MYACLS();
cout << MYACLS::m_si << endl;
cout << myobj.m_si << endl;
cout << pmyobj->m_si << endl;
```

一般来说，“.”或者“->”这种语法都是访问类对象中的成员变量，但静态成员变量压根就不在对象里面保存(而是在数据段里放着呢)，也就是说，存取静态成员变量压根就不需要用到类对象或者类对象指针，所以这里面的这种“.”或者“->”语法只能代表这是一种语法上允许的取巧写法而已。继续在 main 主函数中增加如下代码：

```
MYACLS::m_si = 1;
myobj.m_si = 2;
pmyobj->m_si = 3;
```

设置断点于上面三行代码中的最后一行并进行调试，当程序执行流程停到断点行后，切换到反汇编窗口，如图 4.4 所示。

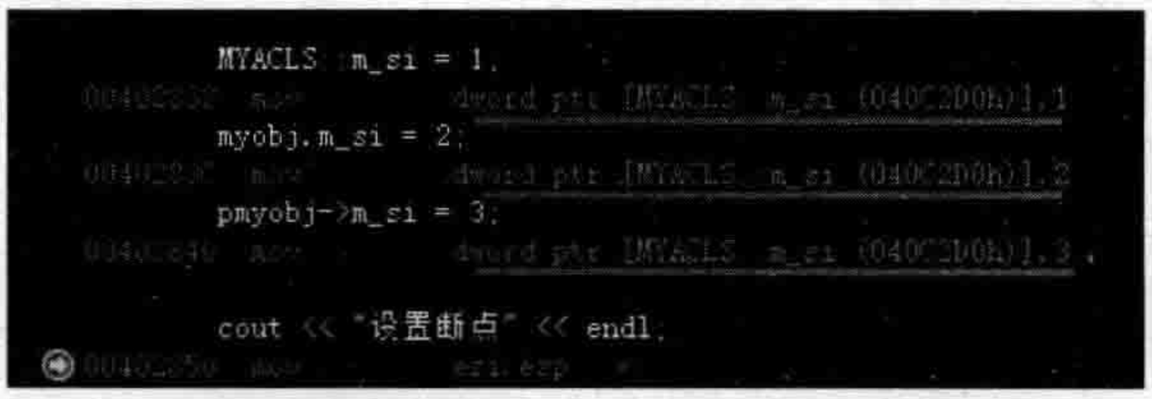

图 4.4　多种访问静态成员变量的方法效果都相同

图 4.4 表明多种给类中静态成员变量赋值的语法，对应的汇编代码都是完全一样的。输出一下成员变量的地址。在 main 主函数中继续增加如下代码：

```
printf("myobj.m_i =  %p\n", &myobj.m_i);
printf("pmyobj->m_i =  %p\n", &pmyobj->m_i);
printf("MYACLS::m_si =  %p\n", &MYACLS::m_si);
printf("MYACLS::m_si =  %p\n", &myobj.m_si);
printf("MYACLS::m_si =  %p\n", &pmyobj->m_si);
```

执行起来，看一看新增加的代码行输出的结果，这里可以多次执行，注意观察：

```
myobj.m_i = 008FFB0C
pmyobj->m_i = 00A4DBE0
MYACLS::m_si = 0016C2D0
MYACLS::m_si = 0016C2D0
MYACLS::m_si = 0016C2D0
```

多次执行可以发现，后面三行的结果始终不发生改变，因为这三行显示的地址实际是写在可执行文件中的。

上面这段程序笔者也在 Linux 操作系统上编译、连接并生成了可执行文件。执行起来，

看一看结果，如图 4.5 所示。

```
kuangxiang@bogon:/mnt/hgfs/linux$ g++ MyProject.cpp -o MyProject
kuangxiang@bogon:/mnt/hgfs/linux$ ./MyProject
0
0
0
myobj.m_i = 0x7ffcbc515480
pmyobj->m_i = 0x83dc20
MYACLS::m_si = 0x601194
MYACLS::m_si = 0x601194
MYACLS::m_si = 0x601194
kuangxiang@bogon:/mnt/hgfs/linux$ ./MyProject
0
0
0
myobj.m_i = 0x7ffdb4d75680
pmyobj->m_i = 0x1b58c20
MYACLS::m_si = 0x601194
MYACLS::m_si = 0x601194
MYACLS::m_si = 0x601194
kuangxiang@bogon:/mnt/hgfs/linux$
```

图 4.5　在 Linux 操作系统下多次执行程序静态成员变量的地址都不会发生变化

利用 nm 命令，看一看可执行文件中这个静态成员变量的地址。输入 nm MyProject，结果如图 4.6 所示(截取部分结果)。

```
0000000000400820 T _start
0000000000601078 D __TMC_END__
0000000000400a46 t _Z41__static_initialization_and_destruction_0ii
0000000000601194 B _ZN6MYACLS4m_siE
                 U _ZNSolsEi@@GLIBCXX_3.4
                 U _ZNSolsEPFRSoS_E@@GLIBCXX_3.4
                 U _ZNSt8ios_base4InitC1Ev@@GLIBCXX_3.4
                 U _ZNSt8ios_base4InitD1Ev@@GLIBCXX_3.4
                 U _Znwm@@GLIBCXX_3.4
```

图 4.6　可执行文件中静态成员变量存放的地址信息

从图 4.6 所示的结果中可以清晰地看到，静态成员变量 m_si 地址就是 601194(十六进制)，所以说，这个静态成员变量地址实际是在可执行文件中就确定了的。

同时也看到了，编译器会自动地将类名和静态成员变量名结合到一起构成一个新的名字(_ZN6MYACLS4m_siE)。这样，如果还有一个类里面也有一个同名的静态成员变量也没关系，通过不同的类名依旧可以区分。

4.4.2　非静态成员变量的存取

非静态成员变量也就是普通成员变量，是存放在类对象中的，需要通过类对象来存取。

在类 MYACLS 中，增加一个 public 修饰的普通成员函数，在其中对成员变量进行一下赋值操作：

```
public:
    void myfunc()
    {
        m_i = 5;
        m_j = 6;
    }
```

在 main 主函数中，继续增加如下代码来调用 myfunc 成员函数：

```
pmyobj->myfunc();
```

在程序员的眼中，看到的是在 myfunc 成员函数里直接给成员变量 m_i、m_j 赋值，但是，站在编译器的角度来看，**编译器是向 myfunc 成员函数中插入了一个隐式的形参 this，这个 this 就表示对象本身**。编译器视角应该是这样的：

```
void MYACLS::myfunc(MYACLS * const this) //注意 const 位置，表示 this 这个参数指向一个对象后不可以指向其他对象
{
    this->m_i = 5;
    this->m_j = 6;
}
```

另外一个笔者要谈的事情就是对非静态成员变量的访问问题。当要访问一个成员变量（寻找一个成员变量地址）时，编译器是把类对象的**首地址**加上该要访问的成员变量的**偏移值**（偏移值概念 4.3.3 小节已讲解过），如图 4.7 所示。

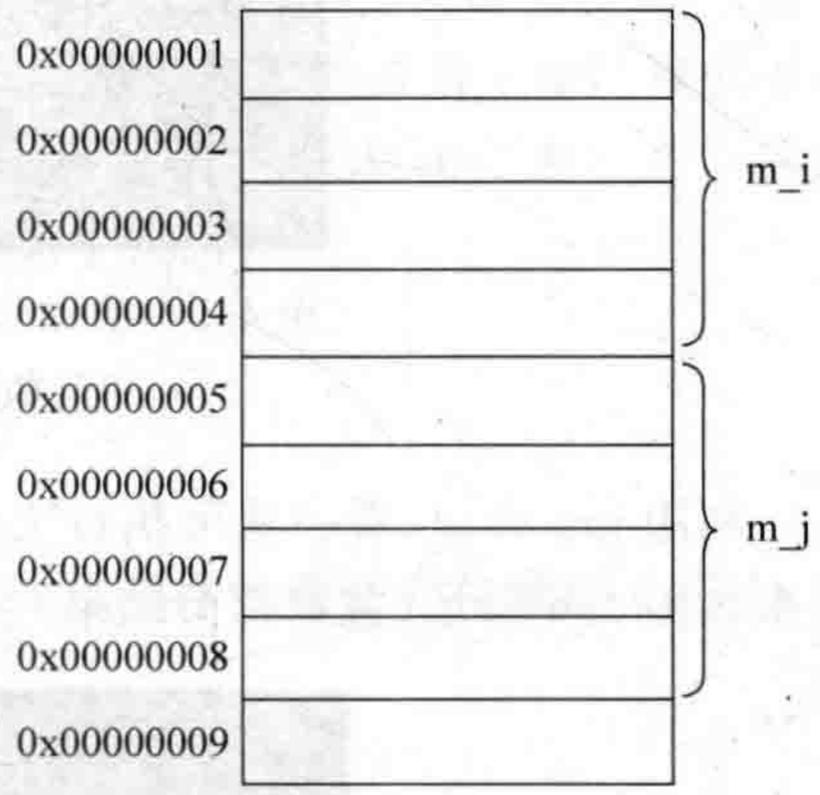

图 4.7　成员变量 m_i、m_j 所占的内存空间示意图

根据图 4.7，想象一下，m_j 的偏移值应该是 4（&MYACLS::m_j），那么 m_j 的地址怎样推算出来？应该是对象的首地址加 4（形如：&myobj + 4）。

设置断点于 MYACLS 类 myfunc 成员函数中任意一行并进行调试，当程序执行流程停到断点行后，切换到反汇编窗口，可以看到如图 4.8 所示。

图 4.8　反汇编窗口下可以看到系统是利用偏移值来找到成员变量地址的

从图 4.8 可以看到，this 代表的对象首地址就是 m_i 成员变量的地址，而 this+4（eax+4），代表的就是 m_j 成员变量的地址。这个偏移值数字 4，是在汇编代码中直接出现的，说明成员变量的偏移值是在编译的时候就已经计算好的，所以存取的效率会很高。

这里再把 m_i 和 m_j 成员变量的偏移值输出一下方便观察。在 main 主函数中增加如下代码：

```
printf("MYACLS::m_i =  %d\n", &MYACLS::m_i); //0
printf("MYACLS::m_j =  %d\n", &MYACLS::m_j); //4
```

执行起来，看一看新增加的两行代码输出的结果：

```
MYACLS::m_i = 0
MYACLS::m_j = 4
```

现在，思考另外一个问题。在 MyProject.cpp 中类 MYACLS 定义的更上面位置，引入

一个新的类 FAC：

```
class FAC
{
public:
    int m_fai;
    int m_faj;
};
```

接着，让 MYACLS 类继承自 FAC 类：

```
class MYACLS : public FAC
{
    ...
};
```

在 MYACLS 类继承 FAC 类之前，MYACLS 类的 m_i 和 m_j 的偏移值是 0 和 4，那么继承之后呢？

执行起来，看一看最后两行计算 m_i 和 m_j 偏移值的语句输出的结果：

```
MYACLS::m_i = 8
MYACLS::m_j = 12
```

从结果可以看到，m_i 和 m_j 的偏移值从 0 和 4，变成了 8 和 12。

设置断点于 MYACLS 类 myfunc 成员函数中任意一行并进行调试，当程序执行流程停到断点行后，切换到反汇编窗口，可以看到如图 4.9 所示。

图 4.9　反汇编窗口下可以看到系统是利用偏移值来找到成员变量地址的

看图 4.9 可以发现，即便有继承关系，这种偏移值也是编译的时候就计算好的（如图中的 eax＋8、eax＋0Ch）。

在 main 主函数中继续增加如下代码行：

```
pmyobj->m_faj = 7;                    //给父类的成员变量赋值
```

设置断点于该行并进行调试，当程序执行流程停到断点行后，切换到反汇编窗口，可以看到如图 4.10 所示。

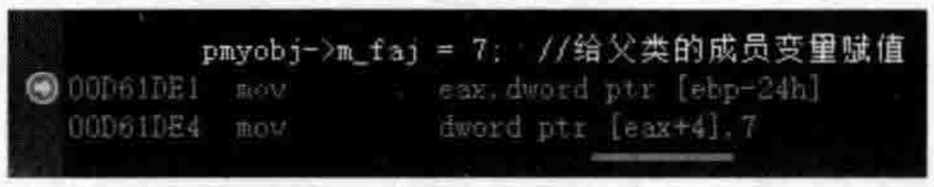

图 4.10　反汇编窗口下看一看父类成员变量的偏移值

所以，这种普通的成员变量，不管只有单独一个类还是单一继承，或者多重继承，不管访问父类的还是子类的成员变量，效率上都一样。

当然，例外的情况是**虚基类**。虚基类中的成员变量的访问，需要一些间接手段，所以这种情况下访问普通成员变量的效率会慢一些，后续会详细讲解虚基类。

在讲解虚基类之前，笔者先抛出一个问题。当前这个范例中，定义了 myobj 对象和 pmyobj 对象指针(MYACLS myobj;和 MYACLS * pmyobj = new MYACLS();)。不知读者是否想过，myobj. m_i 和 pmyobj-> m_i 这两种访问成员变量的方法，一个是通过对象名，一个是通过对象指针，这两者有什么差别吗？

常规情况下没什么差别，刚才通过看反汇编代码也感受到了。但如果是一个**虚基类**，也就是有爷爷类、父亲类、孙子类，孙子类对象要访问爷爷类中的普通成员变量的话，就会**有差别**了。这个差别跟虚函数有点类似：

(1) myobj 这种对象名访问成员变量的方式是在编译期间就计算出来成员变量的偏移值。

(2) pmyobj 这种对象指针是在执行期间才 new 出来一个具体的对象，而且也不知道 new 的是哪种类型(父、子、孙)的对象，无法在编译期间计算成员变量的偏移值，所以需要间接手段来访问成员变量。讲解虚基类时细讲，先有个大概印象即可。

4.5 单一继承下的数据成员布局

看看如下范例。在 MyProject. cpp 的上面，增加如下代码行：

```
class FAC                                    //父类
{
public:
    int m_fai;
    int m_faj;
};
class MYACLS : public FAC                    //子类
{
public:
    int m_i;
    int m_j;
};
```

在 main 主函数中，增加如下代码：

```
printf("FAC::m_fai =  %d\n", &FAC::m_fai);
printf("FAC::m_faj =  %d\n", &FAC::m_faj);
printf("MYACLS::m_fai =  %d\n", &MYACLS::m_fai);
printf("MYACLS::m_faj =  %d\n", &MYACLS::m_faj);
printf("MYACLS::m_i =  %d\n", &MYACLS::m_i);
printf("MYACLS::m_j =  %d\n", &MYACLS::m_j);
```

执行起来，看一看结果：

```
FAC::m_fai = 0
FAC::m_faj = 4
MYACLS::m_fai = 0
MYACLS::m_faj = 4
MYACLS::m_i = 8
MYACLS::m_j = 12
```

观察上面的结果，看一看偏移值，尤其要注意的是第三行和第四行 printf 语句，笔者用的是子类名，而后面的成员变量名是在父类中定义的。既然这个语法行得通，说明两个含义：

(1) 一个子类对象所包含的内容，是它自己的成员加上它父亲成员的总和。

(2) 从偏移值看，在当前的编译器中，父亲成员是先出现的，然后才是子类成员。那么有没有编译器非要倒着排，让子类成员先出现，父亲成员后出现？暂时还没发现。(虚基类除外，虚基类后续专门探讨)，先探讨常规状态。

在 main 主函数中增加如下代码来定义一个 FAC 类对象：

```
FAC facobj;
```

那么，对象 facobj 的数据布局(内存布局)看起来应该如图 4.11 所示。

在 main 主函数中增加如下代码来定义一个子类 MYACLS 对象：

```
MYACLS myaclobj;
```

那么，对象 myaclobj 的数据布局看起来应该如图 4.12 所示。

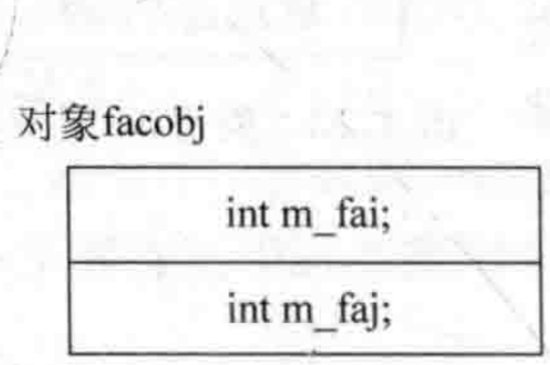

图 4.11　对象 facobj 的数据布局

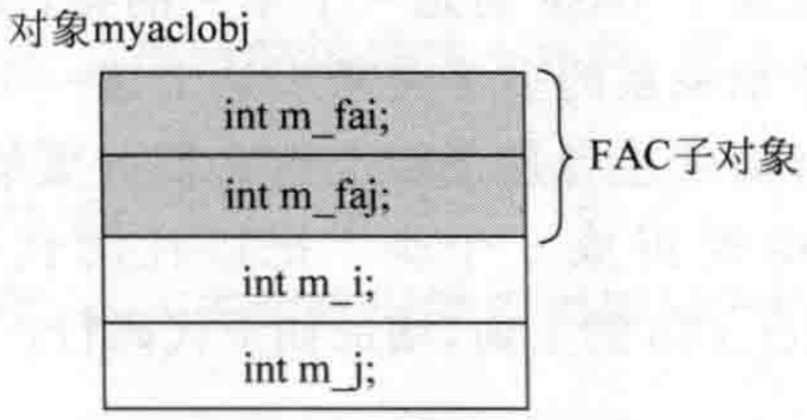

图 4.12　对象 myaclobj 的数据布局

上面的范例还是比较好理解的。现在再看一个稍微特殊一点的范例。把以往的代码先注释掉，在 MyProject.cpp 的上面，增加如下代码行：

```
class Base
{
public:
    int m_i1;
    char m_c1;
    char m_c2;
    char m_c3;
};
```

在 main 主函数中，增加如下代码：

```
cout << sizeof(Base) << endl;
```

输出一下结果，是一个数字 8。因为整个类的成员变量实际上只有 7 字节，那么 sizeof 是 8，根据 4.3.2 节讲解的边界调整与字节对齐知识，可以认为这多出来的 1 字节是边界调整的结果。

在 main 主函数中继续增加代码，输出偏移值具体看一下：

```
printf("Base::m_i1 =  %d\n", &Base::m_i1);
```

```
printf("Base::m_c1 = %d\n", &Base::m_c1);
printf("Base::m_c2 = %d\n", &Base::m_c2);
printf("Base::m_c3 = %d\n", &Base::m_c3);
```

执行起来，看一看结果：

```
8
Base::m_i1 = 0
Base::m_c1 = 4
Base::m_c2 = 5
Base::m_c3 = 6
```

笔者在 Linux 操作系统上也测试过，跟上面的结果完全一样。根据上面的结果，不难绘制出一个类 Base 对象的数据布局图，如图 4.13 所示。

图 4.13 总结起来就是：m_i1 占 4 字节，m_c1、m_c2、m_c3 分别占 1 字节，填充 1 字节用于边界调整。

现在这个 Base 类是一个单一的类，没有继承关系，如果用带继承关系的多个类来表达上述一个类呢（可能因为某些业务需求或者是逻辑上这么拆分更加合理）？笔者准备把 Base 类拆成三个类。把以往的代码先注释掉，在 MyProject.cpp 的上面，增加如下代码行：

类Base对象

图 4.13　类 Base 对象数据布局

```
class Base1
{
public:
    int m_i1;
    char m_c1;
};

class Base2 :public Base1
{
public:
    char m_c2;
};

class Base3 :public Base2
{
public:
    char m_c3;
};
```

现在，Base3 这个子类包含类 Base1、Base2 里所有成员。所以，Base3 类和刚才的 Base 类差不多。但是，问题来了。在 main 主函数中，增加如下代码：

```
cout << sizeof(Base1) << endl; //8
cout << sizeof(Base2) << endl; //12
cout << sizeof(Base3) << endl; //16
```

执行起来，看一看结果：

```
8
12
16
```

因为最终等同于 Base 的类是 Base3 这个类，原来 Base 类的 sizeof 值是 8 字节，现在 Base3 类的 sizeof 值却是 16 字节。不难看到，这种**继承关系一旦引入进来，就多了 8 字节的内存空间占用**。

上面这段程序笔者也在 Linux 操作系统上编译、连接并生成了可执行文件。执行起来，看一看结果，如图 4.14 所示。

```
kuangxiang@bogon:/mnt/hgfs/linux$ g++ MyProject.cpp -o MyProject
kuangxiang@bogon:/mnt/hgfs/linux$ ./MyProject
8
12
12
kuangxiang@bogon:/mnt/hgfs/linux$
```

图 4.14　类 Base1、Base2、Base3 的 sizeof 值比较

观察图 4.14，发现结果和在 Visual Studio 2019 上得到的结果并不相同，图 4.14 中最后一个数字是 12。

不管 12 还是 16，子类的 sizeof 值都不会小于父类的 sizeof 值。

输出一下现在的 Base3 类对象的内存布局。在 main 主函数中继续增加如下输出偏移值的代码：

```
printf("Base3::m_i1 = %d\n", &Base3::m_i1);
printf("Base3::m_c1 = %d\n", &Base3::m_c1);
printf("Base3::m_c2 = %d\n", &Base3::m_c2);
printf("Base3::m_c3 = %d\n", &Base3::m_c3);
```

执行起来，看一看结果：

```
8
12
16
Base3::m_i1 = 0
Base3::m_c1 = 4
Base3::m_c2 = 8
Base3::m_c3 = 12
```

现在，根据这个结果，在 Windows 操作系统下的 Visual Studio 2019 环境中，分别绘制一下 Base1、Base2、Base3 这三个类对象的数据布局，如图 4.15～图 4.17 所示。

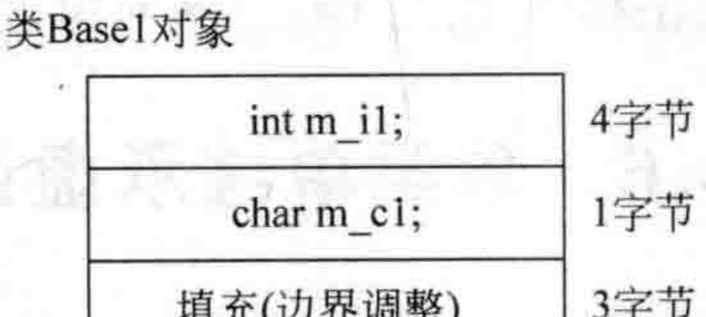

图 4.15　类 Base1 对象数据布局

在 Linux 操作系统上，也重新输出一下结果，如图 4.18 所示。

根据图 4.18，分别绘制一下在 Linux 操作系统下 Base1、Base2、Base3 这三个类对象的数据布局。其中，类

Base1 对象的数据布局和图 4.15 相同；类 Base2 对象的数据布局和图 4.16 相同；类 Base3 对象的布局与 Windows 操作系统下的 Visual Studio 2019 不同，如图 4.19 所示。

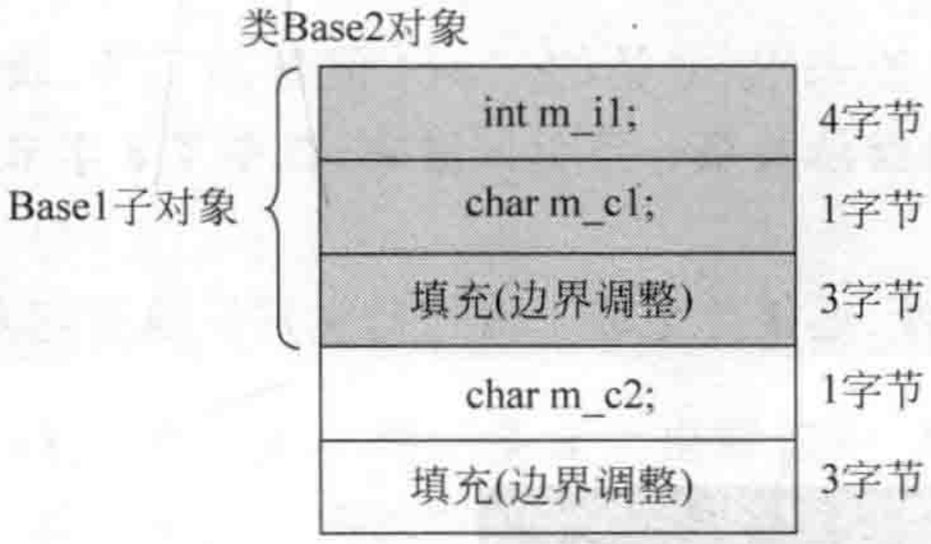

图 4.16 类 Base2 对象数据布局

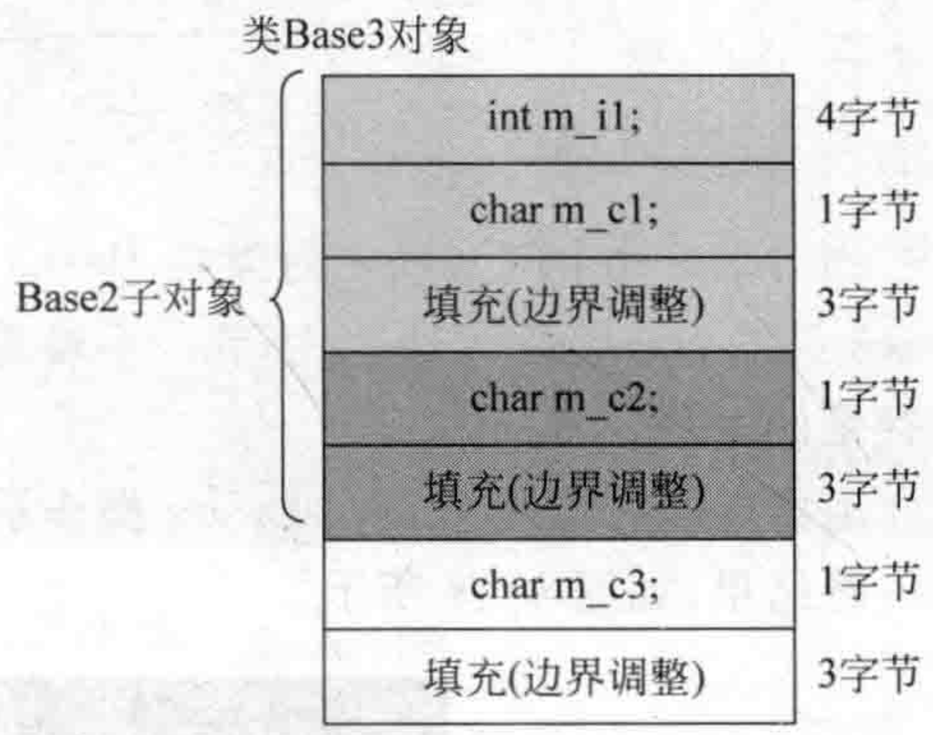

图 4.17 类 Base3 对象数据布局

```
kuangxiang@bogon:/mnt/hgfs/linux$ ./MyProject
8
12
12
Base3::m_i1 = 0
Base3::m_c1 = 4
Base3::m_c2 = 8
Base3::m_c3 = 9
kuangxiang@bogon:/mnt/hgfs/linux$
```

图 4.18 在 Linux 操作系统下类 Base1、Base2、Base3 对象的 sizeof 值和成员变量偏移值

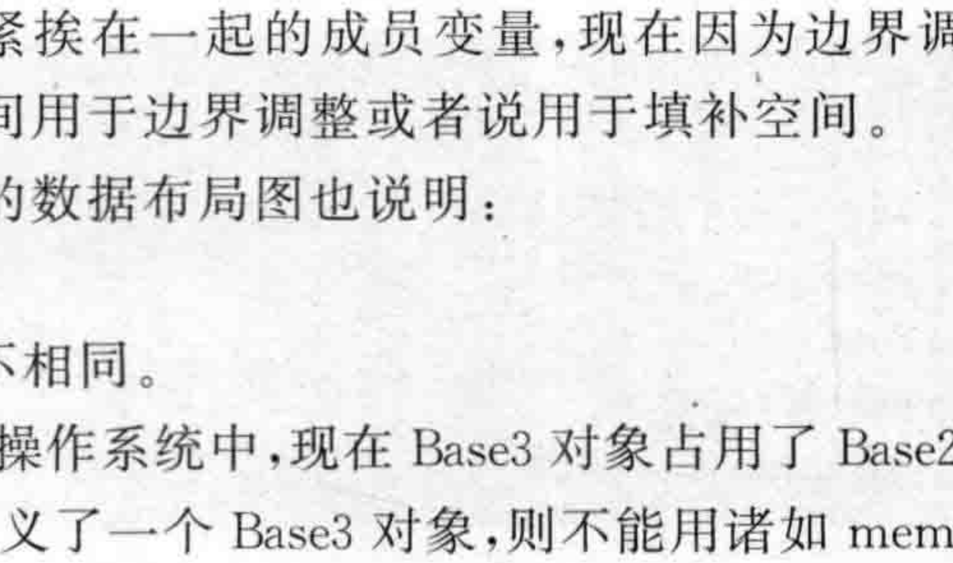

图 4.19 Linux 操作系统下类 Base3 对象数据布局

所以可以看到，继承关系引入后，原本内存紧挨在一起的成员变量，现在因为边界调整原因，不再挨在一起了，并且浪费了许多内存空间用于边界调整或者说用于填补空间。

另外，通过 Linux 和 Windows 操作系统上的数据布局图也说明：

(1) 编译器在不断进步和优化。

(2) 不同厂商的编译器实现的细节可能很不相同。

(3) 内存复制也必须要谨慎。例如在 Linux 操作系统中，现在 Base3 对象占用了 Base2 对象的布局空间，如果定义了一个 Base2 对象，又定义了一个 Base3 对象，则不能用诸如 memcpy 这种内存复制把 Base2 对象内存中的内容往 Base3 对象里复制。因为这两个对象虽然内存大小相同，但是 Base3 类对象多了一个 m_c3 成员，这个 Base2 类对象里并没有这个成员。

4.6 单类单继承虚函数下的数据成员布局

笔者进行的讲解一般都是以 Windows 操作系统下的 Visual Studio 2019 编译器为准(该编译器中解决方案平台选择的是 x86)，其他编译器可能会略有不同，但是，最重要的是

要读者掌握原理和方法，一旦这些对象模型的道理通了，就会一通百通，可以用与笔者相同的手法对其他编译器进行分析。

4.6.1　单个类带虚函数的数据成员布局

看看如下范例。在 MyProject.cpp 的上面，增加如下代码行：

```
class MYACLS
{
public:
    int m_i;
    int m_j;

    virtual void myvirfunc()
    {
    }
    MYACLS()                    //构造函数
    {
        int abc = 1;            //方便加断点
    }
    ~MYACLS()                   //析构函数
    {
        int def = 0;            //方便加断点
    }
};
```

经过前面的学习，读者已经比较清楚，当类中引入虚函数的时候，会有一些额外成本效率的付出：

(1) 编译的时候，编译器会产生类的虚函数表。

(2) 对象中会产生虚函数表指针 vptr，用以指向类的虚函数表。

(3) 编译器会增加或扩展类的构造函数，增加给虚函数表指针 vptr 赋值的代码，让虚函数表指针指向虚函数表。读者已经知道，因为多态的原因，子类和父类的构造函数中都需要分别设定对应的 vptr 值。

(4) 如果是多重继承，如子类继承了两个父类，每个父类如果都有虚函数表，那么每个父类对象都要有一个虚函数表指针 vptr(这也是类中的一个成员变量，只不过是隐藏的)，继承的时候**子类就会把这两个 vptr 都继承过来**，如果子类自己还有额外的虚函数，那么子类与第一个父类共用同一个 vptr(可以回顾 3.4 节)。

(5) 析构函数中似乎也被扩展增加了给虚函数表指针 vptr 赋值的代码，但感觉这个赋值代码似乎和构造函数中的代码相同。

可以看看编译器向类的构造函数和析构函数中增加的代码。在 main 主函数中增加如下代码行：

```
MYACLS aaa;
```

把断点分别设置到 MYACLS 类的构造函数和析构函数中并进行调试，当程序执行流程停到构造函数的断点行后，切换到反汇编窗口，如图 4.20 所示。

```
00031963  mov         ecx,offset _F0B5FBBB_MyProject@cpp (03F026h)
00031968  call        @__CheckForDebuggerJustMyCode@4 (0312CBh)
0003196D  mov         eax,dword ptr [this]
00031970  mov         dword ptr [eax],offset MYACLS::`vftable' (039C54h)
    {
        int abc = 1; //方便加断点
00031976  mov         dword ptr [abc],1
    }
```

图 4.20　编译器向类的构造函数中插入的用于给虚函数表指针赋值的代码

当程序执行流程停到析构函数的断点行后，切换到反汇编窗口，如图 4.21 所示。

```
00031F62  mov         ecx,offset _F0B5FBBB_MyProject@cpp (03F026h)
00031F67  call        @__CheckForDebuggerJustMyCode@4 (0312CBh)
00031F6C  mov         eax,dword ptr [this]
00031F6F  mov         dword ptr [eax],offset MYACLS::`vftable' (039C54h)
        int def = 0; //方便加断点
00031F75  mov         dword ptr [def],0
    }
```

图 4.21　编译器向类的析构函数中插入的用于给虚函数表指针赋值的代码

图 4.21 中，编译器插入的针对虚函数表指针赋值的代码看起来和构造函数中的针对虚函数表指针赋值的代码相同，那么在析构函数中为什么又一次给虚函数表指针赋值呢？请读者也一起思考一下，笔者暂时还没有更好的答案。

目前 MYACLS 类对象的数据布局是什么样子呢？注释掉 main 主函数中原有代码，加入新的代码：

```
cout << sizeof(MYACLS) << endl;
printf("MYACLS::m_i =  %d\n", &MYACLS::m_i);
printf("MYACLS::m_j =  %d\n", &MYACLS::m_j);
```

执行起来，看一看结果：

```
12
MYACLS::m_i = 4
MYACLS::m_j = 8
```

根据执行结果，就能够把 MYACLS 类对象的数据布局画出来(vptr 肯定在最上面)，如图 4.22 所示。

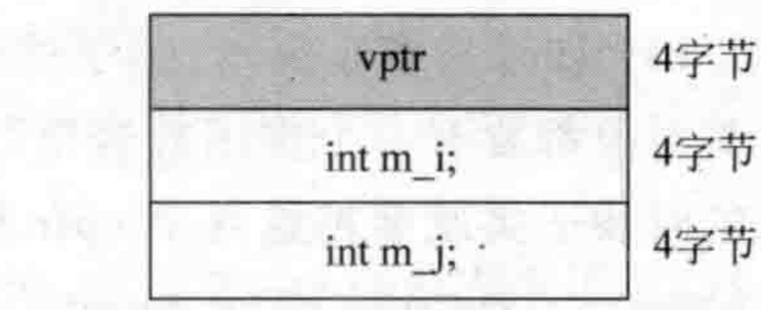

图 4.22　类 MYACLS 对象数据布局

继续在 main 主函数中增加如下代码：

```
MYACLS myobj;
myobj.m_i = 3;
myobj.m_j = 6;
cout << "方便加断点" << endl;
```

设置断点于上面的 myobj.m_j = 6;代码行并进行调试，当程序执行流程停到断点行后，在内存 1 窗口中输入 &myobj 并按 Enter 键以显示 myobj 对象的内存地址，并按 F10 键向下走一行，注意观看内存中变色的位置和变色的内容，如图 4.23 所示。

在图 4.23 中可以看到，将 myobj 对象的首地址作为开始地址，往后面计算，第 9 个字节(偏移值)开始的 4 个字节，在执行了代码行 myobj.m_j = 6;后，这 4 个字节的内容变成了 06000000(6)。

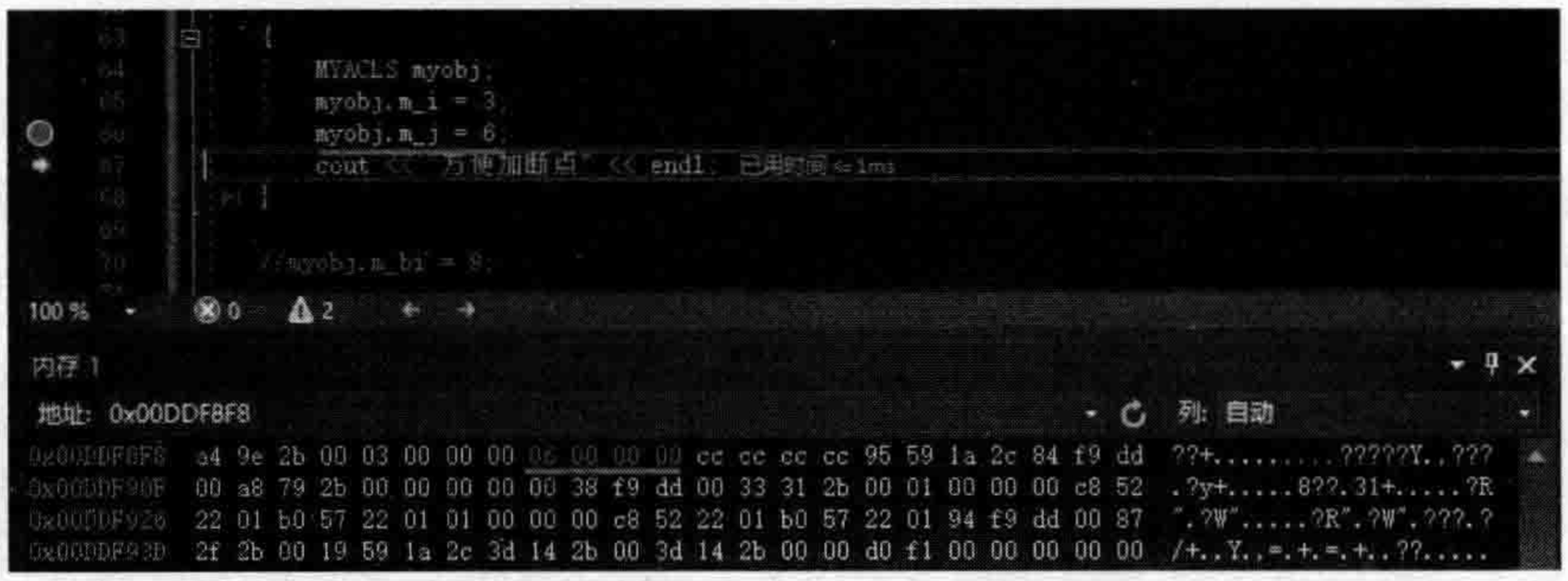

图 4.23　观察类对象数据布局和在内存中的对应关系

4.6.2　单一继承父类带虚函数的数据成员布局

现在如果增加一个基类，基类里也有虚函数。在 MyProject.cpp 的上面，MYACLS 类定义的前面，增加如下代码行来定义 Base 基类：

```
class Base
{
public:
    virtual void mybvirfunc() {}
    int m_bi;
};
```

然后，让 MYACLS 类继承自 Base 基类：

```
class MYACLS :public Base
{
    ...
};
```

在 main 主函数中，注释掉以往代码，加入新代码：

```
cout << sizeof(MYACLS) << endl;
printf("MYACLS::m_bi =  %d\n", &MYACLS::m_bi);
printf("MYACLS::m_i =  %d\n", &MYACLS::m_i);
printf("MYACLS::m_j =  %d\n", &MYACLS::m_j);
```

执行起来，看一看结果：

```
16
MYACLS::m_bi = 4
MYACLS::m_i = 8
MYACLS::m_j = 12
```

看一看类 MYACLS 对象的数据布局，如图 4.24 所示。

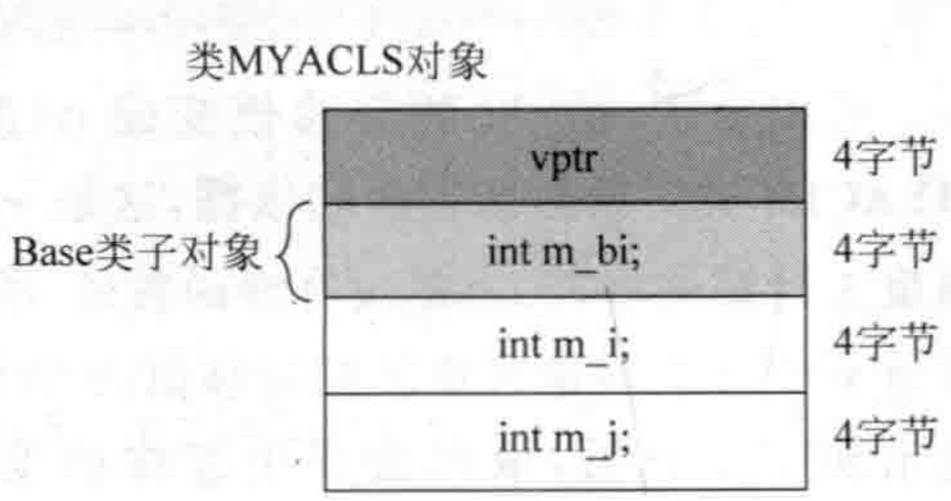

图 4.24　类 MYACLS 对象数据布局

继续在 main 主函数中增加如下代码：

```
MYACLS myobj;
myobj.m_i = 3;
myobj.m_j = 6;
```

```
myobj.m_bi = 9;
cout << "方便加断点" << endl;
```

设置断点于上面的 myobj.m_bi = 9;代码行并进行调试，当程序执行流程停到断点行后，在内存 1 窗口中输入 &myobj 并按 Enter 键以显示 myobj 对象的内存地址，并按 F10 键向下走一行，注意观看内存中变色的位置和变色的内容，如图 4.25 所示。

图 4.25　观察类对象数据布局和在内存中的对应关系

在图 4.25 中可以看到，将 myobj 对象的首地址作为开始地址，往后面计算，第 5 个字节(偏移值)开始的 4 个字节，在执行了代码行 myobj.m_bi = 9;后，这 4 个字节的内容变成了 09000000(9)。

4.6.3　单一继承父类不带虚函数的数据成员布局

现在如果把父类 Base 中的虚函数去掉，那么子类 MYACLS 对象的数据布局会有什么变化吗？将 Base 中的虚函数代码行注释掉：

```
//virtual void mybvirfunc() {}
```

执行起来，看一看结果：

```
16
MYACLS::m_bi = 0
MYACLS::m_i = 8
MYACLS::m_j = 12
```

可以发现，m_bi 的偏移值变成 0(原来是 4)了。但是**这个偏移值是否能作为绘制 MYACLS 类对象数据布局的依据**，这是一个问题。为什么？因为这个偏移值是**站在父类的角度去计算偏移**，父类是没有虚函数的，所以 m_bi 针对于父类对象首地址的偏移值为 0，但是一旦出现了子类后，情况会发生怎样的变化呢？所以，如果用上面的结果数据来绘制子类 MYACLS 对象的数据布局图，有些读者可能认为是这样的，如图 4.26 所示。

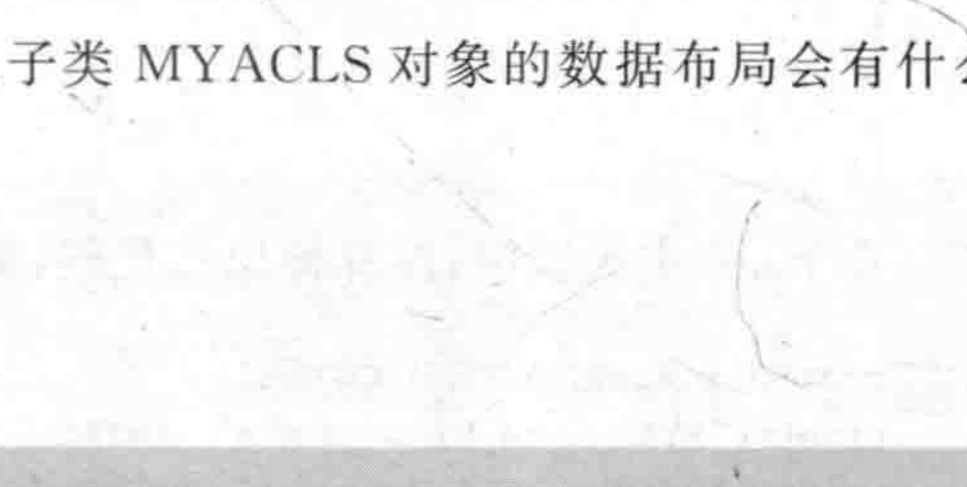

图 4.26　类 MYACLS 对象数据布局

但真的是这样吗？这个问题必须要进一步

测试一下。还是通过程序代码演示，在 main 主函数中现有的代码已经足够。

设置断点于 main 主函数的 myobj. m_bi = 9;代码行并进行调试，当程序执行流程停到断点行后，在内存 1 窗口中输入 &myobj 并按 Enter 键以显示 myobj 对象的内存地址，并按 F10 键向下走一行，注意观看内存中变色的位置和变色的内容，如图 4.27 所示。

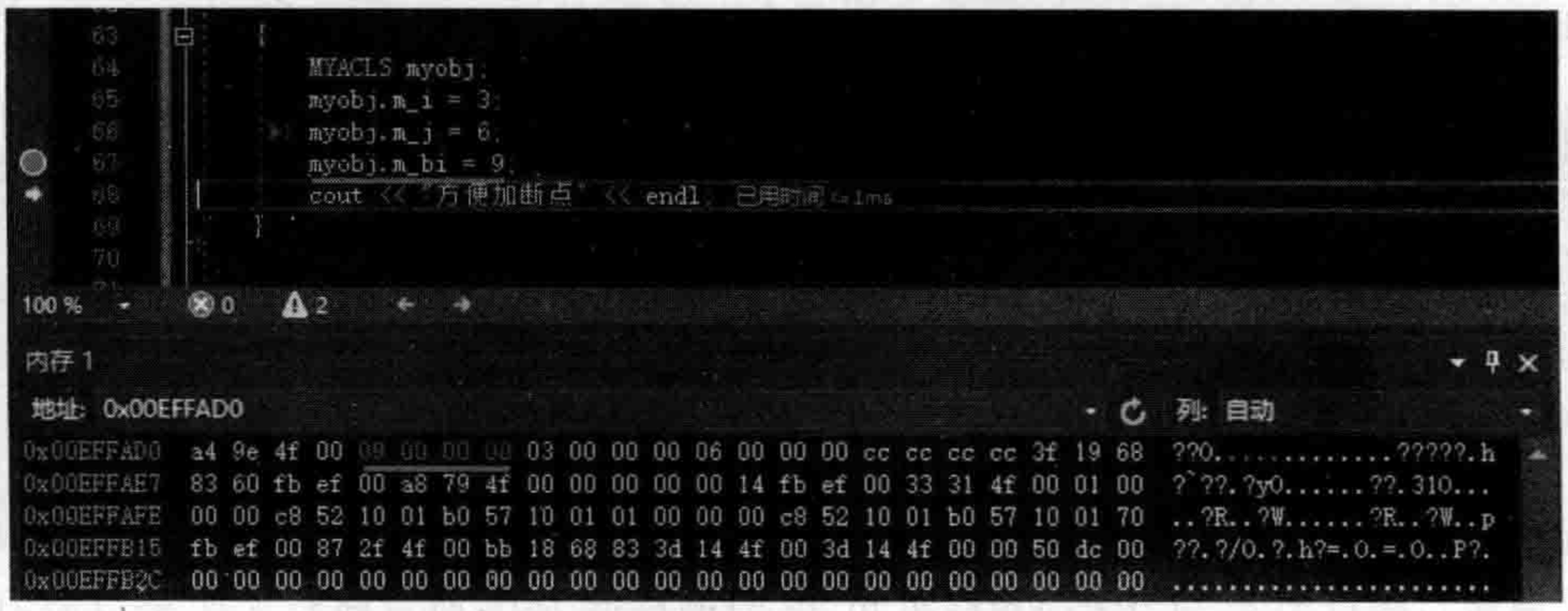

图 4.27　观察类对象数据布局和在内存中的对应关系

通过这个演示，意味着 vptr 还是在类 MYACLS 的开头位置，因此，真正正确的内存布局还是如图 4.24 所示。

所以，**单一继承父类带虚函数的数据成员布局**与**单一继承父类不带虚函数的数据成员布局**实际上是一样的。

4.7　多重继承数据布局与 this 调整深谈

4.7.1　单一继承数据成员布局 this 指针偏移知识补充

上一节最终的范例代码原样保留，本节中要使用。

接着当前的范例，补充讲解一点，在 2.3 节讲解过 this 指针调整的话题。现在情况是父类 Base 没有虚函数，所以该类的对象也就没有虚函数表指针(vptr)，子类 MYACLS 有虚函数，所以该类的对象中会多出一个虚函数表指针，根据上一节讲解的内存布局图，要访问父类对象的 m_bi 成员变量的话，this 指针要调整往下走 4 个字节(参考图 4.24)。

为了验证这件事，向 Base 类和 MYACLS 类增加构造函数，打印 this 指针值。

向 Base 类增加 public 修饰的默认构造函数：

```
public:
    Base()
    {
        printf("Base::Base()构造函数的 this 指针是: %p!\n", this);
    }
```

MYACLS 类原来有默认构造函数，现在向其中增加代码。完整的 MYACLS 类默认构造函数代码如下：

```
public:
```

```
MYACLS()                                    //构造函数
{
    int abc = 1;                            //方便加断点
    printf("MYACLS::MYACLS()构造函数的 this 指针是：%p!\n", this);
}
```

执行起来，看一看结果：

```
16
MYACLS::m_bi = 0
MYACLS::m_i = 8
MYACLS::m_j = 12
Base::Base()构造函数的 this 指针是：00CFF8F4!
MYACLS::MYACLS()构造函数的 this 指针是：00CFF8F0!
```

观察结果，果然可以看到，要访问 Base 类的成员，this 指针要往下跳 4 个字节（**this 指针进行了调整**），跳过图 4.24 中的 vptr，而 m_bi 的偏移值为 0，所以跳过 4 个字节后，直接就能够访问到 m_bi 成员变量。

如果现在 Base 类中要是有虚函数，那么 this 指针还用调整吗？将 Base 类中的虚函数代码行的注释取消：

```
virtual void mybvirfunc() {}
```

执行起来，看一看结果：

```
16
MYACLS::m_bi = 4
MYACLS::m_i = 8
MYACLS::m_j = 12
Base::Base()构造函数的 this 指针是：00EFFE0C!
MYACLS::MYACLS()构造函数的 this 指针是：00EFFE0C!
```

观察结果，可以看到，**this 指针是相同的，不用调整**，因为父类 Base 带有虚函数，所以该类对象有虚函数表指针（vptr），子类 MYACLS 的虚函数表指针可以认为是从父类继承而来的，而且也在内存布局的最上面。另外，观察到 m_bi 偏移值变成了 4，这代表从对象首地址＋4 个字节，就能够访问到 m_bi 成员变量。

4.7.2 多重继承且父类都带虚函数的数据成员布局

在 MyProject.cpp 的上面，新增加一个 Base2 基类：

```
class Base2
{
public:
    virtual void mybvirfunc2() {}
    int m_b2i;
};
```

让 MYACLS 类同时继承 Base 和 Base2 类：

```
class MYACLS :public Base, public Base2
{
    ...
};
```

在 main 主函数中，增加一行打印 m_b2i 成员变量偏移值的语句：

```
printf("MYACLS::m_b2i =  %d\n", &MYACLS::m_b2i);
```

执行起来，看一看结果(摘取显示成员变量偏移值这部分结果)：

```
24
MYACLS::m_bi = 4
MYACLS::m_b2i = 4
MYACLS::m_i = 16
MYACLS::m_j = 20
```

估算一下，因为 MYACLS 类继承两个都有虚函数的父类，那么 MYACLS 类对象应该有两个虚函数表指针(vptr)，一共占 8 字节，加上 4 个 int 成员 16 字节，总和是 24 字节，与上面结果中的字节数是吻合的。

结果中有两个成员变量 m_bi 和 m_b2i 的偏移值都是 4，源于这两个父类对象都有虚函数指针，因为这两个父类是平级的，所以对于每一个父类来讲，它的成员变量都是偏移 4 字节的。2.3 节讲解过多重继承的 this 指针调整问题，所以虽然 m_bi、m_b2i 两个成员变量的**偏移值相同，但是它们分属于两个不同的基类子对象，经过 this 指针调整后，偏移值相同的两个成员变量也可以被正常访问。**

向 Base2 类增加构造函数，打印 this 指针值。

```
public:
    Base2()
    {
        printf("Base2::Base2()构造函数的 this 指针是: %p!\n", this);
    }
```

执行起来，看一看结果：

```
24
MYACLS::m_bi = 4
MYACLS::m_b2i = 4
MYACLS::m_i = 16
MYACLS::m_j = 20
Base::Base()构造函数的 this 指针是: 0137FAF0!
Base2::Base2()构造函数的 this 指针是: 0137FAF8!
MYACLS::MYACLS()构造函数的 this 指针是: 0137FAF0!
```

通过结果可以看到，访问 Base 类成员变量的时候，this 指针是不用调整的，而访问 Base2 类成员变量的时候，this 指针是要跳过 8 个字节的。

显然，根据上面这些数据，还是无法确定内存布局，所以继续完善 main 主函数中的代码，用老手法来找内存布局。所谓用老手法来找内存布局，其实前面早就使用过了，如果再

具体详细地描述一下就是：通过设置断点来调试程序，当程序停留在断点行时，在内存的编辑框中输入对象首地址以显示出对象所代表的内存中内容，然后通过按 F10 键单步执行的方式执行给某个成员变量赋值的代码行，此时内存中的某些数据会发生变化，观察这些变化的数据离对象首地址有多远的距离（单位：字节）。据此，就可以确定某个成员变量的真正偏移值。

在 main 主函数中，增加如下代码行：

```
myobj.m_b2i = 12;
```

通过设置断点并不断地调试，从而确定对象的布局如下：

```
myobj.m_i = 3;                    //17～20 字节
myobj.m_j = 6;                    //21～24 字节
myobj.m_bi = 9;                   //5～8 字节
myobj.m_b2i = 12;                 //13～16 字节
```

根据上面这些调试的结果信息，大概就能把数据布局画出来了。可以估计一下，根据继承的顺序，先继承 Base 类，再继承 Base2 类，那么两个 vptr 指针在上面的应该是继承自 Base 类的，在下面的应该是继承自 Base2 类的，所以数据布局图大概如图 4.28 所示。

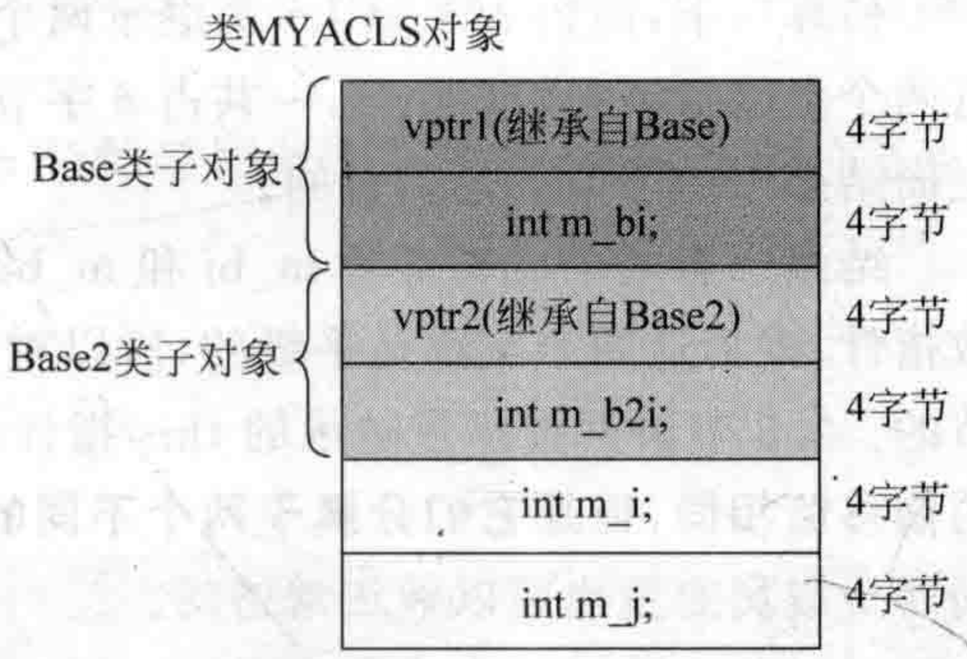

图 4.28　类 MYACLS 对象数据布局

图 4.28 是结合着上面的一些线索和调试来绘制的，能够看到或者推导出如下一些信息：

（1）通过上面 this 指针的打印可以看到：访问 Base 类成员变量 this 指针不用调整，而访问 Base2 类成员变量则 this 指针要跳过 8 个字节。

（2）注意，m_bi 和 m_b2i 的偏移值都是 4。

（3）请想一想，this 指针加这个偏移值就能够访问对应的成员。例如，要访问 m_b2i 成员变量，this 指针调整跳跃了 8 个字节，然后偏移值又是 4 字节，所以从最上面（类 MYACLS 对象）跳过 12 个字节，就能访问到 m_b2i 成员变量，和图 4.28 所绘制的结果一致。

另外，观察一下 myobj 这个 MYACLS 类对象的两个虚函数表指针（vptr）指向的虚函数表（vtbl）的地址，这两个地址似乎也比较接近（反着看：一个是 006d9b8c，一个是 006d9b98）。如图 4.29 所示。

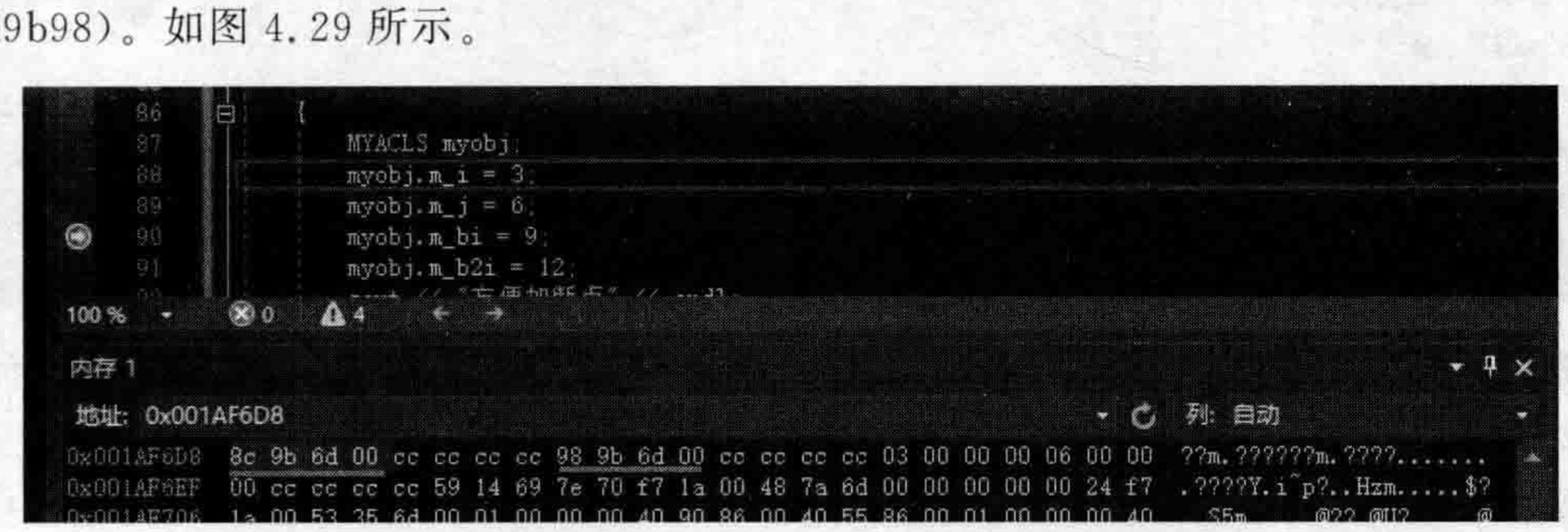

图 4.29　类 MYACLS 对象的两个虚函数表指针指向的两个虚函数表的位置比较接近

这里学习到了一个结论：

当要访问一个类对象中的成员变量时，成员变量的定位是通过两个因素来定位的：①this 指针(根据情况编译器知道是否做调整)；②该成员的偏移值。这种 this 指针的调整、成员偏移值的确定都是需要编译器介入进行的。

现在看一点有意思的事情。在 main 主函数中，增加如下代码(注意代码中的注释)：

```
Base2 * pbase2 = &myobj; //注意结果,myobj 地址为 0x009ffb0c,而 pbase2 给值后是 0x009ffb14,
                         //因为 this 指针调整是往后跳 8 字节,所以这里 myobj 地址值 + 8 字节
                         //后给了 pbase2
```

请想一想，编译器内部肯定是对上面这句代码进行了调整。编译器可能像如下的方式调整的：

```
Base2 * pbase2 = (Base2 * )(  ((char * )&myobj)  +  sizeof(Base)  );
```

在 main 主函数中，增加如下代码：

```
Base * pbase = &myobj;  //pbase 的地址就是 myobj 对象的首地址,因为 this 不需要调整,Base 类
                        //的首地址和 MYACLS 类的首地址相同
```

再看一看下面这段比较有意思的代码，增加在 main 主函数中(仔细阅读代码中的注释)：

```
Base2 * pbase3 = new MYACLS(); //父类指针 new 子类对象,返回的地址是 0x01054bc8,实际这里分
                               //配是 24 字节
MYACLS * psubobj = (MYACLS * )pbase3; //得到的地址是 0x01054bc0,比 pbase3 指向的地址少了 8 字节
delete pbase3; //报异常,据此,可以认为 pbase3 里返回的地址不是分配的首地址,是 this 指针调整
               //后的地址,而 new 分配的真正的内存首地址应该是 psubobj 指向的那个地址
delete psubobj;                          //成功,验证了 psubobj 里才是真正的首地址的说法
```

目前，这个继承结构还不复杂，如图 4.30 所示。

如果出现更复杂的继承结构，例如，Base 类下有一个子类 Base11，MYACLS 类直接继承自 Base11 和 Base2，如图 4.31 所示。

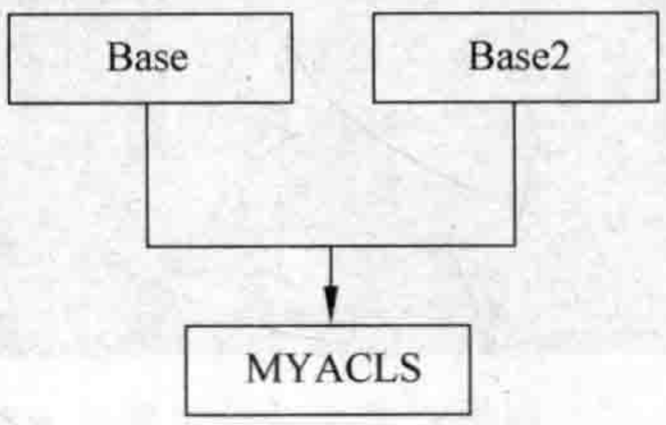

图 4.30　当前的类继承层次关系图

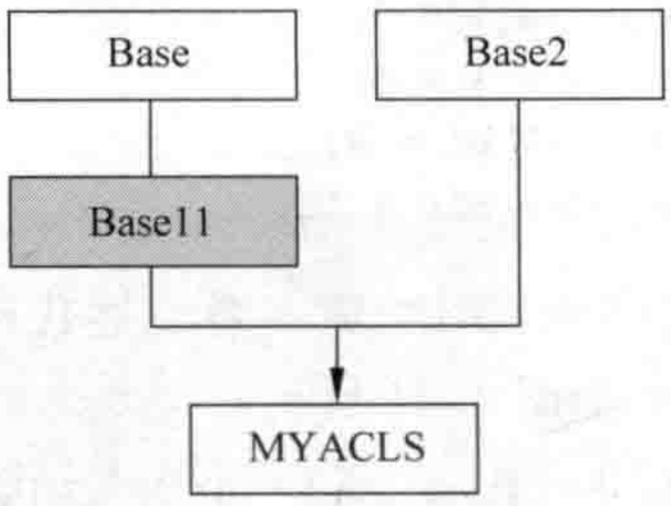

图 4.31　更复杂一点的类继承层次关系图

根据图 4.31，类 MYACLS 对象数据布局很可能是下面这种布局(这里只大概分一下每个子类对象的排列，具体的成员变量等数据就不细分了，留给读者去扩展分析)，如图 4.32 所示。

相信只要掌握笔者所讲述的分析方法，读者也一定能分析出来。希望读者举一反三，根据已经掌握的知识，来学习和推导新知识，这样，对于个人的成长好处是非常巨大的。

另外请注意，虽然目前的继承关系可能比较复杂，但是访问哪个类中的成员变量，会发现成本都差不多，成员位置的计算在编译的时候就已经确定好了，访问成员变量只是做个偏

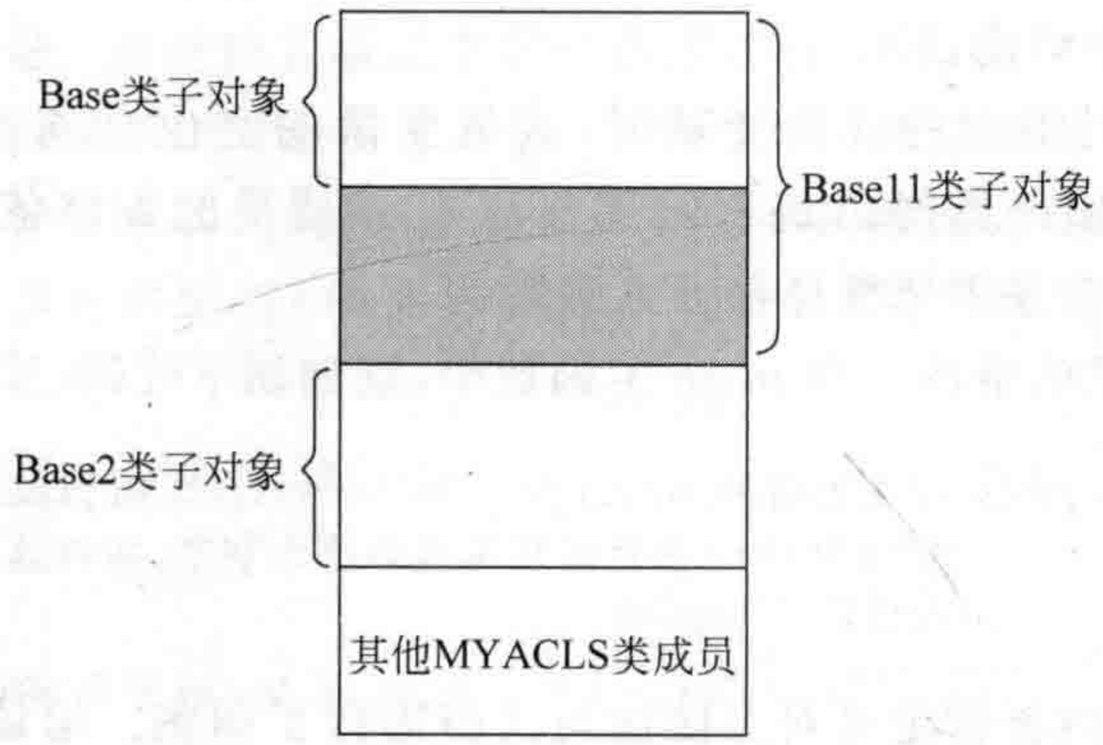

图 4.32　类 MYACLS 对象数据布局

移操作。所以，不管使用类对象也好，对象指针也罢，引用也行，总之访问这些成员变量的成本相同。设置断点，调试，看一看给一些成员变量赋值的反汇编代码，如图 4.33 所示。

```
    {
        MYACLS myobj;
00512D40  lea         ecx,[ebp-2Ch]
00512D43  call        MYACLS::MYACLS (0511110h)
00512D48  mov         dword ptr [ebp-4],0
        myobj.m_i = 3;
00512D52  mov         dword ptr [ebp-1Ch],3
        myobj.m_j = 6;
00512D59  mov         dword ptr [ebp-18h],6
        myobj.m_bi = 9;
00512D60  mov         dword ptr [ebp-28h],9
        myobj.m_b2i = 12;
00512D67  mov         dword ptr [ebp-20h],0Ch
```

图 4.33　利用对象名给对象的成员变量赋值的反汇编代码

如果用一个对象指针来给对象的成员变量赋值，又是什么情形呢？在 main 主函数中，增加如下代码：

```
MYACLS * pmyobj = new MYACLS();
pmyobj->m_i = 3;
pmyobj->m_j = 6;
pmyobj->m_bi = 9;
pmyobj->m_b2i = 12;
```

设置断点，调试，看一看上述代码对应的反汇编代码，如图 4.34 所示。

```
00617145  mov         ecx,dword ptr [ebp-158h]
0061714B  mov         dword ptr [ebp-68h],ecx
        pmyobj->m_i = 3;
0061714E  mov         eax,dword ptr [ebp-68h]
00617151  mov         dword ptr [eax+10h],3
        pmyobj->m_j = 6;
00617158  mov         eax,dword ptr [ebp-68h]
0061715B  mov         dword ptr [eax+14h],6
        pmyobj->m_bi = 9;
00617162  mov         eax,dword ptr [ebp-68h]
00617165  mov         dword ptr [eax+4],9
        pmyobj->m_b2i = 12;
0061716C  mov         eax,dword ptr [ebp-68h]
0061716F  mov         dword ptr [eax+0Ch],0Ch
```

图 4.34　利用对象指针给对象的成员变量赋值的反汇编代码

图 4.34 中，其实[ebp-68h]代表的就是 pmyobj 所指向的 MYACLS 类对象的首地址。

4.8　虚基类问题的提出和初探

4.8.1　虚基类(虚继承/虚派生)问题的提出

笔者在《C++新经典》一书中，对虚基类有比较详细的介绍和讲解，在这里简单回顾一下虚基类的概念。

常规的多重继承如图 4.35 所示。这样继承导致的问题就是 C1 类生成的对象会有两份 Grand 类子对象的备份(C1 从 A1 这里继承了一份 Grand 子对象，又从 A2 这里继承了一份 Grand 子对象)，这容易造成很多问题——空间问题、效率问题、二义性问题(名字冲突)等。

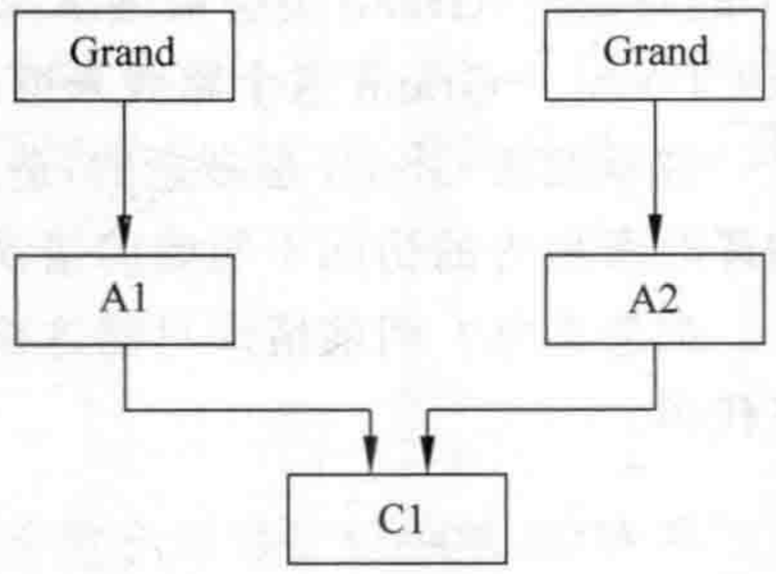

图 4.35　常规的多重继承导致产生出多个爷爷类子对象的问题

在 MyProject.cpp 的上面，增加如下代码行：

```
class Grand //爷爷类
{
public:
    int m_grand;
};
class A1 : public Grand
{
public:
};
class A2 : public Grand
{
public:
};
class C1 :public A1, public A2
{
public:
};
```

在 main 主函数中，加入如下代码：

```
cout << sizeof(Grand) << endl;
cout << sizeof(A1) << endl;
cout << sizeof(A2) << endl;
cout << sizeof(C1) << endl;
```

执行起来，看一看结果：

```
4
4
4
8
```

分析一下结果中的 4、4、4、8。尤其是为什么 C1 的 sizeof 值变成 8 了？就是因为内存中有两份 Grand 类子对象，从而造成有两份 m_grand 成员变量，一份 m_grand 占用 4 字节内存，两份占用的是 8 字节内存。

继续向 main 主函数中增加代码：

```
C1 c1;
c1.m_grand = 12;                    //这句编译时会报错
```

编译报错，系统提示对 m_grand 的访问不明确。为什么不明确呢？就是因为定义类

C1 的对象时，Grand 类会被继承两次，一次是从 C1→A1→Grand 这个路线来继承，一次是从 C1→A2→Grand 这个路线来继承。

继承两次 Grand 是多余的，造成的二义性问题指的就是这种 c1. m_grand = 12；代码编译时所报告的访问不明确的错误。

那怎么办？当前情形只能这样访问 m_grand 成员变量，在 main 主函数中继续增加如下代码：

```
c1.A1::m_grand = 5;
c1.A2::m_grand = 8;
```

设置断点，调试，并观察 c1 对象的内存，如图 4.36 所示。前 4 字节对应的是 A1 路线里的 m_grand，后 4 字节对应的是 A2 路线里的 m_grand，这就证实了在 c1 对象的内存中有两个 m_grand。

图 4.36　c1 对象中包含了两个 m_grand 成员变量

绘制一下 c1 的对象数据布局，如图 4.37 所示。

在图 4.37 中，上面的 m_grand 可以认为是来自于 A1(A1 是继承自 Grand)，下面这个 m_grand 可以认为是来自于 A2(A2 也是继承自 Grand)。

既然继承两次 Grand 是多余的，就需要让 Grand 只被继承一次。

为此，引入了虚基类(虚继承)的概念。读者已经知道，虚基类必须是三层结构，有一个爷爷类，两个父亲类，还有孩子类，一般如图 4.38 所示。

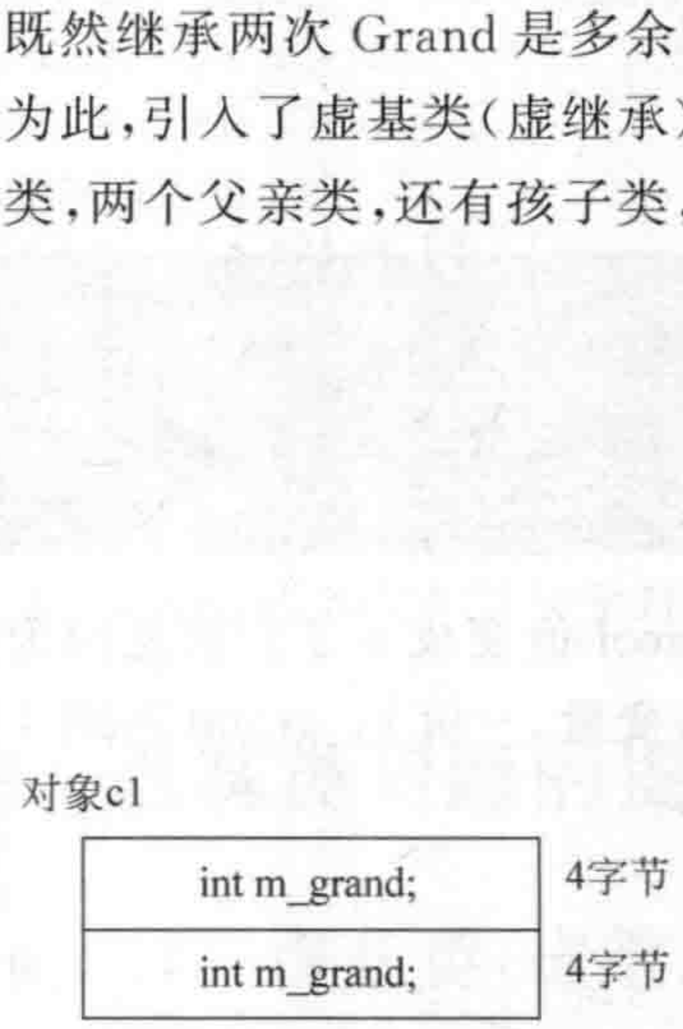

图 4.37　c1 对象数据布局

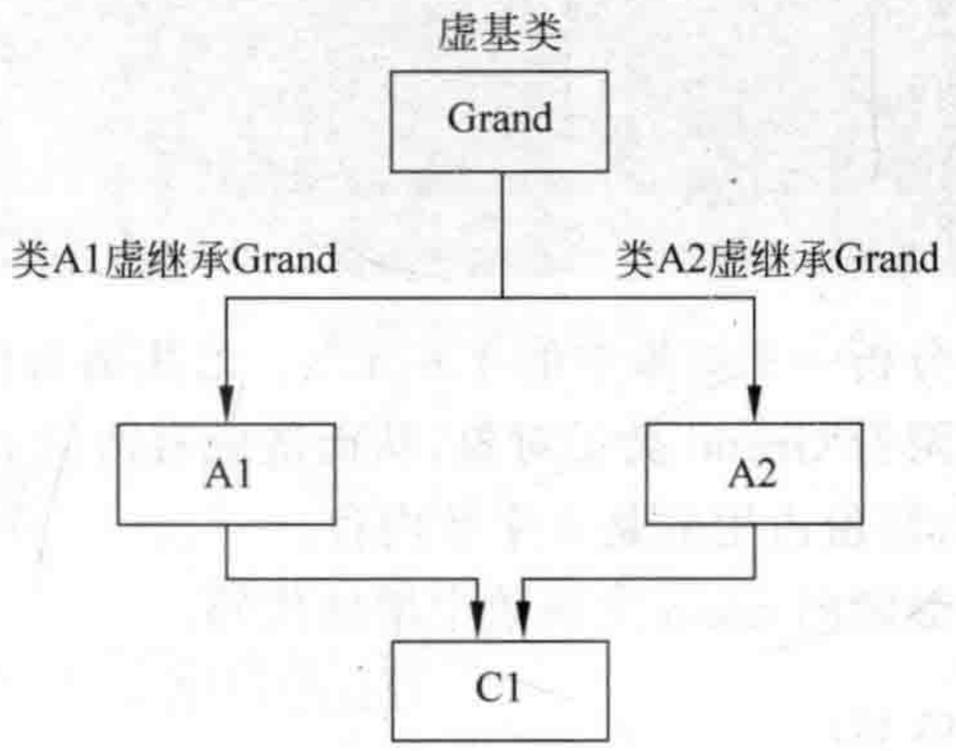

图 4.38　虚继承解决了 Grand 被继承多次的情形

注意图 4.38 中 A1 和 A2 虚继承自 Grand，Grand 就是虚基类，虚基类就代表一件事：在孙子类(C1)对象中，只包含一份 Grand 类子对象，而不管 Grand 在继承体系中出现多少次。这份 Grand 类子对象，可以说，既不来自于 A1，也不来自于 A2，应该看成是一份独立的 Grand 类子对象。

换句话说，如果定义一个 C1 类的对象：

```
C1 c1;
```

那么在 c1 对象的数据布局里只会出现一个 Grand 类子对象。这意味着上面的 m_grand 不会再出现两个了(出现两个的时候一个是来自于 A1，一个是来自于 A2，现在既不来自于 A1 也不来自于 A2，是归属于 C 类对象自己的 Grand 类子对象)。

不要把虚基类(虚继承)和虚函数混为一谈。虚基类解决的是多重继承时 Grand 这种爷爷类的子对象多次出现在 C1 这种孙子类对象中的问题，虚基类这种手段保证了 Grand 这种子对象在 C1 孙子类对象中只有一份。

现在，把以往的所有代码全部注释掉，引入虚基类(虚继承)相关代码。在 MyProject.cpp 的上面，增加如下代码行：

```
class Grand                              //爷爷类
{
public:
    int m_grand;
};
class A1 : virtual public Grand          //注意这里用了 virtual
{
public:
};
class A2 : virtual public Grand          //注意这里用了 virtual
{
public:
};
class C1 :public A1, public A2           //这里不需要 virtual
{
public:
};
```

在 main 主函数中，加入如下代码：

```
C1 c1;
c1.m_grand = 12;                         //这句编译时不会再报错
```

通过上面的代码可以看到，引入虚基类后，就不会出现 m_grand 访问不明确的问题。因为此时只有一个 Grand 类子对象，而不是两个，不会再出现二义性问题。

另外，读者需要知道的是，虚基类的想法很好，但是在编译器中支持虚基类并不是一件轻松的事，不像表面看起来这样容易，每个编译器的实现厂商都可能有自己不同的实现手段。

另外，虚基类出现后，程序员书写代码也会有一定的变化和调整。例如，如果 Grand 类的构造函数带参数，以往是由 A1、A2 这种直接子类在构造函数初始化列表中来初始化

Grand，引入虚基类后，就**必须在 C1 这个孙子类构造函数的初始化列表中初始化 Grand 爷爷类**。相信读者的 C++基础都很好，这些知识应该都已经掌握，所以这里笔者就不多说。

4.8.2 虚基类初探

现在，为了简化问题，首先注释掉 Grand 类中的成员变量 m_grand：

```
//int m_grand;
```

在 main 主函数中，注释掉以往的代码，加入如下代码：

```
cout << sizeof(Grand) << endl;
cout << sizeof(A1) << endl;
cout << sizeof(A2) << endl;
cout << sizeof(C1) << endl;
```

执行起来，看一看结果：

```
1
4
4
8
```

虚基类这种技术的实现和编译器底层密切相关，可能每个平台的实现略有差异，这里就以笔者的当前开发环境 Visual Studio 2019 为主来探讨。

一般来讲，虚基类会涉及两个概念：①**虚基**类表(virtual base table，vbtable)；②虚基类表指针(virtual base table pointer，vbptr)。注意这里都是带字母 b 的，表示 base(基本)的意思。

分析一下上面程序执行的结果，为什么是 1、4、4、8 这几个数字呢？

上述范例代码为了简化问题，类中的内容都保持为空(空类)。读者已经知道，一个纯粹的空类，它的 sizeof 值是 1，那么这里 sizeof 不是 1 的类，肯定是因为继承关系所致。

因为 Grand(爷爷类)是空类，所以 sizeof 值是 1，这好理解。

A1 和 A2 类是属于虚基类层次结构的中间层(父类)，虚继承(virtual)了 Grand 类。它们的 sizeof 值都是 4 个字节，那么这 4 个字节是什么呢？猜测一下不难想象，这应该是一个**指针**，这种**虚继承父类**的子类(A1 和 A2)里面都会被编译器**插入一个虚基类表指针**，这个指针相当于一个成员变量的感觉，因为这个指针是占用类对象尺寸的，也就是 sizeof 的时候，结果等于 4。谈到占用尺寸的问题，这里就与讲解虚函数时的虚函数表指针有点类似(虚函数表指针也是占用类对象尺寸的)。

做一下测试，在 main 主函数中，加入如下代码行：

```
A1 a1;
A2 a2;
cout << "设置断点" << endl;
```

设置断点于 cout 所在行，调试并观察 a1 和 a2 对象所在内存的数据，发现 a1 和 a2 内存中的内容不同，这说明 a1 和 a2 对象里的虚基类表指针指向的位置并不相同。

那么这个虚基类表指针是做什么的，显然它应该是指向虚基类表的，虚基类表后面再谈。

那么，为什么 C1 这个孙子类的 sizeof 值是 8 字节呢？猜一猜，C1 类有 A1 和 A2 两个父类，每个父类里有一个虚基类表指针，这个虚基类表指针会被 C1 类继承，两个虚基类表指针是 2×4=8 字节，所以现在看到的事实是类 C1 的 sizeof 值 8(字节)。

这就是通过简单的观察和分析，推测得到的一个结论。

上一节推测对象数据布局的时候是通过写测试代码来推测，本节通过命令来观察对象数据布局。

在 Visual Studio 2019(或其他 Visual Studio 版本)中打开**开发人员命令行工具**，忘记如何打开的读者可以参考图 2.9。在这个命令行提示符下，可以直接使用 cl 命令行工具(编译链接工具)。

要查看 Grand 类的对象数据布局，在命令行提示符下，进入到 MyProject.cpp 文件所在的路径，并输入 cl /d1 reportSingleClassLayout**Grand** MyProject.cpp，如图 4.39 所示。

图 4.39　Windows 操作系统下使用 cl 工具查看虚基类 Grand 对象数据布局

从图 4.39 中可以看到，虚基类 Grand 是空的，1 字节，没有什么布局可言。

查看 A1 类的对象数据布局，输入 cl /d1 reportSingleClassLayout**A1** MyProject.cpp，如图 4.40 所示。

图 4.40　Windows 操作系统下使用 cl 工具查看类 A1 对象数据布局

查看 A2 类的对象数据布局，如图 4.41 所示。

```
C:\Users\KuangXiang\Desktop\c++\MySolution\MyProject>
C:\Users\KuangXiang\Desktop\c++\MySolution\MyProject>cl /d1 reportSingleClassLayoutA2 MyProject.cpp
用于 x86 的 Microsoft (R) C/C++ 优化编译器 19.22.27905 版
版权所有(C) Microsoft Corporation。保留所有权利。

MyProject.cpp

class A2        size(4):
        +---
 0      | {vbptr}
        +---
        +--- (virtual base Grand)
        +---

A2::$vbtable@:
 0      | 0
 1      | 4 (A2d(A2+0)Grand)
vbi:       class  offset o.vbptr  o.vbte fVtorDisp
           Grand       4       0       4 0
```

图 4.41　Windows 操作系统下使用 cl 工具查看类 A2 对象数据布局

查看 C1 类的对象数据布局，如图 4.42 所示。

```
C:\Users\KuangXiang\Desktop\c++\MySolution\MyProject>cl /d1 reportSingleClassLayoutC1 MyProject.cpp
用于 x86 的 Microsoft (R) C/C++ 优化编译器 19.22.27905 版
版权所有(C) Microsoft Corporation。保留所有权利。

MyProject.cpp

class C1        size(8):
        +---
 0      | +--- (base class A1)
 0      | | {vbptr}
        | +---
 4      | +--- (base class A2)
 4      | | {vbptr}
        | +---
        +---
        +--- (virtual base Grand)
        +---

C1::$vbtable@A1@:
 0      | 0
 1      | 8 (C1d(A1+0)Grand)

C1::$vbtable@A2@:
 0      | 0
 1      | 4 (C1d(A2+0)Grand)
vbi:       class  offset o.vbptr  o.vbte fVtorDisp
           Grand       8       0       4 0
```

图 4.42　Windows 操作系统下使用 cl 工具查看类 C1 对象数据布局

通过利用上述的 cl 工具，简单观察，也分别得到了 Grand、A1、A2、C1 的 sizeof 值分别为 1、4、4、8 的结论。

现在，为了后续演示和观察方便，往各个类中增加一个成员变量。增加了成员变量后的各个类定义内容如下：

```
class Grand                                 //爷爷类
{
public:
    int m_grand;
};
class A1 : virtual public Grand             //注意这里用了 virtual
{
public:
    int m_a1;
```

```
};
class A2 : virtual public Grand          //注意这里用了 virtual
{
public:
    int m_a2;
};
class C1 :public A1, public A2           //这里不需要 virtual
{
public:
    int m_c1;
};
```

执行起来，看一看结果：

```
4
12
12
24
```

注意比较，在往各个类中增加成员变量之前的 sizeof 值是 1、4、4、8。

如何理解上述这几个新的 sizeof 值呢？

因为 Grand(爷爷类)是包含一个 int 类型的成员变量，所以 sizeof 值是 4，这好理解。

A1 类和 A2 类的 sizeof 值是 12，怎样得到的？两个 int 类型的成员变量＋1 个虚基类表指针，一共是 3 个 4 字节＝12 字节。

C1 类的 sizeof 值是 24，怎样得到的？四个 int 类型的成员变量＋2 个虚基类表指针，一共是 6 个 4 字节＝24 字节。

同样，用 cl 命令看一看此时的 Grand 类的对象数据布局，如图 4.43 所示。

```
C:\Users\KuangXiang\Desktop\c++\MySolution\MyProject>cl /d1 reportSingleClassLayoutGrand MyProject.cpp
用于 x86 的 Microsoft (R) C/C++ 优化编译器 19.22.27905 版
版权所有(C) Microsoft Corporation。保留所有权利。

MyProject.cpp

class Grand     size(4):
        +---
 0      | m_grand
        +---
```

图 4.43　Windows 操作系统下使用 cl 工具查看虚基类 Grand 对象数据布局

查看 A1 类的对象数据布局，如图 4.44 所示。

```
C:\Users\KuangXiang\Desktop\c++\MySolution\MyProject>cl /d1 reportSingleClassLayoutA1 MyProject.cpp
用于 x86 的 Microsoft (R) C/C++ 优化编译器 19.22.27905 版
版权所有(C) Microsoft Corporation。保留所有权利。

MyProject.cpp

class A1        size(12):
        +---
 0      | {vbptr}
 4      | m_a1
        +---
        +--- (virtual base Grand)
 8      | m_grand
        +---

A1::$vbtable@:
 0      | 0
 1      | 8 (A1d(A1+0)Grand)
vbi:       class  offset o.vbptr  o.vbte fVtorDisp
           Grand       8       0       4 0
```

图 4.44　Windows 操作系统下使用 cl 工具查看类 A1 对象数据布局

请注意，虚继承和常规继承不同：

(1) 开头 4 个字节是虚基类表指针：1～4 字节。

(2) 接着的 4 个字节是 m_a1：5～8 字节。

(3) 再接下来的 4 个字节才是 m_grand：9～12 字节。

传统继承的情况下，m_grand 应该是排在 m_a1 前面的，而虚继承恰恰相反。**编译器把 Grand 子对象放到了最后**，它这么设计肯定有它的原因。笔者都知道，设计编译器的这些人对细节都是了如指掌的，都有他们的考虑，根据推测，放到最后应该是为了方便访问。A1 对象的数据布局应该如图 4.45 所示。

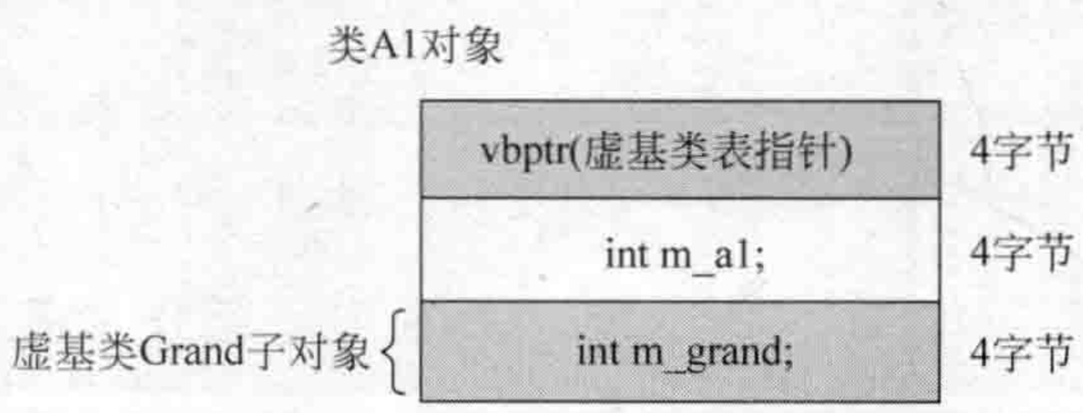

图 4.45　类 A1 对象数据布局

查看 A2 类的对象数据布局，与 A1 很类似，如图 4.46 所示。

```
C:\Users\KuangXiang\Desktop\c++\MySolution\MyProject>cl /d1 reportSingleClassLayoutA2 MyProject.cpp
用于 x86 的 Microsoft (R) C/C++ 优化编译器 19.22.27905 版
版权所有(C) Microsoft Corporation。保留所有权利。

MyProject.cpp

class A2        size(12):
        +---
 0      | {vbptr}
 4      | m_a2
        +---
        +--- (virtual base Grand)
 8      | m_grand
        +---

A2::$vbtable@:
 0      | 0
 1      | 8 (A2d(A2+0)Grand)
vbi:       class  offset o.vbptr  o.vbte fVtorDisp
           Grand       8       0       4 0
```

图 4.46　Windows 操作系统下使用 cl 工具查看类 A2 对象数据布局

随着 C1 类的介入，产生了三层结构，读者已经知道，只有三层结构才能体现出虚基类设计的用处——孙子类对象中只有一个爷爷类子对象。查看 C1 类的对象数据布局，如图 4.47 所示。

从图 4.47 可以看到这 24 个字节的对象数据布局：

- 类 A1 的虚基类表指针：1～4 字节。
- m_a1：5～8 字节。
- 类 A2 的虚基类表指针：9～12 字节。
- m_a2：13～16 字节。
- m_c1：17～20 字节。
- m_grand：21～24 字节。

```
C:\Users\KuangXiang\Desktop\c++\MySolution\MyProject>cl /d1 reportSingleClassLayoutC1 MyProject.cpp
用于 x86 的 Microsoft (R) C/C++ 优化编译器 19.22.27905 版
版权所有(C) Microsoft Corporation。保留所有权利。

MyProject.cpp

class C1        size(24):
        +---
 0      | +--- (base class A1)
 0      | | {vbptr}
 4      | | m_a1
        | +---
 8      | +--- (base class A2)
 8      | | {vbptr}
12      | | m_a2
        | +---
16      | m_c1
        +---
        +--- (virtual base Grand)
20      | m_grand
        +---

C1::$vbtable@A1@:
 0      | 0
 1      | 20 (C1d(A1+0)Grand)

C1::$vbtable@A2@:
 0      | 0
 1      | 12 (C1d(A2+0)Grand)
vbi:       class  offset o.vbptr  o.vbte fVtorDisp
           Grand      20       0       4 0
```

图 4.47　Windows 操作系统下使用 cl 工具查看类 C1 对象数据布局

看一看 C1 对象的数据布局，应该如图 4.48 所示。

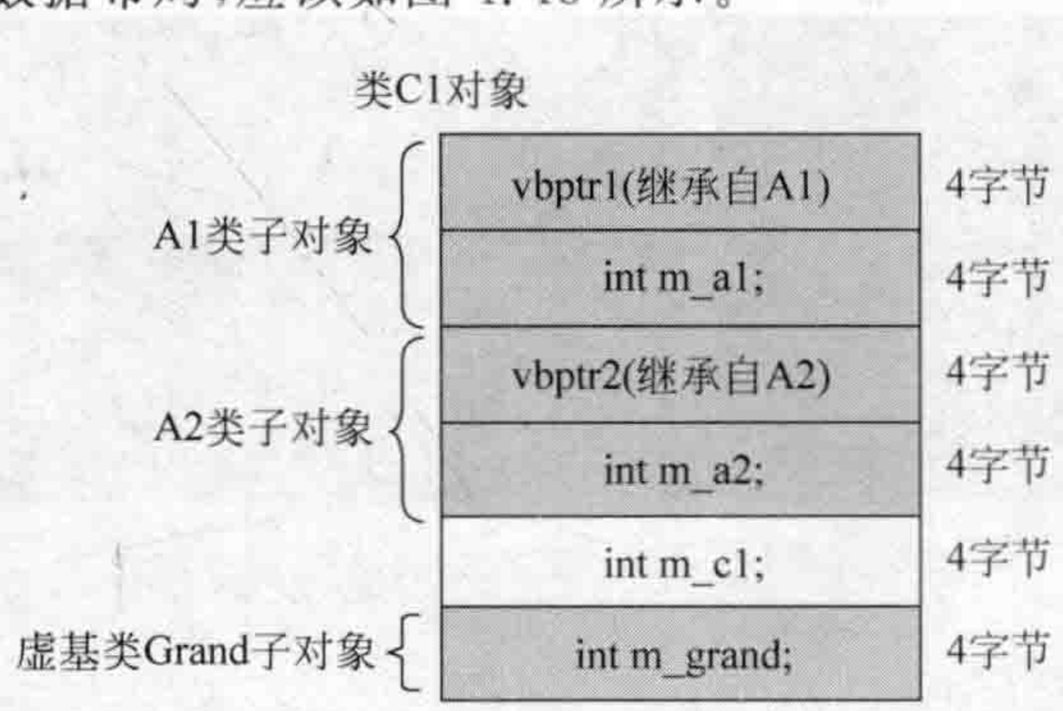

图 4.48　类 C1 对象数据布局

从图 4.48 不难观察到，m_grand 依旧被排在最后。

本节就是提出虚基类问题，并做个初步探索，主要讲述观察到的一些虚基类的内容。

4.9　两层结构时虚基类表内容分析

本节所说的两层结构，指的是有父类 Grand，有子类 A1 和 A2，但没有孙子类 C1 的意思。

虚基类表，一般是在编译的时候，编译器就生成出来了。虚基类表是跟着类走的，依附于类而存在。一般来讲，虚基类表是 8 字节长(编译器就是这样设计的)，4 个字节为一个单位，如果有多个虚基类(class A1 : **virtual** public Grand ,**virtual** public Grand2{…};)，每多一个，虚基类表的长度一般要多加 4 个字节。

4.9.1 虚基类表内容之5～8字节内容分析

上一节最终的范例代码原样保留，本节中要使用。

在图4.45中，已经看到了类A1对象的数据布局。

在main主函数中，加入如下代码行：

```
A1 a1obj;
a1obj.m_grand = 2;
a1obj.m_a1 = 5;
```

读者已经知道，一旦有了虚基类，那么生成该类对象时，对象中就会有一个虚基类表指针(vbptr)，也就是说，对象a1obj中会有一个虚基类表指针(隐藏的成员变量)。

在2.4节讲解过，在有虚基类的情况下，编译器会为类生成一个“合成默认构造函数”。这里，Grand就是虚基类，编译器会为A1类合成一个默认构造函数，并且在这个默认构造函数中，编译器会插入代码，给vbptr这个虚基类表指针赋值(这里就是虚基类表指针得到值的时机)。

将断点设置在A1 a1obj;行，开始调试，当程序执行流程停留在断点行时切换到反汇编窗口，如图4.49所示。

```
        {
            A1 a1obj;
003256E2  push        1
003256E4  lea         ecx,[ebp-14h]
003256E7  call        A1::A1 (0321235h)
            a1obj.m_grand = 2;
003256EC  mov         eax,dword ptr [ebp-14h]    已用时间<=1ms
003256EF  mov         ecx,dword ptr [eax+4]
003256F2  mov         dword ptr [ebp+ecx-14h],2
            a1obj.m_a1 = 5;
003256FA  mov         dword ptr [ebp-10h],5
```

图4.49 生成A1类对象时的反汇编窗口

在图4.49中，注意到创建a1obj对象导致调用了A1::A1构造函数，在**反汇编窗口**中也可以使用快捷键F10单行执行，用快捷键F11(对应“调试”→“逐语句”命令)跳入到函数内部去执行。这里可以使用F11键跳入到A1::A1构造函数中去继续跟踪调试。根据需要配合使用F10、F11键，就可以在A1::A1的构造函数中找到为虚基类表指针赋值的语句行，如图4.50所示。

```
00321EBD  lea         edi,[ebp-0CCh]
00321EC3  mov         ecx,33h
00321EC8  mov         eax,0CCCCCCCCh
00321ECD  rep stos    dword ptr es:[edi]
00321ECF  pop         ecx
00321ED0  mov         dword ptr [this],ecx    已用时间<=1ms
00321ED3  cmp         dword ptr [ebp+8],0
00321ED7  je          A1::A1+32h (0321EE2h)
00321ED9  mov         eax,dword ptr [this]
00321EDC  mov         dword ptr [eax],offset A1::`vbtable' (0329B30h)
00321EE2  mov         eax,dword ptr [this]
```

图4.50 编译器在默认构造函数中插入为虚基类表指针赋值的语句

接着，看看怎样给虚基类(Grand)的成员变量(m_grand)赋值。继续观察图 4.49，针对代码行 a1obj.m_grand = 2;对应的汇编代码，第 1 行 mov eax,dword ptr [ebp-14h]的含义就是把当前对象内存中开始的 4 个字节内容放到 eax 里，可以使用命令“调试”→“窗口”→“寄存器”，打开寄存器窗口观察 eax 的值，如图 4.51 所示。

```
    {
        A1 a1obj;
003256E2  push        1
003256E4  lea         ecx,[ebp-14h]
003256E7  call        A1::A1 (0321235h)
        a1obj.m_grand = 2;
003256EC  mov         eax,dword ptr [ebp-14h]
003256EF  mov         ecx,dword ptr [eax+4]    已用时间<=1ms
003256F2  mov         dword ptr [ebp+ecx-14h],2
        a1obj.m_a1 = 5;
003256FA  mov         dword ptr [ebp-10h],5
100 %
寄存器
EAX = 00329B30 EBX = 00D28000 ECX = 00EFF84C EDX = 00000001 ESI = 003213BB EDI = 00EFF860 EIP = 003256EF

0x00329B34 = 00000008
```

图 4.51　a1obj 对象内存开头的 4 字节内容(虚基类表地址)被放到了 EAX 寄存器中

现在，就等于把虚基类表地址放入到 eax 寄存器里面去了(根据图 4.45，对象 a1obj 内存开始的 4 个字节是虚基类表指针，指向的是虚基类表)。注意，汇编代码中的 ebp-14h 其实代表的就是 a1obj 对象，而[ebp-14h]代表 a1obj 对象首地址中的内容(实际上是前 4 个字节的内容)。

第 2 行 mov ecx,dword ptr [eax+4]的含义是虚基类表首地址，往后移动 4 个字节后(第 5～8 个字节)的内存中的内容给了 ecx 寄存器，执行该行后发现 ecx 中的值变成了 8，如图 4.52 所示。

```
    {
        A1 a1obj;
003256E2  push        1
003256E4  lea         ecx,[ebp-14h]
003256E7  call        A1::A1 (0321235h)
        a1obj.m_grand = 2;
003256EC  mov         eax,dword ptr [ebp-14h]
003256EF  mov         ecx,dword ptr [eax+4]
003256F2  mov         dword ptr [ebp+ecx-14h],2    已用时间<=1ms
        a1obj.m_a1 = 5;
100 %
自动窗口    局部变量    内存 1    内存 2
地址: 0x00329B30
0x00329B30  00 00 00 00 08 00 00 00 00 00 00 00
0x00329B63  00 dc 9e 32 00 10 9f 32 00 01 00 00
0x00329B96  61 72 69 61 62 6c 65 20 27 00 27 20
寄存器
EAX = 00329B30 EBX = 00D28000 ECX = 00000008 EDX = 00000001

0x00EFF854 = CCCCCCCC
```

图 4.52　将虚基类表首地址后移 4 个字节后的内存内容放到了 ECX 寄存器

第 3 行 mov dword ptr [ebp+ecx-14h],2 的含义是什么呢？这里[ebp+ecx-14h]代表 a1obj 对象代表的内存首地址往后移动 8 个字节(因为 ecx 等于 8)后的内存(9～12 字节)，而 2 的含义是把这段内存的内容设置为 2。根据图 4.45，a1obj 对象往后移 8 个字节正好移动到 m_grand 成员变量的地址上去，所以这行汇编代码的含义就是把 2 这个值赋给 m_grand 成员变量。如图 4.53 所示。

图 4.53　将 2 这个值(00 00 00 02)写入到对象 a1obj 所代表内存空间的第 9～12 个字节的位置

所以，观察图 4.53 不难想象，被赋值为 2 的内存就是图 4.54 中最下面的这段内存(m_grand 所代表的内存)：

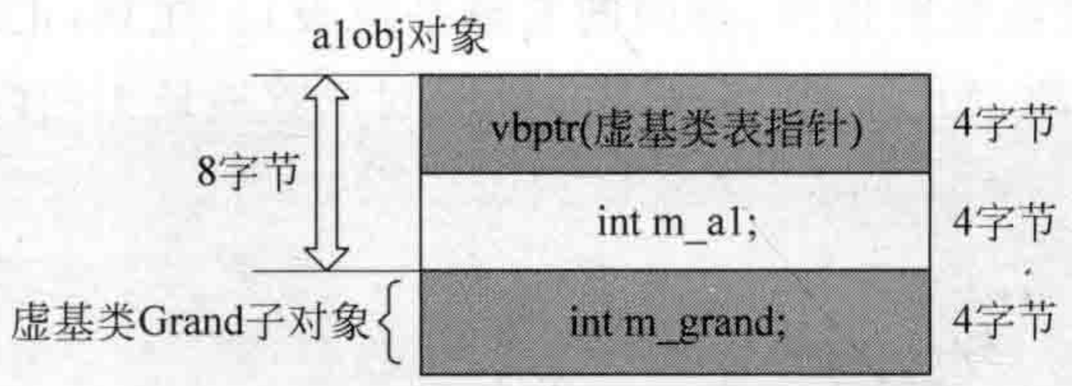

图 4.54　类对象 a1obj 的数据布局

所以，虚基类表中的第 5～8 个字节存储的实际是图 4.54 中的**“虚基类表指针”这个成员变量(vbptr)的首地址和虚基类(Grand)子对象首地址**之间的**偏移量**(两个地址相减后的**差值**)。也就是说，“虚基类表指针”成员变量的首地址＋这个偏移量(通过前面解说汇编代码，本例中这个偏移量的值为 8)就等于这个虚基类对象首地址(其实在这里也就是 m_grand 成员变量的地址)。跳过这个偏移，就能访问到虚基类对象所代表的内存。

4.9.2　各种形式的继承

首先，确认一下当前范例中，Grand、A1、A2、C1 的 sizeof 值分别是 4、12、12、24。

通过上面的学习，对虚基类表第 5～8 字节的内容已经知道是什么了，那第 1～4 字节的内容又是什么呢？

为了研究这个问题，需要增加一些代码。在 MyProject.cpp 的上面，增加一个新类 Grand2：

```
class Grand2
{
public:
    int m_grand2;
};
```

现在 A1 来继承一下 Grand2，但是这里并**不是虚**继承 Grand2，而是普通继承(实继承) Grand2：

```
class A1 : virtual public Grand, public Grand2{ … };
```

此时，Grand、A1、A2、C1 的 sizeof 值分别是 4、16、12、28。

这里尤其要注意的是 A1 的 sizeof 值从 12 变成了 16。在 main 主函数中，继续增加一些代码，来看一看现在的对象布局。

```
A1 a1obj;
a1obj.m_grand = 2;
a1obj.m_grand2 = 6;
a1obj.m_a1 = 5;
```

设置断点，调试观察，用老手法来找内存布局(4.7.2 节对老手法进行过详细的描述)，从而确定对象的布局如下：

```
A1 a1obj;
a1obj.m_grand = 2;                    /13～16 字节
a1obj.m_grand2 = 6;                   //1～4 字节
a1obj.m_a1 = 5;                       //9～12 字节
```

从而推导出虚基类表指针是第 5～8 字节，所以 A1 类对象的数据布局图大概如图 4.55 所示：

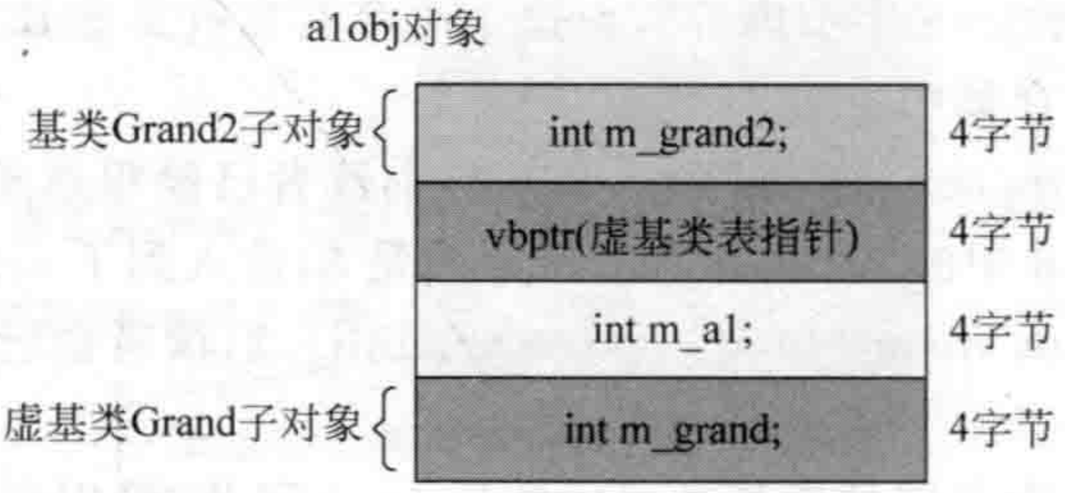

图 4.55　类对象 a1obj 的数据布局

从图 4.55 可以看到，虚基类 Grand 子对象还是在最下面，而基类 Grand2 子对象跑到最上面去了，A1 类对象(m_a1)依旧夹在虚基类表指针 vbptr 和虚基类 Grand 子对象之间。

知道了虚基类表指针是第 5～8 字节后，就可以在调试过程中查看虚基类表的内容了(通过在内存中直接输入虚基类表的地址即可)，如图 4.56 所示。

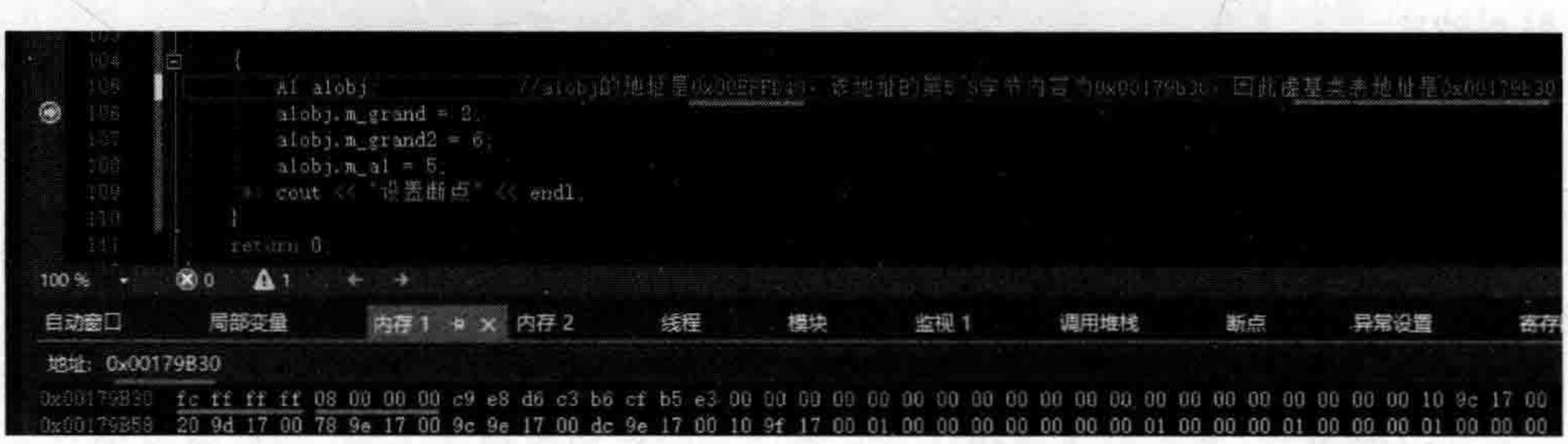

图 4.56　通过对象地址找到虚基类表地址，然后显示虚基类表的内容

从图 4.56 可以看到，第 5～8 字节的内容还是 8，没有变化，但是第 1～4 字节内容发生了改变，以往是 00 00 00 00，现在是 fc ff ff ff，这个值其实是－4，负数一般都是补码存储，所以看起来比较怪。如果向一个 int 型变量中保存一个－4，然后观察一下它的内存，就能看到 fc ff ff ff 这个值。

然后看一下对应的汇编代码，如图 4.57 所示。

```
    {
        A1 a1obj;
00171231  push        1
00171233  lea         ecx,[ebp-18h]
00171236  call        A1::A1 (0171235h)
        a1obj.m_grand = 2;
0017123B  mov         eax,dword ptr [ebp-14h]
0017123E  mov         ecx,dword ptr [eax+4]
00171241  mov         dword ptr [ebp+ecx-14h],2
        a1obj.m_grand2 = 6;
00171248  mov         dword ptr [ebp-18h],6
        a1obj.m_a1 = 5;
0017124F  mov         dword ptr [ebp-10h],5
        cout << "设置断点" << endl;
00171256  mov         esi,esp
00171258  push        offset std::endl<char,std::char_traits<char> > (017142Eh)
```

图 4.57　给 a1obj 对象的 m_grand 成员变量赋值所对应的汇编代码

经过调试、查看寄存器、比对，观察 a1obj.m_grand = 2 这行对应的汇编代码（和上面讲过的 3 行汇编代码非常类似），其实：

（1）第 1 行代码 mov eax,dword ptr [ebp-14h]。又见到了熟悉的 ebp-14h，这里要对 ebp-14h 有一个更细致的理解，ebp-14h 代表的其实是图 4.55 中虚基类表指针指向的地址（也就是对象 a1obj 的第 5～8 字节内容），而这个内容其实就是虚基类表的地址。现在这个地址被放入到了 eax 寄存器中。

（2）第 2 行代码 mov ecx,dword ptr [eax＋4]，读者已经很熟悉了，依旧是虚基类表后 4 个字节的内容（图 4.56 中的 08 00 00 00，其实就是 8）放入到了 ecx 寄存器中。

（3）第 3 行代码 mov dword ptr [ebp＋ecx－14h],2，读者也已经很熟悉了，依旧是给 m_grand 成员变量赋值 2。

不过目前看起来并没有用到虚基类表中的第 1～4 字节，没用到就无须理会。

修改一下类 A1 的继承关系，原来的第一个父亲是虚继承，第二个父亲是普通继承。现在反过来，第一个父亲变成普通继承，第二个父亲变成虚继承：

```
class A1 : public Grand, virtual public Grand2{ … };
```

再次用老手法分析一下 main 主函数中的对象布局：

```
A1 a1obj;
a1obj.m_grand = 2;                    //1～4 字节
a1obj.m_grand2 = 6;                   //13～16 字节
a1obj.m_a1 = 5;                       //9～12 字节
```

从而推导出虚基类表指针是第 5～8 字节，所以 A1 类对象的数据布局图大概如图 4.58 所示。

跟踪调试，发现虚基类表中的内容没有发生变化，依旧是前 4 个字节内容为 fc ff ff ff，后 4 个字节内容为 08 00 00 00，如图 4.56 所示。反汇编代码也大同小异，参考图 4.57，这里不再额外说明。

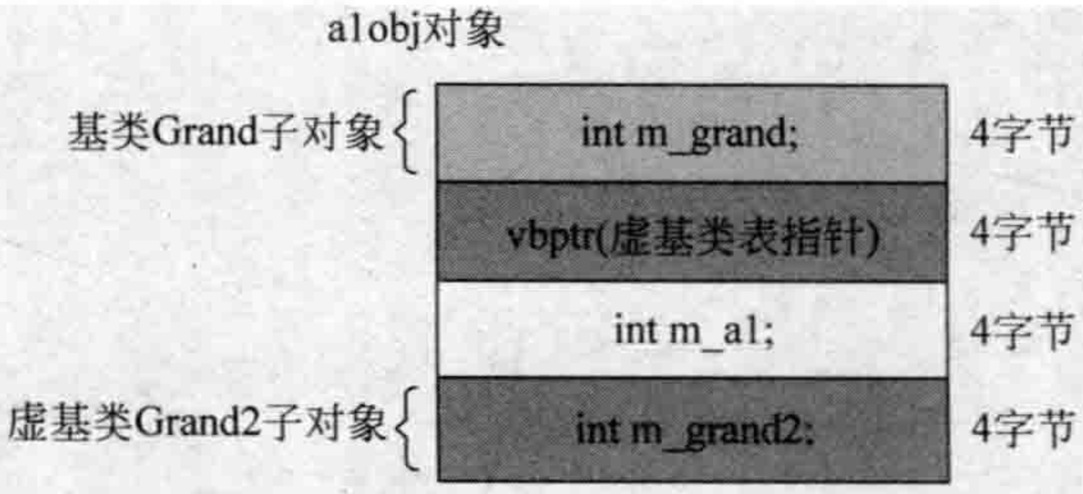

图 4.58　类对象 a1obj 的数据布局

再次修改 A1 类的继承关系，这次继承的两个父亲都变成虚继承：

```
class A1 : virtual public Grand, virtual public Grand2{...};
```

再次用老手法分析一下 main 主函数中的对象布局：

```
A1 a1obj;
a1obj.m_grand = 2;                    //9～12 字节
a1obj.m_grand2 = 6;                   //13～16 字节
a1obj.m_a1 = 5;                       //5～8 字节
```

从而推导出虚基类表指针是第 1～4 字节，所以 A1 类对象的数据布局图大概如图 4.59 所示。

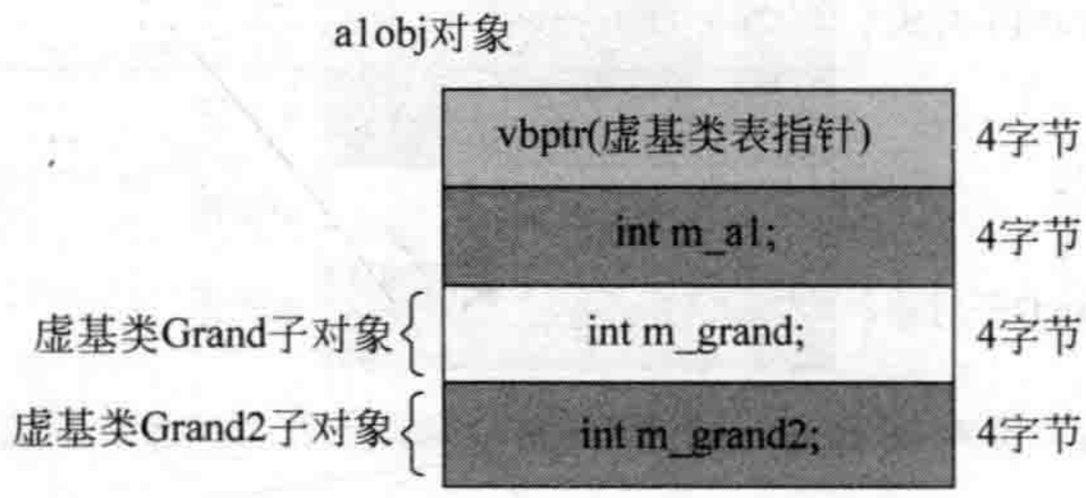

图 4.59　类对象 a1obj 的数据布局

设置断点并调试执行，看一看虚基类表中的内容，如图 4.60 所示。这里关注从虚基类表首地址开始的一共 12 个字节的内容。

图 4.60　继承自两个虚基类时 A1 类对应的虚基类表中的内容

再看一下对应的汇编代码，如图 4.61 所示。

观察图 4.61 可以得到一些结论：

(1) 虚基类表现在应该是有 3 项(如果按每一项有 4 字节计算)。因为图 4.61 的汇编代码用到了取虚基类表中＋4 或者＋8 字节后的内存内容作为偏移值，来计算成员变量 m_grand 和 m_grand2 的首地址并为它们赋值。

```
    {
        A1 a1obj;              //a1obj的地址是0x00EFFD48，该地址的第5~8字节内容为0x00179b30，因此虚基类表地址是0x00179b30
00922084  lea         ecx,[ebp-10h]
00922087  call        A1::A1 (0921235h)
        A1 a2obj;
0092208C  push        1
0092208E  lea         ecx,[ebp-30h]
00922091  call        A1::A1 (0921240h)
        a1obj.m_grand = 2;
00922096  mov         eax,dword ptr [ebp-10h]
00922099  mov         ecx,dword ptr [eax+4]
0092209C  mov         dword ptr [esp+ecx-10h],2
        a1obj.m_grand2 = 6;
009220A4  mov         eax,dword ptr [ebp-10h]
009220A7  mov         ecx,dword ptr [eax+8]
009220AA  mov         dword ptr [esp+ecx-10h],6
        a1obj.m_a1 = 5
009220B2  mov         dword ptr [ebp-14h],5
        cout << "设置断点" << endl;
```

图 4.61　给 a1obj 对象的 m_grand 和 m_grand2 成员变量赋值所对应的汇编代码

(2) 虚基类表中的偏移量信息是按照继承顺序来存放的。

(3) 虚基类子对象一直都放在最下面。

4.9.3　虚基类表内容之 1～4 字节内容分析

在图 4.56 中，看到了虚基类表中第 1～4 字节内容为 fc ff ff ff(－4)的情形，再次参考图 4.58 并把这个图进一步绘制一下，如图 4.62 所示。

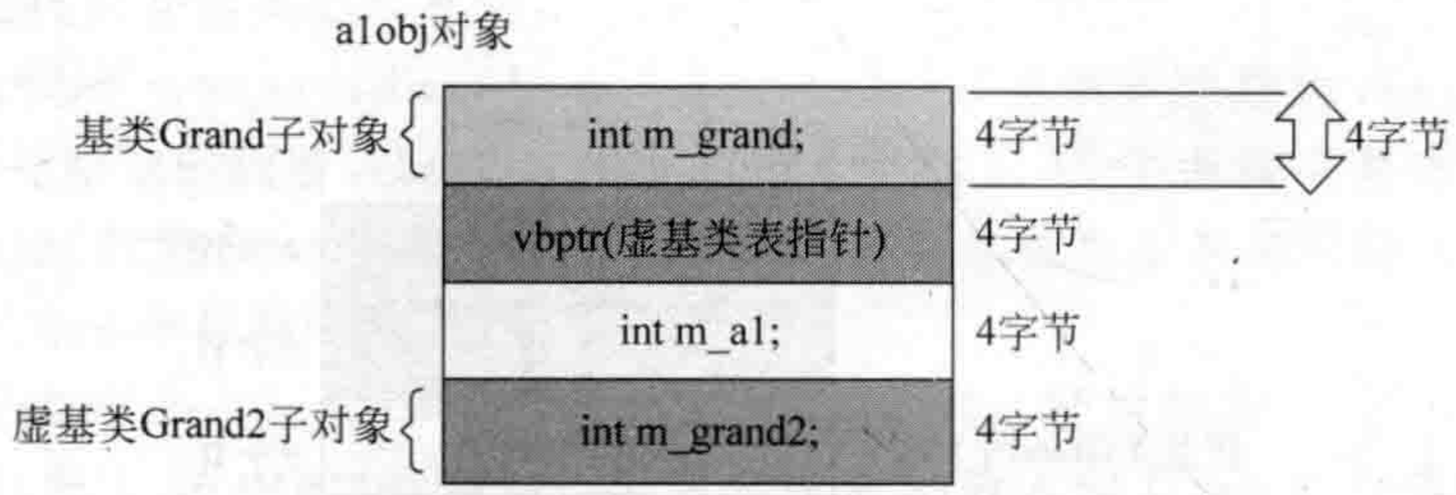

图 4.62　类对象 a1obj 的数据布局(实继承 Grand，虚继承 Grand2)

所以，经过各种比较和思考，虚基类表内容的前 4 个字节应该是对象 a1obj 首地址与“虚基类表指针 vbptr”成员变量的首地址之间的偏移量(差值)。例如，本对象首地址为 0x0085f818，而 vbptr 成员变量的首地址为 0x0085f81C(其实在图 4.62 中就是对象首地址往后移动 4 个字节，即 0x0085f818＋4)，那么，0x0085f818－0x0085f81C，结果就是－4。

上面分析了不少情况，也分析了对应的汇编代码，不知读者是否发现一个规律：**只有对虚基类的成员变量进行处理如赋值的时候，才会用到虚基类表**——取其中的值用作偏移值来进行虚基类成员变量首地址的定位运算。

4.10　三层结构时虚基类表内容分析与虚基类设计原由

4.10.1　三层结构时虚基类表内容分析

上一节研究分析了两层结构时虚基类表的内容，本节再回过头来看一看传统的三层结构(爷爷、父亲、孙子)时虚基类表的内容。为简单清晰起见，本节重新书写范例代码(实际上就是 4.8 节后期的范例代码)。在 MyProject.cpp 的上面，增加如下代码行：

```
class Grand                              //爷爷类
{
public:
    int m_grand;
};

class A1 : virtual public Grand          //注意这里用了 virtual
{
public:
    int m_a1;
};
class A2 : virtual public Grand          //注意这里用了 virtual
{
public:
    int m_a2;
};
class C1 :public A1, public A2           //这里不需要 virtual
{
public:
    int m_c1;
};
```

在 4.8.2 节已经分析过传统虚基类结构的三层对象布局，其中 sizeof(C1)的值是 24。这 24 个字节的布局如图 4.48 所示。

在 main 主函数中，加入如下代码：

```
cout << sizeof(Grand)'<< endl;
cout << sizeof(A1) << endl;
cout << sizeof(A2) << endl;
cout << sizeof(C1) << endl;
C1 c1obj;
c1obj.m_grand = 2;
c1obj.m_a1 = 5;
c1obj.m_a2 = 6;
c1obj.m_c1 = 8;
```

对象 c1obj 的整个 24 字节内存内容如图 4.63 所示。

图 4.63　对象 c1obj 的整个 24 字节内存内容

看一下 vbptr1，在 1～4 字节，地址是 0x005E9B48，内存里的内容如图 4.64 所示。

看一下 vbptr2，在 9～12 字节，地址是 0x005E9B54，内存里的内容如图 4.65 所示。

图 4.64 C1 类所关联的第一个虚基类表的内容

图 4.65 C1 类所关联的第二个虚基类表的内容

现在结合上面的这些内容，可以把图 4.48 进一步完善一下，如图 4.66 所示。

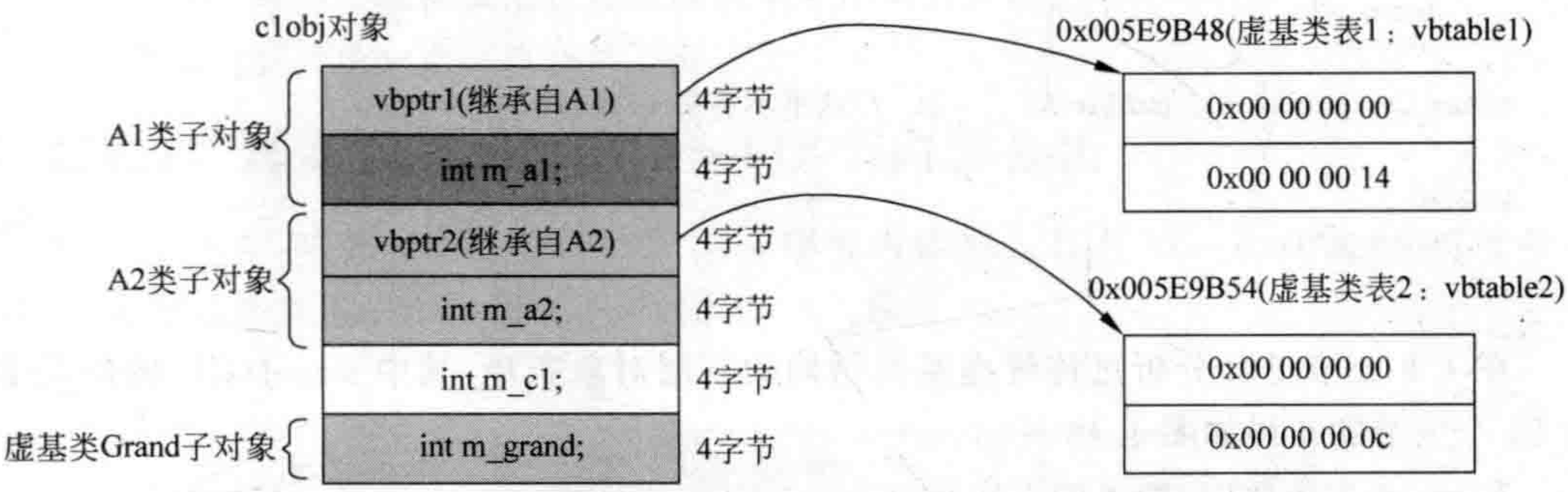

图 4.66 类 C1 对象数据布局以及其所指向的两个虚基类表

图 4.66 中的这种 0x005E9B48、0x005E9B54 就是虚基类表的首地址，这种虚基类表读者已经知道，它编译的时候就已生成好（跟着 C1 类走并依附于该类），这些地址也都是固定的。如果在 main 主函数中创建一个新对象，例如：

```
C1 c2obj;
```

可以发现，c2obj 的虚基类表指针的内容与 c1obj 完全一样，说明 c1obj 和 c2obj 所指向的虚基类表完全相同，这一点读者要认识到。

图 4.67 创建类 C1 对象 c1obj 时的反汇编代码，即将调用类 C1 的构造函数

可以进入到 C1 类的构造函数去看一看。将断点设置在 C1 c1obj;代码行，按 F5 键开始调试，当程序执行流程停到断点行时，切换到反汇编窗口，如图 4.67 所示。

按 F10 键向下走两行，遇到构造函数 C1::C1 时按 F11 键走进该构造函数里面去（这里根据具体编译器不同可能还需要再按一次 F11 键）。

类 C1 构造函数中的汇编代码如图 4.68 所示。

从图 4.68 可以看到，在类 C1 的构造函数里，给这两个虚基类表指针（vbptr1 和 vbptr2）赋值，内容正好应该是刚刚看到的内存（0x005E9B48、0x005E9B54）中的内容，然后这里又调用了 A1 和 A2 类的构造函数。

```
005E1F60  push        ebp     已用时间 <= 1ms
005E1F61  mov         ebp,esp
005E1F63  sub         esp,0CCh
005E1F69  push        ebx
005E1F6A  push        esi
005E1F6B  push        edi
005E1F6C  push        ecx
005E1F6D  lea         edi,[ebp-0CCh]
005E1F73  mov         ecx,33h
005E1F78  mov         eax,0CCCCCCCCh
005E1F7D  rep stos    dword ptr es:[edi]
005E1F7F  pop         ecx
005E1F80  mov         dword ptr [this],ecx
005E1F83  cmp         dword ptr [ebp+8],0
005E1F87  je          C1::C1+3Ch (05E1F9Ch)
005E1F89  mov         eax,dword ptr [this]
005E1F8C  mov         dword ptr [eax],offset C1::`vbtable' (05E9B48h)
005E1F92  mov         eax,dword ptr [this]
005E1F95  mov         dword ptr [eax+8],offset C1::`vbtable' (05E9B54h)
005E1F9C  push        0
005E1F9E  mov         ecx,dword ptr [this]
005E1FA1  call        A1::A1 (05E123Ah)
005E1FA6  push        0
005E1FA8  mov         ecx,dword ptr [this]
005E1FAB  add         ecx,8
005E1FAE  call        A2::A2 (05E128Ah)
005E1FB3  mov         eax,dword ptr [this]
005E1FB6  pop         edi
005E1FB7  pop         esi
005E1FB8  pop         ebx
005E1FB9  add         esp,0CCh
005E1FBF  cmp         ebp,esp
```

图 4.68　类 C1 构造函数中所做的事

在反汇编窗口继续按 F10 键一行一行跟踪代码行的执行，遇到构造函数 A1::A1 时按 F11 键走进该构造函数里面去看一看(这里根据具体编译器不同可能还需要再按一次 F11 键)，如图 4.69 所示。

```
005E1EC0  push        ebp     已用时间 <= 1ms
005E1EC1  mov         ebp,esp
005E1EC3  sub         esp,0CCh
005E1EC9  push        ebx
005E1ECA  push        esi
005E1ECB  push        edi
005E1ECC  push        ecx
005E1ECD  lea         edi,[ebp-0CCh]
005E1ED3  mov         ecx,33h
005E1ED8  mov         eax,0CCCCCCCCh
005E1EDD  rep stos    dword ptr es:[edi]
005E1EDF  pop         ecx
005E1EE0  mov         dword ptr [this],ecx
005E1EE3  cmp         dword ptr [ebp+8],0
005E1EE7  je          A1::A1+32h (05E1EF2h)
005E1EE9  mov         eax,dword ptr [this]
005E1EEC  mov         dword ptr [eax],offset A1::`vbtable' (05E9B30h)
005E1EF2  mov         eax,dword ptr [this]
005E1EF5  pop         edi
005E1EF6  pop         esi
005E1EF7  pop         ebx
005E1EF8  mov         esp,ebp
```

图 4.69　类 A1 构造函数中所做的事

图 4.69 中 A1 的构造函数里似乎也有一个设置虚基类表指针的语句，但这个语句前面有一个 je 跳转语句，这个跳转语句会跳过这里的设置虚基类表指针的语句使其不被执行。构造函数 A2::A2 也是同理。所以，看得出来，给虚基类表指针值的时机实际上是在孙子类构造函数 C1::C1 里面。

把断点设置在 c1obj.m_grand = 2;代码行，跟踪调试执行，看看其对应的反汇编代码，如图 4.70 所示。

```
        C1 clobj;
005E6086  push        1
005E6088  lea         ecx,[ebp-20h]
005E608B  call        C1::C1 (05E111Dh)
        clobj.m_grand = 2;
005E6090  mov         eax,dword ptr [ebp-20h]
005E6093  mov         ecx,dword ptr [eax+4]
005E6096  mov         dword ptr [ebp+ecx-20h],2
        clobj.m_a1 = 5;
005E609E  mov         dword ptr [ebp-1Ch],5
        clobj.m_a2 = 6;
005E60A5  mov         dword ptr [ebp-14h],6
        clobj.m_c1 = 8;
005E60AC  mov         dword ptr [ebp-10h],8
        cout << "设置断点" << endl;
```

eax=0x005E9B48
ecx=0x14=20
已用时间<=1ms

图 4.70　给 clobj 对象的 m_grand 成员变量赋值所对应的汇编代码

这里读者其实也发现了一个问题，vbptr2 实际是没有用到的，只用到了 vbptr1。

所以以往也谈过，这种三层结构解决了只保留一份爷爷子对象的问题，但同时引入了虚基类表指针，额外占用了内存空间（当然编译器编译时还要生成虚基类表等也同样要多占用内存）。同时，程序员要访问虚基类的成员变量，是需要通过虚基类表指针进行偏移计算的，所以在这里读者看得很清楚，上面的代码中，访问 m_a1、m_a2、m_c1 等成员变量的速度明显要比访问 m_grand 快。将来在面试时，若面试官询问：在什么情况下，访问某个成员变量比访问其他成员变量要慢？就可以回答：访问虚基类的成员变量要比访问其他成员变量更慢。

4.10.2　虚基类为什么这样设计

有些读者可能好奇，虚基类引入了很多复杂性，但是虚基类设计得如此复杂，为什么这样设计？

这是一个非常难以回答的问题，C++标准只是规定了一些框架性的东西，而编译器内部的具体实现，每一家编译器开发商都有自己的做法，上面的这些分析针对的是 Visual Studio 2019 编译器开发商微软公司具体针对虚基类的一个实现方法。

其他编译器厂商可能会有不同的实现方法，例如可能把虚基类表里的偏移值放到虚函数表里去，那就不用单独设计虚基类表和虚基类表指针了。总之，实现方法可能千差万别。

有一点必须要承认，编译器开发商对各种 C++模型细节的认识程度要远远超过任何常人，否则他们根本无法开发出编译器来，他们为什么这样实现虚基类，会考虑到非常多的因素。例如，原本的三层结构（虚基类引入之前）中，是有两个 Grand 子对象（参考图 4.35）存在的，虚继承后就变成了一个 Grand 子对象（参考图 4.38），这种两个 Grand 子对象变成一个是怎样变的？C1 对象布局里面父类 A1 和 A2 子对象以及类 C1 子对象中的成员变量如何访问？对象指针之间如何赋值（例如让父类 A1 的对象指针指向子类 C1 对象）？同时还要考虑其他继承关系如 C1 类通过 A1 和 A2 虚继承 Grand 类，但还可能同时普通继承其他的类，如图 4.71 所示这种继承关系图。

如果类 C1 下面还有子类（如 D1 类），那这种虚基类 Grand 的成员变量在对象布局中的位置可能又要放到 D1 类成员变量的后面。

另外，还有父类指针指向子类对象的情形，如下面代码的处理：

```
A2 * pobja2 = new C1();
```

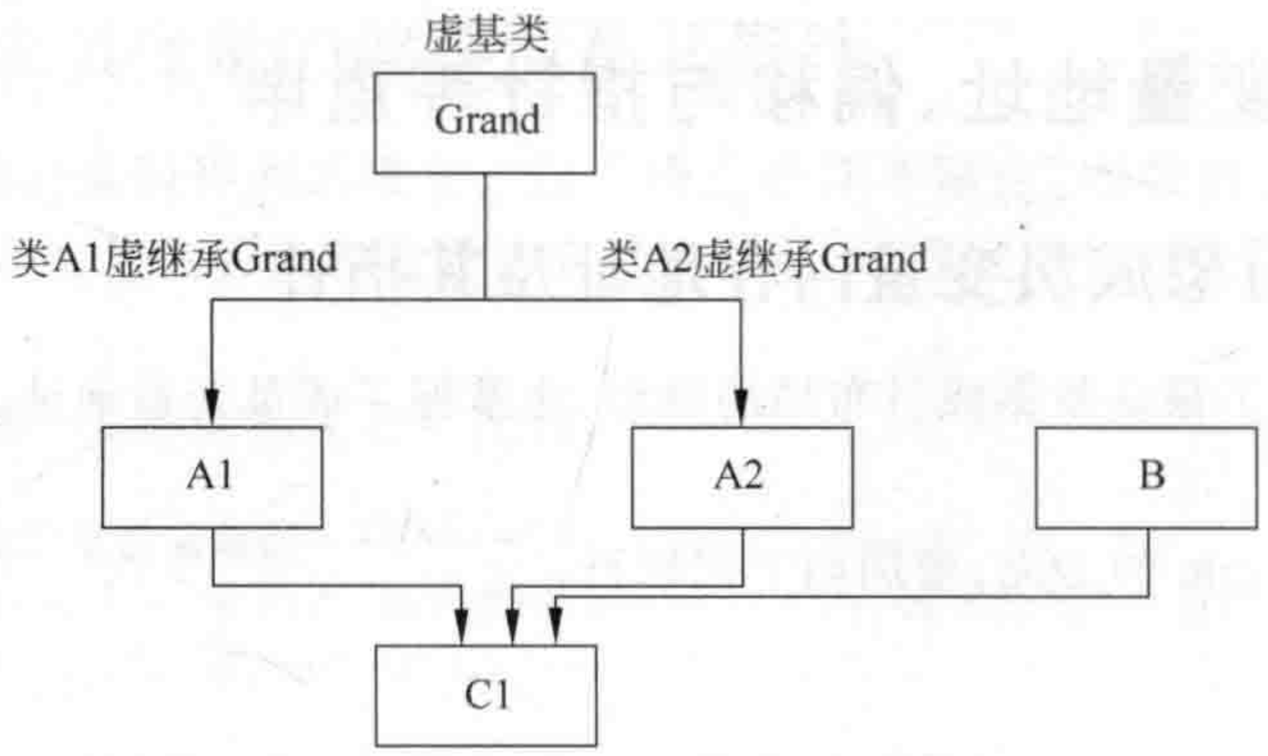

图 4.71　虚继承与普通继承混合在一起

所以，虚基类用起来觉得挺简单，但是编译器内部的实现绝不像表面看上去那样简单。向 main 主函数中继续增加一些代码，让父类指针指向子类对象：

```
A2 * pa2 = &c1obj;
pa2 -> m_grand = 8;
pa2 -> m_a2 = 9;
```

可以在上面这三行代码中的任何一行设置一个断点，开始调试并观察。自然观察得到，pa2 所指向的内存地址并不是 c1obj 对象的首地址(指向的是图 4.66 中的 A2 类子对象的首地址)。例如，c1obj 对象的首地址可能是 0x00adf9fc，而此时，pa2 所指向的内存地址却是 0x00adfa04。0x00adfa04 − 0x00adf9fc = 8(这 8 个字节其实就是图 4.66 中的 A1 类子对象所占的 8 个字节)。切换到反汇编窗口，如图 4.72 所示。

```
        A2* pa2 = &c1obj;
00AF60B3  lea           eax,[ebp-20h]
00AF60B6  test          eax,eax
00AF60B8  je            main+128h (0AF60C8h)
00AF60BA  lea           ecx,[ebp-20h]
00AF60BD  add           ecx,8
00AF60C0  mov           dword ptr [ebp-128h],ecx
00AF60C6  jmp           main+132h (0AF60D2h)
00AF60C8  mov           dword ptr [ebp-128h],0
00AF60D2  mov           edx,dword ptr [ebp-128h]
00AF60D8  mov           dword ptr [ebp-2Ch],edx
        pa2->m_grand = 8;
00AF60DB  mov           eax,dword ptr [ebp-2Ch]
00AF60DE  mov           ecx,dword ptr [eax]
00AF60E0  mov           edx,dword ptr [ecx+4]
00AF60E3  mov           eax,dword ptr [ebp-2Ch]
00AF60E6  mov           dword ptr [eax+edx],8
        pa2->m_a2 = 9;
00AF60ED  mov           eax,dword ptr [ebp-2Ch]
00AF60F0  mov           dword ptr [eax+4],9
        cout << "设置断点" << endl;
```

图 4.72　父类指针指向子类对象，给虚基类成员变量赋值要经过好多步骤

从图 4.72 可以看到，pa2-> m_grand = 8；这行代码对应的汇编代码中出现的 ebp-2Ch 其实就是 pa2，因为 pa2 开始的 4 个字节里保存的是第二个虚基类表的首地址，所以，其实 pa2-> m_grand = 8；这行代码用到的是图 4.66 中的**第二个虚基类表**的第 5～8 个字节做偏移值来确定 m_grand 成员变量的首地址，从而为其赋值。

凡是访问虚基类的成员变量，都要借助虚基类表来查询偏移量，才能确定虚基类成员变量的内存位置，比访问普通的成员要多了好几步，所以访问速度上肯定是要慢一些的。

这里也不可能面面俱到地带着读者把各种情况下的对象数据布局全部分析到，如果有兴趣，可以通过上面这些讲解，举一反三地去研究学习。例如，如果在类 Grand 中有虚函数，或者在类 A1、A2 中有虚函数，那么可能又多出了虚函数表指针等内容，这个由读者自己扩展分析。

4.11 成员变量地址、偏移与指针等重申

4.11.1 对象成员变量内存地址及其指针

前面已经看到了很多数据成员布局的情形，也掌握了成员变量地址、成员变量偏移值的概念，回顾一下。

在 MyProject.cpp 的上面，增加如下代码行：

```
class MYACLS
{
public:
    int m_i;
    int m_j;
    int m_k;
};
```

在 main 主函数中，加入如下代码：

```
MYACLS myobj;
myobj.m_i = myobj.m_j = myobj.m_k = 0;
printf("myobj.m_i =  %p\n", &myobj.m_i); //用对象名.成员变量名，这里输出成员变量的内存
                                         //地址

MYACLS * pmyobj = new MYACLS();
printf("pmyobj->m_i =  %p\n", &pmyobj->m_i);
printf("pmyobj->m_j =  %p\n", &pmyobj->m_j);

int * p1 = &myobj.m_i; //对象成员变量指针
int * p2 = &pmyobj->m_j;
*p1 = 15;
*p2 = 30;
printf("p1 地址 =  %p,p1 值 = %d\n", p1, *p1);
printf("p2 地址 =  %p,p2 值 = %d\n", p2, *p2);
```

执行起来，看一看结果：

```
myobj.m_i = 00AFFDB4
pmyobj->m_i = 00EE4FC8
pmyobj->m_j = 00EE4FCC
p1 地址 = 00AFFDB4,p1 值 = 15
p2 地址 = 00EE4FCC,p2 值 = 30
```

通过上面的演示可以看到：

对象所属的成员变量的表现像一个独立的变量一样，有真正的内存地址，可以往地址中保存值，**可以用一个指针指向对象的成员变量**，也可以通过一个指针来操纵对象的成员变量，如给对象成员变量赋值等。

4.11.2　成员变量的偏移值及其指针

再谈一谈成员变量偏移值的概念。这个概念应该不陌生，因为前面章节已经演示过。读者已经知道，成员变量偏移值和对象没有什么关系，不要把它和对象所属的成员变量混为一谈。

在 main 主函数中，继续加入如下代码：

```
cout << "打印成员变量偏移值 ----------------" << endl;
printf("MYACLS::m_i =  %d\n", &MYACLS::m_i);     //打偏移值用 %d 更醒目一些
printf("MYACLS::m_j =  %d\n", &MYACLS::m_j);
```

执行起来，看一看结果(摘取新增加的代码产生的结果)：

```
打印成员变量偏移值 ----------------
MYACLS::m_i = 0
MYACLS::m_j = 4
```

用成员变量指针来输出偏移值也可以。要知道，**成员变量指针**里面保存的其实是**偏移值**，并不是真正的内存地址。

在 main 主函数中，继续加入如下代码：

```
//成员变量指针
int MYACLS::* mypoint = &MYACLS::m_j;
printf("MYACLS::m_j 偏移值 =  %d\n", mypoint);

mypoint = &MYACLS::m_i;
printf("MYACLS::m_i 偏移值 =  %d\n", mypoint);
```

执行起来，看一看结果(摘取新增加的代码产生的结果)：

```
MYACLS::m_j 偏移值 = 4
MYACLS::m_i 偏移值 = 0
```

4.11.3　成员变量指针和不指向任何成员变量的成员变量指针

从上面的范例中可以看到指向第一个类中成员变量的成员变量指针，可以设置一个断点看看它的值，发现它的值是 0(0x00000000)，如图 4.73 所示。

图 4.73　成员变量指针指向类中第一个成员变量的情形

既然成员变量指针指向了某个成员变量，那么就可以通过一个对象名或者对象指针来访问某个对象的成员变量。在 main 主函数中，继续加入如下代码：

```
myobj.*mypoint = 22;                    //注意写法
pmyobj->*mypoint = 19;                  //注意写法
```

在函数调用中也可以使用成员变量指针作为形参。在 MyProject.cpp 的上面，增加如下代码：

```
void myfunc(int MYACLS::* mempoint, MYACLS& obj)
{
    obj.*mempoint = 260;                //注意写法
}
```

在 main 主函数中，继续加入如下代码：

```
myfunc(mypoint, myobj);                 //注意调用方法
myfunc(mypoint, *pmyobj);               //注意调用方法
//看一看 mypoint 的 sizeof 值
cout << "sizeof(mypoint) = " << sizeof(mypoint) << endl;  //也是 4 字节
```

这时笔者有一点好奇：一个指向 NULL 的成员变量指针是否会和指向一个数据成员的成员变量指针相同呢？

在 main 主函数中，继续加入如下代码：

```
int MYACLS::* mypoint2;
```

设置断点在该代码行的下一行，并开始进行调试，当执行完该代码行后，将鼠标移动到这个成员变量指针上，观察其值，如图 4.74 所示。

图 4.74　未经初始化的成员变量指针编译器默认给其值为 0xcccccccc

从图 4.74 可以看到，未经初始化的成员变量指针，编译器给的默认值是 0xcccccccc。

当给它赋值成 0 或者 NULL，调试时可以观察到其值是 0xffffffff，但如果输出结果的话，结果会显示为－1(0xffffffff 就是－1)。在 main 主函数中，继续加入如下代码：

```
mypoint2 = 0;
mypoint2 = NULL;
printf("mypoint2 = %d\n", mypoint2); //-1(0xffffffff)
```

设置断点在上述两行代码行的下一行，并开始进行调试，当执行完上述赋值代码行后，将光标移动到这个成员变量指针上，观察其值，如图 4.75 所示。

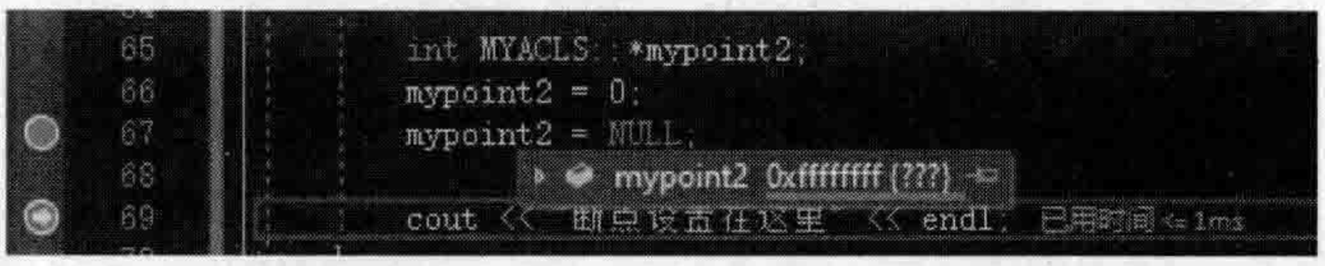

图 4.75　给成员变量指针 0 或者 NULL，则编译会显示其值为 0xffffffff

这说明：赋值给该成员变量指针(mypoint2)一个 0 值或者 NULL，编译器内部给它转换成了－1 而不是 0。编译器的这种行为应该不难理解，因为成员变量指针是 0(偏移值是 0)意味着这个成员变量指针所指向的成员变量是该类对象数据布局中的第一个成员变量(前面再没有其他的人为的或者系统增加进去的成员变量)，如不能有虚函数，不然系统会向类中增加虚函数表指针这种隐含的成员变量。

所以为了区分类对象数据布局中的指向第一个成员变量的指针和一个空的成员变量指针，系统做了处理，一个空的成员变量指针，就让其值为 0xffffffff。

根据代码 mypoint = &MYACLS::m_i;，这表示 mypoint 指向的是类 MYACLS 中的第一个成员变量。而根据代码 mypoint2 = NULL;，这表示 mypoint2 没有指向任何类 MYACLS 中的成员变量，所以代码行 if (mypoint == mypoint2)这种条件显然不会成立。但是下面两行代码，显然条件会成立(因为这两个成员变量指针都指向 m_i 成员变量)：

```
int MYACLS::* mypoint10 = &MYACLS::m_i;
if (mypoint == mypoint10)                    //本条件是成立的
{
    cout << "条件成立" << endl;
}
```

可能有些读者会想，既然 mypoint2 等于－1，那加 1 是不是会变成 0？经过测试，下面三行代码，编译器都会报错。

```
mypoint2 += 1;                               //语法错误，普通指针可以，但成员变量指针不可以
mypoint2++;                                  //语法错误
mypoint2 = ( (&MYACLS::m_i) + 1);   //语法错误
```

既然语法上不允许，那只能作罢。

第5章

函数语义学

针对函数话题，本章针对普通成员函数、虚函数、静态成员函数的调用进行详细的阐述。

针对 C++多态性这个话题，本章从代码实现上和表现形式上给出了详细的定义和解释，让读者对 C++的多态性到底是什么有一个深入的了解，这极其重要，不但能够应付大量针对此话题的面试考题，也能够为读者将来学习设计模式相关知识打下坚实的基础。

本章同时用大量篇幅叙述多重继承情况下带来的复杂问题并详细分析为什么父类的析构函数一定要写成虚析构函数，这经常作为 C++高级程序开发人员最重要的面试考题之一。

此外，本章也对 RTTI、vcall thunk、inline 等问题都有涉及，相信读者在学习后会收益良多。

5.1 普通成员函数调用方法

看看如下范例。在 MyProject.cpp 的上面，增加如下代码行：

```
class MYACLS
{
public:
    int m_i;
    void myfunc(int abc)
    {
        m_i += abc;
    }
};
```

再继续增加一个全局函数，要实现上述成员函数 myfunc 相同的功能，就得多传递一个 MYACLS * 类型的参数进来。看看代码：

```
//全局函数 gmyfunc
void gmyfunc(MYACLS * ptmp, int abc)
{
    ptmp->m_i += abc;
}
```

在 main 主函数中，加入如下代码：

```
MYACLS myacls;
myacls.myfunc(18);                    //调用成员函数
gmyfunc(&myacls, 18);                 //调用全局函数
```

从上面的代码可以看到，一个是调用成员函数，一个是调用全局函数。这两种调用方式的调用效率有什么不同吗？

初看上去凭直觉，总觉得调用成员函数要比调用全局函数慢。

实际上不是这样，C++语言在设计的时候就有这样一个要求：对普通成员函数的调用不应该比对全局函数的调用效率差。所以，编译器内部实际是**将对成员函数 myfunc 的调用转换成了一种对全局函数的调用**。

读者已经知道，成员函数与全局函数一样，有**独立的内存地址**，并且这些函数的内存地址是在编译的时候就确定好的。基于此，编译器把成员函数转换成全局函数似乎也就很简单。不妨站在**编译器视角**看一看**成员函数被编译器转化为全局函数**后的样子：

```
void myfunc(MYACLS * const this, int abc) //编译器会额外安插一个 this 指针，一般放到参数列表的开头
{
    this->m_i += abc;
}
```

这里注意：

(1) 编译器额外增加了一个叫作 this 的形参，是一个指针类型，指向的其实就是生成的那个对象，**该形参对于确定对象所属的某个成员变量在内存中的位置有用**。

(2) 常规成员变量的存取都通过 this 形参来进行，如上述的 **this->** m_i。

(3) 设置断点到 myacls.myfunc(18);代码行，开始调试，当程序执行流程停到断点行时，切换到反汇编窗口，看到如图 5.1 中的地址 0A510F0h 就是这个成员函数的地址。这个地址是编译时就确定好的。

```
    {
        MYACLS  myacls;
        myacls.myfunc(18);    //调用成员函数
00A54732  push        12h
00A54734  lea         ecx,[ebp-0Ch]
00A54737  call        MYACLS::myfunc (0A510F0h)
        gmyfunc(&myacls, 18); //调用全局函数
00A5473C  push        12h
00A5473E  lea         eax,[ebp-0Ch]
00A54741  push        eax
00A54742  call        gmyfunc (0A51055h)
00A54747  add         esp,8
        printf("MYACLS::myfunc地址=%p\n", &MYACLS::myfunc);
00A5474A  push        offset MYACLS::myfunc (0A510F0h)
00A5474F  push        offset string "MYACLS::myfunc\xb5\xd8\xd6\xb7=%p\n" (0A57BCCh)
00A54754  call        _printf (0A5137Fh)
00A54759  add         esp,8
    }
```

图 5.1　调用成员函数对应的反汇编代码

加入如下代码，可以把这个函数地址输出出来：

```
printf("MYACLS::myfunc 地址 = %p\n", &MYACLS::myfunc);  //MYACLS::myfunc 地址 = 00A510F0
```

可以在 Linux 操作系统下编译运行看一下，如图 5.2 所示。

```
kuangxiang@bogon:/mnt/hgfs/linux$ g++ MyProject.cpp -o MyProject
MyProject.cpp: In function 'int main()':
MyProject.cpp:34:54: warning: format '%p' expects argument of type 'void*', but argument 2
   printf("MYACLS::myfunc地址=%p\n", &MYACLS::myfunc);
                                                     ^
kuangxiang@bogon:/mnt/hgfs/linux$ ./MyProject
MYACLS::myfunc地址=0x40087a
kuangxiang@bogon:/mnt/hgfs/linux$
```

图 5.2　Linux 操作系统下输出成员函数地址的代码执行结果

这里再次使用 Linux 操作系统下的 nm 命令，列出可执行文件中的全局变量、函数存放地址等信息，如图 5.3 所示。

图 5.3 显示的成员函数地址为 40087a，与图 5.2 显示的 0x40087a 是相同的结果。这说明该地址是编译的时候就确定好的。

所以，最终这种调用成员函数的代码行 myacls.myfunc(18);可能会变成下面的样子：

```
_ZN6MYACLS6myfuncEi(&myacls,18);
```

也就是说，**调用成员函数最终的感觉还是相当于调用了一个全局函数。**

如果查看反汇编代码也可以看得出来，如图 5.4 所示。

```
kuangxiang@bogon:/mnt/hgfs/linux$ nm MyProject
0000000000601058 B __bss_start
0000000000601058 b completed.7594
                 U __cxa_atexit@@GLIBC_2.2.5
0000000000601048 D __data_start
0000000000601048 W data_start
00000000004006b0 t deregister_tm_clones
0000000000400730 t __do_global_dtors_aux
0000000000600e08 t __do_global_dtors_aux_fini_array_entry
0000000000601050 D __dso_handle
0000000000600e18 d _DYNAMIC
0000000000601058 D _edata
0000000000601060 B _end
0000000000400776 T _Z7gmyfuncP6MYACLSi
000000000040087a W _ZN6MYACLS6myfuncEi
                 U _ZNSt8ios_base4InitC1Ev@@GLIBCXX_3.4
                 U _ZNSt8ios_base4InitD1Ev@@GLIBCXX_3.4
```

图 5.3　用 nm 命令查看成员函数的地址

```
        MYACLS  myacls;
        myacls.myfunc(18);    //调用成员函数
00A54732  push        12h
00A54734  lea         ecx,[ebp-0Ch]
00A54737  call        MYACLS::myfunc (0A510F0h)
```

图 5.4　调用成员变量对应的汇编代码

在图 5.4 中，代码行 myacls.myfunc(18);对应的汇编代码中 lea ecx,[ebp-0Ch]代码行的目的是把对象 myacls 的首地址放到 ecx 寄存器中（相当于把 this 指针放入了 ecx）。例如，如果对象 myacls 的首地址是 0x00B5FDC0，那么寄存器 ecx 中的值就会是 00B5FDC0，然后汇编代码紧接着执行了 call MYACLS::myfunc (0A510F0h)，在调试时可以用 F11 快捷键进入该函数，如图 5.5 所示。

```
class MYACLS
{
public:
    int m_i;
    void myfunc(int abc)
    {
00A51800  push        ebp
00A51801  mov         ebp,esp
00A51803  sub         esp,0CCh
00A51809  push        ebx
00A5180A  push        esi
00A5180B  push        edi
00A5180C  push        ecx
00A5180D  lea         edi,[ebp-0CCh]
00A51813  mov         ecx,33h
00A51818  mov         eax,0CCCCCCCCh
00A5181D  rep stos    dword ptr es:[edi]
00A5181F  pop         ecx
00A51820  mov         dword ptr [this],ecx
00A51823  mov         ecx,offset _F0B5FBBB_MyProject@cpp (0A5C026h)
00A51828  call        @__CheckForDebuggerJustMyCode@4 (0A51212h)
        m_i += abc;
00A5182D  mov         eax,dword ptr [this]
00A51830  mov         ecx,dword ptr [eax]
00A51832  add         ecx,dword ptr [abc]
00A51835  mov         edx,dword ptr [this]
00A51838  mov         dword ptr [edx],ecx
    }
```

图 5.5　调用成员函数时利用 this 参数完成对成员变量的计算

从图 5.5 可以看到，在 myfunc 成员函数中，实际上是先把 ecx 的值给回到 this，后续做 m_i += abc;运算的时候直接使用 this 参与运算。

另外，在 Linux 操作系统下，有个话题没说完，就是成员函数的函数名。已经发现编译器会对成员函数名进行适当的处理，如图 5.3 中看到的成员函数名被处理为 _ZN6MYACLS6myfuncEi。在这里，编译器为其增加了类名，然后还增加了其他一些字母数字组合，这些组合是由编译器内部的一些规则决定的，这些规则同时也能有效地防止该成员函数和其他的函数重名。

无论成员函数也好，成员变量也罢，在处理它们名字的时候，编译器都会向名字中混合很多其他内容，例如上面看到的把类名混合进去，还有一些数字字母等也混合进去，可能这些内容是一些参数类型编码后得到的结果，主要目的应该是让这些名字唯一。

说到这里，想一想，如果父类中有一个成员函数，所继承的子类中又有一个同名的成员函数，从程序员的角度来讲，这两个成员函数同名，但是在真正处理的时候，从编译器的角度来讲，这两个函数名字肯定是不同的。

这些名字可以称为编译器内部名称，一般不应该显示给程序员(开发者)，但如果在某种编译器中或者某种场合下(如编译程序时在出现的报错信息中)看到了，也不应该感到奇怪。

5.2　虚成员函数与静态成员函数调用方法

5.2.1　虚成员函数调用方法

上一节最终的范例代码原样保留，本节中要使用。

在 MYACLS 类中，增加一个 public 修饰的虚函数。代码如下：

```
public:
    virtual void myvirfunc()
    {
        printf("myvirfunc()被调用,this =  %p\n", this);
    }
```

在 main 主函数中，注释掉原有代码，加入如下代码：

```
MYACLS myacls;
myacls.myvirfunc();
MYACLS * pmyacls = new MYACLS;
pmyacls->myvirfunc();
delete pmyacls;
```

这个范例在 3.6 节详细讲过，读者应该比较熟悉。

读者知道，myacls. myvirfunc();和 pmyacls-> myvirfunc();是不同的。用对象名 myacls 来调用虚函数，就像是调用普通成员函数一样，因为编译器很聪明，编译器知道这种调用不是多态，不需要通过虚函数表来查找并调用虚函数。

但是要用对象指针来调用虚函数，则是要通过虚函数表指针查找虚函数表，然后通过虚函数表找到要调用的虚函数的地址，从而调用指定的虚函数。

从编译器视角来看，编译会把下面的代码

```
pmyacls->myvirfunc();
```

转换为大致如下这种代码：

```
(*pmyacls->vptr[0])(pmyacls);
```

(1) 这里 vptr 表示编译器生成的虚函数表指针，指向虚函数表。

(2) 在这种情况下，下标[0]就是虚函数表中的第一项，因为这里虚函数表中只有唯一的虚函数 myvirfunc 的地址。

(3) 这里传递进去一个参数，代表上一节提到的 **this 参数**(this 指针)，这也是编译器给加进去的。这样调用一个成员函数就等价于调用一个普通函数，this 参数有特定的用途，如**确定对象所属的某个成员变量在内存中的位置**。

(4) pmyacls-> vptr[0]前面使用了“*”之后就代表要调用的虚函数的入口地址。

现在，在类 MYACLS 中增加另外一个 public 修饰的虚函数：

```
public:
    virtual void myvirfunc2()
    {
        printf("myvirfunc2()被调用,this = %p\n", this);
    }
```

然后修改一下 myvirfunc 虚函数——在其中调用 myvirfunc2：

```
public:
    virtual void myvirfunc()
    {
        printf("myvirfunc()被调用,this = %p\n", this);
        myvirfunc2();
    }
```

把断点设置在 myvirfunc 函数中的 myvirfunc2();代码行。开始调试，当程序执行流程停在断点行时，切换到反汇编代码，如图 5.6 所示。

```
        printf("myvirfunc()被调用,this = %p\n", this);
00F424AD  mov         eax,dword ptr [this]
00F424B0  push        eax
00F424B1  push        offset string "myvirfunc()\xb1\xbb\xb5\xf7\xd3\xc3,this = %p\n" (0F47B3Ch)
00F424B6  call        _printf (0F4104Bh)
00F424BB  add         esp,8
        myvirfunc2();
00F424BE  mov         eax,dword ptr [this]
00F424C1  mov         edx,dword ptr [eax]
00F424C3  mov         esi,esp
00F424C5  mov         ecx,dword ptr [this]
00F424C8  mov         eax,dword ptr [edx+4]
00F424CB  call        eax
00F424CD  cmp         esi,esp
00F424CF  call        __RTC_CheckEsp (0F41285h)
    }
```

图 5.6　对 myvirfunc2 的调用是通过虚函数表进行的

在图 5.6 中不难发现，对 myvirfunc2 虚函数的调用是通过查虚函数表来进行的。

现在，继续修改 myvirfunc 虚函数。修改后的代码如下：

```
public:
    virtual void myvirfunc()
```

```
{
    printf("myvirfunc()被调用,this =  %p\n", this);
    //myvirfunc2();
    MYACLS::myvirfunc2();
}
```

读者能想象此时此刻,这个写法和上面的写法有什么区别吗？看一看对应的反汇编代码,如图 5.7 所示。

```
        printf("myvirfunc()被调用,this = %p\n", this);
002724AD  mov         eax,dword ptr [this]
002724B0  push        eax
002724B1  push        offset string "myvirfunc()\xb1\xbb\xb5\xf7\xd3\xc3,this = %p\n" (0277B3Ch)
002724B6  call        _printf (027104Bh)
002724BB  add         esp,8
        //myvirfunc2();
        MYACLS::myvirfunc2();
002724BE  mov         ecx,dword ptr [this]
002724C1  call        MYACLS::myvirfunc2 (0271433h)
    }
```

图 5.7　对 MYACLS::myvirfunc2 的调用是直接调用而不再通过虚函数表进行调用

从图 5.7 中可以看到,一旦换了一种写法,这里采用“类名::”的写法,这叫作类范围操作符,用来明确调用 myvirfunc2,可以发现对 myvirfunc2 的调用变成了直接调用,而不再是通过虚函数表来调用了。这种调用方式的写法更有效率,而且**压制住了虚拟机制,不再通过查询虚函数表来调用**。

所以,请读者注意这种调用方法：类范围操作符::虚函数名(参数)。**这种明确调用虚函数的方法等价于直接调用一个普通的成员函数**。

5.2.2　静态成员函数调用方法

在类 MYACLS 中增加一个 public 修饰的静态成员函数：

```
public:
    static void mystfunc()
    {
        printf("mystfunc()被调用\n");
    }
```

在 main 主函数中,注释掉以往的代码,加入如下全新代码：

```
MYACLS myacls;
myacls.mystfunc();
MYACLS * pmyacls = new MYACLS;
pmyacls->mystfunc();
```

对于静态成员函数的调用,无论用对象名来调用,还是用对象指针来调用,效果都是一样的——都会被编译器转换为一般的针对普通函数(非成员函数)的调用形式。

静态成员函数是跟着类走的,所以**调用静态成员函数时编译器是不会插入 this 作为形参的**。

在 Linux 操作系统下,编译一下 MyProject.cpp,然后用 nm 命令,列出可执行文件中的全局变量、函数存放地址等信息,如图 5.8 所示。

```
kuangxiang@bogon:/mnt/hgfs/linux$ g++ MyProject.cpp -o MyProject
MyProject.cpp: In static member function 'static void MYACLS::mystfunc()':
MyProject.cpp:31:43: warning: format '%p' expects a matching 'void*' argument [-Wformat=]
   printf("mystfunc()被调用.this = %p\n");
                                       ^
kuangxiang@bogon:/mnt/hgfs/linux$ nm MyProject
0000000000601060 B __bss_start
00000000006010b8 b completed.7594
                 U __cxa_atexit@@GLIBC_2.2.5
0000000000601050 D __data_start
0000000000601050 W data_start
0000000000400760 t deregister_tm_clones
00000000004007e0 t __do_global_dtors_aux
00000000004007a0 t register_tm_clones
                 U __stack_chk_fail@@GLIBC_2.4
0000000000400730 T _start
0000000000601060 D __TMC_END__
00000000004008ae t _Z41__static_initialization_and_destruction_0ii
0000000000400826 T _Z7myfuncP6MYACLSi
0000000000400934 W _ZN6MYACLS10myvirfunc2Ev
0000000000400959 W _ZN6MYACLS8mystfuncEv
0000000000400902 W _ZN6MYACLS9myvirfuncEv
0000000000400970 W _ZN6MYACLSC1Ev
0000000000400970 W _ZN6MYACLSC2Ev
                 U _ZNSt8ios_base4InitC1Ev@@GLIBCXX_3.4
```

图 5.8　用 nm 命令查看静态成员函数的信息

所以，下面这样的调用：

```
myacls.mystfunc();
pmyacls->mystfunc();
```

将被编译器转换为如下这种相同的调用，这里并不需要 this 参数：

```
_ZN6MYACLS8mystfuncEv();
_ZN6MYACLS8mystfuncEv();
```

4.3.3 节讲解过一种奇异的写法来输出一个类成员变量的偏移值。这里也有类似的写法来调用静态成员函数：

```
((MYACLS *)0)->mystfunc();                    //这样调用静态成员函数没问题
```

如果是调用普通成员函数，一般需要用到对象或者对象指针。例如：

```
obj.myfunc();
```

然后编译器会增加一个 this 参数到现有的参数列表中，这个 this 参数指向的就是这个对象本身，传递到所调用的成员函数里去的主要目的是操作对象所属的成员变量，如给一个成员变量赋值等。当然，如果这个成员函数中**不操作任何成员变量，实际上是根本不需要这个 this 参数的，也就是说，此时是不需要通过对象或者对象指针来调用成员函数**的（言下之意就是应该像全局函数一样，直接通过函数名来调用）。

其实，读者也看到了，调用这种静态成员函数就不需要 this 参数。在 main 主函数中增加如下代码行：

```
((MYACLS *)0)->mystfunc();
```

设置断点在该行并开始调试，当程序执行流程停到该断点行时，切换到反汇编窗口，如图 5.9 所示。

图 5.9 中对静态成员函数的调用根本没涉及 this 参数，但是如果调用一个普通的成员函数，例如在 main 主函数中增加如下代码：

```
        ((MYACLS*)0)->mystfunc();  //这样调用静态成员函数没问题
00A2AE3  call        MYACLS::mystfunc (01A10A0h)
        cout << "断点设置在这里" << endl;
```

图 5.9　直接调用静态成员函数时无须用到 this 参数

((MYACLS *)0) -> myfunc(12);

从语法层面来讲，上面这行代码没有问题，可以编译通过。设置断点在该行并开始调试，当程序执行停到该断点行时，切换到反汇编窗口，如图 5.10 所示。

```
00852B53  call        MYACLS::mystfunc (08510A0h)
        ((MYACLS*)0)->myfunc(12);
00852B58  push        0Ch
00852B5A  xor         ecx,ecx
00852B5C  call        MYACLS::myfunc (08514C9h)
        cout << "断点设置在这里" << endl;
```

图 5.10　调用普通成员函数时需用到 this 参数

代码行((MYACLS *)**0**)-> myfunc(12);中的这个 0，其实就是 this 指针，放入了 ecx 中，图 5.10 中的汇编代码行 xor ecx,ecx 就是将 ecx 清 0，代表的就是这个 this 指针。这里就相当于把这个 0 当作 this 参数传递进普通成员函数调用中去了。

虽然传递到普通成员函数中去的是一个 0，但是只要程序员不调用成员变量，这个 this 参数就不会被使用，程序就不会产生运行时的异常，但是若写下面这样的代码就会产生运行时异常：

```
void myfunc(int abc)
{
    m_i += abc; //这里的 m_i 需要用到 this 指针(this->m_i),而传递进来的 this 指针若为 0,
                //则本行代码执行时会报异常
}
```

但是如果在 myfunc 里调用静态成员函数，则毫无问题，因为静态成员函数中根本不需要用到 this 指针：

```
void myfunc(int abc)
{
    //m_i += abc;
    mystfunc(); //这绝对没有问题
}
```

所以，回头想想，为什么 C++支持下面这种语法？

```
((MYACLS * )0) -> mystfunc();
```

因为**有些成员函数希望支持独立于类对象之外的存取操作**，所以，就出现了这种解决之道，类设计者就搞出了这种奇特的写法：把 0 强制转型为一个类类型指针，从而提供出了一个 this 指针实体，作为一个参数传递给普通成员函数。当然，如果是静态成员函数，这个 this 实体并不作为参数，因为静态成员函数并不需要类对象指针作为隐藏参数传递进函数里去，但是可以作为一种提示(给程序员看的)，提示调用的是哪个类的静态成员函数。

那么，静态成员函数有哪些特性呢？总结如下：

(1) 静态成员函数**没有 this 指针**，这是最重要的一点。

(2) 无法直接存取类中普通的非静态成员变量。

(3) 静态成员函数不能在末尾增加 const 后缀，也不能设置为 virtual。

（4）可以用类对象来调用，但不要求一定要用类对象来调用。看一下反汇编代码就知道，即便是用类对象来调用，实际上还是被转换为了直接调用，参考图 5.9 会发现，所有对静态成员函数的调用都会被转化为“类名::静态成员函数名(……)”的调用形式。

（5）静态成员函数等同于全局函数(非成员函数)，所以一般提供回调函数的场合，可以将这种函数提供为回调函数。

静态成员函数有地址并且是在编译的时候就确定好的。读者不妨尝试用如下代码行将静态成员函数的地址输出出来：

```
printf("MYACLS::mystfunc()地址 = %p\n", MYACLS::mystfunc);
```

在 Linux 下编译并运行程序，输出的静态成员函数地址信息与使用 nm 命令得到的静态成员函数地址信息一致，如图 5.11 所示。

```
kuangxiang@bogon:/mnt/hgfs/linux$ g++ MyProject.cpp -o MyProject
MyProject.cpp: In static member function 'static void MYACLS::mystfunc()':
MyProject.cpp:32:43: warning: format '%p' expects a matching 'void*' argument [-Wformat=]
   printf("mystfunc()被调用,this = %p\n");
                                           ^
^[[Akuangxiang@bogon:/mnt/hgfs/linux$ ./MyProject
mystfunc()被调用
mystfunc()被调用
mystfunc()被调用
mystfunc()被调用
mystfunc()被调用
MYACLS::mystfunc()地址 = 0x400b59
断点设置在这里
kuangxiang@bogon:/mnt/hgfs/linux$ nm MyProject
0000000000602078 B __bss_start
00000000006021f0 b completed.7594
                 U __cxa_atexit@@GLIBC_2.2.5
0000000000602068 D __data_start
0000000000602068 W data_start
0000000000400900 t deregister_tm_clones
0000000000400980 t __do_global_dtors_aux
0000000000601e08 t __do_global_dtors_aux_fini_array_entry
0000000000602070 D __dso_handle
0000000000601e18 d _DYNAMIC
0000000000602078 D _edata
00000000004008d0 T _start
0000000000602078 D __TMC_END__
0000000000400a97 t _Z41__static_initialization_and_destruction_0ii
00000000004009c6 T _Z7gmyfuncP6MYACLSi
0000000000400b34 W _ZN6MYACLS10myvirfunc2Ev
0000000000400aea W _ZN6MYACLS6myfuncEi
0000000000400b59 W _ZN6MYACLS8mystfuncEv
0000000000400b02 W _ZN6MYACLS9myvirfuncEv
0000000000400b70 W _ZN6MYACLSC1Ev
0000000000400b70 W _ZN6MYACLSC2Ev
0000000000400b34 W _ZN6MYACLS10myvirfunc2Ev
0000000000400aea W _ZN6MYACLS6myfuncEi
0000000000400b59 W _ZN6MYACLS8mystfuncEv
0000000000400b02 W _ZN6MYACLS9myvirfuncEv
0000000000400b70 W _ZN6MYACLSC1Ev
0000000000400b70 W _ZN6MYACLSC2Ev
```

图 5.11　输出来的静态成员函数的地址与用 nm 命令查看到的静态成员函数的地址一致

5.3　虚函数地址问题的 vcall 引入

上一节讲了一下虚成员函数(虚函数)，本节要进一步探讨一下这个问题。

看看如下范例。在 MyProject.cpp 的上面，增加如下类 MYACLS 的定义代码：

```
class MYACLS
{
public:
```

```
    virtual void myvirfunc1()
    {
        cout << "虚函数 MYACLS::myvirfunc1()执行了" << endl;
    }
    virtual void myvirfunc2()
    {
        cout << "虚函数 MYACLS::myvirfunc2()执行了" << endl;
    }
};
```

虚函数在内存中也有一个**固定的地址，也是在编译时确定好的**。读者可以在 Linux 操作系统下进行编译并用 nm 命令进行查看，可以查看到下面的字样：

```
0000000000400af0 W _ZN6MYACLS10myvirfunc1Ev
0000000000400b1c W _ZN6MYACLS10myvirfunc2Ev
```

上面两行充分表明虚函数的地址是在编译的时候就确定好的(地址分别为 00400af0 和 00400b1c)。

在 main 主函数中，加入如下代码：

```
printf("MYACLS::myvirfunc()地址 = %p\n", &MYACLS::myvirfunc1);
printf("MYACLS::myvirfunc()地址 = %p\n", &MYACLS::myvirfunc2);
cout << sizeof(MYACLS) << endl;                    //4 字节
MYACLS * pmyobj = new MYACLS();
pmyobj->myvirfunc1();
pmyobj->myvirfunc2();
cout << "断点设置在这" << endl;
```

将断点设置在 cout 所在行，开始调试，当程序执行停在断点行时，将光标放到 pmyobj 上并展开以观察其中的虚函数表指针所指向的虚函数表项，如图 5.12 所示。

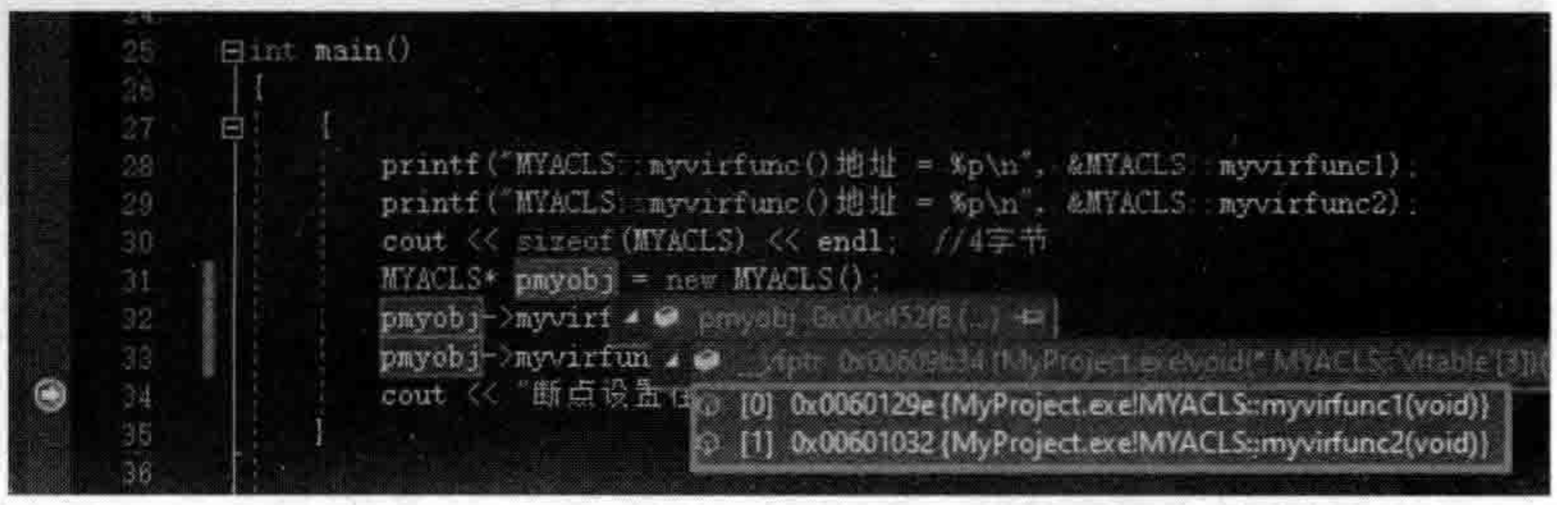

图 5.12　通过 new 出来的一个类对象观察类的虚函数表中的表项

如果查看 pmyobj 所指向的对象地址，在该对象地址中找到虚函数表指针所指向的虚函数表并查看**虚函数表对应的内存地址**中的内容，可以很容易地看到，虚函数表内容中的前 8 个字节为 9e 12 60 00 32 10 60 00，四个为一组，倒序解读，就可以解读为 0x0060129e 和 0x00601032 两组信息，与图 5.12 显示的虚函数表中的两个虚函数地址完全一致。

执行起来，看一看结果：

```
MYACLS::myvirfunc()地址 = 006011C7
MYACLS::myvirfunc()地址 = 006010FF
```

```
4
虚函数 MYACLS::myvirfunc1()执行了
虚函数 MYACLS::myvirfunc2()执行了
```

将输出的结果和图 5.12 比较，可以注意到一个事实，两者显示的两个虚函数的地址信息并不一样。这让人很费解，看来必须要仔细分析一下了。

把断点设置在 main 主函数的任意一个 printf 所在行，开始调试，当程序执行停在断点行时，切换到反汇编代码，如图 5.13 所示。

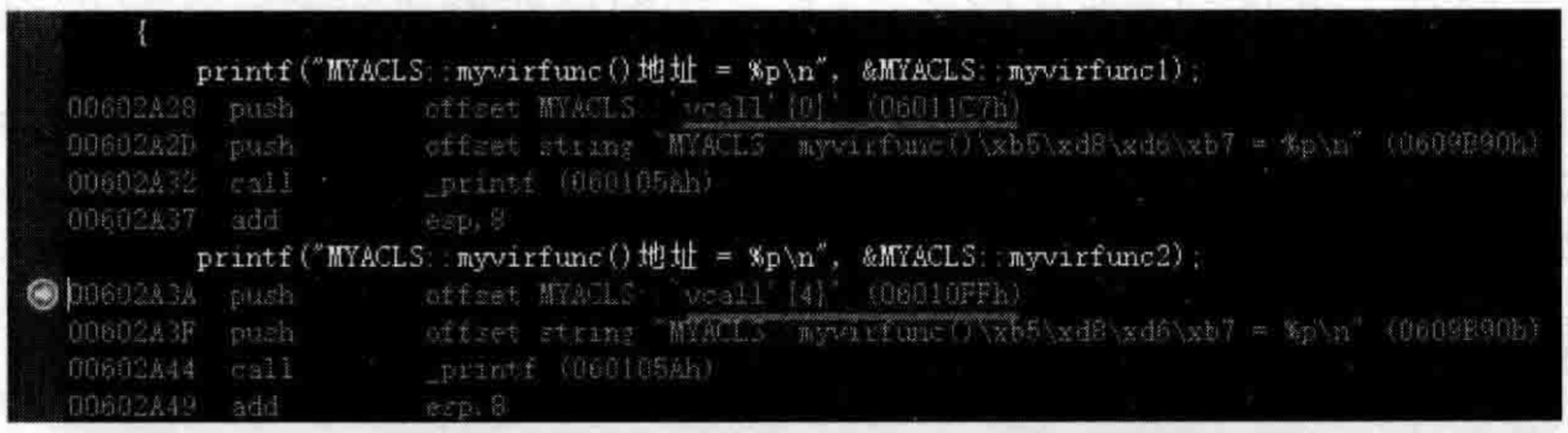

图 5.13　输出虚函数地址的反汇编代码

第一行反汇编代码是一个 push——压栈，看起来是把 MYACLS::'vcall'{0}';压栈，这行汇编代码后面跟一个地址（06011C7h），其实用 printf 语句输出的 &MYACLS::myvirfunc1 地址正是该地址。

在反汇编窗口中找一找该地址，如何找？反汇编窗口左上角有个地址栏，可以直接在其中输入 06011C7h 并按 Enter 键，反汇编窗口会自动定位到该地址，如图 5.14 所示。

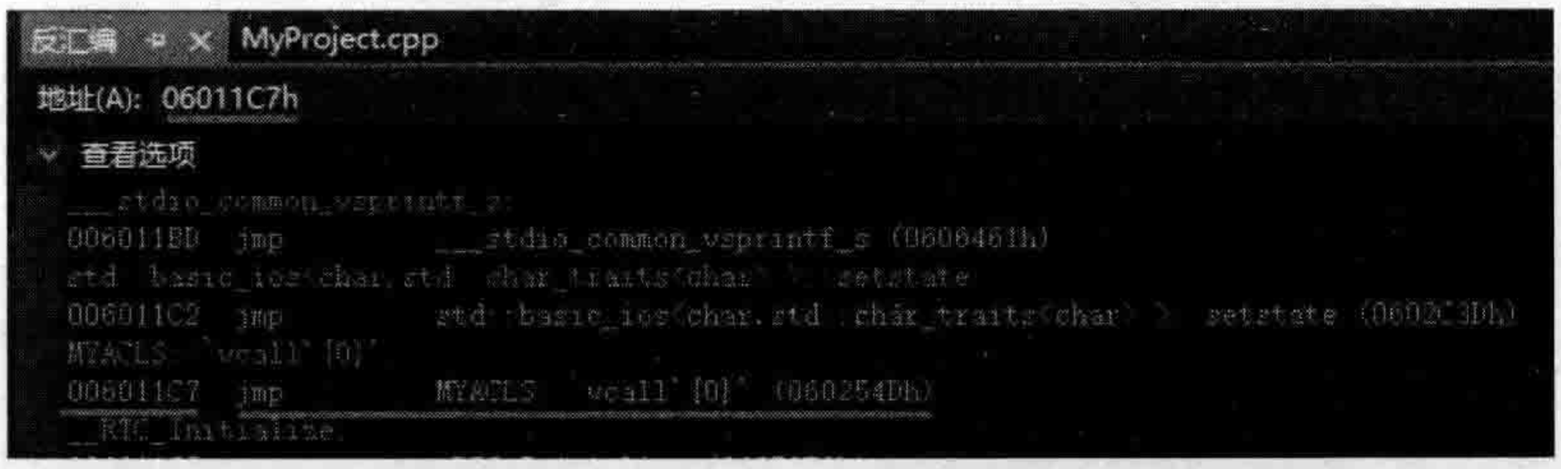

图 5.14　在反汇编窗口跳转到一个指定的地址

图 5.14 中，在反汇编窗口跳转到了 06011C7h 这个地址，其对应的反汇编代码是：

```
jmp  MYACLS::`vcall'{0}'(060254Dh)
```

上面这行代码看起来又跳转到了 060254Dh 地址，继续，在反汇编窗口左上角输入 060254Dh 并按 Enter 键，反汇编窗口自动定位到该地址，如图 5.15 所示。

看着图 5.15 可以猜一下，ecx 中保存的可能是 this 指针，而一般来讲，虚函数表指针是类对象的第一个成员变量，所以 mov 这行汇编代码应该是把虚函数表指针所指向的地址，也就是虚表地址存到了 eax 中。

接着的 jmp 汇编代码应该是跳转到了虚函数表中所指向的某个虚函数去执行。因为是从 MYACLS::`vcall'{0}'跟踪过来的汇编代码，所以估计这里应该是执行虚函数表的第一项所对应的虚函数。当然在图 5.13 中，如果是从 MYACLS::`vcall'{4}'追踪过来，不难

发现，追踪到的汇编代码应该是如图 5.16 所示(注意其中的 eax+4 表示执行的是虚函数表中的第二项所对应的虚函数)。

图 5.15　在反汇编窗口根据 vcall{0}的指引跳转到另外一个指定的地址

图 5.16　在反汇编窗口根据 vcall{4}的指引跳转到另外一个指定的地址

看目前的意思，这里是中转了一下，笔者要特别强调，现在笔者所用的开发环境是 Visual Studio 2019 编译器，其他编译器不一定是这样。该编译器通过一系列的 vcall 函数，而后在 vcall 函数里做跳转，跳转到真正的虚函数去(应该是每个 vcall 函数都唯一对应着一个虚函数)。

所以，下面的代码：

```
printf("MYACLS::myvirfunc()地址 = %p\n", &MYACLS::myvirfunc1);
printf("MYACLS::myvirfunc()地址 = %p\n", &MYACLS::myvirfunc2);
```

输出的是 vcall 函数的地址，而不是真正的虚函数地址。

既然话题说到这，读者可能不禁要问，vcall 是什么？掺和进来的目的是什么？vcall 是编译生成的内容，完整的名字应该叫作 vcall thunk。为什么引入 vcall thunk？可以把它看成一段代码，看成一段函数也行，它有两个作用：

(1) 它能调整 this 指针，例如一个子类继承父类 1 和父类 2，如果父类 2 的指针要调用子类的析构函数，那这个父类 2 的指针就需要利用这里的 vcall thunk 机制调整。后续讲多重继承还会谈到 this 指针调整。

(2) 跳到真正的虚函数中去。vcall thunk 读者简单认识即可，因为编译器的开发者有他们自己的考量，有些考量可能非常细致，没必要太过深究。

本节主要目的就是带着读者看一看 vcall，vcall 调用的是真实的虚函数地址。以后万一见到这个 vcall，能够认识即可。后面章节会进一步给出 vcall 相关的范例代码。

5.4　静动态类型、绑定、坑点与多态体现深谈

本节主要是澄清一些概念，为后续顺畅地讲解作铺垫，因为有些概念容易模糊不清，从而造成后续学习的困扰。

5.4.1　静态类型和动态类型

看看如下范例。在 MyProject.cpp 的上面，增加两个类，有父类，有子类，先写成空类：

```
class Base
{
public:
};
class Derive : public Base
{
public:
};
class Derive2 : public Base
{
public:
};
```

引出两个概念：静态类型与动态类型。

(1) 静态类型：对象定义时的类型，编译期间就确定好的。看下面的代码，注意注释：

```
Base base;                                          //静态类型是 Base
Derive derive;                                      //静态类型是 Derive
Base * pbase;     //别管指向啥，反正定义的时候定义的是 Base *，所以静态类型是 Base *
Base * pbase2 = new Derive();                       //静态类型依旧是 Base *
Base * pbase3 = new Derive2();                      //静态类型依旧是 Base *
```

(2) 动态类型：对象目前所指向的类型(运行的时候才决定类型的)。根据上面的代码行，一般来讲，只有指针或者引用才有动态类型的说法，而且一般都是指父类的指针或者引用。根据上面的代码行，看一看所涉及的变量的动态类型：

- Base 和 Derive **没有动态类型**，因为它们既不是指针也不是引用。
- pbase 也没有动态类型，因为它没有指向任何对象。
- pbase2 有动态类型，它的动态类型是 Derive。
- pbase3 有动态类型，它的动态类型是 Derive2。

另外，动态类型在执行过程中可以改变。例如如下：

```
pbase = pbase2;                                     //pbase 的动态类型是 Derive
pbase = pbase3;                                     //pbase 的动态类型是 Derive2
```

5.4.2 静态绑定和动态绑定

静态绑定：绑定的是静态类型，所对应的函数或属性**依赖于**对象的静态类型，发生在**编译期**。

动态绑定：绑定的是动态类型，所对应的函数或属性依赖于对象的动态类型，发生在运行期。

这两个概念有些晦涩，不好理解，后续笔者会具体举例，届时请读者参照具体实例来理解。这里先给出一些结论：

(1) 普通成员函数是静态绑定，而虚函数是动态绑定。

(2) 缺省参数一般是静态绑定。

后面会有范例阐述这些绑定问题。

5.4.3　继承的非虚函数坑

向 Base 类中增加一个 public 修饰的普通成员函数(不是虚函数)：

```
public:
    void myfunc()
    {
        cout << "Base::myfunc()" << endl;
    }
```

继续向 Derive 类里增加一个 public 修饰的普通成员函数：

```
public:
    void myfunc()
    {
        cout << "Derive::myfunc()" << endl;
    }
```

在 main 主函数中，加入如下代码：

```
Derive derive;
Derive * pderive = &derive;
pderive->myfunc();                          //Derive::myfunc()

Base * pbase = &derive;
pbase->myfunc();                            //Base::myfunc(),这里调用的居然是父类的
                                            //myfunc,是一个陷阱,写程序时一定要注意
```

注意上面代码行中的注释语句，可以看到，pbase-> myfunc()；调用的是 Base 类的 myfunc 成员函数。为什么是这样？

这就是笔者刚才所说的：**普通成员函数是静态绑定**。换句话说，**myfunc 是普通成员函数，这里到底执行哪个 myfunc 成员函数取决于调用者的静态类型，pbase 的静态类型是 Base，所以调用的是 Base 的 myfunc，而 pderive 的静态类型是 Derive，所以调用的是 Derive 的 myfunc 成员函数**。

结论：**不应该在子类中重新定义一个继承来的非虚成员函数**。因为，上面的演示结果绝对是程序员不希望发生的结果。

5.4.4　虚函数的动态绑定

下面完善一下代码，向类 Base 中增加一个 public 修饰的虚函数：

```
public:
    virtual void myvirfunc()
    {
        cout << "Base::myvirfunc()" << endl;
    }
```

继续向 Derive 类里增加一个 public 修饰的虚函数：

```
public:
```

```
    virtual void myvirfunc()
    {
        cout << "Derive::myvirfunc()" << endl;
    }
```

在 main 主函数中，继续增加如下代码行：

```
Base base;
pderive -> myvirfunc();                    //Derive::myvirfunc()
pbase -> myvirfunc();                      //Derive::myvirfunc()
pbase = &base;
pbase -> myvirfunc();                      //Base::myvirfunc()
```

注意上面代码行中的注释语句，可以看到，**虚函数是动态绑定**。换句话说，myvirfunc 是**虚函数，这里到底执行哪个 myvirfunc 取决于调用者的动态类型，这里 pbase 的动态类型开始是 Derive ***，后来又变为 **Base ***，所以调用的也分别是 **Derive** 和 **Base** 的 **myvirfunc**，而 **pderive** 的动态类型是 **Derive ***，所以调用的是 **Derive** 的 **myvirfunc**。

5.4.5 重新定义虚函数的缺省参数坑

修改 Base 类中的虚函数 myvirfunc，为其增加一个缺省参数，并且在函数体中输出该参数的值：

```
public:
    virtual void myvirfunc(int value = 1)
    {
        cout << "Base::myvirfunc(),value = " << value << endl;
    }
```

同样修改 Derive 类中的虚函数 myvirfunc，也为其增加一个缺省参数，并且在函数体中输出该参数的值：

```
public:
virtual void myvirfunc(int value = 2)
    {
        cout << "Derive::myvirfunc(),value = " << value << endl;
    }
```

执行起来，看一看结果，有意思的事情发生了：

```
Derive::myfunc()
Base::myfunc()
Derive::myvirfunc(),value = 2
Derive::myvirfunc(),value = 1
Base::myvirfunc(),value = 1
```

上述结果的第 4 行是执行代码行 pbase-> myvirfunc();产生的结果，可以注意到，执行的是**子类的虚函数**，但是输出的**形参 value 的缺省值却是父类的虚函数形参的缺省值**。

正常来讲执行子类虚函数，这个形参的缺省值应该采用的是子类虚函数中的形参缺省值，可惜，事与愿违。

这就属于函数参数缺省值的静态绑定。上面说过，**缺省参数是静态绑定**（pbase 的静态类型是 Base * ），所以缺省参数绑定到父类 Base 对应的 myvirfunc 的缺省参数上去了（而不管 pbase 指向的是否是子类对象）。

为什么 C++要这样做？主要就是考虑到执行期间效率问题，缺省参数静态绑定好实现，动态绑定就不好实现，可能更慢和更复杂。所以 C++编译器作了这样的取舍。

结论：**不要在子类中重新定义虚函数缺省参数的值**。

5.4.6　C++中的多态性

读者对 C++中耳熟能详的“多态性”这个词，但请仔细想一想，多态性的概念，你真弄清楚了吗？

多态性，显然是一种特性，那怎样体现这种特性？

首先，不管从代码实现上还是从表现形式上，有一个观点是肯定的：

多态必须存在虚函数并且调用虚函数，没有虚函数，绝不可能存在多态。

关于多态性这个概念，笔者认为分两方面谈比较容易说清楚：

（1）从代码实现上。

（2）从表现形式上。

继续谈：

1. 从代码实现上

当调用一个虚函数的时候，调用的路线是不是通过查询虚函数表来找到虚函数入口地址然后去执行虚函数？如果调用的是这个路线，就是多态；如果调用的不是这个路线（而是直接调用），那就不是多态。

所以如果从这个角度理解，那就不用管有没有继承关系，也不用管有没有派生类。

在 MyProject.cpp 的上面，先定义一个 A 类：

```
class A
{
public:
    virtual void myvirfunc() {}
};
```

在 main 主函数中，加入如下代码（注意看代码中的注释）：

```
A * pa = new A();
pa->myvirfunc();                    //这个是多态的

A a;
a.myvirfunc();                      //这个就不是多态的

A * ya = &a;
ya->myvirfunc();                    //这个也是多态
```

上述代码是否是多态，笔者判断的理论依据是对应的汇编代码。设置断点在上面任意一行代码行，开始调试，当程序执行停到断点行时，切换到反汇编窗口，如图 5.17 所示。

```
        pa->myvirfunc(); //这是不是多态，是多态的
00A82C52  mov         eax,dword ptr [ebp-3Ch]
00A82C55  mov         edx,dword ptr [eax]
00A82C57  mov         esi,esp
00A82C59  mov         ecx,dword ptr [ebp-3Ch]
00A82C5C  mov         eax,dword ptr [edx]
00A82C5E  call        eax
00A82C60  cmp         esi,esp
00A82C62  call        __RTC_CheckEsp (0A812F3h)

        A a;
00A82C67  lea         ecx,[ebp-48h]
00A82C6A  call        A::A (0A811D1h)
        a.myvirfunc(); //这个就不是多态的
00A82C6F  lea         ecx,[ebp-48h]
00A82C72  call        A::myvirfunc (0A812D0h)

        A* ya = &a;
00A82C77  lea         eax,[ebp-48h]
00A82C7A  mov         dword ptr [ebp-54h],eax
        ya->myvirfunc(); //这个也是多态
00A82C7D  mov         eax,dword ptr [ebp-54h]
00A82C80  mov         edx,dword ptr [eax]
00A82C82  mov         esi,esp
00A82C84  mov         ecx,dword ptr [ebp-54h]
00A82C87  mov         eax,dword ptr [edx]
00A82C89  call        eax
00A82C8B  cmp         esi,esp
00A82C8D  call        __RTC_CheckEsp (0A812F3h)
```

图 5.17　通过反汇编代码判断多态性的体现

在图 5.17 中，针对调用了虚函数的代码行，只要其对应的汇编代码超过 6 行(只有查询虚函数表来执行虚函数才需要这么多行汇编代码)，对应的汇编代码功能都应该是通过虚函数表指针查询虚函数表中虚函数的地址，然后调用对应的虚函数，从代码实现上，这就是多态性。

2. 从表现形式上(表现形式当然最终也是通过代码体现)

这种情况更符合传统上对多态的理解。

要想从表现形式上看出来是否是多态，要加几条说明：

(1) 有父类有子类(有继承关系)，父类中必须含有虚函数，子类重写父类中的虚函数。

(2) **父类指针指向子类对象**或者**父类引用绑定(指向)子类对象**。

(3) 当以父类的指针或者引用调用子类中重写了的虚函数时，就能看出来多态的表现了，因为调用的是子类的虚函数。

前面代码中，已经演示了父类指针指向子类对象的代码。整理如下：

```
Derive derive;
Base * pbase = &derive;
pderive->myvirfunc(); //Derive::myvirfunc()
```

或者：

```
Base * pbase2 = new Derive();                    //释放内存请读者自行操作,笔者在这里没演示
pbase2->myvirfunc();                             //Derive::myvirfunc()
```

对于父类引用绑定(指向)子类对象，看如下范例代码：

```
Derive derive2;
Base& yinbase = derive2;
yinbase.myvirfunc();                             //Derive::myvirfunc()
```

5.5　单继承虚函数趣味性测试和回顾

5.5.1　单继承下的虚函数

看看如下范例。在 MyProject.cpp 的上面，增加如下代码：

```
class Base
{
public:
    virtual void f() { cout << "Base::f()" << endl; }
    virtual void g() { cout << "Base::g()" << endl; }
    virtual void h() { cout << "Base::h()" << endl; }
};
class Derive : public Base {
public:
    void g() { cout << "Derive::g()" << endl; }
    void myselffunc() {} //只属于 Derive 的函数
};
```

针对上面的代码，图 3.3 绘制出了父类对象和子类对象内存布局示意图。

这里可以进行深一步的研究，给 Derive 类增加一个 public 修饰的新虚函数 i。修改后的 Derive 代码如下：

```
class Derive : public Base {
public:
    virtual void i() { cout << "Derive::i()" << endl; }
    virtual void g() { cout << "Derive::g()" << endl; }
    void myselffunc() {} //只属于 Derive 的函数
};
```

这里笔者想看一看 Derive 类虚函数表项的排列顺序——虚函数 i 会排列在 g 的上面吗？

要分析虚函数表项的排列顺序，可以写代码，也可以借助反汇编查看。这里笔者介绍一下反汇编查看之法。

在 main 主函数中，加入如下代码段：

```
Derive myderive;
Derive * pmyderive = &myderive;
//注意下列虚函数的调用顺序
pmyderive->f();
pmyderive->g();
pmyderive->h();
pmyderive->i();
```

将断点设置在 pmyderive->f();代码行，开始调试程序，当程序运行停在断点行时，切换到反汇编窗口观察，如图 5.18 所示。

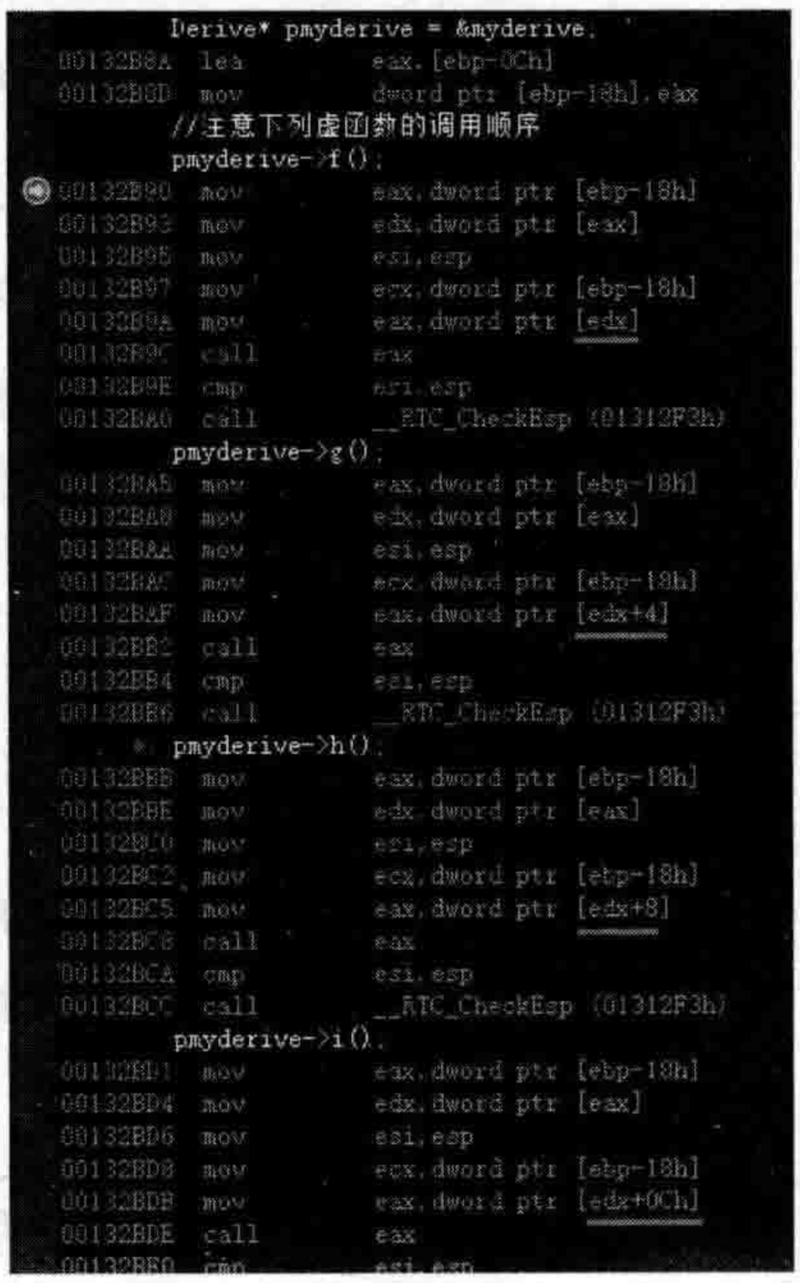

图 5.18　通过代码检测虚函数在虚函数表中的排列顺序

从图 5.18 可以看到，所调用的虚函表项分别是[edx]、[edx＋4]、[edx＋8]、[edx＋0Ch]，这说明这几个虚函数表项是按从上到下的顺序排列的，每次跳跃 4 字节。

在增加了 i 这个新的虚函数后，Base 父类对象和 Derive 子类对象内存布局示意图应该如图 5.19 所示。

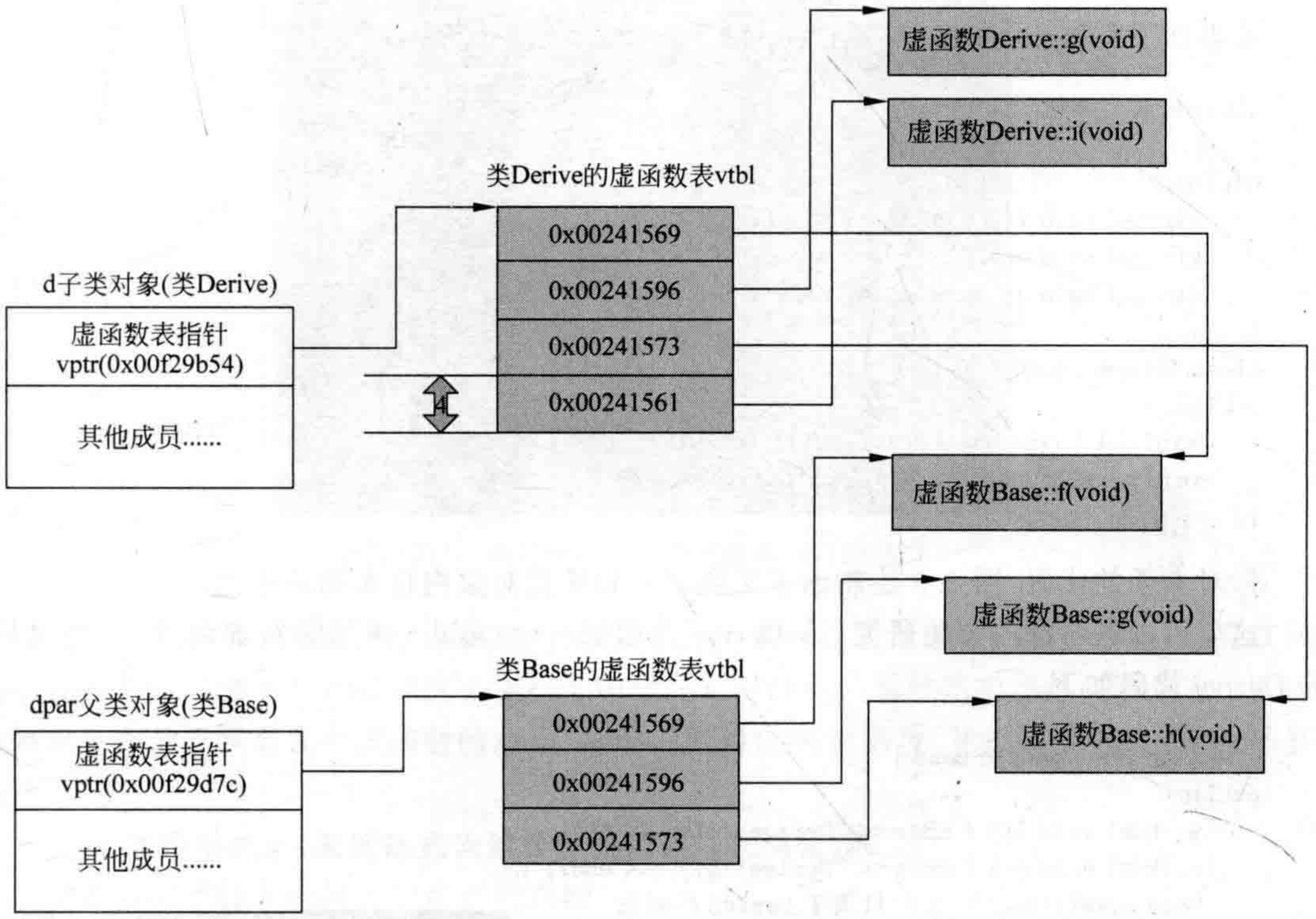

图 5.19　Base 父类对象和 Derive 子类对象内存布局示意图

从图 5.19 中看到了子类 Derive 虚函数表项的排列，首先顺序跟父类一样——f、g、h，然后才是自己的 i，这样排列是比较有好处的，下面会解释好处。现在看一看传统多态表现形式的代码。在 main 主函数中加入如下代码：

```
Base * pb = new Derive();                    //基类指针指向一个子类对象
pb->g();

Derive myderive;
Base& yb = myderive;                         //基类引用的是一个子类对象
yb.g();
```

既然子类的虚函数表的排列顺序与父类一致，所以想一想这种类似 pb-> g();的代码，在编译的时候能不能被编译器编译成如下形式：

```
(*pb->vptr[1])(pb);
```

后面圆括号中的 pb 读者已经知道，是隐含的 this 参数，前面讲解过。这里重点观察下标[1]，可以认为编译器在编译的阶段就能确定这个下标值。为什么？因为不难发现，不管子类也好，父类也好，在虚函数表中，g 这个虚函数对应的就是下标[1]。观察图 5.19 就能

理解。这也就解释了上述子类虚函数表项与父类虚函数表项排列顺序一样(一致)的好处。

所以,要在执行期间调用虚函数,唯一需要知道的内容就是调用哪个虚函数表中的虚函数——父类虚函数表还是子类虚函数表。

5.5.2 回顾和一些小试验

5.3 节曾经说过,虚函数的地址也是编译期间就知道的,固定写在可执行文件中的。

那虚函数表呢? 虚函数表在执行期间大小和内容都不会改变,所以虚函数表也是编译期间就构建出来的,跟执行期没有关系。这在 3.5.2 节介绍的比较清楚了。

下面的事读者也很清楚:在虚函数表中,顺序记录着每个虚函数的首地址。而后,编译器会向类中加入虚函数表指针这个隐含的成员变量,并在类的构造函数中安插代码用于给这个虚函数表指针赋值(这些都是编译的时候做的事)。

现在,根据一些现有的认知做一些实验。在 main 主函数中,增加如下代码:

```
Derive a1;
Derive a2;
Derive * pa3 = new Derive();
```

可以设置断点观察 a1、a2 对象以及 pa3 所指向的对象的内存,会发现这三者内存中的内容完全一样,因为 Derive 的 sizeof 值是 4 字节,这 4 字节正好是虚函数表指针的大小,所以,说明这三者的虚函数表指针指向的是同一个虚函数表。

如果把 Derive 里面的内容全注释掉(只保留其继承自父类 Base),如下:

```
class Derive : public Base {
public:
};
```

在 main 主函数中,继续加入如下代码行:

```
Base b1;
```

通过设置断点观察,不难发现,**即便子类 Derive 不重写任何虚函数,子类仍旧有自己的虚函数表,和父类 Base 的虚函数表不是一个表,尽管它们的表项相同。**

如果某个类中只有一个纯虚函数:

```
class BCXu {
public:
    virtual void pvfunc() = 0;
};
```

在 main 主函数中,加入如下代码:

```
cout << sizeof(Base) << endl;
```

执行起来,输出的 sizeof 值为 4,这说明在类 BCXu 的成员变量中肯定多了一个隐藏的虚函数表指针(占 4 字节),而 BCXu 类也肯定有一个虚函数表。所以得到一个结论:即便是纯虚函数也是在虚函数表中占据一个表项的。

5.6 多继承虚函数深释、第二基类与虚析构必加

5.6.1 多继承下的虚函数

本节的范例其实是接着上一节来的，但为了演示代码的清晰性，笔者把这些代码重新整理一下，完整地列在下面，同时还增加了一个 Base2 类的定义，让子类 Derive 同时继承 Base 和 Base2 两个基类：

```
class Base
{
public:
    virtual void f() { cout << "Base::f()" << endl; }
    virtual void g() { cout << "Base::g()" << endl; }
    virtual void h() { cout << "Base::h()" << endl; }
};

class Base2
{
public:
    virtual void hBase2() { cout << "Base2::hBase2()" << endl; }
};

class Derive : public Base, public Base2 {
public:
    virtual void i() { cout << "Derive::i()" << endl; }
    virtual void g() { cout << "Derive::g()" << endl; }
    void myselffunc() {}                          //只属于 Derive 的函数
};
```

根据以往的学习已经知道，多重继承问题的复杂性其实就体现在后继承的这个基类上。因为如果创建 Derive 对象，从对象的内存布局上来看，后面这个基类所属的 Base2 子对象是排在 Base 子对象之后的，存在 this 指针调整的问题，这在 2.3 节和 4.7 节都说过。

在 main 主函数中，加入如下代码行：

```
Base2 * pb2 = new Derive();
```

此时，pb2 指针指向的地址是经过 this 指针调整后的地址，指向的是 Base2 这个子对象，其实编译的时候，上面这行代码会被编译器转换成如下两行代码：

```
Derive * temp = new Derive();
Base2 * pb2 = (Base2 *)( (char *)(temp) + sizeof(Base));
```

这样调整之后，再访问 Base2 里面的成员变量等，才能正确地访问。

继续回顾下一个问题，如果这个时候 delete pb2;，发现程序的执行报告异常。这在 4.7.2 节已经演示过。但是这个故事没讲完，这里要继续讲。

现在，继续在 main 主函数中加入如下代码行：

```
delete pb2;
```

看起来，用 pb2 来 new 了一段内存，然后用 delete pb2;来释放这段内存，这是很正常很天经地义的一个操作，但眼下程序运行后报告异常，这让人难以理解。那怎样解决这个异常的问题呢？

5.6.2　如何成功删除用第二基类指针 new 出来的子类对象

要这样考虑这件事，虽然是用 Base2 指针，但是 new 的实际是一个 Derive 对象：

```
Base2 * pb2 = new Derive();
```

(1) 对于这种情形来讲，当 delete pb2;时，当然需要释放的是整个 Derive 对象内存，而不可能仅仅是 pb2 所指向的那块内存。

(2) 要实现(1)中的想法，编译器正常的动作应该是得能调用到 Derive 对象的析构函数，因为刚才也说了，实际上 delete 的是 Derive 对象，那么，Derive 对象的析构函数肯定应该被正常调用。

(3) 那么，谁来调用 Derive 对象的析构函数？显然是系统，或者说是编译器。编译器在处理 delete pb2;代码行的时候肯定会把这种 delete 代码解析成若干条汇编语句，其中有一条汇编语句必然是去调用析构函数。但问题的关键是，调用哪个析构函数？是 Base2 的，还是 Derive 的，还是 Base 的？看一看现在的类 Derive 对象数据布局示意图，如图 5.20 所示。

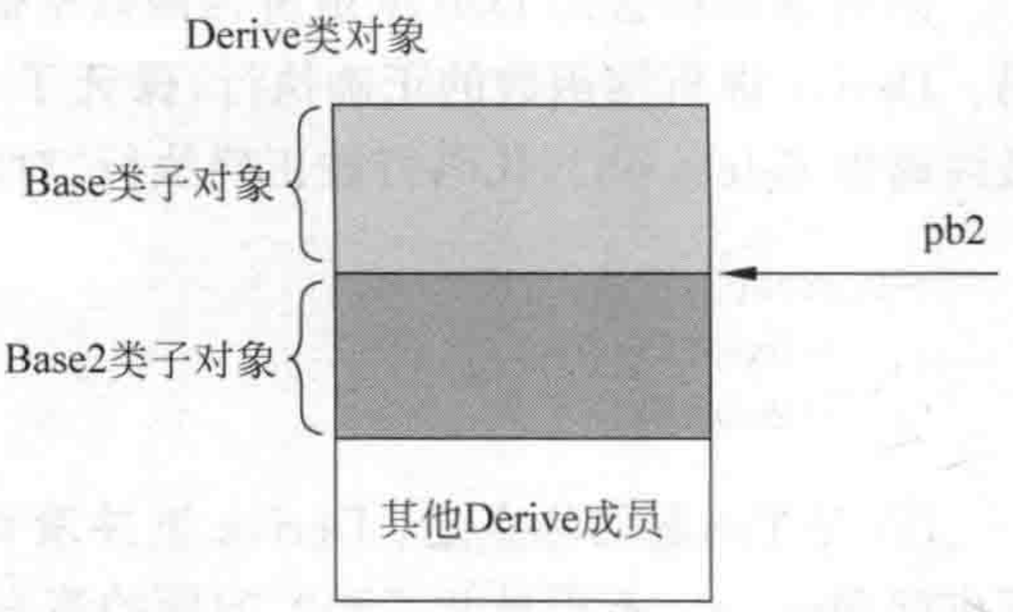

图 5.20　类 Derive 对象数据布局示意图

(4) pb2 显然与 Base 类关系不大，因为 pb2 是一个 Base2 类型的指针，那么调用析构函数应该要么是调用 Base2 的，要么是 Derive 的。根据以往学习过的虚函数知识和 5.4.4 节讲解过的“动态绑定知识”，可以知道“虚函数是动态绑定的”。那么想一想，当执行 delete pb2;代码行时，系统会怎么动作？这里分几点说：

① **如果 Base2 里没有析构函数**，就像现在范例代码的情形，那么编译器就会直接删除以 pb2 开头的这段内存，程序运行一定会报异常，因为这段内存的起始地址压根就不是 new 的时候返回的内存起始地址。

② 如果 Base2 里有一个析构函数，但这个析构函数是一个普通析构函数(非虚析构函数)，那么当 delete pb2;时，这个**析构函数会被系统调用**，但是 delete 的还是 pb2 开头的这段内存，程序运行一定会**报异常**，因为这段内存的起始地址压根就不是 new 的时候返回的内存起始地址。因为析构函数不是虚函数，编译器实施静态绑定，静态绑定就意味着 delete Base2;时删除的内存开始地址就是 pb2 当前指向的位置，所以肯定是错的。

在 Base2 中增加一个 public 修饰的普通析构函数并在其中设置断点：

```
~Base2()
{
    cout << "Base2::~Base2()" << endl;
}
```

开始调试后，发现断点会停止在 Base2 的析构函数中，如果按快捷键 F5（“调试”→“继续”命令）继续运行程序，则会执行到 delete pb2；代码行并报告运行异常。

③ 如果 Base2 里不是一个普通的析构函数而是一个**虚析构**函数，事情发生了转机，可以神奇地发现，delete pb2；执行成功，不会再报告异常了。此时离正确答案很近了。

修改 Base2 类的普通析构函数为虚析构函数：

```
virtual ~Base2()
{
    cout << "Base2::~Base2()" << endl;
}
```

因为 pb2 实际指向的是一个 Derive 对象，这意味着**如果** Derive 里有虚析构函数，那么编译器在处理 delete pb2；代码行时，因为多态的特性，查虚函数表去执行析构函数时执行的肯定是 Derive 的虚析构函数。

然后编译器会向 Derive 虚析构函数中插入调用 Base、Base2 这两个父类析构函数的代码。Derive 虚析构函数的正确执行，**保证了 Base 析构函数和 Base2 析构函数被正确执行**，最终确保 delete pb2；代码行被正确执行，对象被完美释放。所以释放的流程应该是：

```
~Derive()
    ~Base2()
    ~Base1()
```

④ 现在问题的焦点是：Derive 里**并没有**虚析构函数。那么现在的范例程序为什么也不出漏洞（bug）、不报异常了呢？问题的答案正在于 Base2 里的虚析构函数。因为 Base2 里有虚析构函数，在 Base2 里有虚析构函数的情况下，**如果 Derive 里没有虚析构函数，编译器会为 Derive 合成虚析构函数**，就是为了让 Derive 的虚析构函数能够正确调用 Base2 的析构函数和 Base 的析构函数从而正确完成 this 指针的调整和内存的释放。当然，如果 Derive 里有虚析构函数最好（就算 Derive 类中的析构函数不是虚析构函数而只是普通析构函数，因为父亲 Base2 中**有虚析构**函数，所以孩子类 Derive 的析构函数也会**自动成为虚析构函数**），编译器会向 Derive 的析构函数中插入调用 Base 和 Base2 析构函数的代码。

可以观察一下当前范例中，编译器合成的 Derive 的虚析构函数 Derive::~Derive。在开发人员命令行工具中，进入到 MyProject.obj 文件所在的路径，并输入 dumpbin /all MyProject.obj > MyProject.txt，产生 MyProject.txt 文件，打开该文件，如图 5.21 所示。

```
60501020 flags
        Code
        COMDAT; sym= "public: virtual __thiscall Derive::`Derive(void)" (??1Derive@@UAE@XZ)
        16 byte align
        Execute Read

RAW DATA #17
  00000000: 55 8B EC 81 EC CC 00 00 00 53 56 57 51 8D BD 34  U.ì.Ì....SVWQ.?4
  00000010: FF FF FF B9 33 00 00 00 B8 CC CC CC CC F3 AB 59  ???3...?ÌÌÌÌó«Y
  00000020: 89 4D F8 8B 4D F8 83 C1 04 E8 00 00 00 00 5F 5E  .M?.M?.Á.è....._^
  00000030: 5B 81 C4 CC 00 00 00 3B EC E8 00 00 00 00 8B E5  [.?Ì...;ìè.....?
  00000040: 5D C3                                            ]?

RELOCATIONS #17

                                                  Symbol    Symbol
 Offset    Type              Applied To            Index     Name
 --------  ----------------  --------------------  --------  ------
 0000002A  REL32                    00000000       118     ??1Base2@@UAE@XZ (public: virtual __thiscall Base2::`Base2(void))
 0000003A  REL32                    00000000       12F     __RTC_CheckEsp
```

图 5.21　编译器合成 Derive 虚析构函数用以调用 Base2 虚析构函数

这里笔者要重点提一下“**虚析构函数**”这个词。在传统虚函数中，子类的虚函数必须与父类的虚函数同名，才能实现多态。但是“虚析构函数”是一个特殊的存在，因为子类和父类类名不相同，所以它们的析构函数名字肯定不同。但是，只要这个**析构函数**是一个**虚**函数，依旧支持多态。这个，请读者注意。

(5) 一定要记住，对于这种带继承关系的类，new 一个对象后，要成功 delete 该对象，要求读者务必做到不管父类，还是子类，一定要**明确地写出虚析构函数**，千万不要省略掉，否则要么系统崩溃，要么可能导致内存泄露(下一节会实际举例)。

现在修改代码，向所有类中加入**虚析构**函数，从而保证多态的正常运作，保证所有的析构函数都能被正常调用，内存能够正常回收。

Base 类中：

```
public:
    virtual ~Base()
    {
        cout << "Base::~Base()" << endl;
    }
```

Derive 类中：

```
public:
    virtual ~Derive()
    {
        cout << "Derive::~Derive()" << endl;
    }
```

(6) 加入了虚析构函数后，将断点设置在 Derive::~Derive 虚析构函数中，调试程序，当程序运行停在断点行时，切换到反汇编窗口，如图 5.22 所示。

```
00D9296F  mov         dword ptr [this],ecx
00D92972  mov         ecx,offset _F0B5FBBB_MyProject@cpp (0D9F026h)
00D92977  call        @__CheckForDebuggerJustMyCode@4 (0D912E9h)
00D9297C  mov         eax,dword ptr [this]
00D9297F  mov         dword ptr [eax],offset Derive::`vftable' (0D99DF8h)
00D92985  mov         eax,dword ptr [this]
00D92988  mov         dword ptr [eax+4],offset Derive::`vftable' (0D99BA8h)
        cout << "Derive::~Derive()" << endl;
00D9298F  mov         esi,esp
00D92991  push        offset std::endl<char,std::char_traits<char> > (0D91523h)
00D92996  push        offset string "Derive::~Derive()" (0D99CECh)
00D9299B  mov         eax,dword ptr [__imp_?cout@std@@3V?$basic_ostream@DU?$char_traits@D@std@@@1@A (0D9D0D8h)]
00D929A0  push        eax
00D929A1  call        std::operator<<<std::char_traits<char> > (0D91514h)
00D929A6  add         esp,8
00D929A9  mov         ecx,eax
00D929AB  call        dword ptr [__imp_std::basic_ostream<char,std::char_traits<char> >::operator<< (0D9D0A4h)]
00D929B1  cmp         esi,esp
00D929B3  call        __RTC_CheckEsp (0D912F3h)
    }
00D929B8  mov         ecx,dword ptr [this]
00D929BB  add         ecx,4
00D929BE  call        Base2::~Base2 (0D9155Ah)
00D929C3  mov         ecx,dword ptr [this]
00D929C6  call        Base::~Base (0D9157Dh)
00D929CB  mov         ecx,dword ptr [ebp-0Ch]
00D929CE  mov         dword ptr fs:[0],ecx
```

图 5.22　子类的虚析构函数的函数体执行完成后分别调用了 Base2 和 Base 的虚析构函数

从图 5.22 中可以看到，子类先执行了自己的虚析构函数的函数体，然后又去调用 Base2 和 Base 的虚析构函数。

执行起来，看一看结果(注意结果中显示的析构函数的函数体执行顺序)：

```
Derive::~Derive()
Base2::~Base2()
Base::~Base()
```

根据以往学习过的知识和经验，现在可以绘制一下当前这种情况下的 Derive 类相关的虚函数表。在绘制之前，可以在 main 主函数中写一些测试代码，利用测试代码，就可以绘制虚函数表：

```
Base * pbm = new Base();
Base2 * pb222 = new Base2();
cout << "断点设置在这里" << endl;
```

将断点设置在 cout 所在行，调试程序，当程序运行停在断点行时，将鼠标分别放在 pbm 和 pb222 上并展开，观察它们的虚函数表项，如图 5.23 所示。

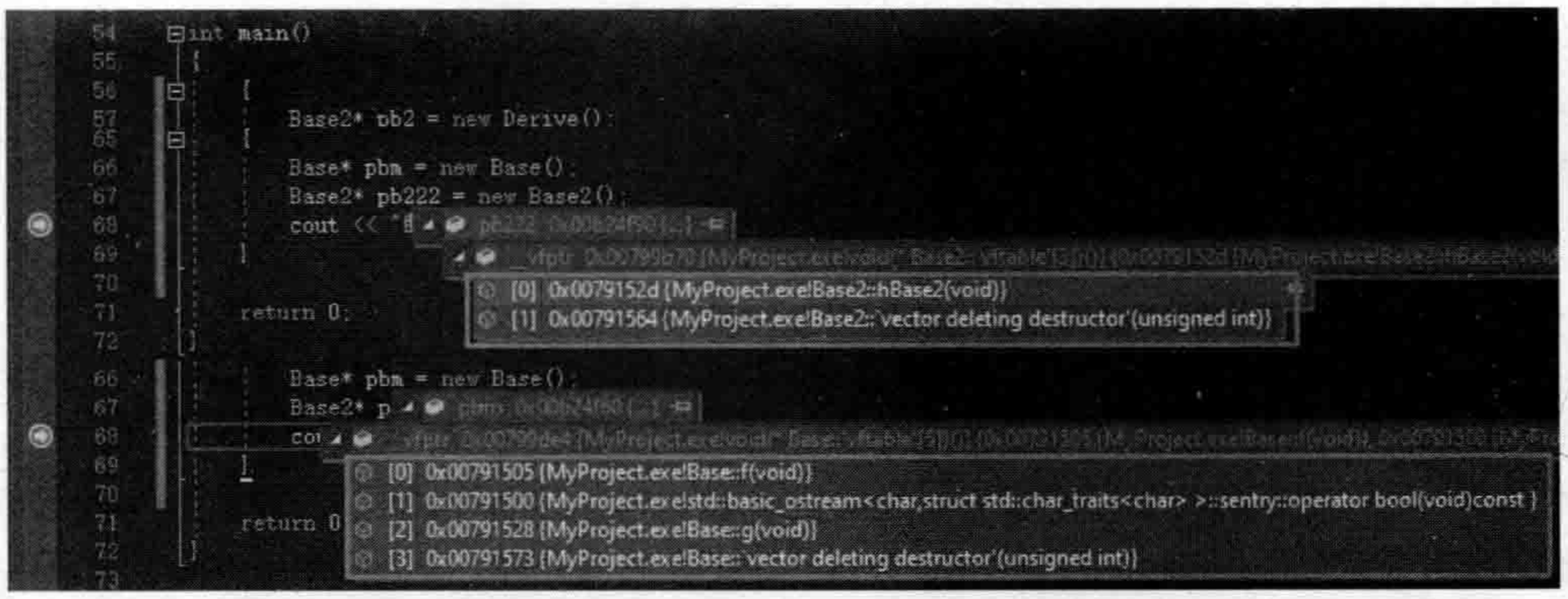

图 5.23　通过 new 创建类对象来观察某个类关联的虚函数表中的各个表项

虽然图 5.23 可能不一定全，但可以大概知道与某个类相关的虚函数表项有哪些。

继续在 main 主函数中增加一些测试代码：

```
Derive * p11212 = new Derive();
p11212->g();                    //Derive::g()
p11212->i();                    //Derive::i(),走虚函数表,查询虚函数表得到虚函数地址并调用虚函数
Derive dddd;
dddd.i();                       //Derive::i(),不走虚函数表,直接调用虚函数
```

上述代码读者可以自行设置断点并进行调试，查看反汇编代码，确定哪些虚函数调用需要通过虚函数表进行，哪些虚函数调用是直接调用，不需要通过虚函数表进行，这些知识笔者在前面已经都详细地演示过了。

根据上面的代码测试，加之各种认知，绘制一下子类 Derive 对象的虚函数表的组织结构图，如图 5.24 所示。

观察图 5.24，发现类 Derive 的虚函数表 2 中的第二项看起来是一串信息。这一长串代码是怎么来的呢？

开始的时候笔者通过设置断点进行观察，在 main 主函数的代码行 p11212-> g();设置

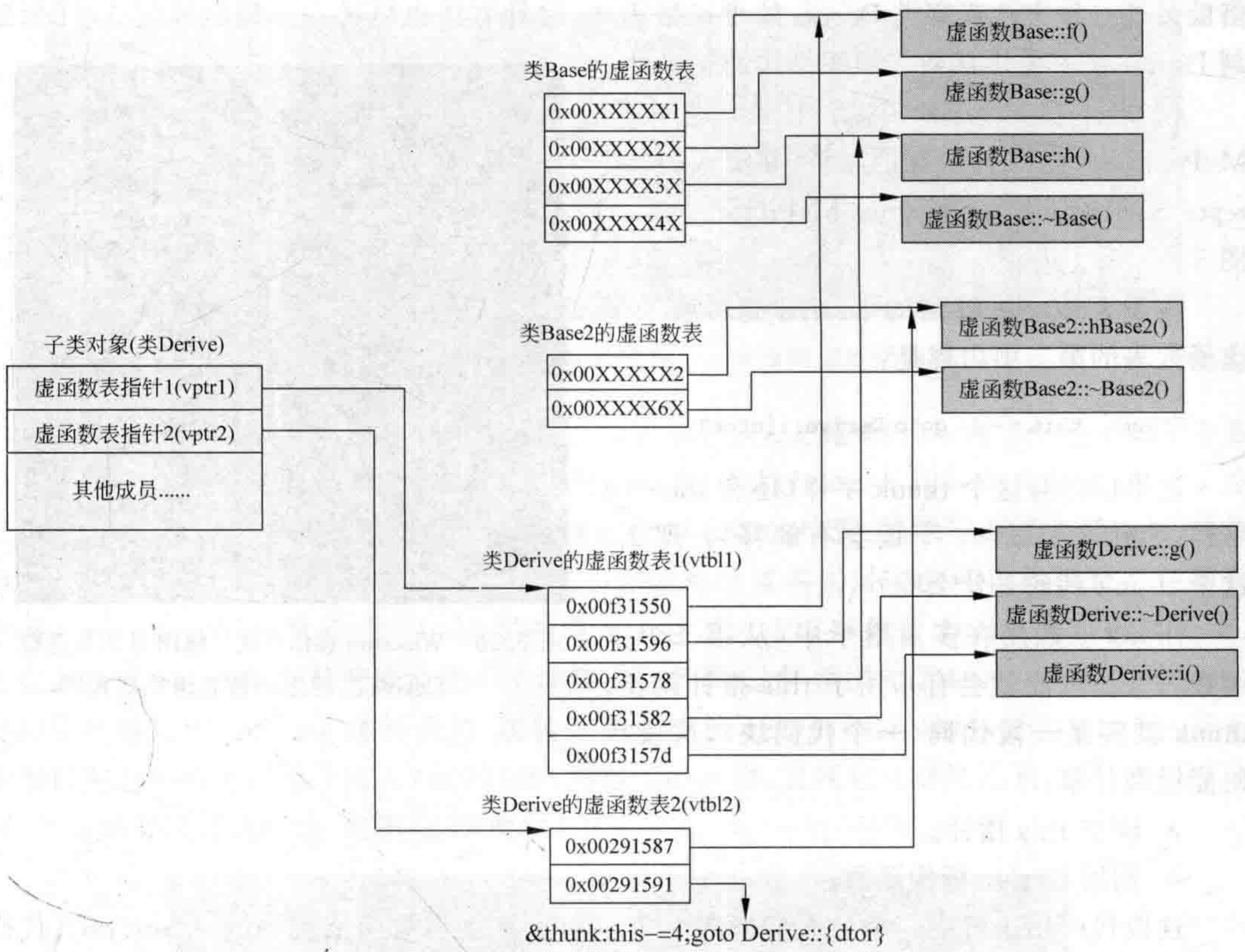

图 5.24　Derive 类对象虚函数表组织结构图

了一个断点，调试程序，当程序运行停在断点行时，将鼠标移动到 Derive * p11212 = new Derive();代码行的 p11212 并展开，查看其中的第二个虚函数表(Base2 类所属的虚函数表)的内容，如图 5.25 所示。

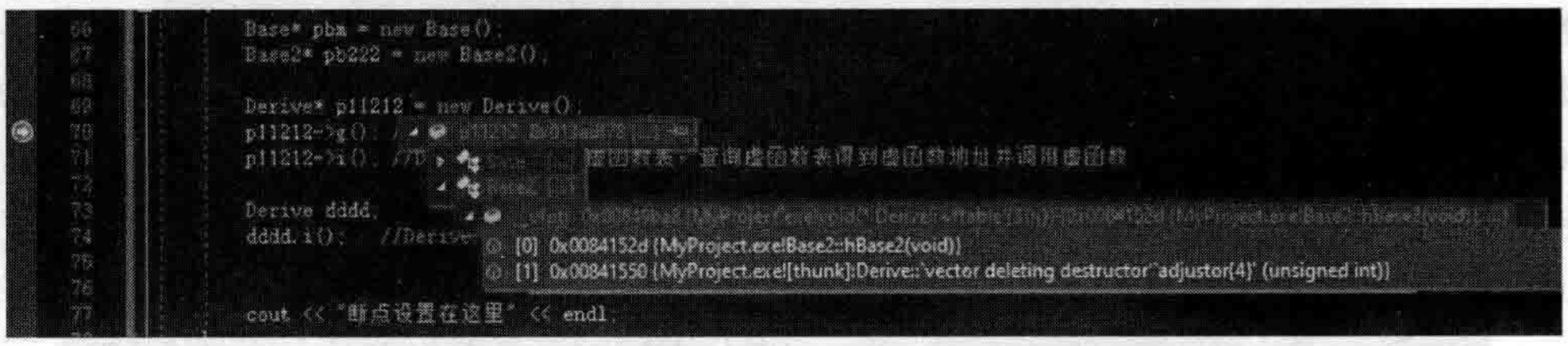

图 5.25　Derive 类的第二个虚函数表中的一个特殊项

在图 5.25 中，可以看到第二个虚函数表中的最后一项，内容是：

```
0x00841550 {MyProject.exe![thunk]:Derive::`vector deleting destructor'adjustor{4}'(unsigned int)}
```

如果读者编译器中显示的信息与图 5.25 显示的信息差异太大，建议用“生成”→“重新生成解决方案”命令来重新编译一下整个项目，因为有时编译器也会显示出一些错乱的信息。

上面这一堆信息中发现有一个关键词：thunk。这个关键词应该引起重视。此外，笔者

借助另外一种方法观察类 Derive 的虚函数表项，这种方法也更进一步地验证图 5.24 所绘制 Derive 类对象虚函数表组织结构的正确性。

使用开发人员命令行工具，进入到 MyProject.cpp 文件所在的路径，并输入 cl /d1 reportSingleClassLayoutDerive MyProject.cpp，如图 5.26 所示。

```
class Derive    size(8):
        +---
 0      | +--- (base class Base)
 0      | | {vfptr}
        | +---
 4      | +--- (base class Base2)
 4      | | {vfptr}
        | +---
        +---

Derive::$vftable@Base@:
        | &Derive_meta
        |  0
 0      | &Base::f
 1      | &Derive::g
 2      | &Base::h
 3      | &Derive::{dtor}
 4      | &Derive::i

Derive::$vftable@Base2@:
        | -4
 0      | &Base2::hBase2
 1      | &thunk: this-=4; goto Derive::{dtor}
```

图 5.26　Windows 操作系统下使用 cl 工具查看 Derive 类的虚函数表组织结构图

在图 5.26 中，看到的 Derive 类的第二个虚函数表的第二项内容是：

```
&thunk: this -= 4; goto Derive::{dtor}
```

这里依然有这个 thunk 字样（还有 this-=4 字样，为什么要减 4？下面会有解释）。那么，这个 thunk 到底是什么呢？

thunk 一般用在多重继承中（从第二个虚函数表开始可能就会有），用于 this 指针调整。**thunk 其实是一段代码（一个代码块）**，这段代码能做两件事：

- 调整 this 指针。
- 调用 Derive 析构函数。

这段代码用来解决一个什么问题呢？正是用来解决当前面范例中的 delete pb2;代码行执行时，可以通过这段代码来成功调用 Derive 的析构函数，从而实现正确的释放内存的目的。或者说的再明确一点，**当执行 delete pb2;代码行时，系统就会调用（执行）thunk 这段代码**。

现在来看一看 thunk 对应的代码块是什么样子，虽然图 5.25 中显示的第二个虚函数表中的最后一项的一堆字符串似乎难以捕捉到有用的内容，但是在这一堆字符串最前面有一个地址 0x00841550（这个地址就是 thunk 块的入口地址），在调试程序时，当断点停留在图 5.25 所示的断点行时可以切换到反汇编窗口，并在该窗口左上角的地址部分输入 0x00841550 并按 Enter 键，可以看到反汇编窗口导航到了该地址对应的代码行，如图 5.27 所示。

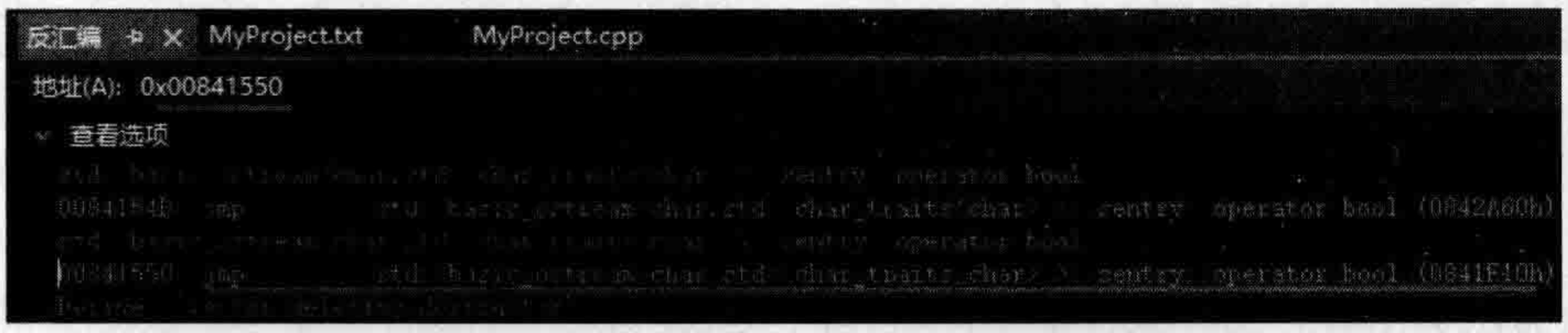

图 5.27　查看虚函数表中 thunk 所对应的代码块

从图 5.27 中看到了地址 0x00841550 对应的是一个 jmp 跳转语句，跳转到的是 0x0841F10h 地址。于是继续在反汇编窗口左上角的地址部分输入 0x0841F10 并按 Enter 键，可以看到反汇编窗口导航到了该地址对应的代码行如图 5.28 所示。

```
反汇编 - MyProject.txt    MyProject.cpp
地址(A): 0x0841F10
查看选项
00841F01  call    __RTC_CheckEsp (08412F3h)
00841F06  mov     esp,ebp
00841F08  pop     ebp
00841F09  ret
--- 无源文件 ---
00841F0A  jmp     dword ptr [__imp_std::basic_ostream<char,std::char_traits<char> >::operator<< (034D0E0h)]
00841F10  sub     ecx,4
00841F13  jmp     Derive::`vector deleting destructor' (0841555h)
00841F18  int     3
```

图 5.28　进一步查看虚函数表中 thunk 所对应的代码块

看图 5.28 中的 sub ecx,4 代码行，这里表示 ecx-4(ecx 减 4)，ecx 根据以往经验，应该是 this 指针，ecx-4 就是在**调整 this 指针**(前面也刚刚说过 thunk 代码块能够调整 this 指针)。为什么这里是－4？参考图 5.20，注意该图中 pb2 的位置，原来 this 指针在 pb2 的位置，如果想要调用 Derive 类的析构函数(因为删除对象要调用对应类的析构函数)，必须要拿到 Derive 对象的首地址，所以 pb2 的位置向上移动 4 个字节就是 Derive 对象的首地址，所以，sub ecx,4 实际就是把地址往小了调整 4 个字节，这样就**指向了 Derive 对象的首地址**。然后紧接着又是一个 jmp 跳转语句，跳转到的应该是 0x0841555h 地址，于是，继续在反汇编窗口左上角的地址部分输入 0x0841555 并按 Enter 键，发现它又继续跳到 0x0842AC0 地址，于是继续跟进该地址，如图 5.29 所示。

```
反汇编 - MyProject.txt    MyProject.cpp
地址(A): 0x0842AC0
查看选项
00842ABD  int     3
00842ABE  int     3
00842ABF  int     3
00842AC0  push    ebp
00842AC1  mov     ebp,esp
00842AC3  sub     esp,0CCh
00842AC9  push    ebx
00842ACA  push    esi
00842ACB  push    edi
00842ACC  push    ecx
00842ACD  lea     edi,[ebp-0CCh]
00842AD3  mov     ecx,33h
00842AD8  mov     eax,0CCCCCCCCh
00842ADD  rep stos dword ptr es:[edi]
00842ADF  pop     ecx
00842AE0  mov     dword ptr [this],ecx
00842AE3  mov     ecx,dword ptr [this]
00842AE6  call    Derive::~Derive (08415BEh)
00842AEB  mov     eax,dword ptr [ebp+8]
00842AEE  and     eax,1
00842AF1  je      Derive::`scalar deleting destructor'+41h (0842B01h)
00842AF3  push    8
00842AF5  mov     eax,dword ptr [this]
00842AF8  push    eax
00842AF9  call    operator delete (08412DAh)
00842AFE  add     esp,8
00842B01  mov     eax,dword ptr [this]
00842B04  pop     edi
00842B05  pop     esi
00842B06  pop     ebx
00842B07  add     esp,0CCh
```

图 5.29　进一步查看虚函数表中 thunk 所对应的代码块

通过层层跟进 thunk 代码块，看到了如图 5.29 所示的信息，这里的汇编代码比较多，但挑重点看即可，可以看到 **Derive::～Derive 析构函数被调用**了(因为前面正确地指向了 Derive 对象的首地址，所以这里就可以正确地调用 Derive 的析构函数。前面讲过，调用类

中的普通成员函数当然也包括虚函数需要传递一个隐含的 this 指针给该成员函数，所以一定要先正确地拿到 Derive 对象的首地址，这就是前面先调整 this 指针的目的）。

上面分析了不少 thunk 代码块所做的事情，这里可以看得出来，编译器的设计者居然**在虚函数表项目中嵌入了一个 thunk 代码块（实际是一个指向一个代码块的指针）**专门处理 delete pb2；这种代码的对象释放问题。以往可能读者会认为虚函数表中只有指向各个虚函数入口地址的指针。

现在继续研究一下 thunk 代码块被调用的过程（前面已经多次说过，执行 delete pb2；代码行时，就会调用这个 thunk 代码块）。

将断点行设置在 delete pb2；所在行，调试程序，当程序执行停留在断点行时切换到反汇编窗口，如图 5.30 所示。

```
        delete pb2;
008468EA  mov         eax,dword ptr [ebp-18h]
008468ED  mov         dword ptr [ebp-124h],eax
008468F3  cmp         dword ptr [ebp-124h],0
008468FA  je          main+0F2h (0846922h)
008468FC  mov         esi,esp
008468FE  push        1
00846900  mov         ecx,dword ptr [ebp-124h]
00846906  mov         edx,dword ptr [ecx]
00846908  mov         ecx,dword ptr [ebp-124h]
0084690E  mov         eax,dword ptr [edx+4]
00846911  call        eax
00846913  cmp         esi,esp
```

图 5.30　通过调用 thunk 代码块实现正确地释放对象

图 5.30 中，edx 其实就是 pb2 所指向的内存地址（也就是 Derive 类第二个虚函数表的首地址），所以 edx＋4 其实就是跳到了图 5.24 中 Derive 类第二个虚函数表的第二项，其实就是 thunk 代码块的首地址。然后紧接着 call eax 正是去调用 thunk 代码块去了。按 F11 快捷键不断跟踪进去，读者就可以看到图 5.29 所示的画面了（thunk 代码块执行的画面）。

继续看图 5.29，刚刚说到，Derive::～Derive 析构函数被调用，跟踪进该析构函数中看一看做了什么。首先执行了 Derive::～Derive 的函数体，然后又调用了 Base2::～Base2 和 Base::～Base，如图 5.31 所示。

```
00842985  mov         eax,dword ptr [this]
00842988  mov         dword ptr [eax+4],offset Derive::`vftable' (0849BA8h)    已用时间<=1ms
        cout << "Derive::~Derive()" << endl;
0084298F  mov         esi,esp
00842991  push        offset std::endl<char,std::char_traits<char> > (0841523h)
00842996  push        offset string "Derive::~Derive()" (0849CECh)
0084299B  mov         eax,dword ptr [__imp_?cout@std@@3V?$basic_ostream@DU?$char_traits@D@std@@@1@A (084D0D8h)]
008429A0  push        eax
008429A1  call        std::operator<<<std::char_traits<char> > (0841514h)
008429A6  add         esp,8
008429A9  mov         ecx,eax
008429AB  call        dword ptr [__imp_std::basic_ostream<char,std::char_traits<char> >::operator<< (084D0A4h)]
008429B1  cmp         esi,esp
008429B3  call        __RTC_CheckEsp (08412F3h)
    }
008429B8  mov         ecx,dword ptr [this]
008429BB  add         ecx,4
008429BE  call        Base2::~Base2 (084155Ah)
008429C3  mov         ecx,dword ptr [this]
008429C6  call        Base::~Base (084157Dh)
008429CB  mov         ecx,dword ptr [ebp-0Ch]
008429CE  mov         dword ptr fs:[0],ecx
008429D5  pop         ecx
008429D6  pop         edi
```

图 5.31　thunk 代码块调用完了 Derive::～Derive 又去调用 Base2::～Base2 和 Base::～Base 析构函数

回头再看图 5.29，在调用了 Derive::～Derive 后，又去调用了系统的 operator delete（这是一个函数）用来真正地释放内存。

总结一下，当执行 delete pb2；代码行的时候，系统工作流程如图 5.32 所示。

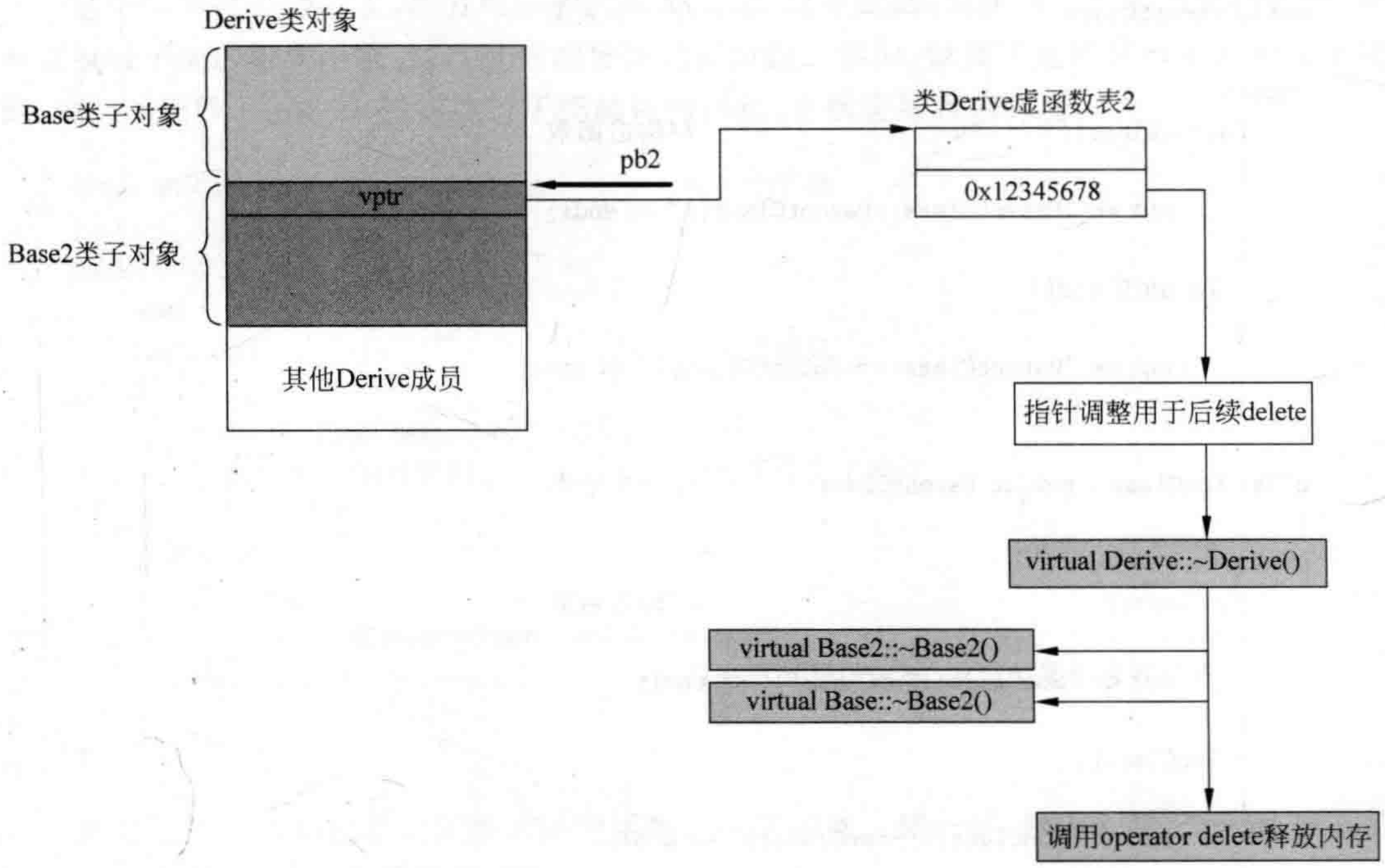

图 5.32　delete pb2;代码行执行后系统工作流程图

图 5.32 比较好理解，在执行了代码行 delete pb2;来 **delete 第二个基类指针**(本意当然是要释放整个 new 出来的内存)。系统释放内存的步骤总结如下：

(1) 通过 Base2 类的虚函数表指针 vptr 去找 Derive 类的虚函数表 2，找到了该虚函数表的第 2 项并调用该项。这第 2 项就是 thunk 代码块的首地址，这个代码块能够调整 this 指针偏移，并能够执行 Derive 对象的虚析构函数，那么就开始执行这个代码块。

(2) 执行 this 指针调整功能。

(3) 调用 Derive::~Derive 虚析构函数。在 Derive::~Derive 中又调用了 Base2::~Base2 和 Base::~Base 两个虚析构函数。

(4) 调用 operator delete 释放内存。

本节用了很大篇幅带着读者走读一下汇编代码，看看用第二基类指针怎么释放整个子类对象。同时，认识了 thunk，其实它就是一段汇编代码，这段汇编代码的**首地址**被放在了虚函数表里作为虚函数表中的一项，这段汇编代码用于 this 指针调整以及执行 Derive 的虚析构函数。当然，在**不需要 this 指针调整的情况下虚函数表中也不会有 thunk 代码块首地址这一项**。

5.6.3　父类非虚析构函数时导致的内存泄露演示

在 C++面试中被问及最多的一个问题是：**一个父类的析构函数为什么必须是虚函数**？其实针对这个问题，5.6.2 节已经给出了非常详细的解释和答案。

本节的目的就是用更浅显的范例来解释“**一个父类的析构函数为什么必须是虚函数**”。在 MyProject.cpp 的上面，增加如下代码行来定义具有父子关系的两个新类：

```
class ParentClass                                    //一个父类
{
public:
    ParentClass()                                    //构造函数
    {
        cout << "ParentClass::ParentClass()" << endl;
    }
    ~ParentClass()
    {
        cout << "ParentClass::~ParentClass()" << endl;
    }
};
class SonClass : public ParentClass                  //一个子类
{
public:
    SonClass()                                       //构造函数
    {
        cout << "SonClass::SonClass()" << endl;
    }
    ~SonClass()
    {
        cout << "SonClass::~SonClass()" << endl;
    }
};
```

在 main 主函数中，加入如下代码来创建一个子类对象：

```
SonClass sonobj;
```

执行起来，看一看新加入的这行代码的结果：

```
ParentClass::ParentClass()
SonClass::SonClass()
SonClass::~SonClass()
ParentClass::~ParentClass()
```

可以看到，创建 sonobj 时，先执行父类的构造函数体，再执行子类的构造函数体，析构 sonobj 时，先执行子类的析构函数体，再执行父类的析构函数体（对象的构造、析构顺序等话题第 6 章会详细讲解）。

现在看起来，上面的结果没有问题。

在 main 主函数中，注释掉刚才的 SonClass sonobj;代码行，加入新代码行，这次用父类指针创建子类对象：

```
ParentClass * parobj = new SonClass;          //创建 SonClass 对象
delete parobj;                                //删除 SonClass 对象
```

执行起来，看一看新加入的这两行代码的结果：

```
ParentClass::ParentClass()
SonClass::SonClass()
ParentClass::~ParentClass()
```

这个结果就有问题了，读者可以看的非常清楚，此时此刻，只执行了**父类的析构函数**，并没有执行子类的析构函数，这就有可能导致内存泄露。例如，像如下这样修改子类的定义代码，则因为删除 parobj 时没有执行子类的析构函数，必然会导致内存泄露：

```
class SonClass : public ParentClass          //一个子类
{
public:
    char * m_p = NULL;
    SonClass()                               //构造函数
    {
        cout << "SonClass::SonClass()" << endl;
        m_p = new char[100];                 //这里分配了内存
    }
    ~SonClass()
    {
        cout << "SonClass::~SonClass()" << endl;
        delete m_p;                          //这里要释放内存,否则会导致内存泄露
    }
};
```

此时的解决方案就是让父类的析构函数变成虚函数。试一试，修改父类 ParentClass 的析构函数定义，让其成为虚析构函数：

```
virtual ~ParentClass()
{
    cout << "ParentClass::~ParentClass()" << endl;
}
```

再次执行程序，看一看结果：

```
ParentClass::ParentClass()
SonClass::SonClass()
SonClass::~SonClass()
ParentClass::~ParentClass()
```

这个结果就正常了，读者可以看的非常清楚，此时此刻，子类的析构函数先执行了，然后**父类的析构函数也执行了**，这就不会有任何问题，内存泄露迎刃而解。这里要说明两点：

(1) 如果不涉及父类子类问题，整个系统中只有单独一个类，那么该类的析构函数是否是虚函数都没有关系。

(2) 有人说如果父类析构函数不设置为虚函数，而只把子类虚构函数设置为虚函数会怎样？读者可以测试一下，结果就是报程序执行异常。为什么会报异常，读者可以自己分析一下。这里给一些提示：

① 子类 SonClass 因为有了一个虚函数，会在成员变量中多出一个虚函数表指针(x86 平台下是 4 字节)，这意味着子类 SonClass 对象的 sizeof 值比父类 ParentClass 对象的 sizeof 值多出 4 字节。

② 因为多出的这 4 字节导致在真正调用系统底层的 operator delete 释放内存时 new 的内存数量和 delete 的内存数量不等，从而使系统产生异常。

③ 如果在 ParentClass 类中增加一个 virtual void f() {}；虚函数定义，则再执行程序，最起码不会再报程序执行异常。

④ 不管怎么说，不建议读者做只给子类析构函数设置为虚函数的尝试，似乎没什么意义。

那么为什么父类用了虚析构函数后，执行结果就正常了？其实 5.6.2 节叙述的比较清楚了，这里再简述一遍。当 delete 指向子类对象的父类指针(parobj)时：

(1) 如果父类的析构函数**不是虚函数**，则不会触发动态绑定，结果就是只会调用父类的析构函数而不会调用子类的析构函数，从而可能导致内存泄露(如果子类的析构函数中存在诸如 delete 这样的代码没被执行的话)。

(2) 如果父类的析构函数**是虚函数，则子类的析构函数一定是虚函数(就算子类析构函数前面不加 virtual 也还是虚析构函数，这是 C++语言的语法规则)**，则会触发系统的动态绑定，因为 new 的实际是一个子类对象，所以先执行的是子类的析构函数，同时编译器还会向子类的析构函数中(具体地说位置应该在子类析构函数的函数体后面)插入调用父类析构函数的代码，最终实现了先调用子类析构函数，再调用父类析构函数，达到了让整个对象完美释放的目的。

5.7 多继承第二基类虚函数支持与虚继承带虚函数

5.7.1 多重继承第二基类对虚函数支持的影响(this 指针调整作用)

本节的范例保持上一节结束时的范例代码，但 main 主函数中的代码全部删除。

多重继承的话题已经探讨了不少，读者已经知道，某个类继承了几个父类，就会有几个虚函数表，而单一继承只会有一个虚函数表。

再次说一下 this 指针调整。回顾一下，调整的目的是做什么？因为要调用一个对象的成员函数，总得需要这个对象或者这个对象的指针，这里单说对象指针。对象指针，指向对象的首地址，也只有**指向对象的首地址，才能正确地调用对象的成员函数**，因为编译器内部是要把这个对象指针(对象首地址)作为 this 参数传递到成员函数中去，然后用这个参数来修改成员函数里面的一些成员变量的值等等，成员变量的地址需要根据这个 this 参数来确定(成员变量的地址＝ this＋成员变量偏移值)。

所以，**this 指针的调整目的就是让对象指针正确地指向对象首地址，从而能正确调用对象的成员函数或者正确确定成员变量的存储位置**，这样理解 this 指针调整即可。

在**多重继承**下，有几种情况，第二或者后续的基类会对虚函数的支持产生影响，也就是说要进行 this 指针的调整。下面看一看有哪几种情况要进行 this 指针的调整。

1. 通过指向第二个基类的指针调用继承类的虚函数时

例如下面的代码：

```
Base2 * pb2 = new Derive();
delete pb2;                                    //这里调用了 Derive 类的虚析构函数
```

上面两行代码的执行细节上一节已经学习过了，因为 pb2 是指向 Derive 对象中的 Base2 子对象(第二基类指针)，所以为了能够正确执行 Derive 的虚析构函数，pb2 必须要调整指向 Derive 对象首地址。通过上一节已经看到，pb2 指向的地址减了 4 字节(图 5.28)，

调整上去，指向了真正 Derive 对象的首地址。

2. 一个指向派生类的指针调用第二个基类中的虚函数时

例如下面的代码：

```
Derive * pd2 = new Derive();
pd2 -> hBase2();
```

设置断点在 pd2-> hBase2()；代码行并开始调试，当程序执行停到该断点行时，切换到反汇编窗口，如图 5.33 所示。

```
        pd2->hBase2();
00706956  mov         ecx,dword ptr [ebp-14h]
00706959  add         ecx,4
0070695C  mov         eax,dword ptr [ebp-14h]
0070695F  mov         edx,dword ptr [eax+4]
00706962  mov         esi,esp
00706964  mov         eax,dword ptr [edx]
00706966  call        eax
00706968  cmp         esi,esp
0070696A  call        __RTC_CheckEsp (07012F3h)
```

图 5.33　通过反汇编窗口观察 this 指针调整问题

在上述代码中，因为 pd2 本身是 Derive * 类型指针，而 hBase2 是第二基类 Base2 的成员函数，要想成功调用 Base2 的 hBase2 成员函数，必须进行 this 指针调整，让 this 指针指向 Base2 类子对象的首地址。与图 5.20 类似，pd2 只有向下走 4 个字节才能指向 Base2 类子对象的首地址，因此，这里 this 是需要＋4 的。图 5.33 中，汇编代码行 add ecx,4 就用于给 this 指针进行＋4 调整。

3. 允许虚函数的返回值类型有所变化时

扩充一下范例代码。在 Base 类中增加如下 public 修饰的成员函数：

```
public:
    virtual Base * clone() const
    {
        return new Base();
    }
```

在 Base2 类中增加如下 public 修饰的成员函数：

```
public:
    virtual Base2 * clone() const
    {
        return new Base2();
    }
```

在 Derive 类中增加如下 public 修饰的成员函数：

```
public:
    virtual Derive * clone() const
    {
        return new Derive();
    }
```

在 main 主函数中，注释掉以往的代码，重新加入如下代码：

```
Base2 * pb1 = new Derive();
Base2 * pb2 = pb1 -> clone(); //执行 Derive::clone()
```

执行调用 clone 成员函数这行代码时，pb1 会调整指向 Derive 对象开始地址，这样调用的才是 Derive 版本的 clone，所以 this 指针应该变小，应该用 sub 汇编语句调整。调试起来

观察一下：在 Base2 * pb2 = pb1->clone();代码行设置断点并开始调试，当程序执行停到该断点行时，将鼠标放在 pb1 变量名上并展开观察，如图 5.34 所示。

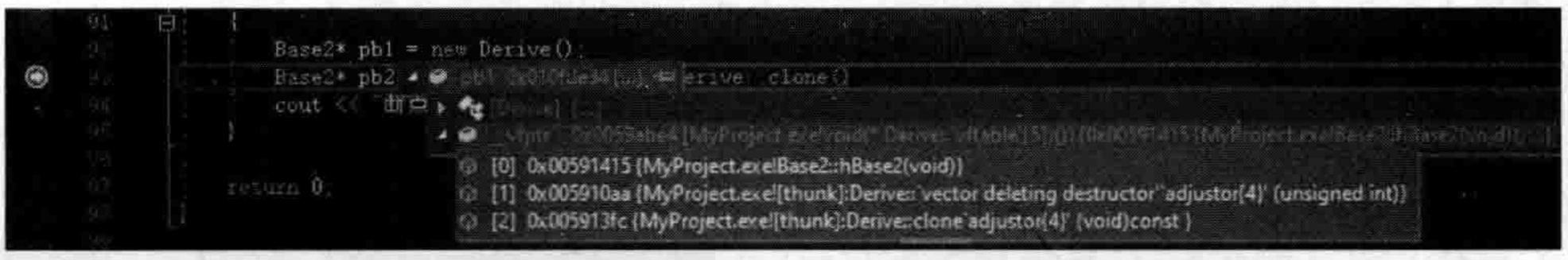

图 5.34　Derive 类的第二个虚函数表中的两个额外的 thunk 项

从图 5.34 中可以看到 Derive 类的第二个虚函数表中有三项，其中后两项都是 thunk 项(代码块)。读者已经知道，thunk 项的一个主要功能就是调整 this 指针偏移值，调整了偏移值后，才能调用 Derive 类的 clone 成员函数。

接着，切换到反汇编窗口，如图 5.35 所示。

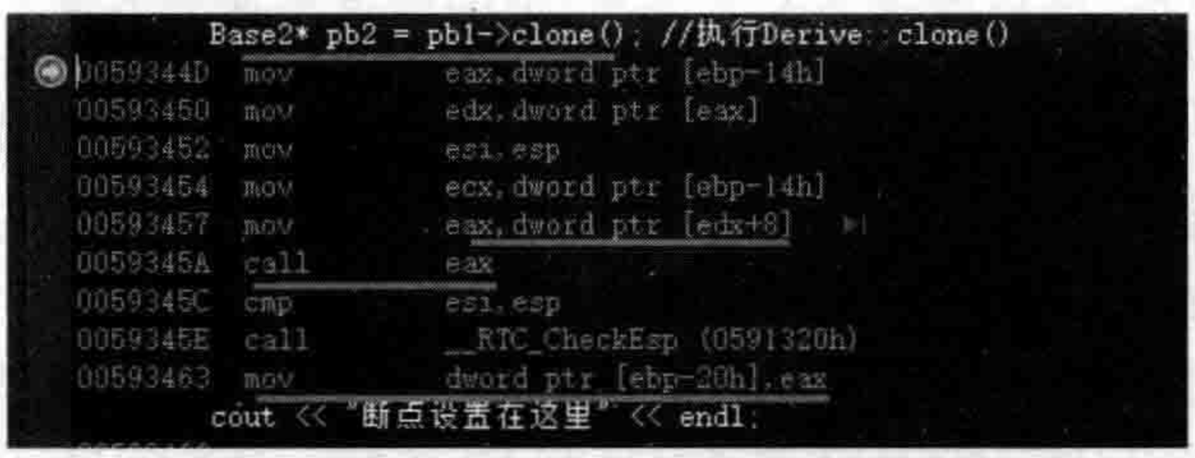

图 5.35　调用 Derive 类的第二个虚函数表中调整 this 偏移值的 thunk 代码块

从图 5.35 中可以看到[edx+8]字样的出现，这意味着寻找 Derive 类第二个虚函数表中的第三项，也就是下标为[2]的代码块项，并接着通过 call 指令去执行这个代码块，按 F11 键跟踪进 call eax 代码行，如图 5.36 所示。

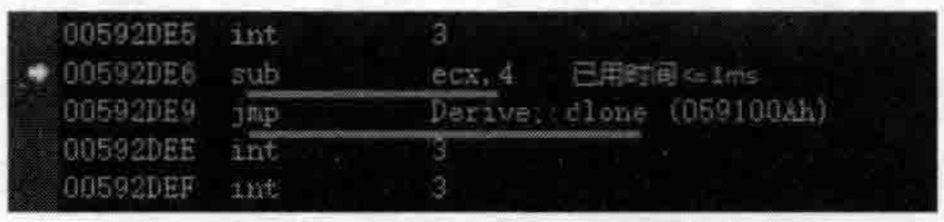

图 5.36　执行 thunk 代码块进行 this 指针调整

从图 5.36 中可以非常清楚地看到，通过调用 sub 指令将 ecx 的内容－4，这意味着 this 指针往回走了 4 个字节，正好走到真实的 Derive 对象的首地址，然后，执行 jmp 行来调用 Derive::clone 成员函数。这里可以跟踪进汇编代码中的 Derive::clone 继续观察，如图 5.37 所示。

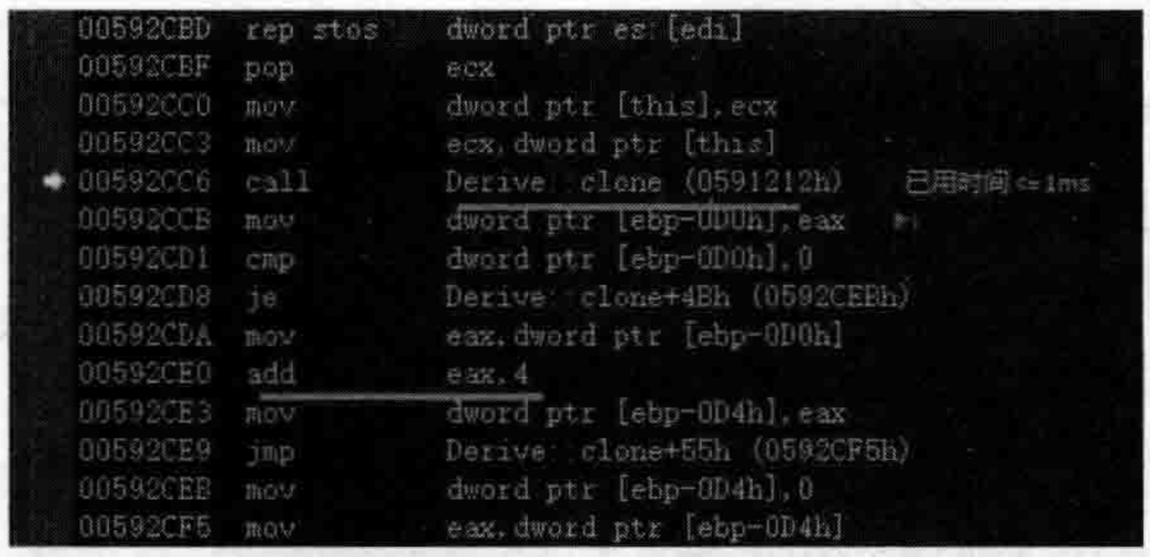

图 5.37　跟踪进 Derive::clone 后继续观察对应的汇编代码

图 5.37 表示已经跟踪进了 Derive::clone 成员函数相关代码段，在其中，首先真正地利用 call 去调用了 Derive::clone 成员函数。执行完 Derive::clone 成员函数后，返回的是一个指向新 Derive 对象的指针，但是因为这个指针是要赋给 pb2(Base2 * pb2 = pb1-> clone();)，pb2 可是一个 Base2 * 类型的指针，所以在赋给 pb2 之前，还要调整一下，也就是指针要往后走 4 个字节，以指向 Base2 子对象。所以图 5.37 中的 add eax,4 就表示指针往后走 4 个字节来进行调整。

用 Shift+F11 键将程序执行流程跳出来后，又返回到了图 5.35 所示的执行流程中，其中的 mov dword ptr [ebp-20h],eax 代码行就是把调整后的 this 指针赋给了 pb2。

请注意，不同的编译器，在这些实现细节上可能各不相同，但道理相通。

5.7.2　虚继承下的虚函数

为了简化问题，把以往的范例代码全部注释掉。这里重新书写：

```
class Base
{
public:
    virtual void f() {}
    virtual ~Base() {}

    int m_basei;
};
class Derive :public virtual Base
{
public:
    virtual ~Derive() {}

    int m_derivei;
};
```

笔者在 main 主函数中通过加几行代码来推测一下对象的数据布局，第 4 章中已经用此法推测过了不少对象的数据布局，但同时既有虚基类又有虚函数的对象数据布局还没研究过，这里就研究一下。在 main 主函数中加入如下测试代码：

```
cout << sizeof(Derive) << endl;                    //16
Derive dobj;
dobj.m_basei = 2;                                  //13~16 字节
dobj.m_derivei = 5;                                //5~8 字节
Derive * pdobj = new Derive();
pdobj -> f();
```

上述代码中，注意 Derive 的 sizeof 值是 16，因为有一个虚基类表指针、一个虚函数表指针，以及两个 int 类型成员变量，一共是 4×4=16(B)。

上面确定了两个成员变量的位置(看代码行中的注释)，接下来面临的问题是确定虚函数表指针的位置。怎样确定？

因为 f 函数是一个虚函数，pdobj-> f();调用走的应该是虚函数表，所以，只要找到虚函

数表首地址，与 pdobj 对象内存的内容比较，就能确定 pdobj 对象内存布局中虚函数表指针的位置。

将断点设置在 pdobj->f();代码行并开始调试，当程序执行停到该断点行时，切换到反汇编窗口，如图 5.38 所示。

在图 5.38 中看到了一行汇编代码 call eax，猜测一下，这个就是调用虚函数 f，这说明在执行 call eax 代码行之前，eax 寄存器中保存着虚函数 f 的入口地址。在当前的调试状态下，多次按 F10 快捷键单步调试程序，一直到图 5.38 中左侧黄色箭头停留在 call eax 代码行为止。此时，观察寄存器中 eax 的值，如图 5.39 所示。

```
        pdobj->f();
00BB1F01  mov       eax,dword ptr [ebp-30h]
00BB1F04  mov       ecx,dword ptr [eax]
00BB1F06  mov       edx,dword ptr [ecx+4]
00BB1F09  mov       eax,dword ptr [ebp-30h]
00BB1F0C  mov       ecx,dword ptr [eax]
00BB1F0E  mov       eax,dword ptr [ebp-30h]
00BB1F11  add       eax,dword ptr [ecx+4]
00BB1F14  mov       ecx,dword ptr [ebp-30h]
00BB1F17  mov       edx,dword ptr [ecx+edx]
00BB1F1A  mov       esi,esp
00BB1F1C  mov       ecx,eax
00BB1F1E  mov       eax,dword ptr [edx]
00BB1F20  call      eax
00BB1F22  cmp       esi,esp
00BB1F24  call      __RTC_CheckEsp (0BB12A8h)
```

图 5.38　通过汇编代码中对虚函数的调用确定虚函数表首地址

```
        pdobj->f();
00BB1F01  mov       eax,dword ptr [ebp-30h]
00BB1F04  mov       ecx,dword ptr [eax]
00BB1F06  mov       edx,dword ptr [ecx+4]
00BB1F09  mov       eax,dword ptr [ebp-30h]
00BB1F0C  mov       ecx,dword ptr [eax]
00BB1F0E  mov       eax,dword ptr [ebp-30h]
00BB1F11  add       eax,dword ptr [ecx+4]
00BB1F14  mov       ecx,dword ptr [ebp-30h]
00BB1F17  mov       edx,dword ptr [ecx+edx]
00BB1F1A  mov       esi,esp
00BB1F1C  mov       ecx,eax
00BB1F1E  mov       eax,dword ptr [edx]
00BB1F20  call      eax     已用时间 <= 1ms
00BB1F22  cmp       esi,esp
00BB1F24  call      __RTC_CheckEsp (0BB12A8h)

    }
00BB1F29  mov       dword ptr [ebp-4],0FFFFFFFFh
100 %
自动窗口    寄存器    局部变量    内存 1    内
EAX = 00BB1168 EBX = 00279000 ECX = 00814FF8 EDX = 00BB
```

图 5.39　通过汇编代码中对虚函数的调用确定虚函数入口 f 的地址

在图 5.39 中，观察寄存器窗口中 EAX 的值，可以看到是 **00BB1168**，这个地址应该就是虚函数 f 的入口地址。

当然，如果在程序执行停留到断点行时直接将光标放在 pdobj 上并展开观察虚函数 f 的入口地址，则更简单，如图 5.40 所示。

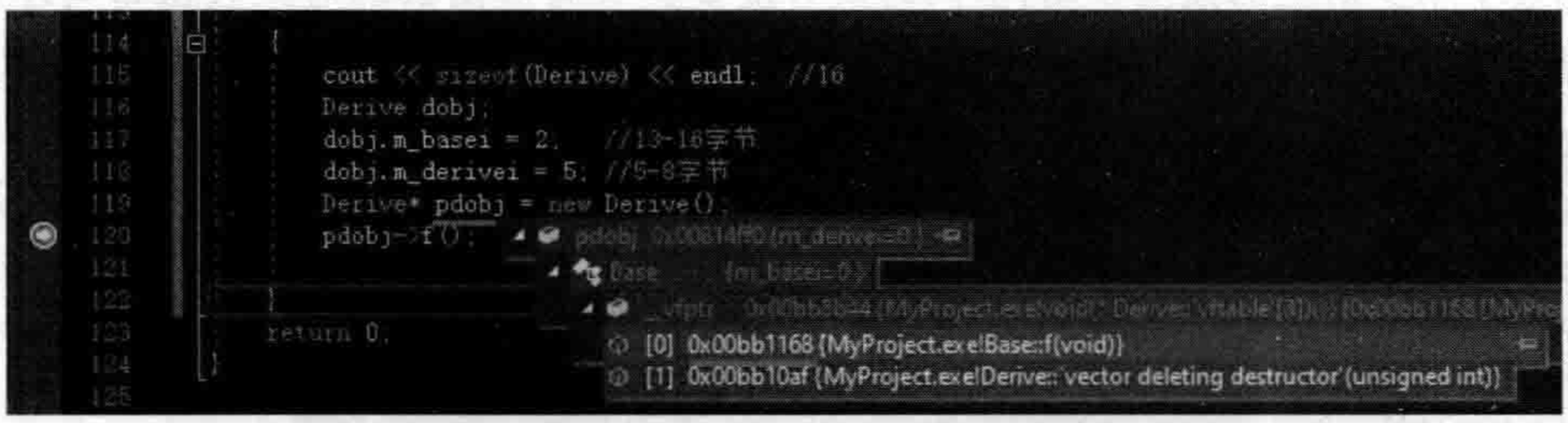

图 5.40　通过 new 一个 Derive 对象可以观察到其虚函数表中某个虚函数的入口地址

在图 5.40 基础之上继续观察，把 pdobj 这个对象指针所属的内存放到“内存 1”窗口中观察，如图 5.41 所示。

根据前面的测试，Derive 对象的 13～16 字节和 5～8 字节都已经确定了。尚不确定的就是 1～4 字节和 9～12 字节，那么哪个会是虚函数表指针所属的位置呢？观察图 5.41，

图 5.41　观察 pdobj 所指向的 Derive 对象内存中的内容

1～4 字节内容是 50 8b bb 00，9～12 字节内容是 44 8b bb 00，在内存窗口中分别观察这两个内存，哪个内存中有 00BB1168 字样，哪个就是虚函数表指针。

如图 5.42 所示。

图 5.42　通过观察内存内容确定虚函数表指针位置

在图 5.42 中可以看到，内存 0x00BB8B44 中有 68 11 bb 00 字样的地址，这说明 0x00BB8B44 是类 Derive 虚函数表的开始地址，因为 0x00BB8B44 对应的是 Derive 类对象的 9～12 字节，因此可以确定，Derive 类对象的 9～12 字节就是虚函数表指针。那么剩下的 1～4 字节就必然是 Derive 类对象的虚基类表指针，所以，Derive 类对象的数据布局应该如图 5.43 所示(图中 vptr 也可以写为 vfptr，都表示虚函数表指针)。

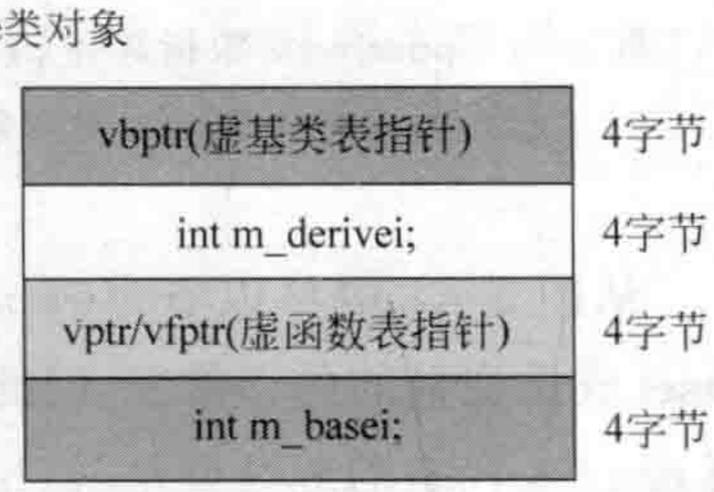

图 5.43　类对象指针 pdobj 所指的 Derive 类对象的数据布局

图 5.43 的布局还是挺反常的。因为有虚基类(虚继承)导致虚函数表指针跑到了数据布局的下面。观看整个图，Derive 类对象的起始地址在最上面从 vbptr 虚基类表指针开始，而 Base 类子对象的起始地址应该是从 vptr 虚函数表指针开始(不要忘记 Base 类里有虚函数)。所

以如果 Base 指针指向 Derive 类对象，则还是存在 this 指针调整问题。

考虑到编译器内部对同时具有虚基类和虚函数情形的对象数据布局支持比较复杂，所以这里做相对简单一些的研究。注释掉 main 主函数中原有代码，加入两行新代码：

```
Base * pbase = new Derive();
pbase->m_basei = 6;
```

笔者原计划将断点设置在 Base * pbase = new Derive();这行并分析相关的汇编代码，但是看到这行代码对应的汇编语句居然多达 30 行，代码行太长，不便理解和分析，于是放弃上面两行代码，重新在 main 主函数中加入下面新的代码行：

```
Derive* pderive = new Derive();
Base* pbase2 = (Base*)pderive;
pbase2->m_basei = 7;
```

将断点设置在 pbase2-> m_basei = 7;代码行并开始调试，当程序执行停到该断点行时，将鼠标分别放在 pderive 变量和 pbase2 变量上观察(这次不看汇编代码因为太过烦琐)。观察到的信息如下：

```
pderive = 0x00965238 {m_derivei = 0 }
pbase2 = 0x00965240 {m_derivei = 0 }
```

可以看到 pderive 和 pbase2 间距是 8 个字节，此时的情形可以在图 5.43 的基础上进一步标注一下，如图 5.44 所示。

此时，可以切换到反汇编窗口，观察代码行 pbase2-> m_basei = 7;对应的汇编代码如图 5.45 所示。

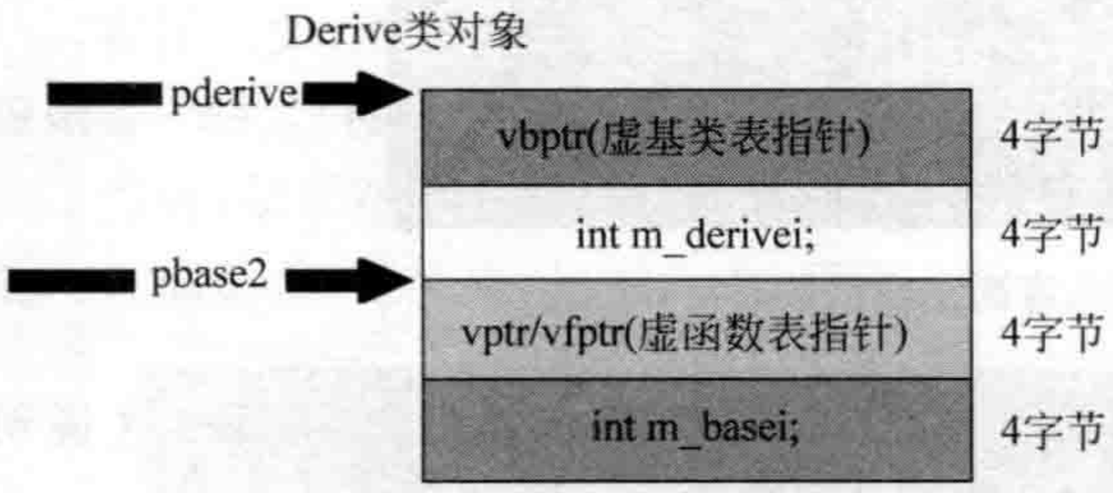

图 5.44 pderive 对象指针和 pbase2 对象指针所指向的 Derive 类对象位置示意

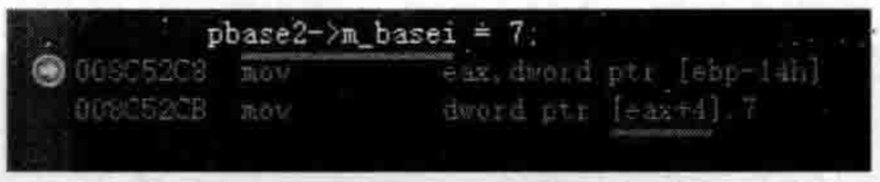

图 5.45 观察通过 pbase2 指针给 m_basei 成员变量赋值的汇编代码

从图 5.44 中可以看出，pbase2 指针往后跳转 4 个字节，所在的内存位置正好是 m_basei 成员变量的内存位置，因此图 5.45 中 eax 中保存的地址就是 pbase2 所指向的地址，而 eax+4 代表的地址就是 m_basei 成员变量的内存地址，符合图 5.44 所绘制的对象数据布局。

另外，很多资料已经指出：虚基类中如果有虚函数，同时也定义有普通成员变量，那么父类和子类之间的偏移调整，就好像进了迷宫深渊一样。所以笔者认为，掌握本节所讲述的知识已经足够，在没有特殊需求的情况下不必研究的太过深入。

5.8　RTTI(运行时类型识别)回顾与存储位置简介

5.8.1　RTTI(运行时类型识别)简单回顾

看看如下范例。在 MyProject.cpp 的上面,增加如下代码行,写一段传统的多态代码:

```
class Base
{
public:
    virtual void f() { cout << "Base::f()" << endl; }
    virtual void g() { cout << "Base::g()" << endl; }
    virtual void h() { cout << "Base::h()" << endl; }
    virtual ~Base() {}
};
class Derive : public Base {
public:
    virtual void g() { cout << "Derive::g()" << endl; }
    void myselffunc() {}                          //只属于 Derive 的函数
    virtual ~Derive() {}
};
```

在 main 主函数中,加入如下代码行:

```
Base * pb = new Derive();                         //基类指针指向一个子类对象
pb->g();

Derive myderive;
Base& yb = myderive;                              //基类引用引用的是一个子类对象
yb.g();
```

相信读者都知道 C++中运行时类型识别(RTTI),因为上面代码中的 pb 也好,yb 也好,并不知道代码运行的时候到底它们指向或者引用的是父类对象还是子类对象,如果想得到它们指向的对象信息,就需要借助 RTTI。有一点要注意,RTTI 要发挥作用,**父类中必须至少要有一个虚函数**,没有虚函数,就无法获得准确的 RTTI 信息。

有了 RTTI 手段,就可以在执行期间查询一个多态指针或者多态引用的信息。

RTTI 的能力通过 typeid 和 dynamic_cast 运算符都能体现。看看如下范例,在 main 主函数中继续增加如下代码:

```
cout << typeid( * pb).name() << endl;
cout << typeid(yb).name() << endl;
```

执行起来,看一看新加入的这两行代码的结果:

```
class Derive
class Derive
```

当然,也可以使用 dynamic_cast 来尝试转换类型,如果转换成功了,说明它指向的就是这个类型。在 main 主函数中继续增加如下代码:

```
Derive * pderive = dynamic_cast < Derive * >(pb);
if (pderive != NULL)
{
    cout << "pb 指向的实际是一个 Derive 类型" << endl;
    pderive -> myselffunc(); //调用属于子类自己的函数
}
```

5.8.2 RTTI 实现原理

C++引入 RTTI 的目的就是让程序在运行时能**根据基类**的指针或者引用来获得这个指针或者引用所指向的对象的实际类型。

刚刚回顾了 RTTI 的功能，演示了两个主要的运算符：typeid(获得类型名)和 dynamic_cast(做安全的类型转换)，现在探究一下 RTTI 的实现原理。

先说一说 typeid。根据上面的用法可以看到，它会返回一个**常量对象的引用**，这个常量对象的类型一般是标准库类型 type_info，type_info 其实就是一个类，里面记录着跟类型有关的信息。在 main 主函数中继续增加如下代码：

```
const std::type_info& tp = typeid( * pb);
```

这些与类型有关的信息，不同的平台可能会有所不同，但大同小异，如类的真实名字总是要有的。可以将光标定位到 type_info 中间，并按 F12 键跳入到 type_info 类的定义中去看一看，如图 5.46 所示(在 Visual Studio 2019 中这个类定义在系统文件 vcruntime_typeinfo.h 中)。

```
65  class type_info
66  {
67  public:
68
69      type_info(const type_info&) = delete;
70      type_info& operator=(const type_info&) = delete;
71
72      size_t hash_code() const noexcept
73      {
74          return __std_type_info_hash(&_Data);
75      }
76
77      bool operator==(const type_info& _Other) const noexcept
78      {
79          return __std_type_info_compare(&_Data, &_Other._Data) == 0;
80      }
81
82      bool operator!=(const type_info& _Other) const noexcept
83      {
84          return __std_type_info_compare(&_Data, &_Other._Data) != 0;
85      }
86
87      bool before(const type_info& _Other) const noexcept
88      {
89          return __std_type_info_compare(&_Data, &_Other._Data) < 0;
90      }
91
92      const char* name() const noexcept
93      {
94          #ifdef _M_CEE_PURE
95          return __std_type_info_name(&_Data, static_cast<__type_info_node*>(__type_info_root_node.ToPointer()));
96          #else
97          return __std_type_info_name(&_Data, &__type_info_root_node);
98          #endif
99      }
100
101     const char* raw_name() const noexcept
102     {
103         return _Data._DecoratedName;
104     }
```

图 5.46　type_info 类的定义

比较典型的接口如图 5.46 中的 name 已经演示过了，还有操作符“==”，用以判断所指向的类型是否相同。在 main 主函数中继续增加如下代码：

```
Base * pb2 = new Derive();
const std::type_info& tp2 = typeid( * pb2);
if (tp == tp2) //条件成立
{
    cout << "很好,类型相同" << endl;
}
```

注意看图 5.46 中 type_info 类的定义代码，可以看到其中有如下两行代码：

```
type_info(const type_info&) = delete;
type_info& operator = (const type_info&) = delete;
```

这说明该类把构造函数、拷贝赋值运算符给删除掉了，意味着对 type_info 这种对象无法进行复制，看起来只能通过现在的代码手段来读 type_info 对象的信息了。

typeid 有很多用法，里面可以包含很多参数。在 main 主函数中继续增加如下代码：

```
cout << typeid(int).name() << endl;
cout << typeid(Base).name() << endl;
cout << typeid(Derive).name() << endl;
Derive * pa3 = new Derive();
cout << typeid(pa3).name() << endl;
```

执行起来，看一看新加入的这几行代码的结果：

```
int
class Base
class Derive
class Derive *
```

当然，上面这几个类型应该说都属于静态类型，而不是多态类型。下面看一看多态类型。不过现在 main 主函数中的代码有点多，为演示方便，先全部注释掉，然后重新加入下面的代码行：

```
Base * pb = new Derive();
Derive myderive;
Base& yb = myderive;
cout << typeid( * pb).name() << endl;          //class Derive
cout << typeid(yb).name() << endl;             //class Derive
Base * pb2 = new Derive();
const std::type_info& tp2 = typeid( * pb2);
cout << tp2.name() << endl;
```

执行起来，看一看结果：

```
class Derive
class Derive
class Derive
```

可以做一个试验：只要把 Base 中的所有虚函数注释掉，再次执行程序，结果就不一样了。不难发现，三行输出结果都变成了 class Base。此刻，读者已经发现了 RTTI 测试无法识别出正确的类型，这说明一个问题，RTTI 的测试(检测)能力应该是与**基类中是否存在虚函数**有关，如果基类中没有虚函数，也就不存在基类的虚函数表，如果没有虚函数表，RTTI 就无法得到正确的结果(无虚函数不多态，也就是说，没有虚函数存在，就不会存在多态的概念。多态是与虚函数、虚函数表息息相关的)。

现在还是要确保基类中有虚函数，这样 typeid 才能够正确拿到类型信息，换句话说，typeid 运算符是在运行过程中**拿到其指向的对象的实际类型**。那么，问题的焦点在于：typeid 是如何在运行过程中拿到这个类型信息的？这个信息是保存在哪里的？

这里要强调一下，不同的编译器可能有不同的实现，这一点有时候是需要解读汇编代码来了解的。这里主要是针对 Visual Studio 2019 编译器来看一看 RTTI 信息的保存位置。

重点研究如下代码行，因为突破口正是在这行：

```
const std::type_info& tp2 = typeid( * pb2);
```

将断点设置在该行并开始调试，当程序执行停到该断点行时，切换到反汇编窗口，如图 5.47 所示。

```
        const std::type_info& tp2 = typeid(*pb2);
00F42C83  mov         eax,dword ptr [ebp-3Ch]
00F42C86  push        eax
00F42C87  call        __RTtypeid (0F414E2h)
00F42C8C  add         esp,4
00F42C8F  mov         dword ptr [ebp-48h],eax
        cout << tp2.name() << endl;
```

图 5.47　typeid 对应的汇编代码

图 5.47 中，重点关注 call __RTtypeid (0F414E2h)代码行，这个事情的重点应该就在__RTtypeid 上，也只有基类存在虚函数表这种情况时，用 typeid 才会出现调用__RTtypeid 这种代码。所以初步估计，用 typeid 获取对象运行时的类型识别信息应该是和虚函数表有直接关系的。

通过 F11 快捷键，跟踪进 call __RTtypeid (0F414E2h)代码行，似乎又需要多次按 F11 键进行跟进和跳转，整体的汇编代码有一定的复杂性，因为笔者不想把这么复杂的事情全盘讲解，那样会花费很多时间，得不偿失。所以笔者换一种方法介绍 RTTI 的保存位置。

笔者翻阅了一些资料，也写了一些代码进行测试核对，而后画了一张图，表示所找到的 type_info 结构信息的保存位置，如图 5.48 所示。

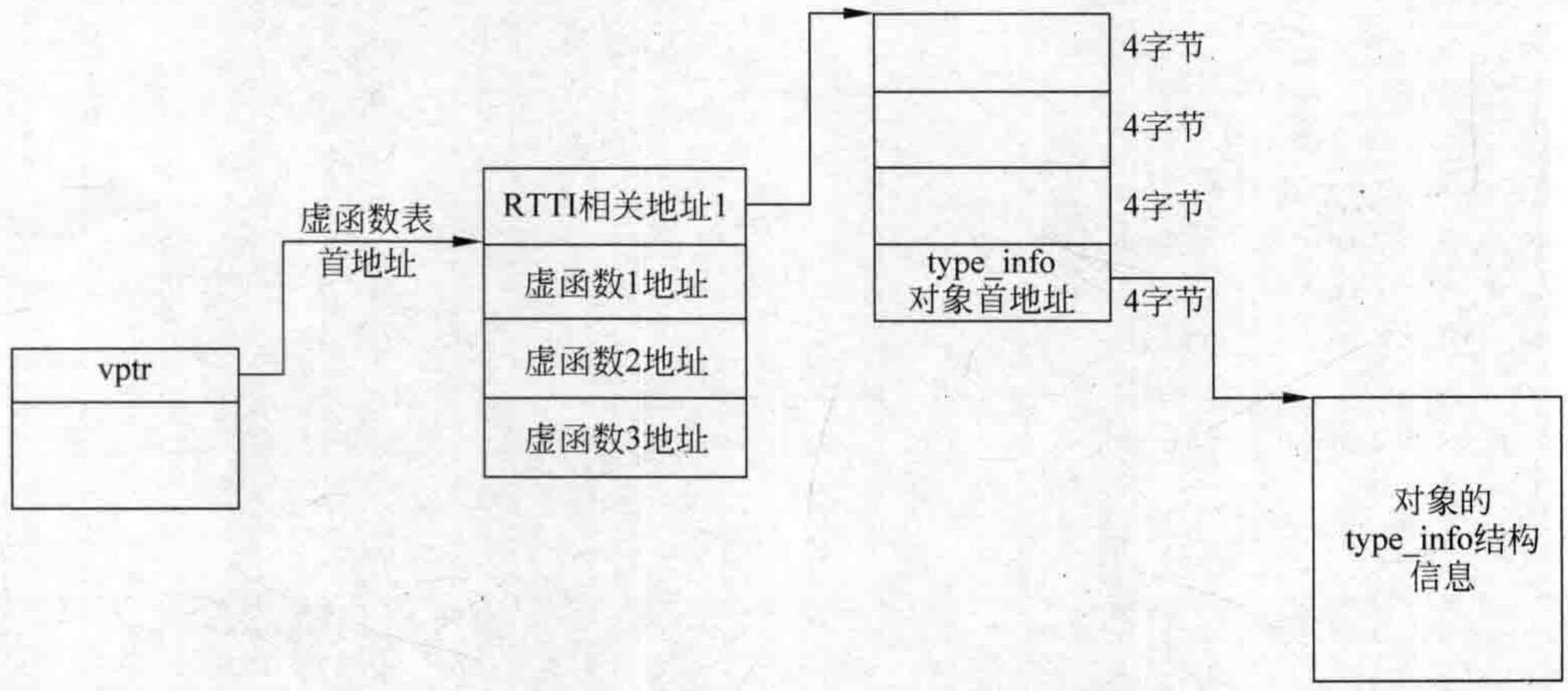

图 5.48　类的 type_info 结构信息保存位置示意图

根据图 5.48,可以写一个程序验证一下其是否是正确的。类似的程序其实前面写过,这里再书写一次。在 main 主函数中,继续输入如下代码:

```
printf("tp2 地址为 : %p\n", &tp2);
long* pvptr = (long*)pb2; //指向对象 pb2 的指针转成 long * 型。大家注意,目前 pb2 对象里只
                          //有虚函数表指针
long* vptr = (long*)(*pvptr);          //指向虚函数表(虚函数表首地址)

printf("虚函数表首地址为:%p\n", vptr);
printf("虚函数表首地址之前一个地址为:%p\n", vptr - 1); //注意,long 在当前这个环境是 4 字
                                                      //节,所以 vptr-1 其实得到的地址
                                                      //是往回走了 4 个地址

//取得这个地址中的内容
long* prttiiinfo = (long*)(*(vptr - 1));
prttiiinfo += 3;                              //跳过 12 个字节
long* ptypeinfoaddr = (long*)(*prttiiinfo);
const std::type_info* ptypeinfoaddrreal = (const std::type_info*)ptypeinfoaddr;
printf("ptypeinfoaddrreal 地址为 : %p\n", ptypeinfoaddrreal);
cout << ptypeinfoaddrreal->name() << endl;  //结果 class Derive
```

执行起来,看一看新加入的这几行代码的结果:

```
tp2 地址为 :007FD150
虚函数表首地址为:007FAB70
虚函数表首地址之前一个地址为:007FAB6C
ptypeinfoaddrreal 地址为 :007FD150
class Derive
```

分析一下上面这段代码可以看到,代码行 prttiiinfo += 3;用于跳过 12 个字节。为什么跳过 12 个字节?实际上这 12 字节是一个结构信息,这个结构信息的内容和编译器对 RTTI 的实现有关。不过这里笔者不准备详细讲解这个问题。另外还有多重继承,虚拟继承之类的不同的对象结构布局,type_info 也可能有各种情形,这个的探索就留给读者,如果读者有兴趣,可以按照笔者的分析方法进行更深入的分析。

回头想想,如果基类中没有虚函数,没有虚函数表,那么取得 RTTI 信息肯定就不是这种方法了,取出来的信息也就不叫运行时类型识别了。

例如如下代码行:

```
cout << typeid(int).name() << endl;          //输出 int
```

如果查看这行代码对应的反汇编代码,根本就不会找到图 5.47 中的"__RTtypeid"字样,所以这时 typeid 取得的类型就不是所谓的运行时类型识别了。

对 RTTI 这个话题有上面这些了解就可以了。

5.8.3　vptr、vtbl 与 RTTI 的 type_info 信息创建时机

在 3.5 节曾经说过 vptr、vtbl 的创建时机,回忆一下。

虚函数表指针 vptr 是跟着对象走的,对象创建出来了,才存在 vptr。对象什么时候创建出来?程序运行的时候创建出来。现在可以把这句话说的更完善一些:vptr 也是属于类

中的一个成员变量，它是编译器在编译的时候就插进去的，然后 vptr 的赋值是在程序运行起来后，程序员创建一个对象的时候，执行编译器插入到构造函数中的赋值代码时才给 vptr 赋值的，让其指向该类所属的虚函数表。

虚函数表 vtbl 是在编译期间就确定好的，跟着类走的。

RTTI 的 type_info 信息实际上也是编译期间就创建好的。程序员能够做的事情就是去读取这些信息。所以不难发现，不管生成多少个类对象，取得的 type_info 所指向的地址都相同。也就是说，这个信息也是基于类的。

总结一下：RTTI 信息的取得，一般都是以虚函数表之前的一个位置为突破口，就算是各家的编译器实现 RTTI 手段各不相同，但一般以这个位置为突破口这一点是比较统一的。有的编译器可能存储的 RTTI 信息比较直接，如图 5.49 所示。

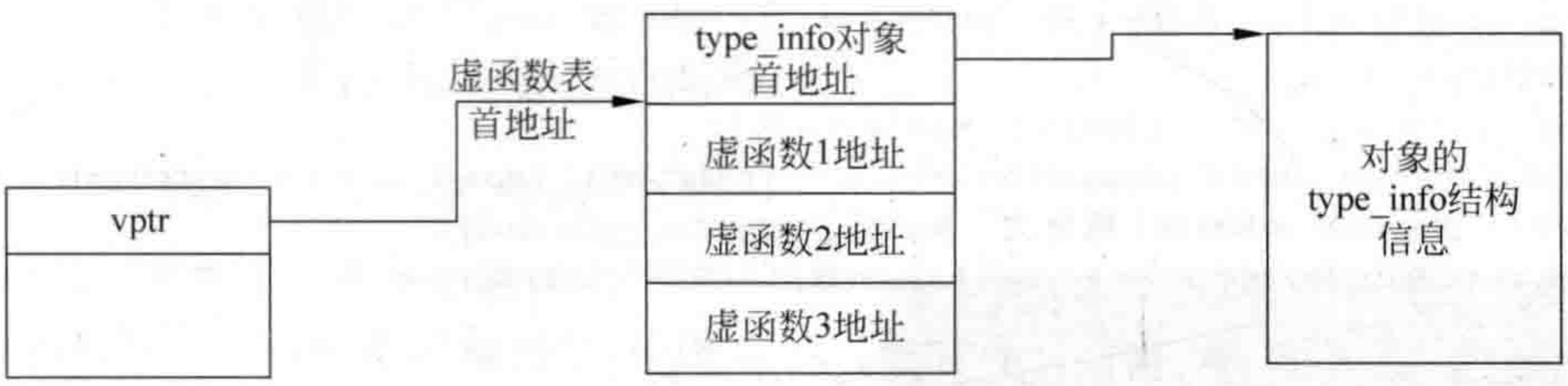

图 5.49　类的 type_info 结构信息保存的另一个位置示意图

5.9　函数调用与继承关系性能

5.9.1　函数调用中编译器的循环代码优化

编译器内部的实现非常复杂，而且各个编译器的开发商所开发的编译器也各不相同，有些编译器很智能，编译出来的程序执行起来很快，有些编译器不够智能，即便是相同的代码，编译出来的可执行程序，执行起来也可能不那么快。

举一个范例，看一看编译器在处理一些代码时的处理速度问题。这里就以 Visual Studio 2019 编译器为例，在 MyProject.cpp 的上面，增加如下代码行：

```
__int64 mytest(int mv)
{
    int i = 0;
    __int64 icout = 0;
    for (int i = 1; i <= 1000000; i++)
    {
        icout += 1;
        if (i == 10000 && mv == 999)
        {
            //printf("---\n");
        }
    }
    return icout;
}
```

在 main 主函数中，加入如下代码：

```
clock_t start, end;                                    //记得包含头文件 # include < ctime >
__ int64 mycount = 1;
start = clock();
for (int i = 1; i <= 1000; i++)
{
    mycount += mytest(6);  //参数如果是固定值,那么就很快,参数如果是一个 i 可变值就很慢
}
end = clock();
cout << "用时(毫秒):" << end - start << endl;
cout << "mycount = " << mycount << endl;
```

执行起来，看一看结果：

```
用时(毫秒):2695
mycount = 1000000001
```

出现上述的运行结果，大概需要 2s。

在 Visual Studio 2019(其他 Visual Studio 版本也类似)下，编译出来的可执行文件一般都分 Debug 版本和 Release 版本。在 Debug 版本，编译器会往里插入很多信息用于程序调试目的，而 Release 版本是发布版(一般用于商业环境中实际使用)，编译器不会插入调试信息，而且还会进行优化，执行效率一般比较高。

一般来讲，在 Visual Studio 2019 开发和执行程序时用的都是 Debug 版本，这样生成的可执行文件就是 Debug 版本的可执行文件，如果要切换成 Release 版本，可以在 Visual Studio 2019 的上面寻找一个下拉列表，如图 5.50 所示。

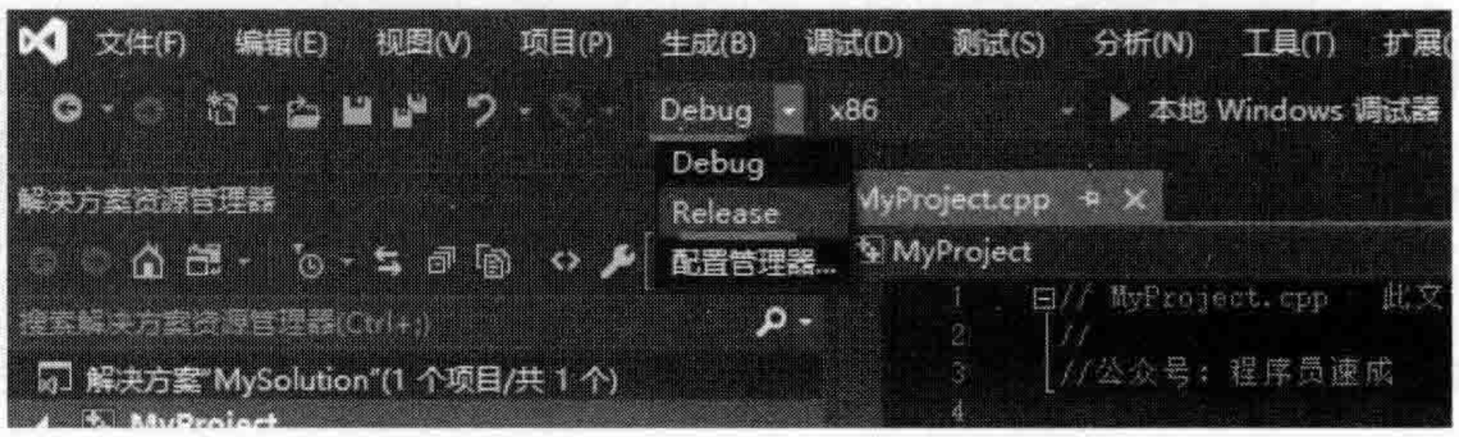

图 5.50　可执行程序的 Debug 和 Release 版本切换下拉列表

在图 5.50 所示的下拉列表中，选择 Release 选项即可。再次用 Ctrl＋F5 快捷键(“调试”→“开始执行(不调试)”命令)来执行程序，此时系统会编译、连接、生成 Release 版本的可执行程序并开始执行。看一看执行结果：

```
用时(毫秒):1
mycount = 1000000001
```

多执行几次会发现，用时要么是 0ms，要么是 1ms，用时非常短。试想一下，就算是运行再快，毕竟数亿次循环只用 1ms，也是不可能的。

观察一下代码在 main 主函数中，for 循环中的 mycount ＋＝ mytest(6);语句行，因为 mytest 里的参数是一个固定值 6，所以估计整个 for 循环在运行时有可能被编译器直接优

化成一条语句：

```
mycount += 循环 1000 次的和值;
```

而对于 mytest 这个函数，其中也有一个 for 循环，编译器也可能对其进行优化，编译器可能会把这种循环求和的语句整个优化成一条语句：

```
icout += 循环 1000000 次的累计值;
```

所以，通过观察感觉编译器很智能，它可能做了两件事：

(1) 优化循环，把循环优化成 1 条语句。

(2) 在编译期间，编译器也是具有运算能力的，有些运算编译器在编译期间就替程序员完成了。

现在修改一下 main 主函数中的代码行 mycount ＋＝ mytest(6);，把这个数字 6 变成变量 i：

```
mycount += mytest(i); //参数如果是固定值,那么就很快,参数如果是一个 i 可变值就很慢
```

执行起来，看一看结果：

```
用时(毫秒):323
mycount = 1000000001
```

通过结果可以看到，它的运行时间突然变成了 300 多毫秒。说明这个修改使编译器的优化能力失效了，编译器将这种**参数固定的函数调用**视为一种不变的表达式，编译器会**尝试做计算**，尝试做优化来取消循环。结果这一修改，参数发生改变，这种情况下编译器就没法做循环优化了。

将上面的代码恢复一下，将 mycount ＋＝ mytest(i);恢复回 mycount ＋＝ mytest(6);。

现在修改 mytest 函数，在其 for 循环中，增加几行代码。增加代码后的 mytest 函数如下：

```
__int64 mytest(int mv)
{
    int i = 0;
    __int64 icout = 0;
    for (int i = 1; i <= 1000000; i++)
    {
        icout += 1;
        if (i == 10000 && mv == 999)
        {
            printf(" --- \n");
        }
    }
    return icout;
}
```

执行起来，看一看结果：

```
用时(毫秒):382
mycount = 1000000001
```

从结果中可以看到,执行时间也显著上升,可以认为,增加了这行 printf 语句,导致编译器没法优化这个 for 循环,从而使执行时间迅速上升。

5.9.2　继承关系深度增加,开销也增加

很多情况下随着继承深度的增加,开销或者说执行时间也会增加,这比较好理解,看看如下范例。在 MyProject.cpp 的上面,增加如下代码行:

```
class A
{
public:
    A()
    {
        cout << "A::A()" << endl;
    }
};
class B :public A
{
public:
};
class C : public B
{
public:
    C()
    {
        cout << "C::C()" << endl;
    }
};
```

在 main 主函数中,加入如下代码行:

```
C c;
```

执行起来,看一看新加入的代码的结果:

```
A::A()
C::C()
```

通过前面章节知识的学习,读者想必已经非常清楚,因为类 A 有一个构造函数,类 C 的构造函数为了调用类 A 的构造函数,必须生成一个类 B 的构造函数,然后在类 B 的构造函数中增加调用类 A 构造函数的代码。

将断点设置在类 C 构造函数的函数体中,开始调试(注意将图 5.50 中的 Release 版本切换回 Debug 版本,否则调试可能不正常),当程序执行停到断点行时,切换到反汇编窗口,如图 5.51 所示。

```
public:
    C()
00B31FD0  push        ebp
00B31FD1  mov         ebp,esp
00B31FD3  sub         esp,0CCh
00B31FD9  push        ebx
00B31FDA  push        esi
00B31FDB  push        edi
00B31FDC  push        ecx
00B31FDD  lea         edi,[ebp-0CCh]
00B31FE3  mov         ecx,33h
00B31FE8  mov         eax,0CCCCCCCCh
00B31FED  rep stos    dword ptr es:[edi]
00B31FEF  pop         ecx
00B31FF0  mov         dword ptr [this],ecx
00B31FF3  mov         ecx,offset _F0B5FBBB_MyProject@cpp (0B3F026h)
00B31FF8  call        @__CheckForDebuggerJustMyCode@4 (0B3128Fh)
00B31FFD  mov         ecx,dword ptr [this]
00B32000  call        B::B (0B3112Ch)
    {
        cout << "C::C()" << endl;
00B32005  mov         esi,esp
```

图 5.51　编译器向类 C 的构造函数中插入调用类 B 构造函数的代码

在图 5.51 中可以看到，编译器向类 C 的构造函数中插入调用类 B 构造函数的代码，图中显示的 0B3112Ch 应该就是类 B 构造函数的地址，在反汇编窗口左上角输入这个地址(0x0B3112C)并按 Enter 键去看一下，如图 5.52 所示。

```
反汇编  MyProject.cpp
地址(A): 0x0B3112C
查看选项
_memset:
00B31127  jmp         _memset (0B35DF9h)
B::B:
00B3112C  jmp         B::B (0B31F70h)
std::basic_ostream<char,std::char_traits<char> >::sentry::sentry:
00B31131  jmp         std::basic_ostream<char,std::char_traits<char> >::sentry::~sentry (0B32420h)
```

图 5.52　观察编译器生成的类 B 构造函数中的代码

在图 5.52 中又显示了一个 jmp B::B(跳转)语句，后面圆括号中显示的地址是 0B31F70h，继续在反汇编窗口左上角输入这个地址(0x0B31F70)并按 Enter 键，如图 5.53 所示。

在图 5.53 中不难看到，类 B 的构造函数中插入了调用类 A 构造函数的代码，所以函数调用比较多，性能上必然要差一些，而且继承关系深度越深，开销就越大。

下面再看一看多重继承导致开销的增加。在 MyProject.cpp 的上面，增加 A1 类的定义：

```
class A1
{
public:
    A1()
    {
        cout << "A1::A1()" << endl;
    }
};
```

```
反汇编  MyProject.cpp
地址(A): 0x0B31F70
查看选项
00B31F6D  int         3
00B31F6E  int         3
00B31F6F  int         3
00B31F70  push        ebp
00B31F71  mov         ebp,esp
00B31F73  sub         esp,0CCh
00B31F79  push        ebx
00B31F7A  push        esi
00B31F7B  push        edi
00B31F7C  push        ecx
00B31F7D  lea         edi,[ebp-0CCh]
00B31F83  mov         ecx,33h
00B31F88  mov         eax,0CCCCCCCCh
00B31F8D  rep stos    dword ptr es:[edi]
00B31F8F  pop         ecx
00B31F90  mov         dword ptr [this],ecx
00B31F93  mov         ecx,dword ptr [this]
00B31F96  call        A::A (0B3119Ah)
00B31F9B  mov         eax,dword ptr [this]
00B31F9E  pop         edi
```

图 5.53　B 类的构造函数中插入了调用 A 类构造函数的代码

现在让类 B 同时继承自类 A 和 A1：

```
class B :public A, public A1
{
public:
};
```

执行起来，看一看结果：

```
A::A()
A1::A1()
C::C()
```

从结果可以看到，多重继承又会去调用第二个基类的构造函数，所以显然也是会增加运行开销的。

5.9.3　继承关系深度增加，虚函数导致的开销增加

为了方便演示，先把类 B 的多重继承去掉，恢复成类 B 只继承自类 A：

```
class B :public A
{
public:
};
```

读者已经知道，虚函数的存在会导致编译器产生虚函数表，在多态中调用虚函数时，是要通过虚函数表寻找并调用虚函数的，这肯定会增加调用开销。

再进一步思考一下，每个类对象会有一个虚函数表指针，在执行类的构造函数时，编译器会往这个构造函数中增加用于给虚函数表指针赋值的代码，所以每多一层继承关系，就会多执行一次对虚函数表指针赋值的代码。

例如，在类 A(不是类 A1，不要加错位置)中增加一个 public 修饰的虚函数：

```
public:
    virtual void myvirfunc() {}
```

在 main 主函数中，增加如下代码行：

```
C *pc = new C();
```

将断点设置在 C 构造函数所在行，开始调试，当程序执行停到断点行时，切换到反汇编窗口，如图 5.54 所示。

从图 5.54 中可以看到，C 类的构造函数中被插入了给虚函数表指针赋值的代码。此时，可以通过快捷键 F10 和 F11 往下执行，直接进入到 B::B 中去观察汇编代码，如图 5.55 所示。

从图 5.55 中可以看到，B 类的构造函数中被插入了给虚函数表指针赋值的代码。同时还看到了调用 A 类构造函数的代码。通过快捷键 F10 和 F11 往下执行，直接进入到 A::A 中去观察汇编代码，如图 5.56 所示。

从图 5.56 中可以看到，A 类的构造函数中被插入了给虚函数表指针赋值的代码。

```
class C : public B
{
public:
    C()
009F2030  push        ebp
009F2031  mov         ebp,esp
009F2033  sub         esp,0CCh
009F2039  push        ebx
009F203A  push        esi
009F203B  push        edi
009F203C  push        ecx
009F203D  lea         edi,[ebp-0CCh]
009F2043  mov         ecx,33h
009F2048  mov         eax,0CCCCCCCCh
009F204D  rep stos    dword ptr es:[edi]
009F204F  pop         ecx
009F2050  mov         dword ptr [this],ecx
009F2053  mov         ecx,offset _F0B5FBBB_MyProject@cpp (09FF026h)
009F2058  call        @__CheckForDebuggerJustMyCode@4 (09F12ADh)
009F205D  mov         ecx,dword ptr [this]
009F2060  call        B::B (09F112Ch)
009F2065  mov         eax,dword ptr [this]
009F2068  mov         dword ptr [eax],offset C::`vftable' (09F9B5Ch)
    {
        cout << "C::C()" << endl;
```

图 5.54　C类的构造函数中插入了给虚函数表指针赋值的代码

```
009F1FDD  lea         edi,[ebp-0CCh]
009F1FE3  mov         ecx,33h    已用时间<=1ms
009F1FE8  mov         eax,0CCCCCCCCh
009F1FED  rep stos    dword ptr es:[edi]
009F1FEF  pop         ecx
009F1FF0  mov         dword ptr [this],ecx
009F1FF3  mov         ecx,dword ptr [this]
009F1FF6  call        A::A (09F11A9h)
009F1FFB  mov         eax,dword ptr [this]
009F1FFE  mov         dword ptr [eax],offset B::`vftable' (09F0B50h)
009F2004  mov         eax,dword ptr [this]
009F2007  pop         edi
009F2008  pop         esi
009F2009  pop         ebx
```

图 5.55　B类的构造函数中插入了给虚函数表指针赋值的代码

```
009F1F39  push        ebx    已用时间<=1ms
009F1F3A  push        esi
009F1F3B  push        edi

class A
{
public:
    A()
009F1F3C  push        ecx
009F1F3D  lea         edi,[ebp-0CCh]
009F1F43  mov         ecx,33h
009F1F48  mov         eax,0CCCCCCCCh
009F1F4D  rep stos    dword ptr es:[edi]
009F1F4F  pop         ecx
009F1F50  mov         dword ptr [this],ecx
009F1F53  mov         ecx,offset _F0B5FBBB_MyProject@cpp (09FF026h)
009F1F58  call        @__CheckForDebuggerJustMyCode@4 (09F12ADh)
009F1F5D  mov         eax,dword ptr [this]
009F1F60  mov         dword ptr [eax],offset A::`vftable' (09F9B3Ch)
    {
        cout << "A::A()" << endl;
```

图 5.56　A类的构造函数中插入了给虚函数表指针赋值的代码

以上看到了在继承层次体系中，会多次执行这种给虚函数表指针 vptr 赋值的语句，继承的深度越深，开销越大。同样，也可以推断出，如果是多重继承，也一样会因为虚函数的引入进一步增加开销，因为又多了虚函数表，又多了虚函数表指针等。这些知识前面都学习过，在这里就不多谈了。

本节主要是想强调，函数的调用、有虚函数的存在、继承深度加深、存在多重继承等情况时，都会对程序的性能有影响。

5.10　指向成员函数的指针和 vcall

5.10.1　不用类对象能否调用类的虚函数和普通成员函数

在 3.2 节中手工调用了虚函数，正常来讲，虚函数的调用也需要 this 指针（this 指针将作为该虚函数的第一个隐藏的参数被传递进虚函数中去），但 3.2 节代码中调用虚函数并没有用到 this 指针，为什么也可以成功调用呢？

这个事情可以这样理解，从本质上来讲，**只要一个函数有地址，就可以直接去调用**（类的普通成员函数包括虚函数都是有独立地址的），如果这个函数需要一些参数，但是调用者调用该函数时没有传递进去这些参数，那么，只要这个函数的函数体代码中没有用到这些参数，那么执行这个函数一般就不会报告异常（或者即便报告异常也会发现这个函数的函数体还是能够被成功执行的），但一旦用到这些参数，程序执行可能就会报告异常。这就解释了 3.2 节中不需要 this 指针也能直接调用虚函数的原因。

对于一个普通的类成员函数，假如这个类成员函数没有任何参数，其实，不需要 this 指针也同样能够直接调用（调用的前提是函数体中不会用到 this 参数）。当然，这种调用是属于一种奇异的编程方法，主要目的是向读者阐述道理。可以写一段代码来不通过创建类对象就能够调用类的普通成员函数。在 MyProject.cpp 的上面，增加如下代码行：

```
//将一个成员函数的地址转换成普通地址
template<typename dst_type, typename src_type>
dst_type pointer_cast(src_type src)
{
    return * static_cast<dst_type * >(static_cast<void * >(&src));
}

class TDF                                          //定义 TDF 类
{
public:
    void myf(int val)
    {
        cout << "myf()成员函数执行了" << endl;
    }
};
```

上面 pointer_cast 这段函数模板代码比较有趣，是笔者通过网络搜集而来。在 main 主函数中，输入如下代码：

```
printf("TDF::myf 的地址是 %p\n", &TDF::myf);
typedef void ( * Func)();
Func fun1 = pointer_cast<Func>(&TDF::myf);
void * func2 = pointer_cast<void * >(&TDF::myf);
fun1();                                            //把成员函数当普通函数调用
```

执行起来，看一看结果：

```
TDF::myf 的地址是 0060142E
myf()成员函数执行了
```

可以看到，TDF 类的普通成员函数 myf 确实是可以直接调用。笔者在实验过程中，尝试给 myf 成员函数随便增加一个参数，发现 myf 函数依旧能够执行，但执行时程序会报告异常。

5.10.2 指向成员函数的指针

在 4.3 节讲解了成员变量的偏移值概念，同时也阐述了当进行成员变量存取时，需要用到类对象的首地址（当然，5.10.1 小节中讲的情况除外，这里指的是常规编程情况下）。

类中成员函数的地址是在编译的时候就确定好的，但是在调用成员函数的时候，是需要通过对象来调用成员函数的，因为成员函数的运行要依赖于一个 this 指针，这个 this 指针其实就是对象的首地址。请记住一个结论：所有这种常规的（非静态的）成员函数，要想调用，都需要一个对象来调用它。

在 MyProject.cpp 的上面，增加类 A 的代码如下：

```
class A
{
public:
    void myfunc1(int tempvalue1)
    {
        cout << "tempvalue1 = " << tempvalue1 << endl;
    }
    void myfunc2(int tempvalue2)
    {
        cout << "tempvalue2 = " << tempvalue2 << endl;
    }
};
```

在 main 主函数中加入如下代码：

```
A mya;
void (A:: * pmypoint)(int tempvalue) = &A::myfunc1; //定义一个成员函数指针并给初值
pmypoint = &A::myfunc2;                             //给成员函数指针赋值
//通过成员函数指针来调用成员函数
(mya. * pmypoint)(15);                              //要使用成员函数指针来调用成员函数，必须要对象介入
A * pmya = new A();
(pmya -> * pmypoint)(20);                           //用对象指针介入来使用成员函数指针调用成员函数
```

这种通过成员函数指针对成员函数的调用，编译器也是要向成员函数中插入当前对象指针作为第一个参数（this 指针），也就是如下：

```
//编译器看来，上面两行调用成员函数的语句转换成如下两行
pmypoint(&mya, 15);
pmypoint(pmya, 20);
```

反观静态成员函数，5.2.2 小节讲解了静态成员函数，当时说过，静态成员函数是跟着类走的，编译器不会向静态成员函数中插入 this 指针作为第一个参数。所以，如果要用函数指针调用静态成员函数，怎样做呢？

在类 A 中增加一个 public 修饰的静态成员函数：

```
public:
    static void mysfunc(int tempvalue)
    {
        cout << "A::mysfunc()静态成员函数执行了,tempvalue = " << tempvalue << endl;
    }
```

在 main 主函数中，加入如下代码：

```
void ( * pmyspoint)(int tempvalue) = &A::mysfunc; //一个普通的函数指针,而不是成员函数指针
pmyspoint(80); //从编译器角度看也是这个代码,不需要有个所谓的 this 指针
```

通过上面的分析可以看到，通过成员函数指针对常规的成员函数的调用成本和通过普通的函数指针来调用静态成员函数的成本差不多，只不过是多了一个参数而已。

5.10.3　指向虚成员函数的指针和 vcall

在类 A 中增加一个 public 修饰的虚函数：

```
public:
    virtual void myvirfunc(int tempvalue)
    {
        cout << "A::myvirfunc()虚成员函数执行了,tempvalue = " << tempvalue << endl;
    }
```

读者已经知道，虚函数也是有地址的，编译的时候就确定好的。

在 main 主函数中，加入如下代码：

```
void (A:: * pmyvirfunc)(int tempvalue) = &A::myvirfunc;
```

可以看到，需要用成员函数指针来获得虚函数的地址，因为虚函数的执行也需要 this 指针。

在 main 主函数中，加入如下代码：

```
A * pvaobj = new A;
pvaobj -> myvirfunc(190);
```

读者已经知道，上面代码中用对象指针来调用虚函数时，执行的路径是通过虚函数表指针来找到虚函数表，通过虚函数表来查找虚函数的地址并执行该虚函数。

继续在 main 主函数中加入代码，用成员函数指针来调用虚函数：

```
(pvaobj -> * pmyvirfunc)(190);
```

执行程序，一切正常，读者已经知道，pvaobj-> myvirfunc(190)；代码行的调用是通过虚函数表完成的，那么(pvaobj-> * pmyvirfunc)(190)；的调用是通过虚函数表完成的吗？实

际上，依旧是通过虚函数表完成的。

设置断点到(pvaobj->*pmyvirfunc)(190);代码行，开始调试，当程序执行停到断点行时，切换到反汇编窗口，如图 5.57 所示。

在图 5.57 中，[ebp-48h]就是对象指针 pvaobj，也就是对象的首地址。[ebp-3Ch]是成员函数指针 pmyvirfunc 所指向的地址。所以看起来是将对象的首地址放入到 ecx 寄存器，然后调用 pmyvirfunc 指向的虚函数，通过 F10 和 F11 快捷键跟入到 call dword ptr [ebp-3Ch]看一看，如图 5.58 所示。

```
        (pvaobj->*pmyvirfunc)(190);
001E65CB  mov         esi,esp
001E65CD  push        0BEh
001E65D2  mov         ecx,dword ptr [ebp-48h]
001E65D5  call        dword ptr [ebp-3Ch]
001E65D8  cmp         esi,esp
001E65DA  call        __RTC_CheckEsp (01E12DAh)
```

图 5.57　通过对象指针和成员函数指针来调用虚函数

```
反汇编 - MyProject.cpp
地址(A): [thunk]:A::`vcall'{0,{flat}}'0
查看选项
001E14A6  jmp         __CxxThrowException@8 (01E6159h)
A::mysfunc:
001E14AB  jmp         A::mysfunc (01E1E10h)
A::`vcall'{0}'
001E14B0  jmp         A::`vcall'{0}' (01E1E0Ch)    已用时间 <= 1ms
```

图 5.58　通过对象指针和成员函数指针来调用虚函数的汇编代码跟踪

在图 5.58 中看到了在 5.3 节讲解过的 vcall(也叫 vcall thunk)。把 vcall 看成一段代码，它能引导程序执行流程跳到真正的虚函数中去。其中的`vcall'{0}'表示虚函数表的第一项。

继续按 F11 键往里面跟踪，如图 5.59 所示。

```
--- 无源文件 ---
001E1E0C  mov         eax,dword ptr [ecx]    已用时间 <= 1ms
001E1E0E  jmp         dword ptr [eax]
```

图 5.59　通过对象指针和成员函数指针来调用虚函数的汇编代码跟踪

在图 5.59 中，将 ecx(对象首地址)开始的 4 个字节内容给了 eax，对象首地址开始的 4 个字节内容其实就是虚函数表指针所指向的虚函数表的首地址，现在 eax 寄存器中保存的就是虚函数表的首地址，继续执行 jmp 跳转。这里继续按 F11 键往里面跟踪，如图 5.60 所示。

```
A::myvirfunc:
001E14BA  jmp         A::myvirfunc (01E28C0h)    已用时间 <= 1ms
001E14BF  int         3
```

图 5.60　通过对象指针和成员函数指针来调用虚函数的汇编代码跟踪

再继续按 F11 键往里面跟踪，就会进入到真正的类 A 的 myvirfunc 虚函数中去执行了，如图 5.61 所示。

所以，对照 5.3 节提到的 vcall 的说法，这里进一步阐述一下 vcall。

vcall 是 virtual call 的缩写，翻译成中文是“虚调用”，它代表一段要执行的代码的首地址，这段代码引导程序的执行流程去执行正确的虚函数(以及进行 this 指针调整等)，或者可以简单地把 vcall 看成虚函数表，如果这样看待的话，那么，`vcall'{0}'代表的就是虚函数表里的第一个虚函数，`vcall'{4}'就代表虚函数表里的第二个虚函数。

```
public:
    virtual void myvirfunc(int tempvalue)
    {
001E28C0  push        ebp
001E28C1  mov         ebp,esp    已用时间<=1ms
001E28C3  sub         esp,0CCh
001E28C9  push        ebx
001E28CA  push        esi
001E28CB  push        edi
001E28CC  push        ecx
001E28CD  lea         edi,[ebp-0CCh]
001E28D3  mov         ecx,33h
001E28D8  mov         eax,0CCCCCCCCh
001E28DD  rep stos    dword ptr es:[edi]
001E28DF  pop         ecx
001E28E0  mov         dword ptr [this],ecx
001E28E3  mov         ecx,offset _F0B5FBBB_MyProject@cpp (01EF026h)
001E28E8  call        @__CheckForDebuggerJustMyCode@4 (01E12D0h)
        cout << "A::myvirfunc()虚成员函数执行了，tempvalue = " << tempvalue << endl;
001E28ED  mov         esi,esp
```

图 5.61　通过对象指针和成员函数指针来调用虚函数的汇编代码跟踪(进入真正的类 A 的虚函数中)

在类 A 原虚函数 myvirfunc **虚函数定义之前**，再增加一个 public 修饰的虚函数(**注意定义位置放在 myvirfunc 之前**)：

```
public:
    virtual void myvirfuncPrev(int tempvalue)
    {
        cout << "A::myvirfuncPrev()虚成员函数执行了,tempvalue = " << tempvalue << endl;
    }
```

断点行不变，依旧设置在(pvaobj->*pmyvirfunc)(190);代码行，开始调试，当程序执行停到断点行时，切换到反汇编窗口，然后再按 F11 键往里面跟踪，如图 5.62 所示。

在图 5.62 中看到的是`vcall'{4}'，而在图 5.58 中看到的是`vcall'{0}'。

继续在 main 主函数中增加如下代码行：

```
printf("%p\n", &A::myvirfunc);
```

取消原来的断点行，设置断点到新加的这个 printf 代码行，开始调试，当程序执行停到断点行时，切换到反汇编窗口，如图 5.63 所示。

```
A::`vcall'{4}'
009814C4  jmp         A::`vcall'{4}' (0981E8Bh)    已用时间<=1ms
009814C9  int         3
009814CA  int         3
009814CB  int         3
```

图 5.62　通过对象指针和成员函数指针来调用虚函数的汇编代码跟踪

```
        printf("%p\n", &A::myvirfunc);
00066660  push        offset A::`vcall'{4}' (0614C4h)
00066665  push        offset string "%p\n" (069C64h)
0006666A  call        _printf (0614C9h)
0006666F  add         esp,8
```

图 5.63　查看输出虚函数地址代码行所对应的汇编代码

通过这种比对和观察，可以更进一步地认识到，这种输出虚函数地址的方式用到了 vcall，输出来的并不是虚函数的真正地址。

所以这里可以完善一下理解，&A::myvirfunc 代表一个地址，这个地址中有一段代码，这个代码中记录的是该虚函数在虚函数表中的一个**偏移值**。有了偏移值，有了具体的对象指针，就能知道调用的是哪个虚函数表里的哪个虚函数了。

如果这个成员函数指针指向的是普通的成员函数呢？在 main 主函数中，继续增加如

下代码行：

```
pmyvirfunc = &A::myfunc2;
(pvaobj->*pmyvirfunc)(33);
```

取消原来的断点行，设置断点在(pvaobj->*pmyvirfunc)(33);代码行，开始调试，当程序执行停到断点行时，切换到反汇编窗口，如图 5.64 所示。

```
        pmyvirfunc = &A::myfunc2;
007B6672  mov         dword ptr [ebp-3Ch],offset A::myfunc2 (07B128Ah)
        (pvaobj->*pmyvirfunc)(33);
007B6679  mov         esi,esp
007B667B  push        21h
007B667D  mov         ecx,dword ptr [ebp-48h]
007B6680  call        dword ptr [ebp-3Ch]
007B6683  cmp         esi,esp
007B6685  call        __RTC_CheckEsp (07B12DAh)
```

图 5.64　通过对象指针和成员函数指针来调用普通成员函数

在图 5.64 中按 F10 键向下执行代码行，在 call dword ptr [ebp-3Ch]代码行按 F11 键跟进去，如图 5.65 所示。

```
A::myfunc2:
007B128A  jmp         A::myfunc2 (07B2030h)    已用时间<=1ms
```

图 5.65　通过对象指针和成员函数指针来调用普通成员函数

从图 5.65 中可以看到，调用普通的成员函数是直接调用的，不存在 vcall 的问题。

所以，在 Visual Studio 2019 中，成员函数指针里保存的可能是一个 vcall(vcall thunk)地址，也可能是一个真正的成员函数地址。如果是一个 vcall 地址，vcall 能够找出正确的虚函数表中的虚函数地址并进行调用。

5.10.4　vcall 在继承关系中的体现

现在引入类 B，作为类 A 的子类，继承关系既然出现，根据以往所说过的，无论父类还是子类，都应该手工增加虚析构函数，为类 A 增加 public 修饰的虚析构函数：

```
public:
    virtual ~A()
    {
    }
```

在 MyProject.cpp 的上面，类 A 定义的下面，增加类 B 的定义，继承自类 A。代码如下：

```
class B :public A
{
public:
    virtual void myvirfunc(int tempvalue)
    {
        cout << "B::myvirfunc()虚成员函数执行了,tempvalue = " << tempvalue << endl;
    }
    virtual ~B()
    {
    }
};
```

在 main 主函数中，加入如下代码：

```
B* pmyb = new B();
void (B:: *pmyvirfunc)(int tempvalue) = &A::myvirfunc; //成员函数指针
(pmyb->*pmyvirfunc)(190);
```

执行起来，看一看新加入的代码的结果：

```
B::myvirfunc()虚成员函数执行了,tempvalue = 190
```

成员函数指针 pmyvirfunc 给的是类 A 的 myvirfunc 虚函数的地址，执行的却是类 B 的 myvirfunc 虚函数，读者可能对此有点不太理解。

设置断点在(pmyb->* pmyvirfunc)(190);代码行，开始调试，当程序执行停到断点行时，切换到反汇编窗口，如图 5.66 所示。

按 F11 键跟踪进代码行 call dword ptr [ebp-6Ch]，如图 5.67 所示。

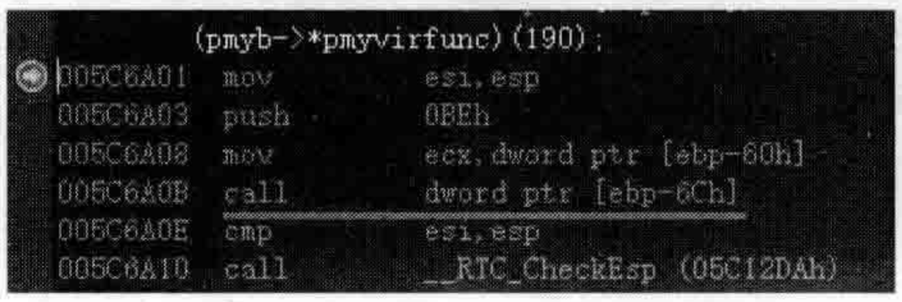

图 5.66　通过对象指针和成员函数指针来调用虚函数

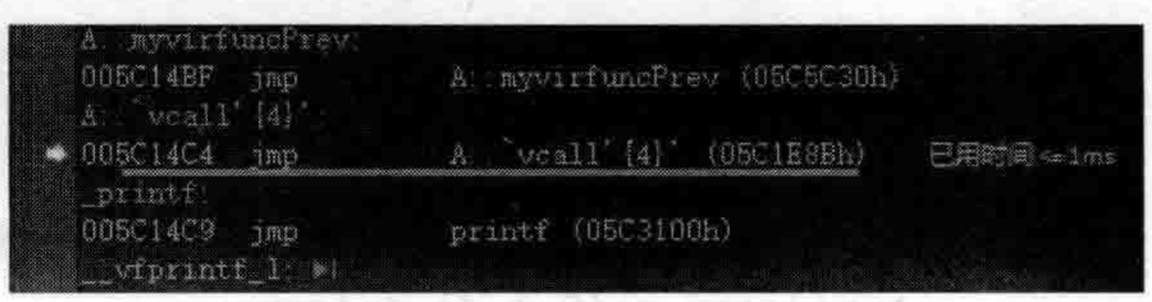

图 5.67　通过对象指针和成员函数指针来调用虚函数的汇编代码跟踪

继续按 F11 键跟踪进代码行 jmp A::`vcall'{4}' (05C1E8Bh)，如图 5.68 所示。

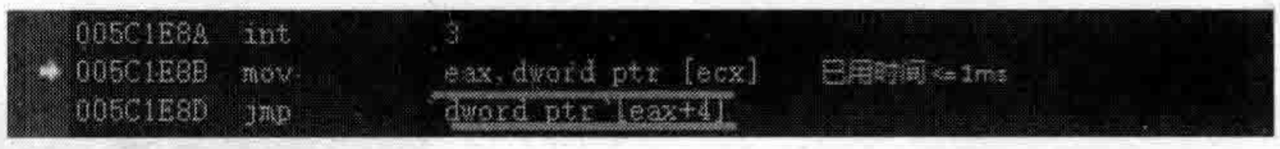

图 5.68　通过对象指针和成员函数指针来调用虚函数的汇编代码跟踪

继续按 F11 键跟踪进代码行 jmp dword ptr [eax+4]，就跳到了 **B::myvirfunc 虚函数**中去了。

成员函数指针给的是类 A 的 myvirfunc 虚函数，结果执行的是类 B 的 myvirfunc 虚函数，这证明了一点：&A::myvirfunc **隐藏着**一个偏移的信息，这个偏移信息与 &B::myvirfunc 隐含着的偏移信息相同(读者是否还记得，5.5.1 节说过：子类虚函数表项与父类虚函数表项排列顺序一样，偏移值实际也就是虚函数表项的排列顺序)。注意这里笔者说的偏移信息就是类似图 5.67 中 vcall 后面{}中的数字，如`vcall'{4}'中的 4 就是偏移值，一般`vcall'{0}'代表虚函数表的第一项，`vcall'{4}'代表虚函数表的第二项，以此类推。每一项 vcall 后面{}中的数字要+4。

也可以试一试将成员函数指针设置为类 B 的 myvirfunc 虚函数首地址并跟踪调试查看反汇编代码：

```
void (B:: * pmyvirfunc)(int tempvalue) = &B::myvirfunc;
```

最终调用的也是类 B 的 myvirfunc 虚函数。说明最终总能够调用到正确的虚函数(类 B 的 myvirfunc 虚函数)。

在 main 主函数中，增加如下两行代码：

```
printf("%p\n", &A::myvirfunc);
printf("%p\n", &B::myvirfunc);
```

执行起来，看一看这两行代码的结果：

```
003F14C4
003F150A
```

上面结果说明这两个 vcall thunk 地址值是不一样的。

5.11 inline 函数回顾和扩展

5.11.1 inline 函数回顾

看看如下范例。在 MyProject.cpp 的上面，增加如下代码行：

```
int myfunc(int testv)
{
    return testv * (5 + 4) * testv;
}
```

在 main 主函数中，加入如下代码：

```
int i = myfunc(12);
cout << i << endl;
```

设置断点到 myfunc 调用行，开始调试，当程序执行停到断点行时，切换到反汇编窗口，如图 5.69 所示。

```
        int i = myfunc(12);
00191918  push        0Ch
0019191A  call        myfunc (019102Dh)
0019191F  add         esp,4
00191922  mov         dword ptr [ebp-8],eax
        cout << i << endl;
```

图 5.69 直接调用一个普通的全局函数

在图 5.69 中可以看到，汇编代码中有一个正常的函数调用行——call myfunc (019102Dh)。

那么，如果把 myfunc 函数设置为 inline 函数会怎样呢？

```
inline int myfunc(int testv)
{
    return testv * (5 + 4) * testv;
}
```

再次调试，当程序执行停到断点行时，切换到反汇编窗口，看到依旧类似于图 5.69。看起来该函数并没有因为增加 inline 关键字而被展开。这是因为在 Visual Studio 2019 中要打开一个开关。

右击 Visual Studio 左侧的“解决方案资源管理器”中的 MyProject 工程名，在弹出的快捷菜单中选择“属性”命令，会弹出一个对话框，在对话框左侧单击“配置属性”→C/C++“优化”子选项，在右侧“内联函数扩展”里选择“只适用于__inline(/Ob1)”并单击“确定”按钮，如图 5.70 所示。

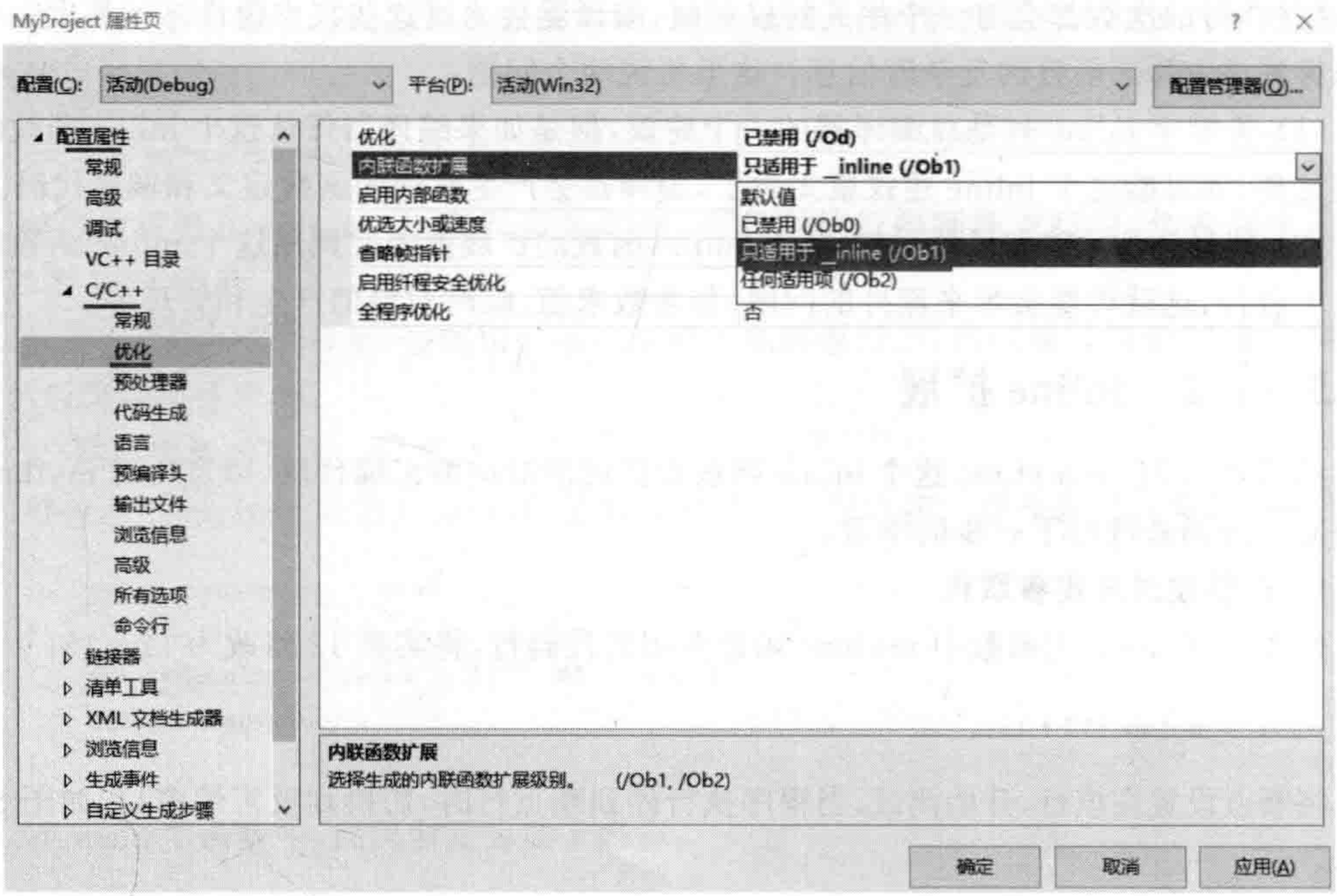

图 5.70　开启内联函数扩展选项

图 5.70 中的关于“内联函数扩展”用于选择生成的内联函数扩展级别，其中的几个选项各有用途，这里简单提一下，不做太细致的研究。

(1) /Ob0：禁用内联扩展。默认情况下，扩展由编译器自行对所有函数进行(通常称为自动内联)。

(2) /Ob1：仅允许对标记为 inline、__inline 或__forceinline 的函数或是在类声明中定义的 C++成员函数中进行扩展。

(3) /Ob2：默认值。允许对标记为 inline、__inline 或__forceinline 的函数或是编译器选择的任何其他函数进行扩展。

再次调试，当程序执行停到断点行时，切换到反汇编窗口，如图 5.71 所示。

```
        int i = myfunc(12);
00A42018  mov         eax,0Ch
00A4201D  imul        ecx,eax,9
00A42020  imul        edx,ecx,0Ch
00A42023  mov         dword ptr [ebp-8],edx
        cout << i << endl;
```

图 5.71　inline 函数被展开

在图 5.71 中可以看到，对 myfunc 函数的调用就被展开了(与图 5.69 进行比较)。一般来讲，看 inline 函数是否被展开，切换到汇编窗口查看是比较稳妥的做法。

大家都知道，inline 这个关键字只是程序员对编译器的一个建议，编译器可以尝试去做，也可以不去做，这取决于编译器的诊断功能，也就是说决定权在编译器，程序员控制不了。

inline 函数既有优点又有缺点，优点是它的执行成本一般比常规的函数调用和函数返回所带来的成本低，提高了程序执行效率，但缺点是会导致代码的膨胀。

当使用了 inline 关键字后，编译器会有比较复杂的测试算法来评估该 inline 函数的复杂度，这个测试算法可能会统计该 inline 函数中如赋值操作、各种函数调用操作等执行的次

数，这些执行的次数都会有一个相关的权重值，编译器会通过这些权重值最终计算出一个和值来得到该 inline 函数的复杂度信息。这里要说两个问题：

（1）关键字 inline 只是对编译器的一个建议，但是如果编译器评估这个 inline 函数的复杂度过高，那可能这个 inline 建议就无效了，编译器会产生常规的函数定义和调用代码。

（2）如果 inline 被编译器采纳，那么 inline 函数的扩展就要在调用这个 inline 函数的那个点上进行，这就可能会带来额外的问题，如参数求值、临时对象的产生和管理等。

5.11.2 inline 扩展

参考图 5.71 中 myfunc 这个 inline 函数被扩展后对应的汇编代码，以及比对 myfunc 函数的 C++源码来进行下一步的学习。

1. 形参被对应实参取代

修改一下 main 主函数中 myfunc 函数调用的代码行，将实参 12 修改为 12＋15：

```
int i = myfunc(12 + 15);
```

将断点设置在该行，开始调试，当程序执行停到断点行时，切换到反汇编窗口，如图 5.72 所示。

在图 5.72 中可以看到，如果实参是一个表达式(12＋15)，编译器会先求值(27 ＝ 0x1Bh)，再替换掉形参。

如果将 main 主函数中的代码再次修改如下，这次 myfunc 函数的实参表达式中将带进来一个变量：

```
int a = 80;
int i = myfunc(a + 15);
```

将断点设置在 myfunc 调用行，开始调试，当程序执行停到断点行时，切换到反汇编窗口，如图 5.73 所示。

```
        int i = myfunc(12+15);
00CD2018  mov         eax,1Bh
00CD201D  imul        ecx,eax,9
00CD2020  imul        edx,ecx,1Bh
00CD2023  mov         dword ptr [ebp-8],edx
        cout << i << endl;
```

图 5.72　inline 函数被展开时会先对实参表达式求值

```
        int a = 80;
003518B8  mov         dword ptr [ebp-8],50h
        int i = myfunc(a + 15);
003518BF  mov         eax,dword ptr [ebp-8]
003518C2  add         eax,0Fh
003518C5  mov         dword ptr [ebp-20h],eax
003518C8  imul        ecx,dword ptr [ebp-20h],9
003518CC  imul        ecx,dword ptr [ebp-20h]
003518D0  mov         dword ptr [ebp-14h],ecx

        cout << i << endl;
```

图 5.73　inline 函数被展开时会先对实参表达式求值

从图 5.73 中可以看到，先计算 a 和 15 的和值(add eax,0Fh)，然后再替换掉形参。

如果在 MyProject.cpp 的上面引入一个全局函数 rtnvalue，代码如下：

```
int rtnvalue()
{
    return 5;
}
```

将 main 主函数中的代码再次修改如下，这次 myfunc 函数的实参表达式中将使用上面函数调用的返回值：

```
int i = myfunc(rtnvalue() + 15);
```

将断点设置在 myfunc 调用行，开始调试，当程序执行停到断点行时，切换到反汇编窗口，如图 5.74 所示。

从图 5.74 中可以看到，先调用 rtnvalue 函数拿到返回值，用该返回值和 15 做运算求和值，然后再替换掉形参。

2. 局部变量的引入

修改一下 myfunc 函数的内容，向函数体中增加一些内容。修改后的完整代码如下：

```
inline int myfunc(int testv)
{
    int tempvalue = testv * (5 + 4) * testv;
    return tempvalue;
}
```

在 main 主函数中，代码恢复为如下内容：

```
int i = myfunc(12);
cout << i << endl;
```

将断点设置在 myfunc 调用行，开始调试，当程序执行停到断点行时，切换到反汇编窗口，如图 5.75 所示。

```
        int i = myfunc(rtnvalue() + 15);
008B3E28  call        rtnvalue (08B1398h)
008B3E2D  add         eax,0Fh
008B3E30  mov         dword ptr [ebp-14h],eax
008B3E33  imul        eax,dword ptr [ebp-14h],9
008B3E37  imul        eax,dword ptr [ebp-14h]
008B3E3B  mov         dword ptr [ebp-8],eax
        cout << i << endl;
```

图 5.74　inline 函数被展开时会先对实参表达式求值

```
        int i = myfunc(12);
00293E28  mov         eax,0Ch
00293E2D  imul        ecx,eax,9
00293E30  imul        edx,ecx,0Ch
00293E33  mov         dword ptr [ebp-14h],edx
00293E36  mov         eax,dword ptr [ebp-14h]
00293E39  mov         dword ptr [ebp-8],eax
        cout << i << endl;
```

图 5.75　inline 函数被展开

观察图 5.75，与图 5.71 进行对比，可以看到，多了两条 mov 汇编代码的操作，把结果值（位于 edx 寄存器中）扔到一个临时地址[ebp-14h]中，这个临时地址代表变量 tempvalue。然后这个临时变量值（结果值）扔回到 eax 寄存器中。所以这也是消耗，多了一个局部变量参与计算。因此，这种局部变量能少用就少用。

3. inline 失败的情形

随着编译器的不断演化和升级改进，inline 能处理的情形也越来越多，inline 成功的机率也越来越高，但有些情况仍然会导致 inline 失败。例如，如果 inline 函数是一个递归调用的函数，inline 可能就会失败，修改 myfunc。修改后的代码如下：

```
inline int myfunc(int testv)
{
    if (testv > 10)
    {
```

```
        testv--;
        myfunc(testv);
    }
    return testv;
}
```

将断点设置在 myfunc 调用行，开始调试，当程序执行停到断点行时，切换到反汇编窗口，如图 5.76 所示。

```
        int i = myfunc(12);
00FA1998  mov         dword ptr [ebp-14h],0Ch
00FA199F  cmp         dword ptr [ebp-14h],0Ah
00FA19A3  jle         main+4Ah (0FA19BAh)
00FA19A5  mov         eax,dword ptr [ebp-14h]
00FA19A8  sub         eax,1
00FA19AB  mov         dword ptr [ebp-14h],eax
00FA19AE  mov         ecx,dword ptr [ebp-14h]
00FA19B1  push        ecx
00FA19B2  call        myfunc (0FA102Dh)
00FA19B7  add         esp,4
00FA19BA  mov         edx,dword ptr [ebp-14h]
00FA19BD  mov         dword ptr [ebp-8],edx
        cout << i << endl;
```

图 5.76　inline 函数因为包含递归调用导致 inline 失败

在图 5.76 中看到了对函数 myfunc 的调用行。这说明 inline 并没有使 myfunc 展开。

可能还有其他情形也会导致 inline 失败，不同的编译器处理结果也不同，在书写 inline 函数时把握的原则就是：代码行尽量少，逻辑尽量简单。这样的函数 inline 的成功率就会比较高，但是到底 inline 是否成功，笔者认为还是看看对应的汇编代码才能够比较确定。

第6章

对象构造语义学

本章对对象的构造、析构进行了细致的讲解，尤其是在有类继承层次的情况，对于父类子对象的构造和析构顺序进行了重点的说明和强调。

对象的复制和析构行为，也是本章的讨论重点。

此外，局部和全局对象、局部静态对象、临时对象等的构造和析构过程，本章都有详细阐述。

6.1 继承体系下的对象构造

6.1.1 对象的构造顺序

对象的构造，简而言之是：从父类到子类，从根源到末端。后面会详细解释。

在进行范例演示之前，请先将图5.70设置的内联函数扩展设置回“默认值”，以免影响后续的代码调试和观察。

在MyProject.cpp的上面，增加如下代码行：

```
class A
{
public:
    A()
    {
        printf("A::A()执行了,this =  %p\n",this);
    };
    virtual ~A() {}
};
class B : public A
{
public:
    B()
    {
        printf("B::B()执行了,this =  %p\n",this);
    };
    virtual ~B() {}
};
class C :public B
{
public:
```

```
    C()
    {
        printf("C::C()执行了,this =  %p\n",this);
    };
    virtual ~C() {}
};
```

在 main 主函数中，加入如下代码行：

```
C c;
```

执行起来，看一看结果：

```
A::A()执行了,this = 0075FE8C
B::B()执行了,this = 0075FE8C
C::C()执行了,this = 0075FE8C
```

将断点设置在 C c;代码行，开始调试，当程序执行停到断点行时，切换到反汇编窗口，如图 6.1 所示。

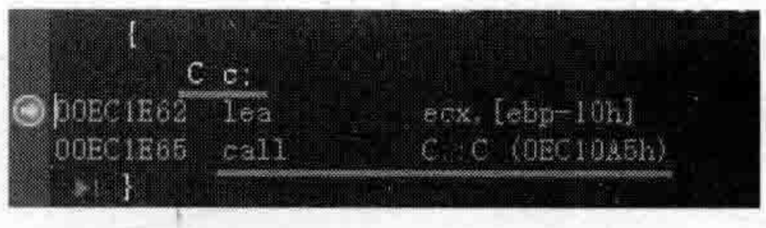

图 6.1 创建一个类对象时观察对构造函数调用的反汇编代码

从图 6.1 中不难看出，对于构造一个 C 类对象，首先调用的是 C 类的构造函数 C::C。按 F11 键跟进 C::C 构造函数，并按 F10 键逐行向下执行，如图 6.2 所示。

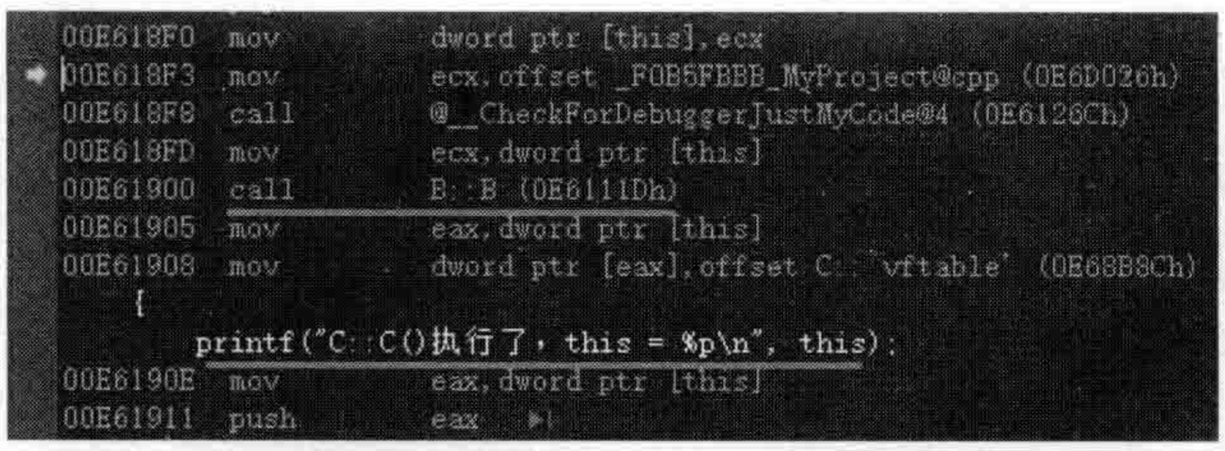

图 6.2 在执行对象的构造函数体之前会先执行父类的构造函数

观察图 6.2，构造函数 C::C 被编译器插入了很多代码，所以，在其执行的过程中，是**先去执行构造函数 B::B**，然后才**执行程序员所写的构造函数 C::C 的函数体**代码。这一点务必要搞清楚执行顺序。

按 F11 键跟进 B::B 构造函数，并按 F10 键逐行向下执行，如图 6.3 所示。

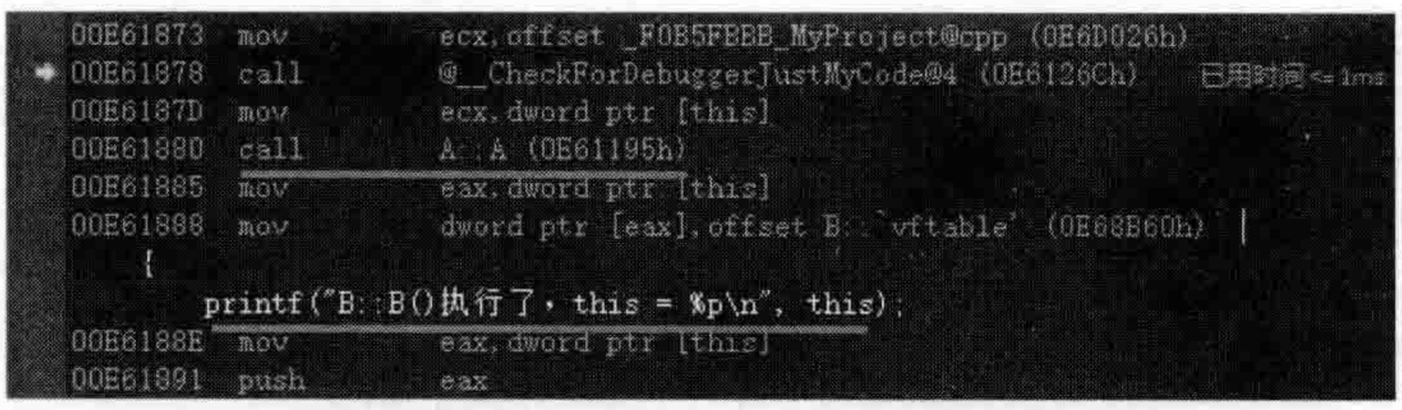

图 6.3 在执行对象的构造函数体之前会先执行父类的构造函数

观察图 6.3，构造函数 B::B 同样被插入了很多代码，所以，在其执行的过程中，是**先去执行构造函数 A::A**，然后才执行程序员所写的构造函数 B::B 的函数体代码。

所以，最终函数的调用关系应该如下(被调用的语句行以缩进形式表示)：

```
C::C()
    B::B()
        A::A()
            printf("A::A()执行了,this =  %p\n",this);
        printf("B::B()执行了,this =  %p\n", this);
    printf("C::C()执行了,this =  %p\n", this);
```

这个调用关系和程序执行得到的结果一致。现在为 C 类增加一个 public 修饰的成员变量：

```
public:
    int m_c;
```

然后为 C 类的构造函数增加初始化列表。现在 C 类的构造函数的代码如下：

```
public:
    C() :m_c(11)
    {
        printf("C::C()执行了,this =  %p\n", this);
    };
```

可以把断点行设置在 C 类内的 C() :m_c(11)代码行(去掉其他断点行)，开始调试，当程序执行停到断点行时，切换到反汇编窗口，如图 6.4 所示。

```
class C :public B
{
public:
    C() :m_c(11)
006B190E  mov         eax,dword ptr [this]
006B1911  mov         dword ptr [eax+4],0Bh
    printf("C::C()执行了，this = %p\n", this);
006B1918  mov         eax,dword ptr [this]
```

图 6.4　注意观察构造函数初始化列表中的代码行在哪里运行

在图 6.4 中，注意观察构造函数初始化列表中的 m_c(11)所在的执行位置，对应的汇编代码是 mov dword ptr [eax+4],0Bh。

此刻，最终函数的调用关系应该如下：

```
C::C()
    B::B()
        A::A()
            printf("A::A()执行了,this =  %p\n",this);
        printf("B::B()执行了,this =  %p\n", this);
    m_c = 1; //成员初始化列表
    printf("C::C()执行了,this =  %p\n", this);
```

从调用关系不难看出，先执行 B 类构造函数的函数体，再执行 A 类构造函数的函数体，再执行 C 类的初始化列表中的代码，最后执行 C 类构造函数的函数体。

可以看到，对象 c 的构造的顺序是：从父类到子类，从根源到末端。父类的构造函数最先执行，所以要认识到，**父类构造函数执行的时候，子类的实体还没有被构造出来**，也就是说，在 C 类构造函数的函数体代码执行完毕之前，这个 C 类对象还不是一个完整的对象，当执行完其中的 A 类构造函数时，只能说这个 A 类子对象构造完毕，然后 B 类构造函数执行完毕，代表着 B 类子对象构造完毕，最终 C 类构造函数执行完毕，**整个 C 类对象构造完毕**，

因为 C 类对象是包含 A 类子对象和 B 类子对象的。

6.1.2 虚函数

根据前面程序执行的结果，三个 this 指针值输出的结果都相同。C 对象中包含着 B 子对象，B 子对象中又包含着 A 子对象。

另外，类 A、B、C 都包含虚析构函数，那么，在各个类的构造函数中，一定有给虚函数表指针赋值的代码被编译器插入进来，看一看它们在哪里。

再次将断点设置在 main 主函数中的 C c;代码行，开始调试，当程序执行停到断点行时，切换到反汇编窗口，按 F11 键跟踪进去，一直跟踪到 A::A 构造函数中，如图 6.5 所示。

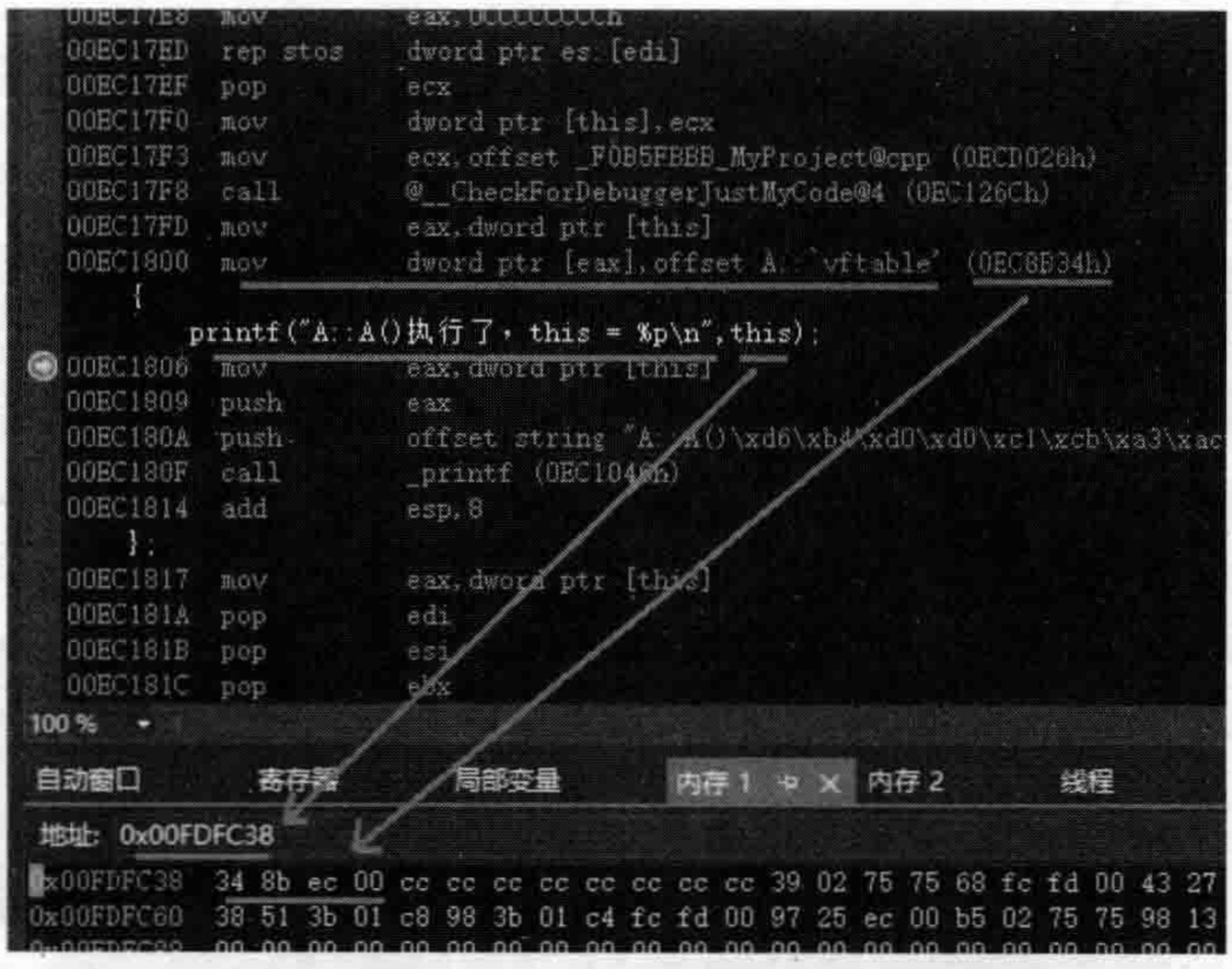

图 6.5 A::A 构造函数的反汇编代码

在图 6.5 中，首先看到的是**给虚函数表指针赋值**的语句（这是编译器插入的语句，注意**插入的位置是在 A::A 构造函数的函数体代码之前**）：

```
mov dword ptr [eax],offset A::`vftable' (0EC8B34h)
```

上面这个赋值其实比较有意思，因为定义的是 C 类对象，所以该对象的虚函数表指针肯定是要指向 C 类的虚函数表，这里赋值后指向的却是 A 类的虚函数表。类似于如下：

```
vptr = A::vftable;
```

不过并没有关系，后续还会有给 vptr 重新赋值的语句。

接着执行了程序员书写的 printf 语句，输出了 this 指针：

```
printf("A::A()执行了,this = %p\n",this);
```

前面说过，执行完这条 printf 语句时，**能拿到 this 地址**了，但这个时候对象还没构造完，**只是构造出了 A 类子对象**。

继续向下跟踪汇编代码，从 A::A 构造函数返回，就回到了 B::B 构造函数中，如图 6.6 所示。

```
00EC187D  mov         ecx,dword ptr [this]
00EC1880  call        A::A (0EC1195h)
00EC1885  mov         eax,dword ptr [this]    已用时间<=1ms
00EC1888  mov         dword ptr [eax],offset B::`vftable' (0EC8B60h)
    {
        printf("B::B()执行了，this = %p\n", this);
00EC188E  mov         eax,dword ptr [this]
00EC1891  push        eax
00EC1892  push        offset string "B::B()\xd6\xb4\xd0\xd0\xc1\xcb\xa3\xacthis = %p\n" (0EC8B68h)
00EC1897  call        _printf (0EC1046h)
00EC189C  add         esp,8
```

图 6.6　B::B 构造函数的反汇编代码

在图 6.6 中，虚函数表指针再一次被赋值了：

```
mov dword ptr [eax],offset B::`vftable' (0EC8B60h)
```

这次赋值就把上次的 vptr 值覆盖了，但遗憾的是，这次赋值是让 vptr 指向 B 类的虚函数表。类似于如下：

```
vptr = B::vftable;
```

显然这也不对，没有关系，后续还会有给 vptr 重新赋值的语句。

接着执行了程序员书写的 printf 语句，输出了 this 指针：

```
printf("B::B()执行了，this = %p\n", this);
```

这个时候对象还是没构造完，**这里又构造出了 B 类子对象**。

继续向下跟踪汇编代码，从 B::B 构造函数返回，就回到了 C::C 构造函数中，如图 6.7 所示。

```
00EC18FD  mov         ecx,dword ptr [this]
00EC1900  call        B::B (0EC111Dh)
00EC1905  mov         eax,dword ptr [this]
00EC1908  mov         dword ptr [eax],offset C::`vftable' (0EC8B8Ch)
};
class C :public B
{
public:
    C() :m_c(11)
00EC190E  mov         eax,dword ptr [this]    已用时间<=1ms
00EC1911  mov         dword ptr [eax+4],0Bh
        printf("C::C()执行了，this = %p\n", this);
00EC1918  mov         eax,dword ptr [this]
00EC191B  push        eax
```

图 6.7　C::C 构造函数的反汇编代码

在图 6.7 中，虚函数表指针第三次被赋值：

```
mov dword ptr [eax],offset C::`vftable' (0EC8B8Ch)
```

这次赋值就把上次的 vptr 值覆盖了，同时，这次对 vptr 的赋值也是 vptr 的最终值。类似于如下：

```
vptr = C::vftable;
```

接着执行了程序员书写的 printf 语句，输出了 this 指针：

```
printf("C::C()执行了，this = %p\n", this);
```

此刻，最终函数的调用关系应该如下：

```
C::C()
    B::B()
        A::A()
            vptr = A::vftable;
            printf("A::A()执行了,this = %p\n",this);
        vptr = B::vftable;
        printf("B::B()执行了,this = %p\n", this);
    vptr = C::vftable;          //给虚函数表指针赋予的最终值
    m_c = 1;                    //成员初始化列表
    printf("C::C()执行了,this = %p\n", this);
```

上面这种调用顺序就能够保证，构造出来的对象的虚函数表指针 vptr 最终指向的是类 C 的虚函数表。所以，千万不要在构造函数内程序员自己的代码中使用诸如 memcpy 或者直接操作内存等手段，去覆盖虚函数表指针这 4 个字节的内存内容，不然程序通过虚函数表来调用虚函数时就可能造成程序运行崩溃。

6.1.3 构造函数中对虚函数的调用

如果在构造函数中调用虚函数，会是什么样的情形呢？

改造一下程序。在类 A 中，加入如下 public 修饰的虚函数：

```
public:
    virtual void myvirfunc()
    {
        printf("A::myvirfunc()执行了!\n");
    };
```

在类 B 中，也加入类似的虚函数，只是 printf 的内容发生了变化：

```
public:
    virtual void myvirfunc()
    {
        printf("B::myvirfunc()执行了!\n");
    };
```

在类 C 中，也加入类似的虚函数，只是 printf 的内容发生了变化：

```
public:
    virtual void myvirfunc()
    {
        printf("C::myvirfunc()执行了!\n");
    };
```

在 main 主函数中，注释掉原有的代码，加入如下新代码：

```
C* mycobj = new C();
mycobj->myvirfunc();
```

从代码实现上，mycobj->myvirfunc();显然是多态，因为是通过虚函数表查询虚函数并执行。

不知读者是否想过，如果在 C 类的构造函数中增加代码来调用 myvirfunc 虚函数，会怎样呢？修改类 C 的构造函数：

```
public:
    C() :m_c(11)
    {
        myvirfunc();     //调用一个虚函数
        printf("C::C()执行了,this =  %p\n", this);
    };
```

想一想，C 类构造函数中针对 myvirfunc 虚函数的调用，调用的是哪个类的 myvirfunc 虚函数呢？因为现在的情况是，C 类对象还没构造完整，而且 A、B、C 三个类都有同名的 myvirfunc 虚函数。

不难发现，如果是在某个类的**构造函数中调用一个虚函数**，那么并**不通过虚函数表**来调用。编译器的做法是：**在构造函数中调用的虚函数从所在类往根类回溯，逐次找这个虚函数，找到哪个，就直接调用（静态方式调用）哪个虚函数，而不是通过虚函数表来调用虚函数。**这是 C++语言的一个规则，编译器做了这种决议，可以尝试着想一想做这种决议的原因：估计原因可能是在构造函数中调用虚函数时，该对象还没有构造完整，因此不宜采用虚函数表机制调用虚函数，迫使编译器做出这种直接调用虚函数的决定。

但笔者认为，如果单纯从调用虚函数的角度来讲，虚函数表指针 vptr 此时（在构造函数函数体内）有有效值了，所以通过虚函数表调用虚函数也应该是安全的，虽然这里是安全的，但试想如果在类 B 的构造函数中也调用了 myvirfunc 虚函数，若通过虚函数表来调用虚函数，调用的会是哪个虚函数呢？不要忘记此时的虚函数表指针 vptr 指向的可是 B 类的虚函数表（vptr = B::vftable;），还没有最终指向 C 类的虚函数表指针（vptr = C::vftable;）。若通过虚函数表来调用虚函数，类 B 构造函数调用的 myvirfunc 肯定是类 B 中的 myvirfunc。另外，如果这个虚函数里用到了一些成员变量，那么这些成员变量要保证先在构造函数中被初始化了，这样才能安全地在虚函数中调用。

总之，最终在 C 类构造函数中调用 myvirfunc 虚函数，调用的是 C 类的 myvirfunc 虚函数，并且是直接调用（虚函数有真实地址，可以直接调用）。

将断点设置在 C 类构造函数的 myvirfunc();代码行，开始调试，当程序执行停到断点行时，切换到反汇编窗口，如图 6.8 所示。

```
class C :public B
{
public:
    C() :m_c(11)
002E54B4  mov         eax,dword ptr [this]
002E54B7  mov         dword ptr [eax+4],0Bh
        myvirfunc(); //调用一个虚函数
002E54BE  mov         ecx,dword ptr [this]
002E54C1  call        C::myvirfunc (02E13B6h)
        printf("C::C()执行了, this = %p\n", this);
002E54C6  mov         eax,dword ptr [this]
002E54C9  push        eax
002E54CA  push        offset string "C::C()\xd6\xb4\xd0\xd0\xc
002E54CF  call        _printf (02E104Bh)
002E54D4  add         esp,8
    };
```

图 6.8　C::C 构造函数中调用虚函数会直接进行调用而不是通过虚函数表

现在往类 A、B、C 中再增加一个 myvirfunc2 虚函数。

在类 A 中，加入如下 public 修饰的虚函数：

```
public:
    virtual void myvirfunc2()
    {
        printf("A::myvirfunc2()执行了!\n");
    };
```

在类 B 中，也加入类似的虚函数，只是 printf 的内容发生了变化：

```
public:
    virtual void myvirfunc2()
    {
        printf("B::myvirfunc2()执行了!\n");
    };
```

在类 C 中，也加入类似的虚函数，只是 printf 的内容发生了变化：

```
public:
    virtual void myvirfunc2()
    {
        printf("C::myvirfunc2()执行了!\n");
    };
```

在每个类的 myvirfunc 虚函数中的第一行都增加调用 myvirfunc2 虚函数的代码：

```
myvirfunc2();
```

断点依旧设置在类 C 构造函数的 myvirfunc();代码行，开始调试，当程序执行停到断点行时，切换到反汇编窗口，按 F11 键跟踪进入到 C 类的 myvirfunc 函数中去，如图 6.9 所示。

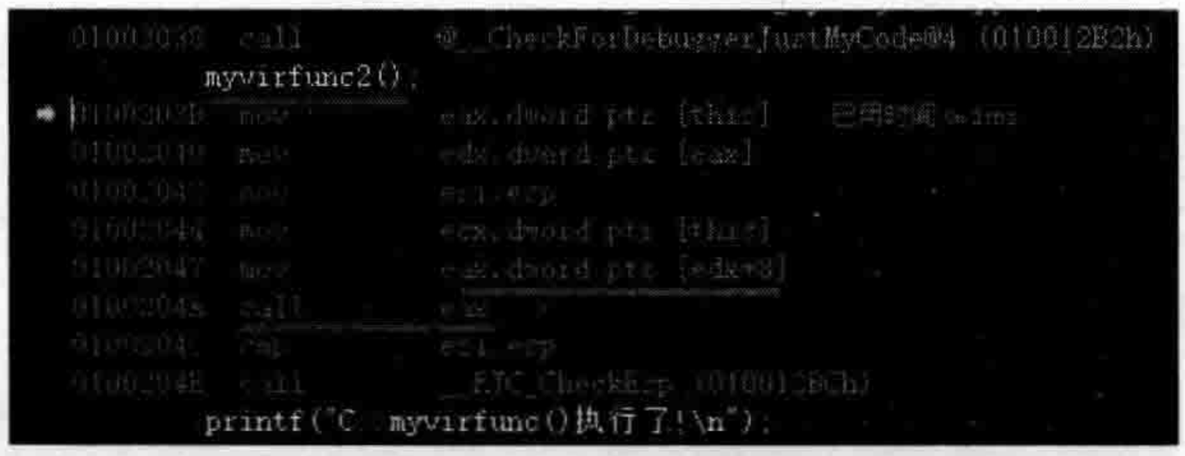

图 6.9　C::C 构造函数中调用虚函数，在该虚函数中调用另外一个虚函数

从图 6.9 中可以看到，对 myvirfunc2 虚函数的调用确实是通过虚函数表来进行的（而不是直接调用。看来，只有在构造函数中出现的虚函数才会获得被直接调用的待遇），这就是编译器的做法。当然，如果通过虚函数表来调用虚函数，那么就一定要注意当前的虚函数表指针指向的是哪个虚函数表（因为在构造类 C 对象的过程中，虚函数表指针被赋值了 3 次）：指向哪个类的虚函数表，调用的就是哪个类对应的虚函数（读者可以试试，在类 B 的构造函数中调用 myvirfunc，在 myvirfunc 中调用 myvirfunc2，这里调用的 myvirfunc2 是通过虚函数表调用，而且调用的是 B 类的 myvirfunc2 虚函数）。

总结：笔者建议，**不要在类的构造函数中调用虚函数**，这个结论同样适合类的析构函数——**不要在类的析构函数中调用虚函数**。遵循这些规则，能够开发出更加健壮的应用程序。在开发的过程中，笔者建议，当遇到有可能是陷阱的地方（自己对程序的某些写法有怀疑或者疑虑的时候），尽量绕开不去触碰，也是很好的解决问题之道。

上面分析了多层次继承时构造函数的执行步骤，有虚函数夹杂在类中的时候虚函数表指针的初始化时机，以及有初始化列表时初始化列表的执行时机。有了这些知识作为基础，以后面临更复杂的继承结构（如虚基类）掺和进来时，就可以自行进行深入分析。同时，掌握了这些知识，也能避免无意中写出一些错误的代码。

如果读者有兴趣进一步深入挖掘，笔者就把虚基类夹杂虚函数这种继承体系下的对象构造的步骤流程留给读者去探索，因为道理都是相通的，可能分析的步骤要烦琐一些，这里笔者就不对这种情况进行额外的分析，因为已经没有太多的新知识了。

6.2　对象复制语义学与析构函数语义学

6.2.1　对象的默认复制行为

在 MyProject.cpp 的上面，增加如下代码行：

```
class A
{
public:
    int m_i, m_j;
};
```

在 main 主函数中，增加如下代码：

```
A aobj;
aobj.m_i = 15;
aobj.m_j = 20;

A aobj2 = aobj;

A aobj3;
aobj3.m_i = 13;
aobj3.m_j = 16;
aobj2 = aobj3;
```

设置断点并调试观察，代码执行后不难发现，A aobj2 = aobj；代码行可以把 aobj 对象的成员变量值原样不动地复制到 aobj2 中，而 aobj2 = aobj3；代码行可以把 aobj3 对象的成员变量值原样不动地复制到 aobj2 中。

这说明系统有一些默认的对象复制行为。当程序员不写自己的拷贝构造函数和拷贝赋值运算符时，这些默认的对象复制行为会发挥作用。

6.2.2　拷贝赋值运算符与拷贝构造函数

如果只是简单的复制操作，那么对象默认的复制行为可能就够了，而且对象默认的复制

行为执行效率也高(编译器不需要额外调用程序员所写的代码)。既然够了,程序员就没必要提供自己的拷贝构造函数和拷贝赋值运算符。

这种对象默认的复制行为如果不安全或者不正确,那么就需要程序员自己书写拷贝构造函数和拷贝赋值运算符,例如在构造对象的时候需要 new 一块内存空间来做一些事情(如 2.8.3 节所说的深拷贝问题)。

现在看一看程序员自己书写的拷贝构造函数和拷贝赋值运算符。在类 A 中增加如下代码:

```
public:
A& operator = (const A& tmp)
{
    cout << "A::operator = (const A&)拷贝赋值运算符执行了" << endl;
    return * this;
}
A(const A& tmptime)
{
    cout << "A::A(const A&)拷贝构造函数执行了" << endl;
}
```

如果现在编译项目,会出现错误提示,因为程序员提供了拷贝构造函数,导致编译器不会生成“合成默认构造函数”,那么 A aobj;、A aobj3;这些代码行就会报语法错。这需要程序员为类 A 定义一个默认构造函数。代码如下:

```
public:
    A()
    {
        cout << "A::A()缺省构造函数执行了" << endl;
    }
```

现在,将断点设置在 A aobj3;代码行,开始调试,当程序执行停到断点行时,将鼠标放在 aobj2 上,观察代码行 A aobj2 = aobj;的执行结果,如图 6.10 所示。

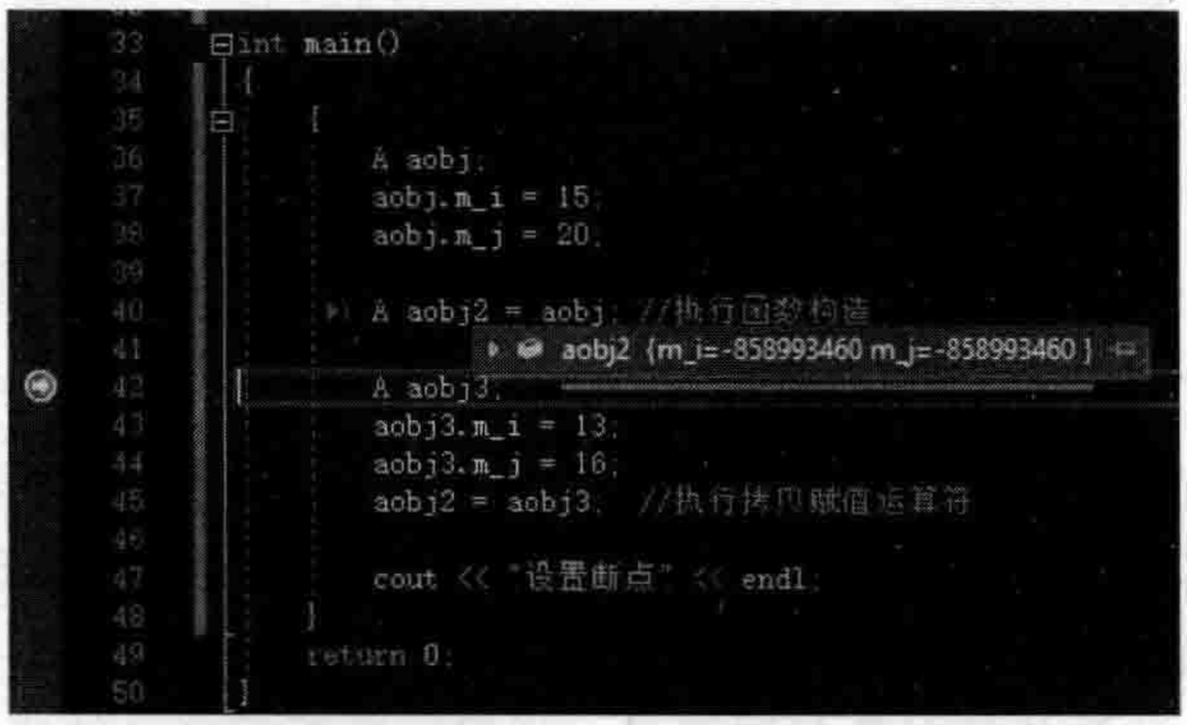

图 6.10 拷贝构造对象

在图 6.10 中发现了问题,在执行了代码行 A aobj2 = aobj;之后,aobj2 对象里成员变量的值都不正常了。这是因为程序员提供了自己的拷贝构造函数和拷贝赋值运算符取代了系统缺省的对象复制行为,那么,程序员就必须要在书写的拷贝构造函数和拷贝赋值运算符

中增加给对象成员有效值的代码。

完善类 A 的拷贝赋值运算符和拷贝构造函数：

```
public:
    A& operator = (const A& tmp)
    {
        m_i = tmp.m_i;
        m_j = tmp.m_j;
        cout << "A::operator = (const A&)拷贝赋值运算符执行了" << endl;
        return * this;
    }
    A(const A& tmptime)
    {
        m_i = tmptime.m_i;
        m_j = tmptime.m_j;
        cout << "A::A(const A&)拷贝构造函数执行了" << endl;
    }
```

再次运行程序并设置断点进行观察，一切正常，无论拷贝构造对象还是对对象进行赋值，都可以得到正确的结果。

6.2.3 如何禁止对象的拷贝构造和赋值

如果在实际工作中，遇到了不希望对象被拷贝构造和被赋值的需求，怎么办呢？

(1) 只需要在类 A 定义中把拷贝赋值运算符和拷贝构造函数私有起来，只声明，不需要写函数体。代码如下：

```
private:
    A& operator = (const A& tmp);
    A(const A& tmptime);
```

(2) 也可以利用 C++ 11 新标准中引入的语法：=delete;，直接把拷贝赋值运算符和拷贝构造函数标记为禁用。代码如下：

```
public:
    A& operator = (const A& tmp) = delete;
    A(const A& tmptime) = delete;
```

6.2.4 析构函数语义

1. 析构函数被合成

通常来讲，如果一个类中不写析构函数，那么生成的目标文件中也不会有该类的析构函数。

针对现在的范例，可以编译一下并查看目标文件中是否有析构函数～A，查看方法在 2.4 节已经介绍过(使用 dumpbin 将.obj 目标导出成.txt 文本文件查看)。实际情况是无法找到析构函数～A。

那么，什么情况下编译器会为类合成出一个析构函数呢？

(1) 如果当前类继承一个基类，基类带析构函数，那么编译器就会为当前类合成出一个

析构函数。这个析构函数的存在意义是因为它要调用基类的析构函数。

在 MyProject.cpp 的上面，类 A 定义的前面，增加如下新类 JI：

```
class JI
{
public:
    virtual ~JI()
    {
        cout << "JI::~JI()析构函数执行了" << endl;
    }
};
```

现在让类 A 继承自类 JI：

```
class A :public JI
{
    ...
};
```

在 main 主函数中，注释掉原有的代码，加入如下代码行(代码行用{}以尽早结束 aobj 对象的作用域方便调试观察)：

```
{
    A aobj;
}
```

现在重新编译一下项目，通过 dumpbin 将 MyProject.obj 目标文件中的信息导出成 .txt 文件并查找，可以找到类 A 的析构函数～A 了。

当然，也可以设置断点来查看。将断点行设置在 A aobj;代码行，当程序执行停到断点行时，切换到反汇编窗口，如图 6.11 所示。

从图 6.11 中不难看到，在超出了 aobj 的作用域后，系统会调用类 A 的析构函数～A。按 F11 键跟踪进该析构函数，如图 6.12 所示。

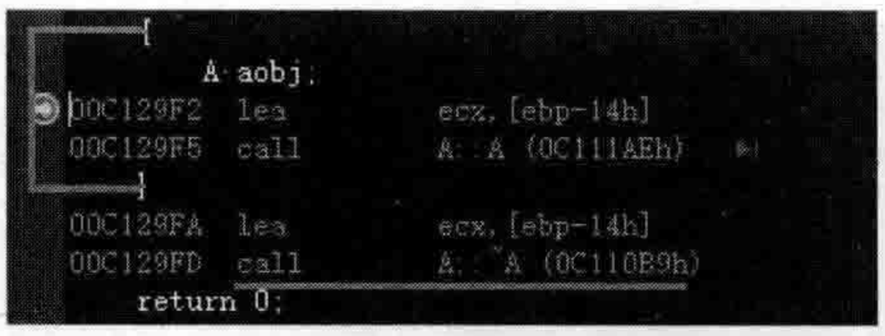

图 6.11 超出对象 aobj 作用域后会调用类 A 的析构函数～A

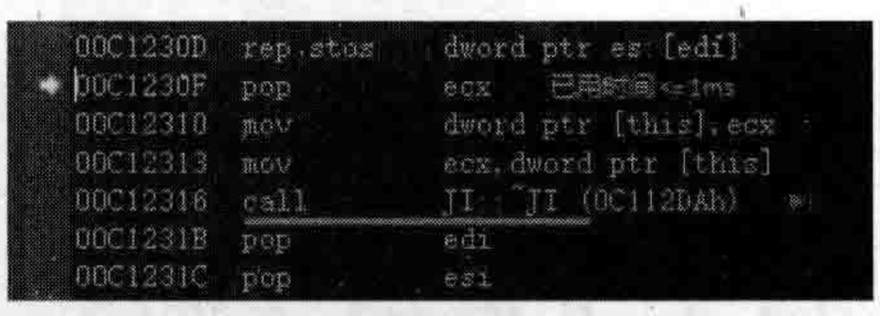

图 6.12 超出对象 aobj 作用域后会调用当前类 A 以及其父类的析构函数

在图 6.12 中，类 A 的析构函数调用了其父类 JI 的析构函数～JI。

(2) 如果当前类的成员变量是一个类类型成员变量，并且这个类类型带有析构函数，编译器也会为当前类合成出一个析构函数。这个析构函数的存在意义就是要调用类类型成员变量所属类的析构函数。

在 MyProject.cpp 的上面，增加两个新类的定义：

```
class ParC
```

```
{
public:
    virtual ~ParC()
    {
        cout << "ParC::~ParC()虚析构函数执行了" << endl;

    }
};
class MemC
{
public:
    ParC m_j;
};
```

在 main 主函数中，注释掉以往代码，增加如下新代码：

```
{
    MemC mobj;
}
```

重新编译一下项目，通过 dumpbin 将 MyProject.obj 目标文件中的信息导出成.txt 文件并查找，就可以找到 MemC 类的析构函数～MemC 了。

同理，如果将断点设置在 MemC mobj;代码行并跟踪调试，在反汇编窗口一样能够看到系统调用了类 MemC 的析构函数～MemC，如图 6.13 所示。

按 F11 键跟踪进该析构函数，如图 6.14 所示。

```
    {
        MemC mobj;
00F128E2  lea         ecx,[ebp-0Ch]
00F128E5  call        MemC::MemC (0F11393h)
    }
00F128EA  lea         ecx,[ebp-0Ch]
00F128ED  call        MemC::~MemC (0F113CFh)
    return 0;
```

图 6.13　超出对象 mobj 作用域后会调用类 MemC 的析构函数～MemC

```
00F1226D  rep stos    dword ptr es:[edi]
00F1226F  pop         ecx
00F12270  mov         dword ptr [this],ecx    已用时间<=1ms
00F12273  mov         ecx,dword ptr [this]
00F12276  call        ParC::~ParC (0F11145h)
00F1227B  pop         edi
00F1227C  pop         esi
```

图 6.14　MemC 的析构函数～MemC 中会调用成员变量 m_j 所属类 ParC 的析构函数～ParC

从图 6.14 中不难看到，当对象 mobj 超出了作用域后，调用了类 MemC 的析构函数～MemC，在～MemC 析构函数中，调用了类类型成员变量 m_j 所属的 ParC 类的析构函数～ParC。

2. 析构函数被扩展

当然，如果程序员书写了自己的析构函数，自然就不需要编译器来合成了，此时编译器就帮助程序员扩展析构函数中的代码，向其中增加调用另一个相关类析构函数的代码。

(1) 先看一下如果当前类的成员变量是一个类类型成员变量，并且这个类类型带析构函数，编译器如何扩展当前类的析构函数。

为 MemC 类增加一个 public 修饰的析构函数：

```
public:
    ~MemC()
```

```
{
    cout << "MemC::~MemC()析构函数执行了" << endl;
}
```

执行起来，看一看结果：

```
MemC::~MemC()析构函数执行了
ParC::~ParC()虚析构函数执行了
```

从结果中可以看到，先执行了类 MemC 的析构函数，然后又执行了 ParC 类的析构函数，注意执行的顺序。

可以设置断点看一下，在 MemC 类析构函数的函数体中设置断点，开始调试，当程序执行停到断点行时，切换到反汇编窗口，如图 6.15 所示。

图 6.15　类 MemC 的析构函数体执行完后调用 ParC 类的析构函数

从图 6.15 中不难看到，编译器会向 MemC 类的析构函数中插入调用类 ParC 析构函数的代码，但是代码插入的**位置是在 MemC 类析构函数的函数体代码之后**。

(2) 再看一下如果当前类继承自一个基类，基类带析构函数，那么编译器如何扩展当前类的析构函数。

为类 A 增加一个 public 修饰的析构函数：

```
public:
    ~A()
    {
        cout << "A::A()析构函数执行了" << endl;
    }
```

注释掉 main 主函数中原来的代码，增加如下代码：

```
{
    A aobj;
}
```

执行起来，看一看结果：

```
A::A()缺省构造函数执行了
A::A()析构函数执行了
JI::~JI()析构函数执行了
```

从结果中可以看到，先执行了 A 类的析构函数，然后又执行了 JI 类的析构函数，注意执行的顺序。换句话说，就是**先执行该类自己的析构函数体，然后才执行父类的析构函数体**。

可以设置断点看一下，在 A 类析构函数的函数体中设置断点，开始调试，当程序执行停到断点行时，切换到反汇编窗口，如图 6.16 所示。

```
00DE233F  mov         dword ptr [eax],offset A::`vftable' (0DE9B5Ch)
        cout << "A::A()析构函数执行了" << endl;
00DE2345  mov         esi,esp
00DE2347  push        offset std::endl<char,std::char_traits<char> > (0DE12D0h)
00DE234C  push        offset string "A::A()\xce\xf6\xb9\xb9\xba\xaf\xca\xfd\xd6\xb4\xd0\xd0\xc1\xcb" (0DE9B84h)
00DE2351  mov         eax,dword ptr [__imp_?cout@std@@3V?$basic_ostream@DU?$char_traits@D@std@@@@1@A (0DED0D4h)]
00DE2356  push        eax
00DE2357  call        std::operator<<<std::char_traits<char> > (0DE1230h)
00DE235C  add         esp,8
00DE235F  mov         ecx,eax
00DE2361  call        dword ptr [__imp_std::basic_ostream<char,std::char_traits<char> >::operator<< (0DED0A0h)]
00DE2367  cmp         esi,esp
00DE2369  call        __RTC_CheckEsp (0DE12ADh)
    }
00DE236E  mov         ecx,dword ptr [this]
00DE2371  call        JI::~JI (0DE12DAh)
00DE2376  mov         ecx,dword ptr [ebp-0Ch]
00DE2379  mov         dword ptr fs:[0],ecx
```

图 6.16　类 A 的析构函数体执行完后调用 JI 类的析构函数

从图 6.16 中不难看到，编译器会向 A 类的析构函数中插入调用 JI 类析构函数的代码，但是代码插入的**位置是在 A 类析构函数的函数体代码之后**。

对象析构时执行析构函数的顺序与对象构造时执行构造函数的顺序正好相反，对象构造时是**先执行父类的构造函数体再执行子类的构造函数体**，而对象析构时是**先执行子类的析构函数体再执行父类的析构函数体**。

不妨再给 JI 类增加一个 public 修饰的构造函数：

```
public:
    JI()
    {
        cout << "JI::JI()缺省构造函数执行了" << endl;
    }
```

执行起来，看一看结果：

```
JI::JI()缺省构造函数执行了
A::A()缺省构造函数执行了
A::A()析构函数执行了
JI::~JI()析构函数执行了
```

如果有需要，读者可以自行设置断点并跟踪调试观察。

另外，如果引入了虚基类，那么虚基类中构造函数和析构函数的执行时机分析就留给读者自行探索。虚基类的引入，通常都会引入更多的复杂性，同时，也会对程序执行效率有一定的影响。这两点要认识清楚。

6.3 局部对象、全局对象的构造和析构

6.3.1 局部对象的构造和析构

在 MyProject.cpp 的上面，增加如下代码行：

```
class A
{
public:
    A()
    {
        cout << "A::A()缺省构造函数执行了" << endl;
    }
    ~A()
    {
        cout << "A::~A()析构函数执行了" << endl;
    }
    int m_i;
};
```

在 main 主函数中，增加如下代码(注意用{}将代码行括住以方便调试和观察局部对象离开作用域时的表现)：

```
{
    A obja;
}
```

将断点设置在 A obja;代码行，开始调试，当程序执行停到断点行时，切换到反汇编窗口，如图 6.17 所示。

```
    {
        A obja;
00A12792  lea         ecx,[ebp-0Ch]
00A12795  call        A::A (0A1118Bh)
    }
00A1279A  lea         ecx,[ebp-0Ch]
00A1279D  call        A::~A (0A1137Ah)

    return 0;
00A127A2  xor         eax,eax
}
```

图 6.17 查看构造和析构一个对象时编译器增加的调用构造和析构函数的代码

在图 6.17 中可以很明显地看到编译器增加的代码：在定义 obja 对象时，编译器帮助程序员调用了类 A 的构造函数，当离开 obja 对象作用域后，编译器帮助程序员调用了类 A 的析构函数。

将 main 主函数中原有代码注释掉，加入全新的代码如下：

```
A obja;
int mytest = 1;
if (mytest == 0)
{
    return 0;
}
```

上面这段代码有一个 if 条件判断语句，当条件满足时，会执行 return 0;语句，这是一条出口语句。一旦条件满足，执行了 return 0;就会从当前的函数中返回(对于 main 主函数，从当前函数中返回意味着整个程序运行的结束)。那么在返回时，对象 obja 就离开了其作

用域，类 A 的析构函数是否会被调用呢？

```
    A obja;
00192792  lea         ecx,[obja]
00192795  call        A::A (019118Bh)
    int mytest = 1;
0019279A  mov         dword ptr [mytest],1
    if (mytest == 0)
001927A1  cmp         dword ptr [mytest],0
001927A5  jne         main+61h (01927C1h)
    {
        return 0;
001927A7  mov         dword ptr [ebp-0E4h],0
001927B1  lea         ecx,[obja]
001927B4  call        A::~A (019137Ah)
001927B9  mov         eax,dword ptr [ebp-0E4h]
001927BF  jmp         main+79h (01927D9h)
    }

    return 0;
001927C1  mov         dword ptr [ebp-0F0h],0
001927CB  lea         ecx,[obja]
001927CE  call        A::~A (019137Ah)
001927D3  mov         eax,dword ptr [ebp-0F0h]
}
```

图 6.18　凡是函数出口的位置都会被插入调用 obja 对象析构函数的代码

将断点设置在 if (mytest == 0)代码行，当程序执行停到断点行时，切换到反汇编窗口，如图 6.18 所示。

从图 6.18 中可以看到，只要是任何一个 main 函数出口(返回)的地方，都要调用类 A 的析构函数用以正确释放 obja 对象，目前上面这段代码有两个从 main 函数返回的位置(一个是 if 条件成立后的 return 语句，一个是 main 函数执行结束时的 return 语句)，这意味着有两次调用类 A 析构函数的代码生成。

这里得到的一个结论就是：只要出了对象的作用域，编译器总会在适当的地方插入调用对象析构函数的代码。所以，编译器确实在背后为程序员做了很多事。

通过上面的讲解可以明白一个道理，这种局部对象现用现定义，尽量把对象定义在**需要立即用到它的代码段的附近**(现用现定义，这也是 C++和 C 语言的一个典型不同处)，这样可以避免编译器产生不必要的对象构造和析构的代码，也可以避免编译器调用不必要的对象构造函数和析构函数。试想，如果像传统 C 语言那样，在函数开头就定义一个类对象则：

(1) 所有该函数的出口处，编译器都要插入对对象析构函数的调用代码。

(2) 如果因为 if 条件成立提前退出了该函数，则会导致该对象的析构函数被调用一次，而如果在此之前，根本就没用过该局部对象，那么对该对象析构函数的调用完全没必要。

例如如下代码行(注意看其中的注释)：

```
void myfunc()
{
    A obja; //obja 定义在这里并不合适
    if (1 == 1)
    {
        //这里会被编译器插入调用 obja 对象析构函数的代码，影响执行效率，完全没必要
        return;
    }
    //这里才会用到 obja 对象，所以 obja 对象定义在这里才合适
    obja.m_i = 10;
    cout << "obja.m_i 的值为" << obja.m_i << endl;
    return;
}
```

另外值得注意的是，当生成一个 A 类对象的时候，其成员变量 m_i 的值是随机的，程序员应该在构造函数初始化列表或者构造函数中为 m_i 成员变量赋初值，如图 6.19 所示。

```
47      A obja;
48      int my ▸ obja {m_i=-858993460} ▸
49      if (mytest == 0)
50      {
51          return 0;
52      }
```

图 6.19　创建一个类对象时其成员变量值随机，应在构造函数初始化列表或者构造函数中为成员变量赋初值

6.3.2 全局对象的构造和析构

在 main 主函数的前面，定义一个全局对象：

```
A g_aobj;
```

在上述行设置一个断点，并在 main 主函数的第一行代码也设置一个断点。调试时可以发现，在程序执行流程进入到 main 主函数之前，这个 g_aobj 对象就已经生成了。实际上编译器在调用 main 主函数之前是做了很多事情的，包括定义全局的 g_aobj 对象，定义这个对象时，肯定是调用了这个对象的构造函数。

这样就可以保证这个**全局对象在 main 主函数中可以直接使用**（因为这个对象已经被构造出来了）。

在 main 主函数中，注释原有的代码，加入如下新代码：

```
g_aobj.m_i = 6;
```

上面代码行没有任何问题，可以正常执行。那么，什么时候这个全局对象会被析构掉呢？在 main 主函数中最后的右大括号“}”所在行设置一个断点，在类 A 析构函数的函数体中也设置一个断点，开始调试，当程序执行停到断点行时，如图 6.20 所示。

当整个 main 主函数执行完毕后，按 F10 键向下继续执行程序，才会执行到类 A 的析构函数来析构全局对象 g_aobj。

在 4.2 节讲解过进程内存空间布局，读者已经知道，全局对象（全局变量）是保存在数据段里的。前面虽然定义了这个全局对象，但是并没有给这个全局对象中的成员变量任何初值，刚才演示局部对象 obja 的时候，如果不给该局部对象中的 m_i 成员变量初值（参考图 6.19），那么 m_i 成员值将变成一个随机值，那么全局对象呢？

取消所有断点，将新断点设置在 main 主函数的 g_aobj.m_i = 6;代码行，开始调试，当程序执行停到断点行（还没有执行该行）时，将鼠标停留在 g_aobj 位置并观察，如图 6.21 所示。

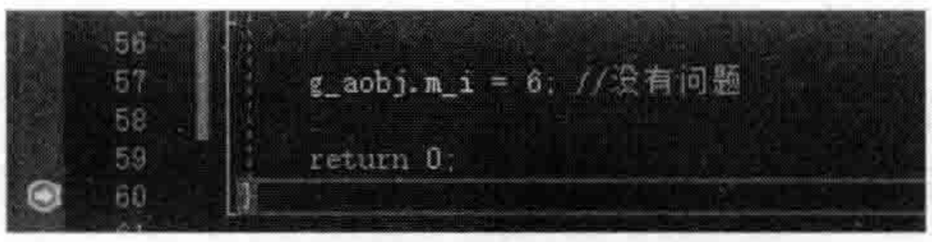

图 6.20 全局对象的析构工作在 main 主函数执行完毕后进行

图 6.21 全局对象的数值类成员变量在未给初值的情况下会被设置为 0

从图 6.21 中可以看到，全局对象的成员变量（一般指数值类型成员变量）在没给初值的情况下，编译器会给一个默认值 0，这是跟局部对象不同的地方。

在 4.2 节也曾说过，这种全局对象的内存地址是在编译阶段确定的，每次程序执行，这些全局对象的内存地址都相同。

在 main 主函数中，继续增加如下代码：

```
printf("g_aobj 对象的地址 = %p\n", &g_aobj);
```

```
A jbobj;
printf("jbobj 对象的地址 = %p\n", &jbobj);
```

执行起来，看一看结果：

```
A::A()缺省构造函数执行了
g_aobj 对象的地址 = 0041E138
A::A()缺省构造函数执行了
jbobj 对象的地址 = 00FDFE4C
A::~A()析构函数执行了
A::~A()析构函数执行了
```

多次执行，每次执行 g_aobj 对象的地址都相同，但 jbobj 对象的地址则每次执行都会不同。

上面这段程序笔者也在 Linux 操作系统上编译、连接并生成了可执行文件。使用 nm 命令列出可执行文件中 g_aobj 存放的位置，如图 6.22 所示。

```
kuangxiang@bogon:/mnt/hgfs/linux$ g++ MyProject.cpp -o MyProject
kuangxiang@bogon:/mnt/hgfs/linux$ ./MyProject
A::A()缺省构造函数执行了
g_aobj对象的地址=0x602194
A::A()缺省构造函数执行了
jbobj对象的地址=0x7ffc4953afb0
A::~A()析构函数执行了
A::~A()析构函数执行了
kuangxiang@bogon:/mnt/hgfs/linux$ nm MyProject
0000000000602080 B __bss_start
0000000000602190 b completed.7594
                 U __cxa_atexit@@GLIBC_2.2.5
0000000000602070 D __data_start
0000000000602070 W data_start
0000000000400940 t deregister_tm_clones
00000000004009c0 t __do_global_dtors_aux
0000000000601df8 t __do_global_dtors_aux_fini_array_entry
0000000000602078 D __dso_handle
0000000000601e08 d _DYNAMIC
0000000000602080 D _edata
00000000006021a0 B _end
0000000000400c34 T _fini
00000000004009e0 t frame_dummy
0000000000601de8 t __frame_dummy_init_array_entry
0000000000400ed0 r __FRAME_END__
0000000000602194 B g_aobj
0000000000602000 d _GLOBAL_OFFSET_TABLE_
0000000000400b4a t _GLOBAL__sub_I__Z6myfuncv
                 w __gmon_start__
```

图 6.22　在 Linux 下使用 nm 命令查看全局对象 g_aobj 在可执行文件中分配的内存地址

在图 6.22 中，注意到了如下这行：

```
0000000000602194 B g_aobj
```

上面这行中的 B 表示 BSS 段，图 4.1 中曾经绘制过这个段，其中保存的数据是未初始化的全局变量、初始化为 0 的全局变量等。虽然程序员的代码中没有初始化该全局变量（指全局对象 g_aobj），但实际上，编译器会**插入代码来将其成员变量初始化为 0**，所以在这里是将 g_aobj 归类为初始化为 0 的全局变量的。在程序运行起来后，在 main 函数之前，就会执行到编译器插入的给全局对象所在内存的所有内容全部置 0 的代码，所以，如图 6.20 所示，在 main 主函数中看到的 m_i 成员变量的值，就已经是 0 了。

现在总结一下全局对象 g_aobj 的初始化和释放过程：

（1）全局对象 g_aobj 获得地址（全局对象的地址在可执行文件中是确定的，并且装载到内存运行的时候就直接按照这个地址来为该全局对象分配内存。所以这种全局对象分配内存和在堆、栈中分配内存不一样：堆中的内存通过 delete 来回收，栈中的内存只要离开变量作用域就会自动回收，而全局对象的内存在程序运行期间一直存在）。

（2）把全局对象 g_aobj 的**内存内容清 0**（静态初始化）。

（3）调用全局对象 g_aobj 所对应的类 A 的构造函数。

（4）执行 main 主函数。

（5）当整个 main 主函数执行完毕后（这意味着可执行程序也即将退出），调用全局对象 g_aobj 所对应的类 A 的析构函数。

（6）整个可执行程序执行完毕。

有两件事情读者要理解：

（1）在程序执行流程进入到 main 主函数之前，以及执行完 main 主函数之后，程序都还会做许多其他的事情。整个程序并不是真的从 main 函数才开始执行，也并不是退出 main 函数（从 main 主函数返回）后程序执行就立即结束。

（2）从上面的流程可以看出，编译器会给程序员的代码中插入很多扩充代码和扩充数据，用来在 main 主函数执行之前，以及 main 主函数执行结束之后做很多事。例如，main 之前对全局对象的静态初始化，对全局对象构造函数的调用，main 之后对全局对象析构函数的调用等。这一点读者要认识到。

6.4 局部静态对象、对象数组构造析构和内存分配

6.4.1 局部静态对象的构造和析构

在 MyProject.cpp 的上面，增加如下代码行：

```
class A
{
public:
    A()
    {
        cout << "A::A()缺省构造函数执行了" << endl;
    }
    ~A()
    {
        cout << "A::~A()析构函数执行了" << endl;
    }
    int m_i;
};

void myfunc()
{
    static A s_aobj1;
    static A s_aobj2;
    printf("s_aobj1 的地址是 %p\n", &s_aobj1);
```

```
    printf("s_aobj2 的地址是 %p\n", &s_aobj2);
}
```

main 主函数先保持为空。

现在在类 A 的构造函数中设置一个断点，开始调试，并注意观察，可以得到结论：

如果程序员在代码中不调用 myfunc 函数，那么根本不会触发类 A 的构造函数，也就是说，myfunc 函数中的静态局部变量就跟不存在一样，根本不会被构造出来。

现在在 main 主函数中加入如下代码来调用两次 myfunc 函数：

```
myfunc();                    //调用一次
myfunc();                    //又调用一次
```

开始调试，并注意观察，可以得到结论：

第一次调用 myfunc 函数时，s_aobj1 和 s_aobj2 分别被构造，因此调用了 2 次类 A 的构造函数。但是，第二次调用 myfunc 函数时，类 A 的构造函数不会再被执行，也就是说，这种局部静态对象只能被构造一次，就算多次调用 myfunc 也一样。

执行起来，看一看结果：

```
A::A()缺省构造函数执行了
A::A()缺省构造函数执行了
s_aobj1 的地址是 0009D14C
s_aobj2 的地址是 0009D154
s_aobj1 的地址是 0009D14C
s_aobj2 的地址是 0009D154
A::~A()析构函数执行了
A::~A()析构函数执行了
```

多次执行程序不难发现，每次执行显示的 s_aobj1 地址和上次运行相同，s_aobj2 也同样如此。

难道 s_aobj1、s_aobj2 的地址也是编译的时候确定好的吗？真的是确定好的。就算是没调用构造函数构造这个对象，也可以发现这个对象其实也是有有效内存地址的，在编译的时候它的内存地址就确定了，只是到运行的时候，或者应该说是运行到对应的代码行时，它的内存才会依据事先约定好的地址分配出来。至于具体的内存分配细节，可能跟具体的编译器和操作系统都有关系。如果读者真想探究，笔者推荐《程序员的自我修养——链接、装载与库》，里面对可执行文件的结构，以及如何装载、运行等有比较详细的介绍。

在 Linux 操作系统上运行一下该程序，如图 6.23 所示。

从图 6.23 中也可以看到，多次执行得到的 s_aobj1 和 s_aobj2 的地址是一样的。

接着，使用 nm 工具在可执行文件中找一找 s_aobj1 和 s_aobj2 的地址信息，如图 6.24 所示。

从图 6.24 中可以看出，s_aobj1 和 s_aobj2 地址是事先确定好的，当这个可执行文件装载到内存开始运行后，这个地址也就被映射到内存中去了。所以从这个角度来说，s_aobj1 和 s_aobj2 这两个局部静态对象，在编译的时候就确定好了开始地址和大小。图 6.24 中局部静态对象名字之前的 b 标记也与图 6.22 类似，表示 BSS 段，也就是该对象的内存来自于 BSS 段。

```
1 192.168.1.126 +
kuangxiang@bogon:/mnt/hgfs/linux$ g++ MyProject.cpp -o MyProject
kuangxiang@bogon:/mnt/hgfs/linux$ ./MyProject
A::A()缺省构造函数执行了
A::A()缺省构造函数执行了
s_aobj1的地址是0x6021d0
s_aobj2的地址是0x6021d4
s_aobj1的地址是0x6021d0
s_aobj2的地址是0x6021d4
A::~A()析构函数执行了
A::~A()析构函数执行了
kuangxiang@bogon:/mnt/hgfs/linux$ ./MyProject
A::A()缺省构造函数执行了
A::A()缺省构造函数执行了
s_aobj1的地址是0x6021d0
s_aobj2的地址是0x6021d4
s_aobj1的地址是0x6021d0
s_aobj2的地址是0x6021d4
A::~A()析构函数执行了
A::~A()析构函数执行了
kuangxiang@bogon:/mnt/hgfs/linux$
```

图 6.23　在 Linux 下演示局部静态对象的构造和析构

```
kuangxiang@bogon:/mnt/hgfs/linux$ nm MyProject
0000000000602090 B __bss_start
00000000006021b0 b completed.7594
                 U __cxa_atexit@@GLIBC_2.2.5
                 U __cxa_guard_abort@@CXXABI_1.3
                 U __cxa_guard_acquire@@CXXABI_1.3
                 U __cxa_guard_release@@CXXABI_1.3
0000000000602080 D __data_start
0000000000602080 W data_start
00000000004009d0 t deregister_tm_clones
0000000000400a50 t __do_global_dtors_aux
0000000000601df8 t __do_global_dtors_aux_fini_array_entry
0000000000602088 D __dso_handle
0000000000601e08 d _DYNAMIC
0000000000602090 D _edata
00000000006021d8 B _end
0000000000400cd4 T _fini
00000000006021d0 b _ZZ6myfuncvE7s_aobj1
00000000006021d4 b _ZZ6myfuncvE7s_aobj2   ry_entry
0000000000400f70 r __FRAME_END__
0000000000602000 d _GLOBAL_OFFSET_TABLE_
0000000000400bf3 t _GLOBAL__sub_I__Z6myfuncv
                 w __gmon_start__
```

图 6.24　局部静态对象在可执行文件中是有固定地址的

去掉其他断点行，在函数 myfunc 中第一个 printf 行设置一下断点，开始调试，当程序执行停到断点行时，把鼠标放在 s_aobj1 或者 s_aobj2 上，如图 6.25 所示。

```
25  void myfunc()
26  {
27      static A s_aobj1;
28      static A s_aob ▸ s_aobj1 {m_i=0} =
29      printf("s_aobj1的地址是%p\n", &s_aobj1);
30      printf("s_aobj2的地址是%p\n", &s_aobj2);
31  }
```

图 6.25　局部静态对象的内存也会被初始化为 0

从图 6.25 中可以看到，局部静态对象的内存默认也会被设置为 0。

去掉其他断点行，在 main 主函数中最后的右大括号“}”所在行设置一个断点，在类 A 析构函数的函数体中也设置一个断点，开始调试，这里的情景与图 6.20 非常类似，就不赘述。所得到的结论就是：类 A 的析构函数在 main 主函数执行结束后才会被调用，这意味着在 main 主函数执行结束后，s_aobj1 和 s_aobj2 这两个局部静态对象才会被释放。

这里值得一说的是系统如何保证诸如 static A s_aobj1;这样的代码行所定义的 s_aobj1 对象只被初始化一次，换句话说，编译器如何保证 s_aobj1 静态局部对象只被构造一次（只调用一次构造函数）。

为了方便观察，改造一下 myfunc 函数：

```
const A& myfunc()
{
    static A s_aobj1;
    printf("s_aobj1 的地址是 %p\n", &s_aobj1);
    return s_aobj1;
}
```

去掉其他断点行，在 myfunc 函数的第一行 static A s_aobj1;设置断点，开始调试，当程序执行停到断点行时，切换到反汇编窗口，如图 6.26 所示。

```
    static A s_aobj1;
00222867  mov         eax,dword ptr [_tls_index (022D1C0h)]
0022286C  mov         ecx,dword ptr fs:[2Ch]
00222873  mov         edx,dword ptr [ecx+eax*4]
00222876  mov         eax,dword ptr ds:[0022D150h]
0022287B  cmp         eax,dword ptr [edx+104h]
00222881  jle         myfunc+0ABh (02228CBh)
00222883  push        22D150h
00222888  call        __Init_thread_header (02210FFh)
0022288D  add         esp,4
00222890  cmp         dword ptr ds:[22D150h],0FFFFFFFFh
00222897  jne         myfunc+0ABh (02228CBh)
00222899  mov         dword ptr [ebp-4],0
002228A0  mov         ecx,offset s_aobj1 (022D14Ch)
002228A5  call        A::A (02211A9h)
    static A s_aobj1;
002228AA  push        offset `myfunc'::`2'::`dynamic atexit destructor for 's_aobj1'' (0228C80h)
002228AF  call        _atexit (02212B2h)
002228B4  add         esp,4
002228B7  mov         dword ptr [ebp-4],0FFFFFFFFh
002228BE  push        22D150h
002228C3  call        __Init_thread_footer (02211B3h)
002228C8  add         esp,4
    printf("s_aobj1的地址是%p\n", &s_aobj1);
002228CB  push        offset s_aobj1 (022D14Ch)
002228D0  push        offset string "s_aobj1\xb5\xc4\xb5\xd8\xd6\xb7\xca\xc7%p\n" (022AC7Ch)
002228D5  call        _printf (022105Fh)
002228DA  add         esp,8
    return s_aobj1;
002228DD  mov         eax,offset s_aobj1 (022D14Ch)
}
```

图 6.26　查看局部静态对象初始化对应的汇编代码

在图 6.26 中，最先映入眼帘的有调用类 A 构造函数的代码(call A::A (02211A9h))，有返回 s_aobj1 对象地址的代码(mov eax, offset s_aobj1 (022D14Ch))，这个地址(022D14Ch)前面说过，是编译的时候就确定好的。

可以把这个对象地址(022D14Ch)放到内存中查看一下，如图 6.27 所示(注意 A 类对象的 sizeof 是 4 字节)。

图 6.27　查看 s_aobj1 局部静态对象内存中的内容

保持“内存 1”窗口处于打开状态，这样可以随时看到图 6.27 所示的内存中内容的变化。在反汇编窗口中，重复按 F10 键让程序逐行往下执行，当执行到图 6.26 中的 call __Init_thread_header (02210FFh)汇编代码行时，会发现图 6.27 所示的 4 字节内存右侧的 4 个字节值发生了改变(从 00 00 00 00 变成了 ff ff ff ff)，如图 6.28 所示。

在反汇编窗口中，重复按 F10 键让程序继续逐行往下执行，当执行到图 6.26 中的 call __Init_thread_footer (02211B3h)汇编代码行时，图 6.28 中 s_aobj1 局部静态对象内存右侧 4 个字节内存的内容又发生了变化(从 ff ff ff ff 变成了 01 00 00 80)，如图 6.29 所示。

图 6.28 执行 call __Init_thread_header (02210FFh)汇编代码行会影响 s_aobj1 局部静态对象内存(4 字节)右侧 4 个字节内存的内容

图 6.29 执行 call __Init_thread_footer (02211B3h))汇编代码行会再次影响 s_aobj1 局部静态对象内存右侧内存的内容

按 F5 键("调试"→"继续"命令)让程序继续执行一直遇到下次停在断点处,因为在 main 主函数中调用了两次 myfunc 函数,所以,断点会**再次**停留在 myfunc 函数的 static A s_aobj1;代码行。此时,再次观察 0x0022D14C 内存所在的 4 个字节右侧的那 4 个字节,已然变成了 01 00 00 80。其实,01 00 00 80 是一个标记,标记这个局部静态对象有没有被构造出来,第一次执行 myfunc 函数时构造并且做了一个标记,第二次比较这个标记是否已经有了,有了就不再构造该局部静态对象了,直接跳过 static A s_aobj1;这一行代码。怎么跳过的,看图 6.26 中的:

```
mov   eax,dword ptr ds:[0022D150h]   //这里是设置了标记 01 00 00 80 的内存的地址,执行后 eax
寄存器的内容: EAX = 80000001
cmp   eax,dword ptr [edx + 104h]
jle   myfunc + 0ABh (02228CBh)
```

就是用上面这三行汇编代码进行标记判断和跳到 02228CBh 内存位置继续执行,而 02228CBh 内存位置正好是 printf("s_aobj1 的地址是%p\n", &s_aobj1);代码行对应的位置,如图 6.30 所示。

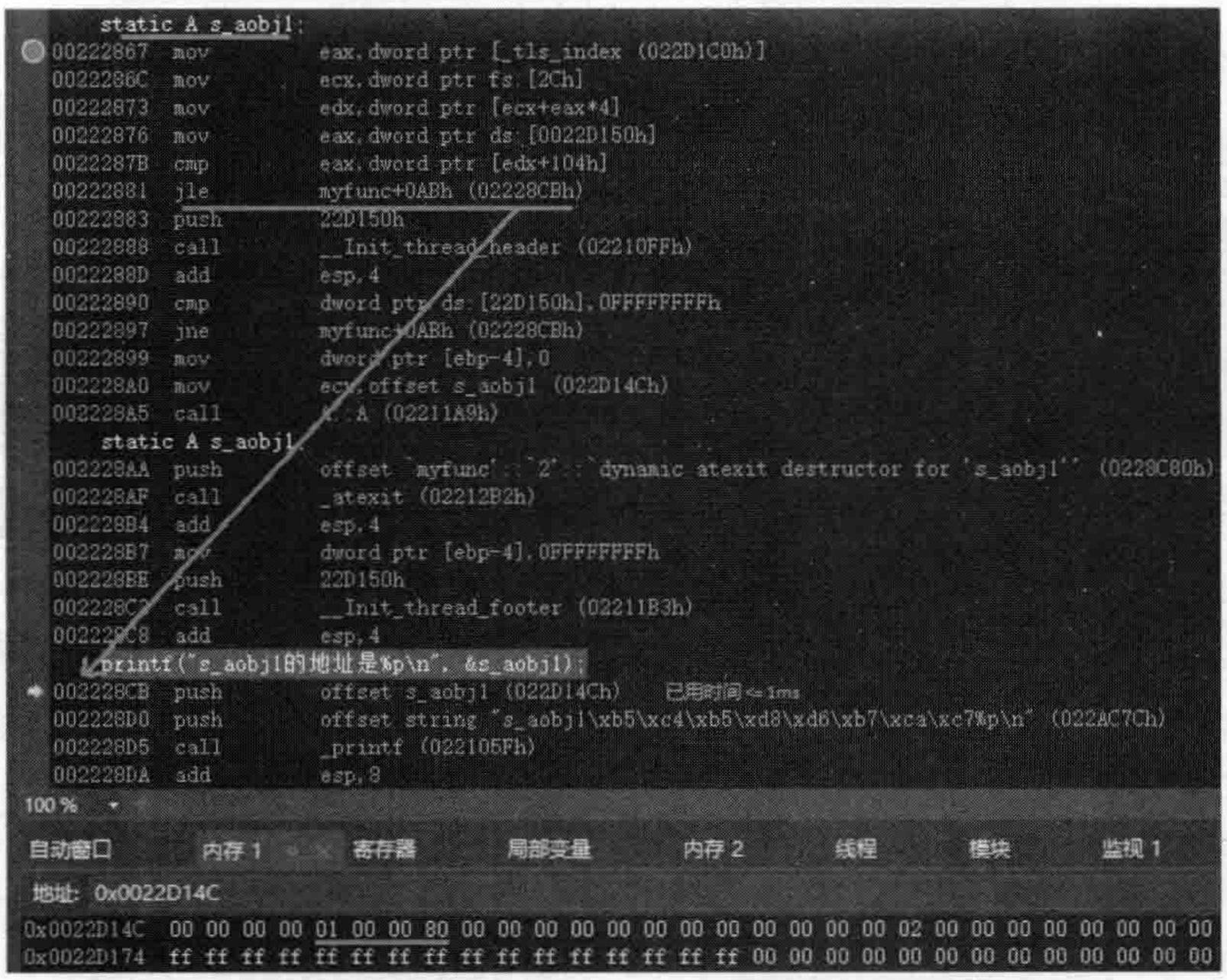

图 6.30 通过设置和比较一些标记编译器可以判断出 s_aobj1 局部静态对象是否被构造过

所以，编辑器就是通过这种标记的方式，在第二次执行 static A s_aobj1;代码行时直接跳过从而不会再次构造 s_aobj1，从而**防止 s_aobj1 局部静态对象被构造多次**。

还有一个问题就是，s_aobj1 局部静态对象被构造出来后，还需要在适当的位置进行释放(析构)。所以，编译器要把已经成功构造的局部静态对象记录起来，以供最后调用析构函数来析构它。那么，编译器是用什么手段记录这些已经成功构造的局部静态对象呢？这个问题可以参考图 6.26，但为了看的更清晰，请看图 6.31。

```
    static A s_aobj1;
00222867  mov         eax,dword ptr [_tls_index (022D1C0h)]
0022286C  mov         ecx,dword ptr fs:[2Ch]
00222873  mov         edx,dword ptr [ecx+eax*4]
00222876  mov         eax,dword ptr ds:[0022D150h]
0022287B  cmp         eax,dword ptr [edx+104h]    已用时间<=1ms
00222881  jle         myfunc+0ABh (02228CBh)
00222883  push        22D150h
00222888  call        __Init_thread_header (02210FFh)
0022288D  add         esp,4
00222890  cmp         dword ptr ds:[22D150h],0FFFFFFFFh
00222897  jne         myfunc+0ABh (02228CBh)
00222899  mov         dword ptr [ebp-4],0
002228A0  mov         ecx,offset s_aobj1 (022D14Ch)
002228A5  call        A::A (02211A9h)
    static A s_aobj1;
002228AA  push        offset `myfunc'::`2'::`dynamic atexit destructor for 's_aobj1'' (0228C80h)
002228AF  call        _atexit (02212B2h)
002228B4  add         esp,4
002228B7  mov         dword ptr [ebp-4],0FFFFFFFFh
002228BE  push        22D150h
002228C3  call        __Init_thread_footer (02211B3h)
002228C8  add         esp,4
    printf("s_aobj1的地址是%p\n", &s_aobj1);
002228CB  push        offset s_aobj1 (022D14Ch)
002228D0  push        offset string "s_aobj1\xb5\xc4\xb5\xd8\xd0\xb7\xca\xc7%p\n" (022AC7Ch)
```

图 6.31　观看系统如何正确记录 main 主函数执行结束后要释放的局部静态对象

从图 6.31 中可以看到，当第一次执行 myfunc 函数中的 static A s_aobj1;代码行时，看到了如下两行重要的汇编代码：

```
push    offset `myfunc'::`2'::`dynamic atexit destructor for 's_aobj1''(0228C80h)
call    _atexit (02212B2h)
```

这两行汇编代码的大概意思就是调用 _atexit 登记一些信息，告诉系统在 main 主函数执行完毕后要执行一段代码。哪段代码呢？就是 push 末尾圆括号中对应的地址 0228C80h 所代表的这段代码。地址 0228C80h 所代表的代码段中包含了哪些内容呢？可以在反汇编窗口左上角输入这个地址观察所对应的代码内容，如图 6.32 所示。

```
00228C80  push        ebp
00228C81  mov         ebp,esp
00228C83  sub         esp,0C0h
00228C89  push        ebx
00228C8A  push        esi
00228C8B  push        edi
00228C8C  lea         edi,[ebp-0C0h]
00228C92  mov         ecx,30h
00228C97  mov         eax,0CCCCCCCCh
00228C9C  rep stos    dword ptr es:[edi]
00228C9E  mov         ecx,offset s_aobj1 (022D14Ch)
00228CA3  call        A::~A (02213CFh)
00228CA8  pop         edi
00228CA9  pop         esi
```

图 6.32　main 主函数执行结束后，整个程序退出之前，释放 s_aobj1 局部静态对象时要执行的代码段

在图 6.32 中可以看到，s_aobj1 静态局部对象对应的首地址 022D14Ch 是有的，有了这个地址，就可以确保后面调用类 A 的析构函数(call A::~A (02213CFh))能够成功执行来释放 s_aobj1 对象。

6.4.2　局部静态对象数组的内存分配

局部静态对象的构造和析构前面详细讲解了，其实局部静态对象数组也很类似。

修改 myfunc 函数，修改后的代码如下：

```
void myfunc()
{
    static A s_aobj[5];
    printf("s_aobj 数组的首地址是 %p\n", s_aobj);
}
```

其他代码不变。执行起来，看一看结果：

```
A::A()缺省构造函数执行了
A::A()缺省构造函数执行了
A::A()缺省构造函数执行了
A::A()缺省构造函数执行了
A::A()缺省构造函数执行了
s_aobj 数组的首地址是 0062D148
s_aobj 数组的首地址是 0062D148
A::~A()析构函数执行了
A::~A()析构函数执行了
A::~A()析构函数执行了
A::~A()析构函数执行了
A::~A()析构函数执行了
```

读者都知道，数组的内存空间是连续的。从运行结果看，调用了 5 次类 A 的构造函数，也调用了 5 次类 A 的析构函数。多次执行可以发现，数组首地址一直都不变。

在 Linux 操作系统上运行一下该程序，如图 6.33 所示。

```
kuangxiang@bogon:/mnt/hgfs/linux$ g++ MyProject.cpp -o MyProject
kuangxiang@bogon:/mnt/hgfs/linux$ ./MyProject
A::A()缺省构造函数执行了
A::A()缺省构造函数执行了
A::A()缺省构造函数执行了
A::A()缺省构造函数执行了
A::A()缺省构造函数执行了
s_aobj数组的首地址是0x6021d0
s_aobj数组的首地址是0x6021d0
A::~A()析构函数执行了
A::~A()析构函数执行了
A::~A()析构函数执行了
A::~A()析构函数执行了
A::~A()析构函数执行了
```

图 6.33　在 Linux 操作系统下演示局部静态对象数组的构造和析构

从图 6.33 中也可以看到，多次执行得到的 s_aobj 数组的首地址是一样的。

接着，使用 nm 工具在可执行文件中找一找 s_aobj 数组的首地址信息，如图 6.34 所示。

在 Linux 下，可以用 g++辅助-c 参数生成目标文件，而后使用 objdump 工具来查看该目标文件(二进制文件)中的信息。objdump 辅助-h 参数可以显示目标文件头部的一些信息，借此可以看一看局部静态对象数组 s_aobj 的大小，如图 6.35 所示。

从图 6.35 中可以看到，.bss 段显示的是 24，代表 24 字节，这 24 字节的空间里包括上面定义的静态局部数组(static A s_aobj[5];)所占用的内存空间大小。

所以能够看得出来，全局变量、局部静态变量等所占用的内存大小信息，一般都在目标文件或者在可执行文件里存着呢。

```
kuangxiang@bogon:/mnt/hgfs/linux$ nm MyProject .
0000000000602090 B __bss_start
00000000006021b0 b completed.7594
                 U __cxa_atexit@@GLIBC_2.2.5
                 U __cxa_guard_abort@@CXXABI_1.3
                 U __cxa_guard_acquire@@CXXABI_1.3
                 U __cxa_guard_release@@CXXABI_1.3
0000000000602080 D __data_start
0000000000602080 W data_start
00000000004009d0 t deregister_tm_clones
0000000000400a50 t __do_global_dtors_aux
0000000000601df8 t __do_global_dtors_aux_fini_array_entry
0000000000400c36 W _ZN1AD2Ev
                 U _ZNSolsEPFRSoS_E@@GLIBCXX_3.4
                 U _ZNSt8ios_base4InitC1Ev@@GLIBCXX_3.4
                 U _ZNSt8ios_base4InitD1Ev@@GLIBCXX_3.4
00000000006020a0 B _ZSt4cout@@GLIBCXX_3.4
                 U _ZSt4endlIcSt11char_traitsIcEERSt13basic_ostreamIT_T0_ES6_@@GLIBCXX_3.4
00000000006021c0 b _ZStL8__ioinit
                 U _ZStlsISt11char_traitsIcEERSt13basic_ostreamIcT_ES5_PKc@@GLIBCXX_3.4
00000000006021d0 b _ZZ6myfuncvE6s_aobj
kuangxiang@bogon:/mnt/hgfs/linux$
```

图 6.34　局部静态对象数组在可执行文件中是有固定首地址的

```
kuangxiang@bogon:/mnt/hgfs/linux$ g++ -c MyProject.cpp
kuangxiang@bogon:/mnt/hgfs/linux$ ls
MyProject  MyProject.cpp  MyProject.o
kuangxiang@bogon:/mnt/hgfs/linux$ objdump -h MyProject.o

MyProject.o:     file format elf64-x86-64

Sections:
Idx Name          Size      VMA               LMA               File off  Algn
  0 .group        00000008  0000000000000000  0000000000000000  00000040  2**2
                  CONTENTS, READONLY, EXCLUDE, GROUP, LINK_ONCE_DISCARD
  1 .group        00000008  0000000000000000  0000000000000000  00000048  2**2
                  CONTENTS, READONLY, EXCLUDE, GROUP, LINK_ONCE_DISCARD
  2 .text         00000173  0000000000000000  0000000000000000  00000050  2**0
                  CONTENTS, ALLOC, LOAD, RELOC, READONLY, CODE
  3 .data         00000000  0000000000000000  0000000000000000  000001c3  2**0
                  CONTENTS, ALLOC, LOAD, DATA
  4 .bss          00000024  0000000000000000  0000000000000000  000001d0  2**4
                  ALLOC
  5 .rodata       0000005f  0000000000000000  0000000000000000  000001d0  2**3
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  6 .text._ZN1AC2Ev 0000002b  0000000000000000  0000000000000000  00000230  2**1
                  CONTENTS, ALLOC, LOAD, RELOC, READONLY, CODE
  7 .text._ZN1AD2Ev 0000002b  0000000000000000  0000000000000000  0000025c  2**1
                  CONTENTS, ALLOC, LOAD, RELOC, READONLY, CODE
  8 .gcc_except_table 00000013  0000000000000000  0000000000000000  00000287  2**0
                  CONTENTS, ALLOC, LOAD, READONLY, DATA
  9 .init_array   00000008  0000000000000000  0000000000000000  000002a0  2**3
                  CONTENTS, ALLOC, LOAD, RELOC, DATA
 10 .comment      00000036  0000000000000000  0000000000000000  000002a8  2**0
                  CONTENTS, READONLY
 11 .note.GNU-stack 00000000  0000000000000000  0000000000000000  000002de  2**0
                  CONTENTS, READONLY
 12 .eh_frame     00000128  0000000000000000  0000000000000000  000002e0  2**3
                  CONTENTS, ALLOC, LOAD, RELOC, READONLY, DATA
kuangxiang@bogon:/mnt/hgfs/linux$
```

图 6.35　利用 objdump 工具查看目标文件头部的信息

这里想聊一下这种局部静态对象数组的内存分配问题。例如定义一个非常非常大的静态数组，修改一下 myfunc 函数的代码。如下：

```
void myfunc()
{
    //static A s_aobj[5];
    static A s_aobj[1000'0000];     //中间有'的数字是 C++14 新标准中支持的，'是数字分隔符
    printf("s_aobj 数组的首地址是 %p\n", s_aobj);
}
```

现在如果执行该程序(注意，main 主函数中调用了 myfunc 函数)，系统给不给这么大的

s_aobj 局部静态对象数组分配内存呢？

这个静态数组所需要的内存空间很庞大(大概需要几十兆字节)，如果程序运行起来就分配内存，这就比较浪费了，所以这个时候编译器有它的智慧和特殊的处理。实际上，程序运行起来后，编译器并没有给这个局部静态数组分配实际的内存。测试一下看，将 main 主函数中的代码全部注释掉，增加如下新代码，其中包含一个 while(1)循环，目的是防止 main 主函数执行结束从而导致整个进程执行结束：

```
int main()
{
    myfunc();
    printf("开始进入 while 循环\n");
    while (1)
    {
    }
    return 0;
}
```

然后，将类 A 中的构造函数和析构函数都注释掉，因为其中的 cout 语句会影响构造对象的速度。

使用快捷键 Ctrl+F5(“调试”→“开始执行(不调试)”命令)将程序执行起来。

在 Windows 操作系统下，启动任务管理，看看 MyProject. exe 这个进程占用了多大的内存，如图 6.36 所示。

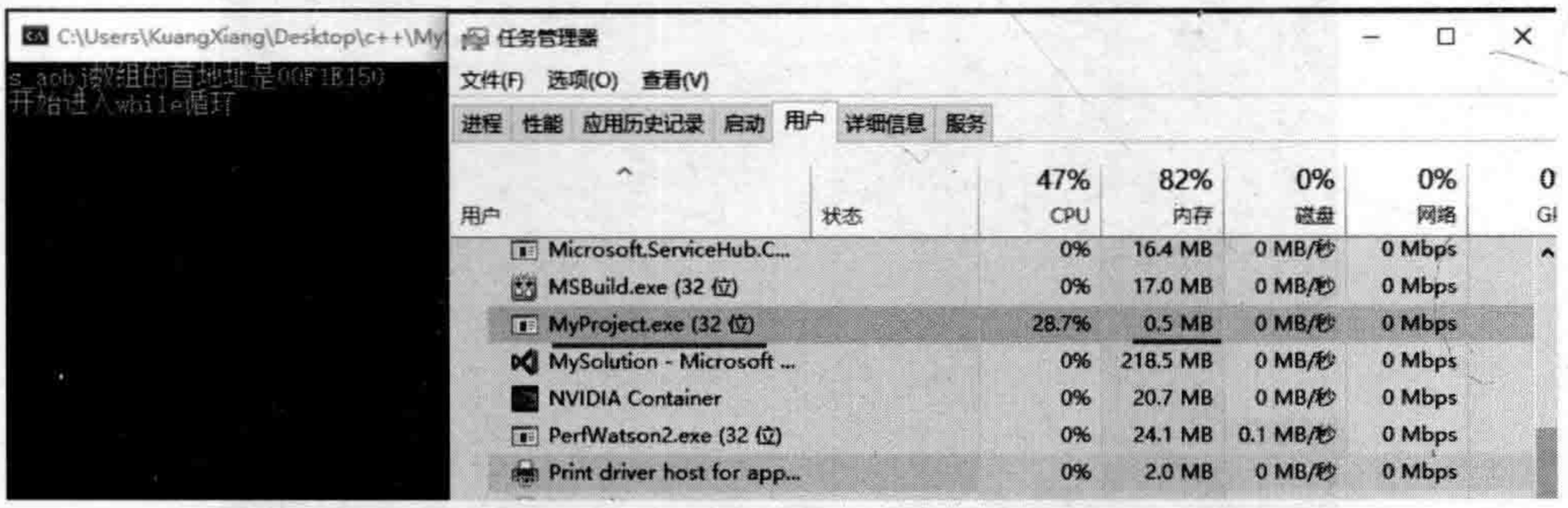

图 6.36　利用任务管理器查看 MyProject. exe 进程所占用的内存空间

从图 6.36 中不难看出，进程 MyProject. exe 只占了 0.5MB 内存空间，这说明 static A s_aobj[1000'0000];语句肯定没占内存，因为这个内存大概需要几十兆字节。

所以系统很智能，当它发现程序员没有针对这个静态对象数组中的对象做什么有用的事情时，它果断地不给这个对象数组分配实际的物理内存。

这个时候读者可能就有疑问了：什么时候会为这个静态对象数组分配实际的物理内存呢？当然是针对这个对象数组做点有用的事的时候。什么叫有用的事呢？改造一下 myfunc 便知：

```
void myfunc()
{
    static A s_aobj[1000'0000];
    for (int i = 0; i < 1000'0000; i++)
```

```
    {
        s_aobj[i].m_i = i;     //这对于数组来说,一定算一件有用的事
    }
    printf("s_aobj 数组的首地址是 %p\n", s_aobj);
}
```

从上面的代码中可以看到,每个数组中的元素保存的 m_i 成员变量值都不同(因为都不同,编译器的智能和优化能力就发挥不出来)。再次按 Ctrl+F5 键运行程序。启动任务管理,看看这次 MyProject.exe 进程占用了多大的内存,如图 6.37 所示。

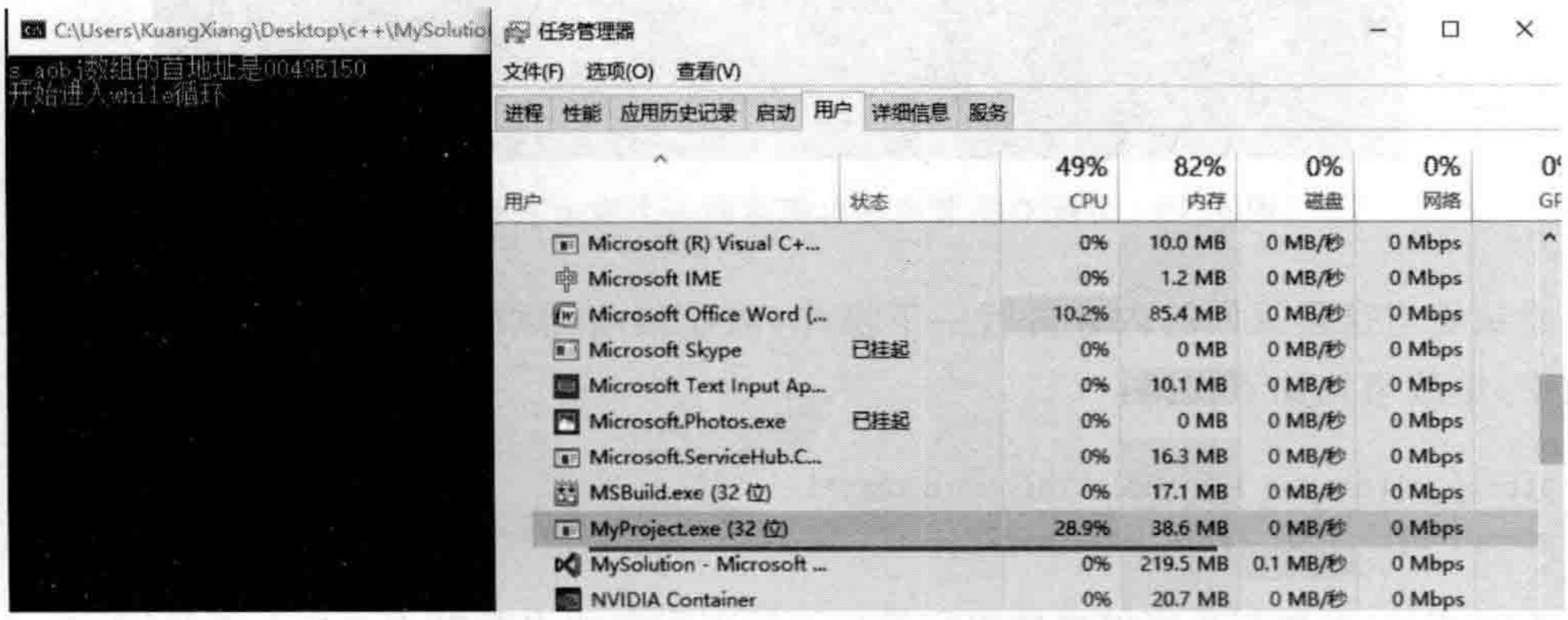

图 6.37　利用任务管理器再次查看 MyProject.exe 进程所占用的内存

从图 6.37 中不难看出,这次进程 MyProject.exe 占了 38.6MB 内存,这说明 static A s_aobj[1000'0000];语句一定是占了内存的。

还是那句话,不同操作系统下的不同编译器可能有不同的处理方式和处理技巧,有些技巧可能超出想象的智能,笔者在这里只是抛砖引玉,带一带路,如果读者有兴趣,可以继续深入挖掘下去。

6.5　new、delete 运算符与内存高级话题

笔者所著的《C++新经典》一书中的“内存高级话题”一章,对 new、delete 等内存知识有非常详细的讲解,强烈建议读者在学习本节的时候能够详细去读一下,必然会收获多多,事半功倍。

《C++新经典》一书中的“内存高级话题”一章所覆盖的问题如下:

(1) new、delete 的进一步认识。

(2) new 细节探秘与重载类内 operator new、delete。

(3) 内存池概念、代码实现和详细分析。

(4) 嵌入式指针概念及范例、内存池改进版。

(5) 重载全局 new、delete,定位 new 及重载等。

使用 malloc 和 new 分配 0 字节。

在 main 主函数中加入如下代码行:

```
void* p = malloc(0);
char* q = new char[0];
```

读者都知道，new 内部调用的就是 malloc，所以上面两行代码其实几乎等价。

上面的写法让人感到奇怪。当然，一般来讲，不会这样写代码，因为分配 0 字节内存违背常规的认知。

但是也不难发现，这样写代码从语法上确实是允许的，而且从跟踪调试的结果来看，p 和 q 也都返回了有效的内存地址，如果 6.38 所示。

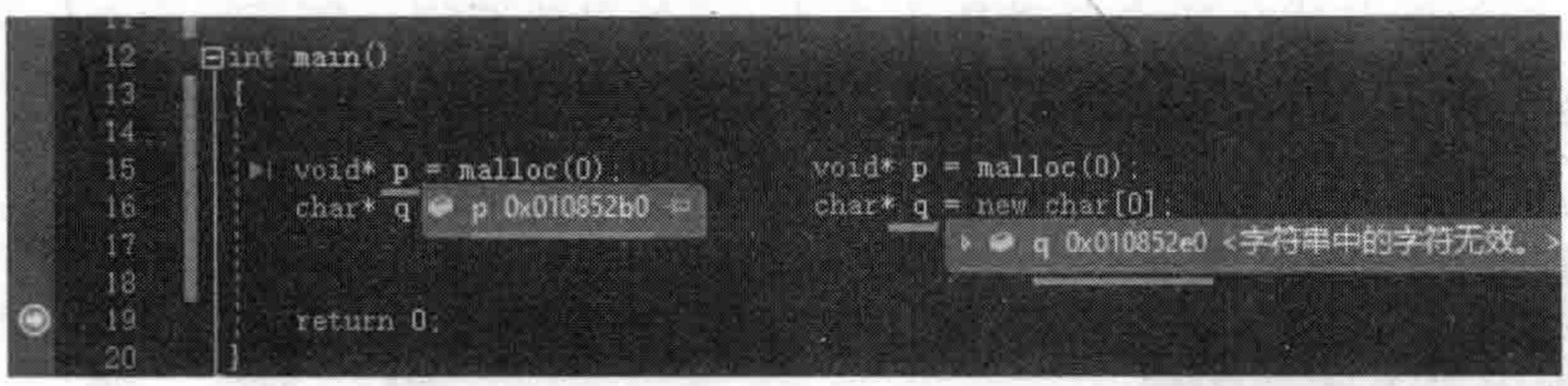

图 6.38　分配 0 字节内存也能返回一个看上去有效的地址值

尝试着对这段返回的内存进行一下操作，向这段内存中复制一个字符串。在 main 主函数中，继续增加如下代码：

```
strcpy_s((char *)p, 100, "This is a test1!");
strcpy_s(q, 100, "This is a test2");
```

跟踪调试，发现似乎能够成功将字符串复制到指定的内存中。

老手程序员和新手程序员最大的区别就是：老手程序员对于不会或者没弄明白的内容可以不去用，但是一般不会用错，新手程序员一般正好反过来——发现系统没有报什么异常会觉得这种用法是正确的。

上面这两行 strcpy_s 代码复制一个这么长的字符串到 malloc(0)或者 new char[0]分配的内存，而且没报错，表面看起来似乎没什么问题，但实际上很可能已经危机四伏，也就是说这个程序可能岌岌可危，随时面临执行中出现各种逻辑错误甚至随时崩溃。因为这两句 strcpy_s 很可能把一段本来不属于程序员能够操作的内存地址中的内容给覆盖掉了，这种覆盖也许程序不会立即报错，但是程序运行的隐患和暗疾已经出现了。

malloc(0)、new char[0]这样的写法到底编译器给分配了几个字节，不管是去猜，还是写程序去测，还是一批人集体讨论等，效果都不大。笔者认为，若真想搞明白这种写法到底分配了多少字节，就去读对应的 malloc 或者 new 的源码，别无它法。

继续在 main 主函数中加入释放内存的代码：

```
free(p);
delete[] q;
```

上面这两行代码，无论执行哪行，都会使程序崩溃。当然，如果前面没有那两行 strcpy_s 语句，则这两行释放内存的代码不会执行崩溃。所以，大概可以感觉出来，strcpy_s 肯定是把内存破坏掉了，导致释放内存时一些信息错乱，株连了 free 和 delete 的执行。

所以，在实际写代码中，**不要写出 malloc(0)、new char[0]这种诡异代码**。别看它们能执行成功，但这个内存只要进行一些写操作就可能导致 free 时报错。

有资料显示，malloc(0)、new char[0]可能返回空指针，也可能返回一个有效指针(取决

于编译器),也有说法是 malloc(0)、new char[0]返回一个有效地址(指针)是语言本身的要求。但是,就算是返回一个有效地址,也不能去用这个地址,不能修改其中的内容。

6.6　临时性对象的详细探讨

6.6.1　拷贝构造函数相关的临时性对象

在 MyProject.cpp 的上面,增加如下代码行:

```
class A
{
public:
    A()
    {
        cout << "A::A()构造函数执行" << endl;
    }
    A(const A& tmpobj)
    {
        cout << "A::A()拷贝构造函数执行" << endl;
    }
    ~A()
    {
        cout << "A::~A()析构函数执行" << endl;
    }
};
A operator + (const A& obj1, const A& obj2)
{
    A tmpobj;                          //调用一次构造函数
    //这里可以做一些加法计算
    //.....
    return tmpobj; //执行后调用了一次拷贝构造函数、一次析构函数,说明有临时对象生成
}
```

在 main 主函数中,加入如下代码:

```
A myobj1;                              //调用一次构造函数
A myobj2;                              //调用一次构造函数
A resultobj = myobj1 + myobj2;
```

执行起来,看一看结果:

```
A::A()构造函数执行
A::A()构造函数执行
A::A()构造函数执行
A::A()拷贝构造函数执行
A::~A()析构函数执行
A::~A()析构函数执行
A::~A()析构函数执行
A::~A()析构函数执行
```

从上面结果中不难发现 operator＋的 return tmpobj；代码行产生了一个临时对象，因为结果中看到了一次拷贝构造函数，这个临时对象就是通过 tmpobj 拷贝构造产生的，用来放置从 operator＋返回的结果。

对于 main 主函数中的代码行 A resultobj = myobj1 + myobj2;，刚才说了，从 operator＋返回了一个临时对象，实际上如果按常规思考，这个临时对象应该再次调用拷贝构造函数来拷贝构造出 resultobj 对象，但实际是编译器在这里做了一个优化——这个临时对象就直接构造到了 resultobj 的预留空间里，这样就相当于省了一次拷贝构造和一次析构。

从上面的结果中可以看到，执行了 4 次构造函数（拷贝构造函数也算构造函数）和 4 次析构函数。

在 Visual Studio 2019 中，设置一下优化选项（设置方法可以参考图 2.28，把“优化”从原来的“已禁用 (/Od)”修改为“最大优化（优选速度）(/O2)”），再设置一下基本运行时检查（设置方法可以参考图 2.29，把“基本运行时检查”从原来的“两者(/RTC1，等同于 /RTCsu) (/RTC1)”修改为“默认值”）。

然后在 operator ＋中加一行 printf 语句用来做一个输出标记：

```
A operator + (const A& obj1, const A& obj2)
{
    A tmpobj; //调用一次构造函数
    //这里可以做一些加法计算
    //.....
    printf("----------\n");
    return tmpobj; //执行后调用了一次拷贝构造函数、一次析构函数，说明有临时对象生成
}
```

执行起来，看一看结果：

```
A::A()构造函数执行
A::A()构造函数执行
A::A()构造函数执行
----------
A::～A()析构函数执行
A::～A()析构函数执行
A::～A()析构函数执行
```

比较两次的输出结果，发现少了一次复制构造和一次析构。

看起来编译器优化后，还让人产生了疑惑，resultobj 对象是怎么构造出来的？为什么现在的结果中只有三次构造函数的执行呢？

为了探究这个问题，继续向 operator ＋中增加一些代码，输出一下这些对象的地址：

```
A operator + (const A& obj1, const A& obj2)
{
    A tmpobj; //调用一次构造函数
    printf("tmpobj 的地址为 %p\n", &tmpobj);

    //这里可以做一些加法计算
```

```
    //......
    printf("----------\n");
    return tmpobj; //执行后调用了一次拷贝构造函数、一次析构函数,说明有临时对象生成
}
```

在 main 主函数中也增加一些代码,输入一下对象的地址。修改后的 main 主函数代码如下:

```
A myobj1;                          //调用一次构造函数
printf("myobj1 的地址为 %p\n", &myobj1);
A myobj2;                          //调用一次构造函数
printf("myobj2 的地址为 %p\n", &myobj2);
A resultobj = myobj1 + myobj2;
printf("resultobj 的地址为 %p\n", &resultobj);
```

执行起来,看一看结果:

```
A::A()构造函数执行
myobj1 的地址为 001CF9CE
A::A()构造函数执行
myobj2 的地址为 001CF9CD
A::A()构造函数执行
tmpobj 的地址为 001CF9CF
----------
resultobj 的地址为 001CF9CF
A::~A()析构函数执行
A::~A()析构函数执行
A::~A()析构函数执行
```

从结果中不难发现,编译器优化之后,竟然直接把 tmpobj 的地址拿来做 resultobj 的地址了。这种优化是比较强悍的,也就是省了一次复制构造和一次析构。而且这几个对象的地址离得都很近。

现在把刚才设置的优化选项和基本运行时检查选项恢复回去,以免对后续内容的讲解或者程序的调试产生影响:

(1) 把“优化”从“最大优化(优选速度)(/O2)”恢复为“已禁用 (/Od)”)。

(2) 把“基本运行时检查”从原来的“默认值”恢复为“两者(/RTC1,等同于 /RTCsu)(/RTC1)”。

6.6.2　拷贝赋值运算符相关的临时性对象

向类 A 中增加一个 public 修饰的拷贝赋值运算符:

```
public:
    A& operator = (const A& tmpaobj)
    {
        cout << "A::operator = ()拷贝赋值运算符执行" << endl;
        return * this;              //返回一个该对象的引用
    }
```

在 operator＋中,先把 2 条 printf 语句注释掉。

在 main 主函数中，把 3 条 printf 语句注释掉，还要把对象 resultobj 的定义的赋值操作分成两行来写。调整后的 main 主函数代码如下：

```
A myobj1;                                    //调用一次构造函数
A myobj2;                                    //调用一次构造函数
A resultobj;
resultobj = myobj1 + myobj2;
```

执行起来，看一看结果：

```
A::A()构造函数执行
A::A()构造函数执行
A::A()构造函数执行
A::A()构造函数执行
A::A()拷贝构造函数执行
A::~A()析构函数执行
A::operator=()拷贝赋值运算符执行
A::~A()析构函数执行
A::~A()析构函数执行
A::~A()析构函数执行
A::~A()析构函数执行
```

可以将断点设置在 resultobj = myobj1 + myobj2;代码行，并跟踪调试查看结果。不难发现，operator+中 return tmpobj;返回的临时对象调用了拷贝赋值运算符赋给了 resultobj，然后，这个临时对象自己析构掉了。上述结果中的如下两行就是这个意思：

```
A::operator=()拷贝赋值运算符执行
A::~A()析构函数执行
```

所以，这个拷贝赋值就不如上面的拷贝构造，上面的拷贝构造是直接把临时对象构造到 resultobj 的预留空间，执行性能上就提高了一些。

重复前面的步骤，开启优化选项，设置基本运行时检查。

把 operator+里的 2 条 printf 语句和 main 主函数中的 3 条 printf 语句注释取消。现在完整的 operator+和 main 主函数中的代码应该如下：

```
A operator + (const A& obj1, const A& obj2)
{
    A tmpobj;                                //调用一次构造函数
    printf("tmpobj 的地址为 %p\n", &tmpobj);

    //这里可以做一些加法计算
    //......
    printf("----------\n");
    return tmpobj; //执行后调用了一次拷贝构造函数、一次析构函数，说明有临时对象生成
}
int main()
{
    A myobj1;                                //调用一次构造函数
```

```
    printf("myobj1 的地址为 %p\n", &myobj1);
    A myobj2;                          //调用一次构造函数
    printf("myobj2 的地址为 %p\n", &myobj2);

    //A resultobj = myobj1 + myobj2;
    A resultobj;
    resultobj = myobj1 + myobj2;
    printf("resultobj 的地址为 %p\n", &resultobj);

    return 0;
}
```

在类 A 的拷贝赋值运算符中也增加一条 printf 语句。增加后的完整代码如下：

```
public:
    A& operator = (const A& tmpaobj)
    {
        cout << "A::operator = ()拷贝赋值运算符执行" << endl;
        printf("tmpaobj 的地址为 %p\n", &tmpaobj);
        return * this;              //返回一个该对象的引用
    }
```

执行起来，看一看结果：

```
A::A()构造函数执行
myobj1 的地址为 008FF963
A::A()构造函数执行
myobj2 的地址为 008FF962
A::A()构造函数执行
A::A()构造函数执行
tmpobj 的地址为 008FF960
----------
A::operator = ()拷贝赋值运算符执行
tmpaobj 的地址为 008FF960
A::~A()析构函数执行
resultobj 的地址为 008FF961
A::~A()析构函数执行
A::~A()析构函数执行
A::~A()析构函数执行
```

从结果中不难看到，这个临时对象调用拷贝赋值运算符把内容给了 resultobj 后，自己就析构(释放)掉了。和开启优化选项之前相比，省了一次拷贝构造和一次析构。

所以，这种也是产生了临时对象，然后临时对象自己析构掉了。

通过上面的讲述，综合而言：

```
A resultobj = myobj1 + myobj2;
```

和

```
A resultobj;
resultobj = myobj1 + myobj2;
```

比起来，上面的写法(定义和初始化写在一行)效率更高一些。

现在把刚才设置的优化选项和基本运行时检查选项**恢复回去**，以免对后续内容的讲解和程序的调试产生影响。

6.6.3 直接运算产生的临时性对象

在 operator＋中，再次把 2 条 printf 语句注释掉。

在 main 主函数中为了简单，将原有的代码注释掉，修改为如下全新的代码：

```
A myobj1;                          //调用一次构造函数
A myobj2;                          //调用一次构造函数
myobj1 + myobj2;
```

执行起来，看一看结果：

```
A::A()构造函数执行
A::A()构造函数执行
A::A()构造函数执行
A::A()拷贝构造函数执行
A::~A()析构函数执行
A::~A()析构函数执行
A::~A()析构函数执行
A::~A()析构函数执行
```

可以将断点设置在 myobj1 ＋ myobj2;行，跟踪调试，可以看到，在执行完该行后，类 A 的析构函数执行了两次。上述结果中的如下两行就是这个意思：

```
A::~A()析构函数执行
A::~A()析构函数执行
```

这说明 myobj1 和 myobj2 这两个对象相加产生了一个临时对象(另一个临时对象是在 operator＋里产生的，并没有计算在这里)，并且这个临时对象立即被析构了。

回想前面写过的代码行：

```
A resultobj = myobj1 + myobj2;
```

这个代码行的情况是临时对象被构造到了 resultobj 中，并没有立即析构，而是 resultobj 的生命周期到了后才析构。而 myobj1＋myobj2 这种临时对象是立即析构，因为没有一个对象来保存(接收)myobj1＋myobj2 返回的这个结果对象(这个结果对象是一个临时对象)。

如果开启优化选项，那么结果如下：

```
A::A()构造函数执行
A::A()构造函数执行
A::A()构造函数执行
A::~A()析构函数执行
A::~A()析构函数执行
A::~A()析构函数执行
```

比优化之前少一次复制构造和一次析构，估计 operator＋中那个临时对象直接构造到 tmpobj 里面去了。

关于临时对象，还有点内容要说，需要调整一下代码。

在类 A 中，增加一个 public 修饰的成员变量：

```
public:
    int m_i;
```

在类 A 的拷贝构造函数中，增加一行代码：

```
m_i = tmpobj.m_i;
```

在 operator＋中，return 语句之前，增加一行代码：

```
tmpobj.m_i = obj1.m_i + obj2.m_i;
```

在 main 主函数中，注释掉以往的代码，增加如下全新的代码：

```
A myobj1;                                    //调用一次构造函数
myobj1.m_i = 1;
A myobj2;                                    //调用一次构造函数
myobj2.m_i = 2;
printf("(myobj1 + myobj2).m_i =  %d\n", (myobj1 + myobj2).m_i); //临时对象,打印其 m_i 值
```

执行起来，能够正常输出结果。主要输出结果项如下：

```
(myobj1 + myobj2).m_i = 3
```

刚才说过，myobj1＋myobj2 这种临时对象是立即析构的，因为这两个对象相加后的结果并没有保存。那么这里不禁要思考一下，上面的 printf 语句就是输出这个临时对象的 m_i 成员变量值的，如果这个临时对象在 printf 语句执行完之前就析构，那 printf 语句输出出来的就是已经被析构的对象的成员变量，这显然不对。所以，**实际上临时对象被析构是 printf 这行语句的最后一步**，这样就能保证 printf 输出的临时对象的 m_i 成员变量值有效。

将断点设置在上面的 printf 行，开始调试，当程序执行停到断点行时，切换到反汇编窗口，如图 6.39 所示。

```
    printf("(myobj1 + myobj2).m_i = %d\n", (myobj1 + myobj2).m_i); //临时对象，打印其m_i值
00FF2AC3  lea         eax,[myobj2]
00FF2AC6  push        eax
00FF2AC7  lea         ecx,[myobj1]
00FF2ACA  push        ecx
00FF2ACB  lea         edx,[ebp-0F0h]
00FF2AD1  push        edx
00FF2AD2  call        operator+ (0FF124Eh)
00FF2AD7  add         esp,0Ch
00FF2ADA  mov         dword ptr [ebp-104h],eax
00FF2AE0  mov         eax,dword ptr [ebp-104h]
00FF2AE6  mov         ecx,dword ptr [eax]
00FF2AE8  push        ecx
00FF2AE9  push        offset string "(myobj1 + myobj2).m_i = %d\n" (0FF9B7Ch)
00FF2AEE  call        _printf (0FF1050h)
00FF2AF3  add         esp,8
00FF2AF6  lea         ecx,[ebp-0F0h]
00FF2AFC  call        A::~A (0FF1398h)
```

图 6.39　临时对象在被 printf 语句使用完后才释放

在图 6.39 中可以明显地看到，临时对象的析构是在调用完了 printf 语句后才进行的，这表示 printf 语句可以安全地输出这个临时对象的 m_i 成员变量值。

继续观察一下类似的情况。在 main 主函数中继续加入如下代码：

```
if ((myobj1 + myobj1).m_i > 3 || (myobj1 + myobj2).m_i > 5)
{
    cout << "条件成立" << endl;
}
```

取消其他断点，将断点设置在这条新加入的 if 语句行，并调试观察，不难看到，仅仅这条 if 语句行就会产生如下这么多行输出：

```
A::A()构造函数执行
A::A()拷贝构造函数执行
A::~A()析构函数执行
A::A()构造函数执行
A::A()拷贝构造函数执行
A::~A()析构函数执行
A::~A()析构函数执行
A::~A()析构函数执行
```

这里不深入探究这条 if 判断语句导致的针对临时对象的处理。因为判断条件的原因导致 if 语句中“||”的左右两端的条件(关系运算)都进行了判断，因此产生了一堆针对构造、拷贝构造、析构函数的调用。如果把这条 if 语句的第一个条件((myobj1 + myobj1).m_i > 3)修改一下，让其为 true，这样系统就不用再判断第二个条件((myobj1 + myobj2).m_i > 5)了，如此可以节省大量的 A 类的构造、拷贝构造、析构函数的调用。修改 main 主函数中刚刚的 if 语句如下：

```
if ((myobj1 + myobj1).m_i > 1 || (myobj1 + myobj2).m_i > 5)
```

再次看一看这条 if 语句的执行产生的输出：

```
A::A()构造函数执行
A::A()拷贝构造函数执行
A::~A()析构函数执行
A::~A()析构函数执行
```

可以看到，因为(myobj1 + myobj2).m_i > 5 这个条件判断没有执行，因此输出结果节省了不少内容。

总之，临时对象会耗费很多系统成本，而且这些成本可能往往程序员并没有注意到，在写程序的时候，是否一个不留神就写出了一个临时对象呢！

同时也需要明白，编译器往往会在必要的地方帮助程序员插入代码来产生临时对象供编译器完成程序员的代码要实现的意图。

1. 临时对象被摧毁

写程序时还是要多留心，尽量少用这种临时对象，另外有的时候一旦没用好，还可能用错。这里写一段用错的代码：

```
const char * p = (string("123") + string("456")).c_str(); //这就有问题,临时对象被摧毁了
printf("p =  %s \n", p);        //无法打印正确结果,无效内存
```

下面的代码是可以正确输出结果的：

```
string aaa = (string("123") + string("456"));
const char * q = aaa.c_str();
printf("q =  %s \n", q);
```

2. 临时对象因绑定到引用而被保留

如果临时对象绑定到一个引用上，看如下代码：

```
const string& aaa = (string("123") + string("456"));
printf("aaa =  %s\n", aaa.c_str());
```

上面两行代码可以输出正确的结果。所以可以发现，如果临时对象被绑定到一个引用上，那么这个临时对象的生命周期就会变成这个引用的生命周期(引用的生命周期在这里看成是一个正常的局部变量的生命周期)。

另外也看到了，当开启了编译器优化选项后，编译器的优化能力就会介入程序运行中，帮助程序员少产生了很多临时对象，少调用了很多构造、拷贝构造和析构函数，这对提升程序运行效率有重要和积极的意义，这方面编译器开发商就要各显其能了。但是值得一提的是，编译器的优化能力介入后，会不会导致原本计划在构造、拷贝构造和析构函数中执行的代码因为优化的原因没有去执行(没有按照程序员的本意去执行)从而使程序执行产生错误？这值得思考和注意，开发的过程中也要进行严格的测试。

第7章

模板实例化语义学

本章详细阐述了模板实例化时的各种细节问题，这些细节问题也许是读者以往未曾注意甚至是根本不知道的，所以，非常有必要好好学习和掌握一下。

7.1 模板及其实例化详细分析

7.1.1 函数模板

在 MyProject.cpp 的上面，增加如下代码行：

```
template<class T>
T funcadd(const T& a, const T& b)
{
    T addhe = a + b;
    return addhe;
}
```

在 main 主函数中，增加如下代码行：

```
cout << funcadd(12, 14) << endl;
```

执行起来，看一看结果：

```
26
```

可以看到，程序执行输出了正确的结果。

读者在学习模板的时候都知道，上述代码中针对 T 类型的推断，是编译器在编译的时候根据针对 funcadd 的调用来确定的。

可能许多读者对这句话理解的不太深入，那么就一起观察一下。

编译整个项目，然后参考 2.4 节在开发人员命令行工具下使用 dumpbin 工具将 MyProject.obj 导出成 MyProject.txt。

在 MyProject.txt 中寻找 funcadd 字样，不难发现如图 7.1 所示。

在 Linux 操作系统上编译、链接并生成可执行文件。使用 nm 命令，也可以看到 funcadd 字样，如图 7.2 所示。

```
1940   60501020 flags
1941            Code
1942            COMDAT; sym= "int __cdecl funadd<int>(int const &,int const &)" (??$funadd@H@@YAHABH0@Z)
1943            16 byte align
1944            Execute Read
```

图 7.1　函数模板被实例化后的情形

```
kuangxiang@bogon:/mnt/hgfs/linux$ g++ MyProject.cpp -o MyProject
kuangxiang@bogon:/mnt/hgfs/linux$ nm MyProject
0000000000601078 B __bss_start
0000000000601190 b completed.7594
                 U __cxa_atexit@@GLIBC_2.2.5
0000000000601068 D __data_start
0000000000601068 W data_start
00000000004008c0 t deregister_tm_clones
0000000000400940 t __do_global_dtors_aux
00000000006000f8 t __do_global_dtors_aux_fini_array_entry
0000000000601070 D __dso_handle
00000000004009fc t _Z41__static_initialization_and_destruction_0ii
0000000000400a4f W _Z6funcaddIiET_RKS0_S2_
                 U _ZNSolsEi@@GLIBCXX_3.4
                 U _ZNSolsEPFRSoS_E@@GLIBCXX_3.4
```

图 7.2　利用 nm 命令观察可执行文件中函数模板被实例化后的情形

如果把 main 主函数中仅有的 cout 代码行注释掉，那么在目标文件中就不会找到 funcadd 字样了。

通过这个演示就能够很清楚地看到：编译器在编译的时候根据针对 funcadd 的调用来确定函数模板中 T 的类型。同时笔者还需要补充一句：如果并没有针对 funcadd 的调用代码，那么编译器不会产生任何与 funcadd 有关的代码，就好像函数模板 funcadd 从未存在过一样。

通过上面 main 主函数中这行代码(这个调用)可以观察到，编译器实际上产生了一个函数，但这个函数的名字和参数读者可要注意，这个函数的名字不是 funcadd，而是 funcadd <int>。完整一点说，函数的名字和参数如下：

```
int funcadd<int>(int const &, int const &)
```

然后编译器根据这些类型再生成具体针对这些类型的函数体代码。

7.1.2　类模板的实例化分析

类模板的实例化都不陌生，看一个范例。在 MyProject.cpp 的上面，增加如下代码行：

```
template<class T>
class ATPL
{
public:
    T m_i, m_j;
    ATPL(T tmpi = 0, T tmpj = 0)          //构造函数
    {
        m_i = tmpi;
        m_j = tmpj;
    }
};
```

上面定义了一个类模板，但并没有在 main 主函数中用到这个类模板。

编译，用 dumpbin 将.obj 文件导出成.txt 文件并搜索，发现找不到 ATPL 字样，这说明编译器对整个类模板的定义视而不见，没有任何反应，就跟不存在 ATPL 一样。

1. 类模板中的枚举类型

现在向 ATPL 类模板中增加一个枚举类型：

```
public:
    enum ECURRSTATUS
    {
        E_CS_Busy,
        E_CS_Free,
    };
```

当然，如果现在编译的话，编译器还是会对这个类模板视而不见。现在要想使用这个类中的枚举类型，就必须要写代码通过类模板实例化这个枚举类型。在 main 主函数中加入如下代码：

```
ATPL<int>::ECURRSTATUS myenum; //注意要加这个<int>
```

编译，用 dumpbin 将.obj 文件导出成.txt 文件并搜索，发现找不到 ATPL 字样。说明编译器很可能把上面这个代码行优化没了，因为程序中根本没用到 myenum 这个变量。在 main 主函数中再加一句代码：

```
myenum = ATPL<int>::E_CS_Busy;
```

编译，用 dumpbin 将.obj 文件导出成.txt 文件并搜索 ATPL 字样，只找到一个 ATPL 字样，这说明编译器仅仅是将整个“ATPL<int>::ECURRSTATUS”这么长一串内容看成一个枚举类型而已。**跟 ATPL 类模板本身，几乎可以说是没有什么关系。**

继续在 main 主函数中增加代码：

```
ATPL<int>::ECURRSTATUS myenum2;
myenum2 = ATPL<int>::E_CS_Free;
```

编译，用 dumpbin 将.obj 文件导出成.txt 文件并搜索 ATPL 字样，依旧只能搜索到一个 ATPL 字样——ATPL<int>::ECURRSTATUS。

继续在 main 主函数中增加代码：

```
ATPL<double>::ECURRSTATUS myenum3;
myenum3 = ATPL<double>::E_CS_Free;
```

编译，用 dumpbin 将.obj 文件导出成.txt 文件并搜索 ATPL 字样，这次可以搜索到两个 ATPL 字样——ATPL<int>::ECURRSTATUS 和 ATPL<double>::ECURRSTATUS，这表示有两种枚举类型了。

2. 类模板中的静态成员变量

为了演示观察方便，将 main 主函数中**所有代码行先注释掉**。

在类模板中增加一个静态成员变量的声明：

```
public:
    static int m_sti;
```

在类模板定义的下面，定义并初始化这个静态成员变量：

```
template <class T> int ATPL<T>::m_sti = 10;
```

编译，用 dumpbin 将. obj 文件导出成. txt 文件并搜索，没有发现 ATPL 字样和 m_sti 字样。

在 main 主函数中，使用一下 m_sti，增加如下代码：

```
ATPL<int>::m_sti = 18;
cout << ATPL<int>::m_sti << endl;
```

编译，用 dumpbin 将. obj 文件导出成. txt 文件并搜索，就会发现有一个 int 类型变量，名字叫作 ATPL<int>::m_sti。

在 main 主函数中，继续增加代码：

```
ATPL<float>::m_sti = 21;
cout << ATPL<float>::m_sti << endl;
```

编译，用 dumpbin 将. obj 文件导出成. txt 文件并搜索就会发现，对于编译器来讲，会产生两个不同类型的变量，一个名字叫 ATPL<int>::m_sti，另外一个名字叫 ATPL<float>::m_sti。

现在在类模板 ATPL 中再增加一个静态成员变量的声明，只不过这个静态成员变量的类型用的是类型模板参数所代表的类型：

```
public:
    static T m_sti2;
```

在类模板定义的下面，定义并初始化这个静态成员变量：

```
template <class T> T ATPL<T>::m_sti2 = 10;
```

上面代码行要注意一下，因为给了一个初值 10，那么日后在指定 m_sti2 类型的时候程序员就一定要指定和 10 兼容的类型，不然编译就会报错。

例如，后续如果是这样写代码就会出现编译错误：

```
ATPL<string>::m_sti2; //编译错误,因为<>中是 string 类型
```

但这样写代码就没有问题：

```
ATPL<int>::m_sti2; //没有问题,因为<>中是 int 类型
```

这里就可以看到类模板和类的不同之处：只有给进来具体类型，有些语法是否正确或者错误才能看得出来，所以有一些语法的检查，如果牵扯到这种类型模板参数的，那么编译器必须遇到这种实例化代码（如 ATPL<string>::m_sti2）的时候，才能检查出问题来。

在 main 主函数中继续增加如下代码：

```
ATPL<int>::m_sti2 = 132;
ATPL<float>::m_sti2 = 1050;
cout << ATPL<int>::m_sti2 << endl;
cout << ATPL<float>::m_sti2 << endl;
```

执行起来，看一看结果：

```
18
21
132
1050
```

结果正常。编译，用 dumpbin 将.obj 文件导出成.txt 文件并搜索就会发现，对于编译器来讲，会产生出如下几个类型的变量：

```
static int ATPL<int>::m_sti
static int ATPL<float>::m_sti
static int ATPL<int>::m_sti2
static float ATPL<float>::m_sti2
```

请注意，具体的 ATPL 模板相关的类并没有被实例化出来，编译器只会把上面的内容**当成 4 个变量**去看待。

3. 类模板的实例化

为了演示观察方便，将 main 主函数中**所有代码行先注释掉**。

有了类模板，那么类是如何被实例化出来的呢？在 main 主函数中加入如下代码：

```
ATPL<int>* pobj = NULL;
```

编译，用 dumpbin 将.obj 文件导出成.txt 文件并搜索就会发现，没有产生任何 ATPL 相关的内容，也就是说根本没实例化出来任何类，就好像这个类模板不存在一样。所以这种指针的用法，不会实例化出具体类的。

注释上面这行代码，加入如下代码：

```
const ATPL<int>& yobj = 0; //因为构造函数允许缺省参数，这里有隐式类型转换
```

在类模板的构造函数中设置断点并调试，发现确实能调用类模板的构造函数。如果构造函数都调用了，显然这个类模板肯定被实例化出来了。

编译，用 dumpbin 将.obj 文件导出成.txt 文件并搜索，找 ATPL 字样，不难发现，编译器已经实例化出来了这个 ATPL<int>类了（标志就是 ATPL<int>:: ATPL<int>这种名字），如图 7.3 所示（构造函数）。

```
1798         0 number of line numbers
1799  60501020 flags
1800           Code
1801           COMDAT; sym= "public: __thiscall ATPL<int>::ATPL<int>(int,int)" (??0?$ATPL@H@@QAE@HH@Z)
1802           16 byte align
1803           Execute Read
1804
```

图 7.3　利用类模板实例化出类的情形（可以看到实例化成类后的构造函数）

上面这种写法其实类似于下面：

```
ATPL<int> tmpobj(0);
const ATPL<int> &yobj = tmpobj;
```

综合而言，上面不但实例化出来了 ATPL<int>类，同时了也定义了属于该类的对象，因为模板类型确定了，这个对象所占用的内存空间也是确定的。那 m_i 和 m_j 两个成员变

量都是4字节，加起来是8字节。

4. 成员函数的实例化

现在向ATPL类模板中增加两个public修饰的成员函数：

```
public:
    void func1() const { cout << "ATPL::func1()" << endl; }
    void func2() const { cout << "ATPL::func2()" << endl; }
```

从前面的代码已经看到ATPL<int>类被实例化出来了，那么与这个类相关的这两个成员函数一定会被实例化出来吗？或者说有必要被实例化出来吗？

编译，用dumpbin将.obj文件导出成.txt文件并搜索，并没有找到func1或者func2字样，这说明编译器虽然实例化出来了ATPL<int>类，但是并没有实例化出来类中的func1、func2等成员函数。因为做这种没意义的实例化会花费大量的时间和空间，编译器觉得不值得。

怎么才能实例化出成员函数来呢？当然是在代码中有对这些成员函数调用的代码出现时，这并不难想象。

在main主函数中，继续增加如下代码：

```
yobj.func1();
```

执行起来，看一看结果：

```
ATPL::func1()
```

编译，用dumpbin将.obj文件导出成.txt文件并搜索，一定能够找到实例化后的func1。

7.1.3　多个源文件中使用类模板

在一个项目中，可能有多个源文件(.cpp文件)，每个源文件可能都要用到这个类模板，所以，一般在做实际项目的时候，是把类模板定义、实现等所有的相关内容都放到一个.h头文件中去(对于普通的类，定义和实现不能都放在一个头文件中，这是普通类和类模板的区别之一)。

基于这个想法，把刚才的类模板ATPL放到了一个专门的头文件mytemplate.h中去，并在MyProject.cpp的前面，包含该头文件。

```
#include "mytemplate.h"
```

而mytemplate.h的完整内容如下：

```
#ifndef __MYTEMPLATE__
#define __MYTEMPLATE__
using namespace std;
template<class T>
T funadd(const T& a, const T& b)
{
    T addhe = a + b;
```

```
    return addhe;
}

template<class T>
class ATPL
{
public:
    enum ECURRSTATUS
    {
        E_CS_Busy,
        E_CS_Free,
    };

public:
    T m_i, m_j;
    ATPL(T tmpi = 0, T tmpj = 0)          //构造函数
    {
        m_i = tmpi;
        m_j = tmpj;
    }

public:
    static int m_sti;
    static T m_sti2;

public:
    void func1() const { cout << "ATPL::func1()" << endl; }
    void func2() const { cout << "ATPL::func2()" << endl; }
};
template<class T> int ATPL<T>::m_sti = 10;
template<class T> T ATPL<T>::m_sti2 = 10;
#endif
```

在 MyProject.cpp 的上面，增加一个叫作 myfunc 的全局函数：

```
void myfunc()
{
    ATPL<int> myobj;
    myobj.m_sti2 = 18;
    cout << myobj.m_sti2 << endl;
}
```

在 main 主函数中，注释掉以往所有代码，保持为空。

编译，用 dumpbin 将.obj 文件导出成.txt 文件并搜索 ATPL 字样，可以发现，编译器实例化出了 ATPL<int>类，参考图 7.3。

为什么会实例化出了 ATPL<int>类呢？是因为 myfunc 函数中有这样一句代码：ATPL<int> myobj;。即便是程序中没调用过 myfunc 函数，但事实证明，编译器依然实例化出了 ATPL<int>类。

这里将一个新的文件 myfunc.cpp(已经写好，放在了与 MyProject.cpp 相同的目录下)加入到当前的 Visual Studio 项目中来。如何加入？

在解决方案资源管理器中右击项目下面的“源文件”文件夹，并在弹出的快捷菜单中选

择“添加”→“现有项”命令，如图 7.4 所示。

图 7.4　往一个项目中增加一个已经存在的.cpp 源程序文件

在弹出的对话框中选择 myfunc.cpp 并单击“添加”按钮，就可以把 myfunc.cpp 文件添加到“源文件”这个文件夹中来，这也就相当于把文件添加到项目中来了，如图 7.5 所示。

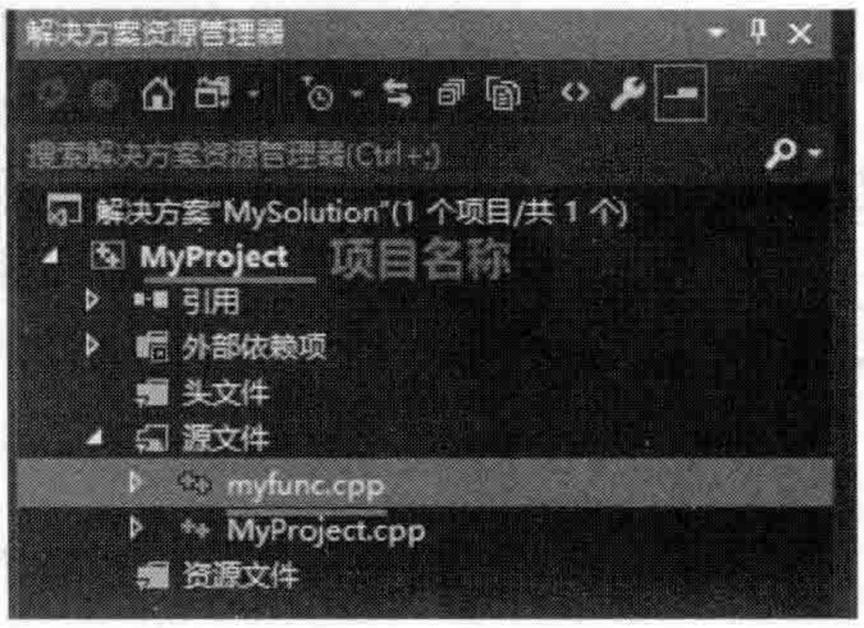

图 7.5　myfunc.cpp 源程序文件被增加到项目 MyProject 中来

现在整个项目（工程）就包括 MyProject.cpp 和 myfunc.cpp 两个源程序文件了。myfunc.cpp 文件的内容如下：

```
#include <iostream>
#include "mytemplate.h"
using namespace std;
int ftest()
{
    ATPL<int> myobj;
    myobj.m_sti2 = 21;
    cout << myobj.m_sti2 << endl;
    return 15;
}
```

编译一下工程，用 dumpbin 分别将 MyProject.obj 导出成 MyProject.txt，将 myfunc.obj 导出成 myfunc.txt：

```
dumpbin /all MyProject.obj > MyProject.txt
dumpbin /all myfunc.obj > myfunc.txt
```

前面说过了，MyProject.txt 中经过确认，编译器实例化出了 ATPL<int>类。那么在 myfunc.txt 中经过寻找，也证实了编译器实例化出了 ATPL<int>类，这是因为在 myfunc.cpp 的 ftest 函数中代码行 ATPL<int> myobj;的存在，虽然 **ftest 函数并没有被调用**。

现在相当于在两个.obj 文件（MyProject.obj 和 myfunc.obj）中有两个相同的 ATPL<int>

类了，那么编译器一般会这样处理：在**编译**的时候可能在多个.obj 文件产生了**多个相同的ATPL<int>类**，但是**链接的时候会只保留一个** ATPL<int>类的实体，其余的会忽略掉。

1. 虚函数的实例化

在 mytemplate.h 中向以往的类模板 ATPL 中增加两个 public 修饰的虚函数：

```
public:
    virtual void virfunc1() { cout << "ATPL::virfunc1()" << endl; }
    virtual void virfunc2() { cout << "ATPL::virfunc2()" << endl; }
```

编译一下工程，用 dumpbin 将 MyProject.obj 导出成 MyProject.txt，在其中查找 virfunc1 和 virfunc2 会发现，虽然从未调用过 virfunc1 和 virfunc2，但能够找到这两个虚函数。为什么呢？其实不难理解：因为有虚函数，所以需要产生虚函数表，而虚函数表里是各个虚函数的地址，既然**需要各个虚函数的地址，那么必须要实例化出每个虚函数来**。

2. 显式实例化

可以在一个.cpp 源文件的上面位置（任何函数之外）写如下的代码对一个类进行显式实例化：

```
template class ATPL<double>;
```

编译一下工程，用 dumpbin 将 MyProject.obj 导出成 MyProject.txt，在其中查找 ATPL<double>字样，能找到很多结果。

上面这种语法叫**类模板的显式实例化**，这种语法似乎会把该类模板中的所有内容（类、成员函数、虚函数、静态成员变量等）都实例化出来，不管是否调用过这些成员函数等。

当然，也可以实例化单独的成员函数，注释掉上面的 template class ATPL<double>;代码行，然后同样在一个.cpp 源文件的上面位置（任何函数之外）写如下的代码：

```
template void ATPL<double>::func2() const;
```

编译一下工程，用 dumpbin 将 MyProject.obj 导出成 MyProject.txt，在其中查找 ATPL<double>字样，看得出来，这个类 ATPL 本身并没有被实例化出来（因为找不到该类的构造函数字样 ATPL<double>::ATPL<double>(double,double)），但是这个成员函数确实被实例化出来了。

这里只是展示这种显式实例化的写法，语法上允许。但是这种显式的实例化到底有什么用，读者以后面对不同的编译器时可能会用到，在这里因为 Visual Studio 2019 这款编译器已经把自动实例化做的很好，所以常规情况下不需要显式实例化这种写法，但是也不排除以后实际开发中遇到的类模板过大（代码过多），程序员会受到每次修改代码编译起来时间过长等问题的困扰，这种情况有可能需要显式实例化的写法。读者以后如果遇到了可能会有更深入的体会，这里就不多谈了。

7.2 炫技写法

在面试的时候，不免会碰到一些古里古怪的问题，个别面试官是为了炫技或通过难为你以得到一种快感。但是不管遇到哪种情况，作为应聘者，都需要微笑面对，从容应答，否则这份工作一定不会属于你。

7.2.1　不能被继承的类

读者都知道，C++11 中提供 final 关键字使得类不能够被继承。那么，如何在不用 final 关键字的情况下，实现一个不能被继承的类呢？

听说可以用**友元函数＋虚继承**来实现，在这里就试一试。但这种实现方式有它的副作用：友元是破坏封装的。虚继承的消耗是比较大的，只有非常必要的情况下才去使用。

在 MyProject.cpp 的上面，增加如下代码行：

```
class A
{
private:
    A() {}
    friend class B;                          //B 可以调用 A 的私有构造函数
};
class B : public A
{
public:
    int m_b;
};
class C :public B
{
public:
    int m_c;
};
```

在 main 主函数中加入如下代码：

```
B myobjb;
myobjb.m_b = 15;

C myobjc;
myobjc.m_c = 20;
```

编译一下，可以成功，因为 B 类是 A 类的友元类，B 类构造函数可以调用 A 类构造函数，所以能成功构造出 B 对象来。

但是，如果希望 B 类不能被继承，现在显然没达到目的，因为 B 类现在能够被 C 类继承。怎么办呢？

解决办法就是用 B 类虚继承 A 类。代码如下：

```
class B : virtual public A //虚继承 A
{
    ...
}
```

现在再次编译，会发现 C myobjc;代码行变得不合法了。

读者都知道，虚基类 A 的构造函数是由孙子来调用，也就是由 C 类来调用的，但是虚基类 A 的构造函数是私有的，只有友元类 B 能调用，孙子类 C 无法调用，这样就无法创建出 C 类对象来。

最终结果就是B类对象可以被正常构造，但是继承B的C类对象无法被正常构造，可以认为这个B类就是最终想得到的无法被继承的类。

7.2.2 类外调用私有虚成员函数

有些人比较喜欢炫技，所以会写出下面的代码，笔者完全不提倡这种写法。现在，拿到这里进行讲解，目的是避免将来读者真碰到这样的考题时会感到茫然无措。

注释掉以往的所有代码，在MyProject.cpp的上面，增加如下代码行：

```
class A
{
private:
    virtual void virfunc()
    {
        myfunc();
    }
    void myfunc()
    {
        cout << "A::myfunc()运行了" << endl;
    }
};
```

观看上面代码不难看到，虚函数virfunc是私有的。私有的虚函数是否能被调用呢？其实能调用，于是有人就写出了非常难看的代码。在main主函数中，加入如下代码：

```
A aobj;
(reinterpret_cast<void(*)()>(**(int**)(&aobj)))();
```

执行起来，看一看结果：

```
A::myfunc()运行了
```

根据结果可以看到，正常调用了虚函数。

毫无疑问，上面的代码很难看，但实际上做的事情就是通过虚函数表调用虚函数，这里所有用到的知识前面都讲解过。回忆一下3.2节，曾经详细写过手工调用虚函数的代码，不知读者是否还记得。

其实，上面这段这么难看的代码就是手工调用虚函数的代码，把这些代码都写到一起，结果就这么长，这么难看。把这一行难看的代码拆解一下看。

(1) A是空类，但因为其中有虚函数，因此sizeof值是4字节，这4字节正好是虚函数表指针。这就好办了：

```
long* pvptr = (long*)&aobj;                    //把对象aobj地址拿到
```

(2) 拿到虚函数表指针：

```
long* vptr = (long*)(*pvptr);                  //把对象aobj地址中的内容(虚函数表指针)拿到,
现在vptr就指向了虚函数表
```

(3) 定义一个函数指针类型：

```
typedef void( * Func)(void);                    //定义一个函数指针类型
```

(4) 将函数指针 f 赋值为第一个虚函数地址：

```
Func f = (Func)vptr[0];                         //将函数指针 f 赋值为第一个虚函数地址
```

(5) 调用虚函数：

```
f();                                            //调用虚函数
```

到此为止，main 主函数中这行难看的代码分解一下，就是上面这几句。

这里笔者就不多讲，也不提倡这样写。

总之，这种通过虚函数表指针直接调用虚函数的写法，就好像一个房间本来是有门的，但非不从门走，而从窗户进。绕过了编译器的检查机制，对于研究虚函数表这个话题和技术有意义，但对实际应用没有什么意义。

后　记

本书写到这里，就要结束了。

笔者相信，无论什么层次、什么实力的C++程序员，都能从这本书中有所收获。

C++方面的知识可以说是博大精深，每个人即使终其一生不断地学习，也只能学到其中的一部分甚至很少的一部分。更何况每个人的寿命有限，黄金的学习时间更有限，从二十几岁毕业，到35岁之间只有十几年，这十几年时间绝大部分人可能会面临恋爱、结婚、生子，孩子的供养需要一笔不小的费用支出，当你35岁之后，你们的父母的年纪有60～70岁了，他们面临着老迈多病，需要赡养、陪伴等，所以，每个人的黄金学习时间真的不长。

怎样用最短的时间，最有效率的学习，挣更多的钱来养家，来提高生活品质，笔者相信这是绝大多数程序员都必须要面对和思考的现实问题。如果能够在固定长度的时间里，学到更牢固、更高品质的知识，就会比其他人更具有社会竞争力，更容易谋得好职位，拿到好薪水。

这个时代有非常多的诱惑，例如有人告诉你学学汇编语言，这样基础更牢固；学学算法，这样让自己更高端；学学编译原理、学学计算机原理……

你会发现需要学的东西实在太多太多了，学了哪样都有收获，但是每种收获你都需要投入大量的时间和精力才能换来，都是要付出代价的。

忍住诱惑，不要人云亦云，不要听别人鼓动一下你就去学一个东西。想清楚当前自己最需要什么，什么是当下最紧迫需要掌握的，学的目的是要干什么，其他不紧急的，以后再说。

在想清楚自己要学什么之后，接下来的问题就是怎么样能够在有限的时间里学到更牢固、更高品质的知识。笔者认为最重要的就是**选对教材（书籍），跟对老师**。教材、老师从来不缺，但是，高品质的教材和能够把最重要的知识很好地传递给学习者的老师，是非常稀缺的。读者一定要仔细寻找，慎重选择。

笔者希望把好运送给每位读者，祝你们都能找到合适的图书和心仪的老师，能够让自己的学习之路走得更通畅，在职场中能够顺心如意！

书中主题 C++对象知识 对象结构的发展及演化、所占空间、各种构造函数的语义等、成员初始化列表等。

C++虚函数 虚函数表、虚函数表指针、多重继承虚函数表分析、辅助工具介绍等。

C++数据语义 数据成员布局和存取、多种继承关系下数据成员的布局、多重继承结构时虚基类表内容分析、this指针调整等。

C++函数语义 成员函数调用方式、静态动态绑定、多态性体现、虚析构函数成因剖析、inline函数详解等。

C++对象构造语义 对象构造顺序、对象复制及析构语义、局部和全局对象的构造与析构，临时对象、模板实例化等。

环境标准 程序可以运行在Visual Studio、g++编译环境。书中全部实例通过Visual Studio 2019编译环境演示、部分实例也通过g++进行差异化演示。

学习资源 全部实例源代码可以扫描右方二维码或者关注“程序员速成”公众号获取。

图书查询·扩展阅读

书圈

清华社官方微信号

扫 我 有 惊 喜

上架指导：计算机/编程/C++

ISBN 978-7-302-55126-3

9 787302 551263 >

定价：79.00元